FACULTY OF ENGINEERING
AND ARCHITECTURE

Management and Analysis of Short Texts: A Graph-Based Approach

Maxime Deforche

Doctoral dissertation submitted to obtain the academic degree of
Doctor of Computer Science Engineering

Supervisors

Prof. Guy De Tré, PhD* - Prof. Antoon Bronselaer, PhD* - Ilse De Vos, PhD**

* Department of Telecommunications and Information Processing
Faculty of Engineering and Architecture, Ghent University

** Department of Literary Studies
Faculty of Arts and Philosophy, Ghent University

Month 20XX

GHENT
UNIVERSITY

ISBN null
NUR 984, 995
Wettelijk depot: null

Members of the Examination Board

Chair

Honorary Prof. Ronny Verhoeven, PhD, Ghent University

Other members entitled to vote

Prof. Walter Daelemans, PhD, Universiteit Antwerpen
Prof. Kristoffel Demoen, PhD, Ghent University
Prof. Jefrey Lijffijt, PhD, Ghent University
Joachim Nielandt, PhD, Digitaal Vlaanderen

Supervisors

Prof. Guy De Tré, PhD, Ghent University
Prof. Antoon Bronselaer, PhD, Ghent University
Ilse De Vos, PhD, Ghent University

Dankwoord

This will be written after the first/internal defence.

Table of Contents

Dankwoord i

Table of Contents iii

Summary vii

Samenvatting xi

List of Figures xv

List of Tables xxi

List of Algorithms and Listings xxv

List of Acronyms xxvii

1 Introduction 1

1.1 The Evolution of Data . . . 2
1.2 Big Data: Challenges and Opportunities . . . 3
1.2.1 The Four V's of Big Data . . . 3
1.2.2 Generating Value from Big Data . . . 7
1.2.3 Challenges of Textual Data Analysis . . . 9
1.3 Byzantine Book Epigrams . . . 10
1.3.1 The Database of Byzantine Book Epigrams . . . 12
1.3.2 Computational Challenges of Analysing Byzantine Book Epigrams . . . 13
1.3.3 The Interconnected Texts Project . . . 14
1.4 Objectives and Research Questions . . . 15
1.4.1 Data Management and Textual Representation . . . 15
1.4.2 Short Text Similarity . . . 16
1.4.3 Context-Aware Short Text Similarity . . . 17
1.4.4 Analysis of Short Texts . . . 18
1.5 Outline . . . 19
1.6 Publications . . . 20
References . . . 22

2 A Graph Data Model for Short Interconnected Texts 31
2.1 Introduction . . . 32
2.2 Related Work . . . 34
2.3 Preliminaries . . . 36
2.3.1 Text and Textual Decomposition . . . 37
2.3.2 Graphs and Graph Databases . . . 37
2.4 Representing a Textual Corpus as a Graph . . . 39
2.4.1 Hierarchical Decomposition of Textual Documents . . . 39
2.4.2 Formal Description of the Graph-Based Text Model . . . 40
2.4.3 Constructing the Graph Representation from a Textual Corpus . . . 42
2.5 Modelling Metadata and Additional Textual Properties . . . 45
2.5.1 Contextual Metadata . . . 45
2.5.2 Textual Properties . . . 46
2.6 Practical Applicability . . . 46
2.6.1 Example Dataset and Database Setup . . . 46
2.6.2 Identifying Important Textual Elements . . . 48
2.6.3 Interconnection Identification and Visualisation . . . 50
2.6.4 Pattern Matching . . . 51
2.7 Conclusions and Future Work . . . 53
References . . . 55

3 A Hierarchical Orthographic Similarity Measure for Short Texts 61
3.1 Introduction . . . 62
3.2 Related Work . . . 64
3.3 Preliminaries . . . 67
3.3.1 Similarity Measures . . . 67
3.3.2 Fuzzy Sets . . . 68
3.3.3 Fuzzy Similarity Relations . . . 69
3.3.4 Fuzzy Graphs and Fuzzy Labelled Property Graphs . . . 70
3.4 Hierarchical Orthographic Similarity Measure . . . 70
3.4.1 String-Based Similarity Measure . . . 71
3.4.2 Node-Based Similarity Measure . . . 74
3.4.3 Full Hierarchical Similarity Calculation . . . 80
3.5 Experiments . . . 83
3.5.1 Dataset . . . 83
3.5.2 Experimental Setup . . . 84
3.5.3 Results . . . 86
3.5.4 Practical Applicability . . . 93
3.6 Conclusions and Future Work . . . 98
References . . . 100

4 A Context-Aware and Responsible Similarity Measure for Short Texts 107
4.1 Introduction . . . 108
4.2 Related Work . . . 110
4.3 Preliminaries . . . 113
4.3.1 Aggregation Operators . . . 113
4.3.2 The Ordered Weighted Averaging Operator . . . 114
4.3.3 Set-Based Similarity Measures . . . 114
4.4 Context-Aware and Responsible Fusion of Similarity Scores . . . 115
4.4.1 OWA-Based Fusion of Similarity Scores . . . 116
4.4.2 Confidence Score Computation . . . 118
4.4.3 Integration into the Graph-Based Text Model . . . 120
4.5 Experiments . . . 121
4.5.1 Dataset . . . 122
4.5.2 Similarity Measures . . . 123
4.5.3 Experimental Setup . . . 127
4.5.4 Results . . . 128
4.5.5 Practical Applicability . . . 133
4.6 Conclusions and Future Work . . . 135
References . . . 139

5 Ranking of Satisfaction-Confidence Pairs 145
5.1 Introduction . . . 146
5.2 Related Work . . . 148
5.3 Preliminaries . . . 150
5.3.1 L-grades . . . 150
5.3.2 Order Relations . . . 154
5.4 Ranking L-grades . . . 155
5.4.1 Full Dominance and Partial Ordering . . . 156
5.4.2 Ranking Based on Lexicographic Ordering . . . 156
5.4.3 Ranking Based on Global Preference . . . 159
5.4.4 Distance Between L-grades . . . 163
5.5 Practical Applicability . . . 165
5.5.1 Ranking Similarity-Confidence Pairs . . . 165
5.5.2 Ranking Geographical Query Results . . . 169
5.6 Conclusions and Future Work . . . 175
References . . . 178

6 Context-Aware and Responsible Short Text Clustering 183
6.1 Introduction . . . 184
6.2 Related Work . . . 186
6.3 Preliminaries . . . 189

6.3.1 K-Medoids . . . 189
6.3.2 Hierarchical Agglomerative Clustering . . . 191
6.3.3 Fuzzy C-Medoids . . . 193
6.4 Context-Aware and Responsible Clustering of Short Texts . . . 195
6.4.1 Similarity and Confidence Score Extraction . . . 195
6.4.2 Confidence Integration Strategies . . . 196
6.5 Experiments . . . 200
6.5.1 Dataset . . . 201
6.5.2 Experimental Setup . . . 201
6.5.3 Results . . . 207
6.5.4 Practical Applicability . . . 216
6.5.5 Discussion . . . 226
6.6 Conclusions and Future Work . . . 227
References . . . 229

7 Conclusions and Future Work 235
7.1 Summary of Contributions . . . 236
7.2 Answers to the Research Questions . . . 237
7.2.1 Data Management and Textual Representation . . . 237
7.2.2 Short Text Similarity . . . 239
7.2.3 Context-Aware Short Text Similarity . . . 240
7.2.4 Analysis of Short Texts . . . 242
7.3 Practical Applicability for the Database of Byzantine Book Epigrams . . . 245
7.4 Short Text Analysis in the Age of Large Language Models . . . 246
7.5 Future Work . . . 247
References . . . 249

Summary

Today, data are generated at a scale and pace without precedent, driven by sensors, online platforms, commercial transactions, and, most recently, generative artificial intelligence. This phenomenon, widely known as *big data*, poses new challenges, the four most prevalent of which are *volume*, *velocity*, *variety*, and *veracity*. The first two concern the quantity and the speed of data production and processing, and modern distributed database architectures have largely addressed both. Variety, the many formats data can take, is more demanding. Schemaless data stores now accommodate everything from structured records to free text, images, and video, at least where storage and management are concerned, but the accurate querying of such partially structured data is still difficult. Veracity, the trustworthiness of data, remains largely unresolved. Data are often incomplete, noisy, or uncertain, or they originate from sources that cannot be fully trusted, and this unreliability propagates through preprocessing and analytical pipelines, undermining the insights drawn from the data. This thesis focuses on one prominent form of varied data, namely text, and on the veracity concerns that accompany its management and analysis.

Text is one of the most prevalent and most varied forms of data, with no fixed structure, widely differing lengths, and a multitude of styles. Modern natural language processing techniques have made remarkable progress in analysing text, largely through vector embeddings that encode the meaning of words and documents. These embeddings, however, rest on statistical patterns learned from vast bodies of text, and their performance deteriorates sharply under two conditions that coincide in the setting studied here. Short texts, such as titles, poems, lyrics, and posts on social media platforms, offer too little context for reliable semantic analysis. Low-resource languages, in turn, lack the training material and the language technology on which such embeddings are built. The difficulty deepens when the contextual metadata accompanying these texts, a potentially valuable source of additional information, are themselves incomplete, imprecise, or absent. Any analysis drawing on those metadata inherits their veracity concerns. Established techniques therefore fail to deliver reliable results for collections of short texts in low-resource languages with incomplete metadata, despite the valuable insights such collections may hold.

In this thesis we respond to these limitations by researching computational (intelligence) techniques for managing and analysing precisely such collections. Four research questions structure the work, asking how short texts and their metadata can best be represented, how the similarity between texts can be quantified, how uncertain context can refine that similarity, and how the refined similarity scores can serve querying, ranking, and clustering. Three design principles run through all four answers. First, the techniques require no language-specific resources, so they remain usable where none exist. Second, they are interpretable, with every computed score traceable to the factors that produced it. Third, they address veracity explicitly, reporting how reliable each result is instead of hiding that uncertainty in a single overall score. Together, these principles

shape a set of tools that follow a decision-support philosophy: they assist domain experts in their search for answers rather than replacing the experts themselves.

Our contributions begin with the modelling of texts and their metadata. We propose a graph-based data model that represents a collection of short texts as a labelled property graph, in which each text is organised as a hierarchy of discourse units, ranging from individual words through sentences or verses to complete texts. Identical content appearing in several texts is captured by one shared node, except at the top level, where each complete text retains its own node to remain uniquely identifiable. This turns the interconnections between texts into an explicit feature of the graph rather than a property that must be uncovered through pairwise comparison. At the same time, the graph imposes no fixed schema, so it accommodates the metadata associated with each text, whose structure and completeness differ from one text to the next. Managed in a graph database, the representation lends itself to declarative pattern matching, visual exploration, and graph-based analysis at any level of the hierarchy.

Building on this representation, we introduce a hierarchical orthographic similarity measure that quantifies how similar two texts are in their written form alone. The measure works bottom-up through the hierarchy. We compare the smallest units, the individual words, through a character-based edit distance, and each element at a higher level through an edit distance over its ordered child nodes, with substitution costs derived from the similarities established at the level below. A score between two text elements consequently reflects both how similar their constituent parts are and how those parts are arranged. Because every intermediate score remains available, each final score can be traced to the exact elements responsible for it. We record the results as graded relationships in the graph, where they are ready for querying and visualisation.

The written form alone, however, captures only part of the similarity between texts. We therefore enrich the textual similarity with contextual metadata, such as dates, locations, and authorship, through a context-aware and responsible fusion mechanism, where “responsible” means that veracity is explicitly considered. At its core lies an adapted ordered weighted averaging (OWA) operator, a fuzzy aggregation technique that combines inputs according to their ordered values rather than their source. Following a predefined weight vector, the operator merges the textual similarity score with the similarities obtained for the individual metadata fields. Missing metadata are handled dynamically by aggregating only over the fields that are present and rescaling the corresponding weights. This mechanism uses the available metadata to their fullest extent, imputes no values artificially, and produces a valid aggregated similarity regardless of how many fields are known. In addition, every aggregated similarity is accompanied by a confidence score expressing how far it can be trusted given the completeness of the metadata, since a high similarity based on complete metadata should be interpreted differently from one based on incomplete metadata. Each comparison of two texts hence yields a satisfaction-confidence pair, expressing how similar the texts and their metadata are (satisfaction) and how reliable that assessment is (confidence). We formalise such pairs as *L-grades*, which allows them to be used in logic reasoning. This explicit treatment of veracity sets our approach apart from conventional fusion techniques, which deliver a single score without revealing how confident one should be in it.

Two analytical techniques build on these L-grades. Because no canonical ordering of satisfaction-confidence pairs exists, we propose a family of ranking strategies designed specifically for L-grades, ranging from strict

lexicographic orderings to parameterised global preference functions. Each strategy is formally proven to induce a total order or a total preorder on any collection of L-grades. The balance between similarity and confidence thereby becomes an explicit and adjustable choice, grounded in a formal ordering, that reflects the priorities of the domain expert or the preferences of stakeholders rather than a rule fixed in advance. Complementing these rankings, we introduce novel clustering strategies, built on the K-Medoids, hierarchical agglomerative clustering, and fuzzy C-Medoids algorithms, that group texts on the basis of the context-aware similarities. The confidence values are integrated in two ways: they either shape the outcome through the distance calculations or expose the uncertainty of the resulting cluster assignments.

We implement and evaluate all techniques on the Database of Byzantine Book Epigrams (DBBE), a corpus of just over thirteen thousand short poems found in the margins of medieval Greek manuscripts. This corpus brings together every research challenge addressed in this thesis. The poems are brief, orthographically inconsistent, and heavily interconnected. They are written in Byzantine Greek, a language for which hardly any digital resources exist, and their contextual metadata are frequently incomplete or imprecise. To date, the study of these texts has depended almost entirely on manual curation, in which scholars identify interconnections, group related texts, and record metadata by hand. That process is inevitably shaped by individual judgement, and it yields a single interpretation of the corpus. Our techniques open new possibilities for this work, as they can propose candidate groupings of related texts, bring hidden connections to light, and point out metadata that one text lacks but a related text possesses. Beyond such suggestions, they support exploratory analysis, allowing experts to adjust parameters, pursue their own research questions, and consider alternative interpretations. In this way, the techniques assist scholars without replacing them, in a manner that is transparent, interpretable, and responsible.

The evaluation across all contributions indicates that the techniques can deliver this support. The hierarchical similarity measure recovers the expert-curated groupings to a large extent, performing best at the finer levels of the text hierarchy and substantially outperforming an existing two-level string-matching baseline. Enriching this textual similarity with metadata does not improve the peak performance of the measure, which is to be expected, since the groupings were established primarily on the basis of the written content of the texts. The enrichment does, however, sharpen the distinction between texts that merely look alike and texts that are genuinely related. Pairs that resemble one another in written form but disagree in their metadata are pushed apart, which improves precision considerably, while the genuinely related pairs are still retrieved, so that recall is preserved. The accompanying confidence scores then signal how far each assessment can be trusted. Metadata thus contribute discriminative information that the texts themselves cannot provide. Clustering points in the same direction, although the effect varies across algorithms. Driven by the context-aware similarities, the two medoid-based algorithms approximate the expert-curated partition closely, with K-Medoids improving on the purely textual baseline and fuzzy C-Medoids remaining close to it, while hierarchical agglomerative clustering loses some agreement. The confidence values can be integrated into the distance calculations, which at a moderate weight even improves that agreement slightly, or can be reported afterwards, in which case the partition itself is left untouched. Throughout, tuned configurations are reported as best-case results, consistent with the intended use in which experts calibrate parameters to their own priorities rather than adopting a fixed setting. Alongside these quantitative findings, querying and visualising the graph make patterns and

interconnections immediately accessible. The ranking strategies, in turn, order candidate matches by similarity, by confidence, or by any balance between the two. Because every step is configurable, from the OWA weights to the trade-off between similarity and confidence, the proposed tools open research opportunities well beyond the questions examined in this thesis.

Viewed in its entirety, the thesis delivers a coherent, interpretable, and mathematically rigorous set of tools for analysing short texts affected by variety and veracity concerns. Together they cover the whole path from representing the texts, through quantifying and contextually enriching their similarity, to ranking and grouping them. Unlike conventional embedding-based natural language processing techniques, whose output often remains unreliable for low-resource languages and is hard to verify, our techniques are deterministic, transparent, and veracity-aware. They are moreover adaptable, accommodating additional inputs in the fusion process, such as semantic measures, as well as alternative clustering algorithms. Although inspired by a single historical corpus, they can be applied to other collections of short, interconnected texts with incomplete metadata, from posts on social media platforms to product reviews and song lyrics. Their performance in such domains, however, remains to be evaluated. What carries over regardless of the corpus is the combination that motivated this work: an analysis that requires no training data and that makes its own uncertainty visible.

Samenvatting

Onder impuls van sensoren, onlineplatformen, commerciële transacties en, sinds kort, generatieve artificiële intelligentie worden gegevens tegenwoordig gegenereerd op een schaal en in een tempo zonder precedent. Dit fenomeen, algemeen bekend als *big data*, brengt nieuwe uitdagingen met zich mee. De vier meest prominente zijn volume, snelheid (*velocity*), verscheidenheid (*variety*) en betrouwbaarheid (*veracity*). De eerste twee betreffen de hoeveelheid gegevens en de snelheid waarmee die geproduceerd en verwerkt moeten worden. Moderne gedistribueerde databankarchitecturen hebben beide uitdagingen grotendeels opgelost. Verscheidenheid, de vele vormen die gegevens kunnen aannemen, stelt hogere eisen. Schemaloze databanksystemen kunnen inmiddels vrijwel elk formaat aan, van gestructureerde records over vrije tekst tot afbeeldingen en video, althans wat opslag en beheer betreft. Het nauwkeurig bevragen van dergelijke gedeeltelijk gestructureerde gegevens blijft echter moeilijk. Voor de vierde uitdaging, de betrouwbaarheid van gegevens, bestaat tot op heden geen alomvattend en toereikend antwoord. Gegevens zijn immers vaak onvolledig of onzeker, bevatten ruis of zijn afkomstig uit bronnen die niet volledig te vertrouwen zijn. Die onbetrouwbaarheid werkt door in de voorverwerking en de analyse, en ondermijnt zo de inzichten die uit de gegevens worden afgeleid. Dit proefschrift richt zich op een prominente vorm van gevarieerde gegevens, namelijk tekst, en op de betrouwbaarheidsproblemen die het beheer en de analyse ervan met zich meebrengen.

Tekst behoort tot de meest voorkomende en meest gevarieerde vormen van gegevens en informatie. Tekst heeft bovendien geen vaste structuur en komt voor in sterk uiteenlopende lengtes en talloze stijlen. Met moderne technieken voor natuurlijke taalverwerking (*natural language processing*) is opmerkelijke vooruitgang geboekt in de analyse van tekst, grotendeels dankzij *vector embeddings* die de betekenis van woorden en documenten numeriek voorstellen. Die *embeddings* berusten echter op statistische patronen die uit enorme hoeveelheden tekst zijn afgeleid. Hun prestaties verslechteren sterk onder twee omstandigheden die in de context van dit onderzoek samenvallen. Korte teksten, zoals titels, gedichten, songteksten en berichten op socialemediaplatformen, bieden te weinig context voor een betrouwbare statistische analyse. Talen met beperkte digitale bronnen (*low-resource languages*) beschikken op hun beurt niet over het tekstmateriaal en de taaltechnologie waarop dergelijke *embeddings* steunen. De moeilijkheid neemt nog toe wanneer de contextuele metadata bij deze teksten, een potentieel waardevolle bron van aanvullende informatie, zelf onvolledig of onnauwkeurig zijn of volledig ontbreken. Elke analyse die op deze metadata steunt, erft hun betrouwbaarheidsproblemen. Gevestigde technieken slagen er daarom niet in betrouwbare resultaten te leveren voor verzamelingen van korte teksten in talen met beperkte digitale bronnen, zeker niet wanneer de metadata bij die teksten onvolledig zijn. Zulke verzamelingen kunnen niettemin waardevolle inzichten bevatten.

Als antwoord op deze beperkingen onderzoeken we in dit proefschrift computationele (intelligentie)technieken voor het beheer en de analyse van precies zulke verzamelingen. Vier onderzoeksvragen karakteriseren het werk:

(i.) hoe korte teksten en hun metadata het best kunnen worden voorgesteld, (ii.) hoe de similariteit tussen teksten kan worden gekwantificeerd, (iii.) hoe onzekere context die similariteit kan verfijnen en (iv.) hoe de verfijnde similariteitsscores kunnen worden ingezet voor het bevragen, rangschikken en clusteren van teksten. Drie ontwerpprincipes vormen de basis van de antwoorden op deze vragen. Ten eerste vereisen de gebruikte technieken geen taalspecifieke bronnen of hulpmiddelen, zodat ze bruikbaar blijven waar die ontbreken. Ten tweede zijn ze interpreteerbaar, doordat elke berekende score herleidbaar is tot de factoren die eraan ten grondslag liggen. Ten derde behandelen ze betrouwbaarheid expliciet, door bij elk resultaat te vermelden hoe betrouwbaar het is in plaats van de onzekerheid te verbergen in één totaalscore. Samen geven deze principes vorm aan een reeks instrumenten die de filosofie van beslissingsondersteuning (*decision support*) volgen. Deze instrumenten staan domeinexperts bij in hun zoektocht naar antwoorden, in plaats van die experts te vervangen.

Onze bijdragen beginnen bij het modelleren van teksten en hun metadata. We stellen een graafgebaseerd gegevensmodel voor dat een verzameling korte teksten voorstelt als een gelabelde eigenschappengraaf (*labelled property graph*). Elke tekst is daarin georganiseerd als een hiërarchie van tekstuele eenheden (*discourse units*), gaande van afzonderlijke woorden over zinnen of verzen tot volledige teksten. Identieke inhoud die in meerdere teksten voorkomt, wordt samengebracht in één gedeelde node, behalve op het hoogste niveau, waar elke volledige tekst zijn eigen node behoudt en zo afzonderlijk herkenbaar blijft. Daardoor worden de onderlinge verbanden tussen teksten een expliciet kenmerk van de graaf in plaats van een eigenschap die via paarsgewijze vergelijking moet worden blootgelegd. Tegelijk legt de graaf geen vast schema op, zodat ook de metadata bij elke tekst een plaats krijgen, hoezeer hun structuur en volledigheid van tekst tot tekst ook verschillen. Doordat deze voorstelling in een graafdatabank wordt beheerd, leent ze zich voor declaratieve patroonherkenning (*pattern matching*), visuele verkenning en graafgebaseerde analyse op elk niveau van de hiërarchie.

Voortbouwend op deze voorstelling introduceren we een hiërarchische orthografische similariteitsmaat die uitsluitend op basis van de geschreven vorm kwantificeert hoezeer twee teksten op elkaar lijken. De maat werkt van onder naar boven door de hiërarchie heen. De kleinste eenheden, de afzonderlijke woorden, vergelijken we aan de hand van een karaktergebaseerde editeerafstand (*edit distance*). Voor elk element op een hoger niveau berekenen we een editeerafstand over zijn geordende kindnodes, met substitutiekosten die worden afgeleid uit de similariteiten die op het niveau eronder zijn vastgesteld. Een score tussen twee tekstelementen weerspiegelt bijgevolg zowel hoezeer de samenstellende eenheden op elkaar lijken als hoe ze zijn geordend. Doordat alle tussenscores beschikbaar blijven, kan elke eindscore worden herleid tot de precieze elementen waarop ze steunt. De resultaten leggen we vast als gegradeerde relaties in de graaf, waar ze meteen beschikbaar zijn voor bevraging en visualisatie.

De geschreven vorm verklaart slechts een deel van de similariteit tussen teksten. Daarom verrijken we de tekstuele similariteit met contextuele metadata, zoals datering, locatie en auteurschap, binnen een contextafhankelijke en verantwoorde aggregatiemethode. “Verantwoord” betekent hier dat betrouwbaarheid expliciet wordt meegenomen. Centraal in deze methode is een aangepaste operator voor *ordered weighted averaging* (OWA), een aggregatietechniek uit de vaagverzamelingenleer die invoerwaarden combineert op basis van hun positie in de rangschikking in plaats van hun herkomst. De operator voegt de tekstuele similariteitsscore samen met de similariteitsscores die voor de afzonderlijke metadatavelden zijn berekend, volgens een vooraf bepaalde gewichtsvector. Ontbrekende metadata worden dynamisch opgevangen door enkel te aggregeren over de

velden die wel aanwezig zijn en de gewichten van de overblijvende rangposities te herschalen. De beschikbare metadata worden zo ten volle benut, zonder dat waarden kunstmatig worden ingevuld, en de berekening levert een geldige similariteitswaarde op, ongeacht hoeveel velden bekend zijn. Daarnaast wordt elke geaggregeerde similariteit vergezeld van een betrouwbaarheidsscore, die uitdrukt in hoeverre die similariteit kan worden vertrouwd, gezien de volledigheid van de metadata. Een hoge similariteit die op volledige metadata steunt, moet immers anders worden geïnterpreteerd dan een even hoge similariteit die op onvolledige metadata steunt. Elke vergelijking van twee teksten levert daarmee een tevredenheid-betrouwbaarheidspaar op, dat aangeeft hoe gelijkaardig de teksten en hun metadata zijn (tevredenheid) en hoe betrouwbaar die beoordeling is (betrouwbaarheid). Dergelijke paren formaliseren we als *L-grades*, waardoor ze gebruikt kunnen worden voor logisch redeneren. Deze expliciete behandeling van betrouwbaarheid onderscheidt onze aanpak van conventionele aggregatietechnieken, die één score opleveren zonder aan te geven hoe betrouwbaar die is.

Deze *L-grades* gebruiken we in twee analytische technieken. Omdat er geen standaardordening van tevredenheid-betrouwbaarheidsparen bestaat, stellen we een familie van rangschikkingsstrategieën voor die specifiek voor *L-grades* zijn ontworpen, gaande van strikte lexicografische ordeningen tot geparametriseerde globale voorkeursfuncties. Voor elke strategie bewijzen we formeel dat ze om het even welke verzameling *L-grades* volledig ordent, als totale orde of als totale preorde. Het evenwicht tussen similariteit en betrouwbaarheid wordt daardoor een expliciete en aanpasbare keuze, verankerd in een formele orderelatie. Die keuze weerspiegelt de prioriteiten van de domeinexpert of de voorkeuren van de belanghebbenden, in plaats van een vooraf vastgelegde regel te volgen. In aanvulling op deze rangschikkingen introduceren we nieuwe clusteringtechnieken die voortbouwen op de *K-Medoids*, *hierarchical agglomerative clustering* en *fuzzy C-Medoids* algoritmen, en die teksten groeperen op basis van de contextafhankelijke similariteitsscores. De betrouwbaarheidsscores worden daarbij op twee manieren verwerkt. Ofwel bepalen ze het resultaat mee, doordat ze in de afstandsberekeningen worden opgenomen, ofwel worden ze achteraf gerapporteerd, waardoor de onzekerheid van de resulterende clustertoewijzingen zichtbaar wordt.

We implementeren en evalueren alle technieken op de teksten van de Database of Byzantine Book Epigrams (DBBE), een corpus van iets meer dan dertienduizend korte gedichten die zijn gevonden in de marges van middeleeuwse Griekse manuscripten. Dit corpus verenigt alle onderzoeksuitdagingen die in dit proefschrift aan bod komen. De gedichten zijn kort, orthografisch inconsistent en sterk onderling verbonden. Ze zijn geschreven in Byzantijns Grieks, een taal waarvoor nauwelijks digitale bronnen bestaan, en hun contextuele metadata zijn vaak onvolledig of onnauwkeurig. De studie van deze teksten steunt tot nog toe bijna volledig op handmatige curatie, waarbij onderzoekers onderlinge verbanden identificeren, verwante teksten groeperen en metadata stuk voor stuk registreren. Dat proces wordt onvermijdelijk gekleurd door individueel oordeel en levert één mogelijke interpretatie van het corpus op. Onze technieken openen hier nieuwe mogelijkheden, doordat ze groeperingen van samenhangende teksten voorstellen, verborgen verbanden blootleggen en metadata aanwijzen die in de ene tekst ontbreken, maar in een gerelateerde tekst wel bekend zijn. Naast dergelijke suggesties maken ze verkennende analyse mogelijk, waarin onderzoekers zelf parameters aanpassen, hun eigen onderzoeksvragen nastreven en alternatieve interpretaties afwegen. Zo ondersteunen de technieken de onderzoeker op een transparante, interpreteerbare en verantwoorde manier, zonder zijn rol over te nemen.

De evaluatie van alle bijdragen wijst erop dat de technieken die ondersteuning ook daadwerkelijk kunnen bieden. De hiërarchische similariteitsmaat reconstrueert de door experts samengestelde groeperingen in grote mate, met de beste resultaten op de fijnere niveaus van de teksthiërarchie en met een aanzienlijke voorsprong op een bestaande tweelagige baseline voor stringvergelijking. Het verrijken van deze tekstuele similariteiten met metadata verhoogt de piekprestatie van de similariteitsmaat niet, wat te verwachten valt, aangezien de groeperingen in de eerste plaats op basis van de geschreven inhoud van de teksten zijn vastgesteld. De verrijking verscherpt wel het onderscheid tussen teksten die louter op elkaar lijken en teksten die daadwerkelijk met elkaar verwant zijn. Paren die qua geschreven vorm op elkaar lijken, maar waarvan de metadata niet overeenstemmen, worden uit elkaar gedreven, wat de *precision* aanzienlijk verhoogt. Paren die orthografisch inconsistent zijn, maar veel metadatawaarden delen, worden juist naar elkaar toe getrokken, waardoor de *recall* op peil blijft. De bijbehorende betrouwbaarheidsscores geven vervolgens aan in hoeverre elke beoordeling kan worden vertrouwd. Metadata dragen zo onderscheidende informatie aan die de teksten zelf niet kunnen leveren. De clusteringexperimenten wijzen in dezelfde richting, al verschilt het effect van algoritme tot algoritme. Gestuurd door de contextafhankelijke similariteiten benaderen de twee *medoid*-gebaseerde algoritmen de door experts samengestelde partitie het dichtst, waarbij *K-Medoids* het zuiver tekstuele baselineresultaat verbetert en *fuzzy C-Medoids* er nauw bij aansluit. *Hierarchical agglomerative clustering* boet daarentegen wat overeenstemming in. De betrouwbaarheidsscores kunnen op twee manieren worden ingezet. Opgenomen in de afstandsberekeningen verbeteren ze die overeenstemming bij een gematigd gewicht zelfs licht, en achteraf gerapporteerd laten ze de partitie zelf ongewijzigd. In het hele proefschrift worden geoptimaliseerde configuraties gerapporteerd als *best-case*-resultaten, in lijn met het beoogde gebruik, waarin experts de parameters op hun eigen prioriteiten afstemmen in plaats van een vaste configuratie over te nemen. Naast deze kwantitatieve bevindingen maken bevraging en visualisatie van de graaf de onderliggende patronen en verbanden onmiddellijk toegankelijk. De rangschikkingsstrategieën ordenen op hun beurt kandidaat-overeenkomsten op similariteit, op betrouwbaarheid of op eender welk evenwicht daartussen. Omdat elke stap configureerbaar is, van de OWA-gewichten tot de afweging tussen similariteit en betrouwbaarheid, openen de voorgestelde instrumenten onderzoeksmogelijkheden die veel verder reiken dan de vragen die in dit proefschrift zijn behandeld.

In zijn geheel bekeken levert dit proefschrift een samenhangend, interpreteerbaar en wiskundig onderbouwd instrumentarium op voor de analyse van korte teksten die worden gekenmerkt door verscheidenheid en betrouwbaarheidsproblemen. Samen bestrijken de bijdragen het volledige traject van het voorstellen van de teksten over het kwantificeren en contextueel verrijken van hun similariteit tot het rangschikken en groeperen ervan. Voor talen met beperkte digitale bronnen leveren conventionele, op *embeddings* gebaseerde technieken voor natuurlijke taalverwerking vaak uitvoer op die onbetrouwbaar en moeilijk te verifiëren is. Onze technieken zijn daarentegen deterministisch en transparant, en behandelen betrouwbaarheid expliciet. Ze zijn bovendien aanpasbaar, onder andere doordat de aggregatiemethode aanvullende invoer kan opnemen, zoals semantische similariteitsmaten, en doordat de clustering ook op andere algoritmen kan steunen. Hoewel ze door één historisch corpus zijn geïnspireerd, kunnen ze worden toegepast op andere verzamelingen van korte, onderling verbonden teksten met onvolledige metadata, van berichten op socialemediaplatformen tot productrecensies en songteksten. Hoe ze in dergelijke domeinen presteren, moet echter nog worden geëvalueerd. Wat wel overdraagbaar is, ongeacht het corpus, is een analyse die geen trainingsgegevens vereist en die onzekerheden expliciet zichtbaar maakt.

List of Figures

1.1 Volume of data generated per year according to the IDC Global DataSphere report [15] and Huawei Data Storage report [16], expressed in Zettabytes (ZBs). 4

1.2 Example of an epigram with id 19512 [78] found in the margin of a Byzantine manuscript. . . 11

2.1 Examples of text representations: (a) an illustrative vector embedding for the words *cat*, *dog*, and *bird*, indicating that the words *cat* and *dog* are semantically closer to each other than to the word *bird*, and (b) a Knowledge Graph (KG) representation of the sentence "*The cat sat on the sofa and took a nap*" based on Natural Language Processing (NLP)-derived information, where entities are represented as nodes and edges represent semantic relationships such as actions or properties. 36

2.2 A graph-based text representation for the texts in Example 2.1. `Word` nodes are indicated in green and display their `text` property, whereas `Sentence` nodes are indicated in yellow, and `Text` nodes, including their numeric `id` property, are indicated in blue. Values of the `rank` property are displayed on the edges. 44

2.3 Neo4j Bloom visualisation of the subgraph containing the first verses of a subset of epigrams. `Verse` nodes are indicated in yellow, `Word` nodes are indicated in green and display their `text` property, and the `CONTAINS` relationships are shown as edges including their `rank` property. 51

2.4 Neo4j Bloom visualisation of the subgraph containing the epigrams with ids 18540 and 18542, both belonging to type 2150. `Epigram` nodes are indicated in blue and display their `id` property, `Verse` nodes are indicated in yellow, `Word` nodes are indicated in green and display their `text` property, and the `CONTAINS` relationships are shown as edges including their `rank` property. 52

3.1 Illustrations of semantic similarity approaches: (a) an illustrative vector embedding for the words *cat*, *dog*, and *bird*, where the angle θ between the vectors for *cat* and *bird* indicates their degree of similarity, with the smaller angle θ' between *cat* and *dog* indicating greater similarity, and (b) a fragment of an illustrative semantic network organised as a taxonomy, showing the concepts *cat*, *dog*, and *tree* and their relations to higher-level categories. 66

3.2 A graphical overview of the steps involved in computing the hierarchical similarity measure between textual components in the graph model. Starting from a textual corpus represented by the graph model from Chapter 2, a string-based score is calculated between the text values stored in the nodes representing the smallest textual components (green section). The pairwise scores between all other nodes at the same level are then computed hierarchically using a node-based similarity measure (red section). The result is a textual graph extended with similarity scores between all textual components at every level of the hierarchy. 72

3.3 A visual representation of the edit operations required to transform the string `RELEVANT` into the string `ELEPHATN`. Deletions are indicated in red, replacements in orange, insertions in green, and transpositions by a pair of crossing arrows. 74

3.4 The graph-based representation of the two texts in Example 3.2. The yellow nodes represent `Sentence` nodes and show their `id` property, whereas the green nodes represent the `Word` nodes and show their `text` property. The values of the `rank` property are shown on the directed edges between different levels of the graph, and the `similarity` values are shown on the undirected edges between word nodes at the same level. Only the relevant similarity scores are displayed: the similarity between the words “test” and “tset”, and the similarity between the sentence nodes for the case $r = 1$. 79

3.5 The same representation as in Figure 3.4, for the alternative scenario in which the similarity threshold r equals 0.7. The replacement cost between the words “test” and “tset” is now 0, which raises the similarity between the sentence nodes. 80

3.6 Subset of the graph-based text representation for the texts in Example 3.3 after computing the similarities between all elements. `Word` nodes are shown in green and display their `text` property; `Sentence` nodes are shown in yellow; `Text` nodes are shown in blue and display their numeric `id` property. The values of the `rank` property are shown on the directed edges (grey) between different levels of the graph, and the `similarity` values are shown on the undirected edges (black) between same-level textual elements. For clarity, only the similarities relevant to the minimum edit distance computation between the two texts are displayed, namely those between nodes from different texts whose similarity score is below 1. 82

3.7 F_1-scores as a function of the α-threshold for all parameter configurations of the proposed similarity measure and both configurations of the Two-Level string matcher. The left plot corresponds to the verse subset and the right plot to the epigram subset. 89

3.8 Precision-recall curves for the default configuration of the proposed similarity measure and the Damerau configuration of the Two-Level string matcher. The verse subset is displayed on the left and the epigram subset on the right. The remaining configurations are omitted for clarity, as they exhibit similar patterns. 89

3.9 Visualisation of the result of the query in Listing 3.2 using Neo4j Bloom. All SIMILAR_TO relationships between Epigram nodes with a similarity score of at least 0.95 are visualised. For clarity, small groups of two or three connected epigram nodes, or nodes with no remaining connections, are omitted from the visualisation. The SIMILAR_TO relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of this similarity. More fine-grained analysis can be performed by zooming into specific areas of the visualisation as well as by interacting with the graph and by expanding nodes into their hierarchical components. The green labels indicate the identifiers of two epigrams that belong to different groups in the visualisation, which are further examined in the following section. 95

3.10 Visualisation of the result of the query in Listing 3.3 using Neo4j Bloom. The hierarchical decompositions of epigrams 22826 and 21182 into their constituent verses and words are shown. Word nodes are indicated in green, Verse nodes in yellow, and Epigram nodes in blue. The edges between same-level textual elements represent the SIMILAR_TO relationships and display the computed similarity scores. Other edges, which connect nodes at different levels, represent the CONTAINS relationships that capture the hierarchical decomposition of the epigrams. The SIMILAR_TO relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of this similarity. 97

4.1 Overview of the computation of the aggregated similarity and confidence score for the similarity tuple $\mathbf{s} = (0.6, 1.0, \diamond, 0.7)$ and weight vector $\mathbf{w} = (0.3, 0.2, 0.3, 0.2)$ from Example 4.1. The restriction of the similarity tuple $\bar{\mathbf{s}}$ is indicated in green, the restriction of the weight vector $\overline{\mathbf{w}}$ is indicated in blue, and the residual weight vector $\overline{\mathbf{w}}^c$ is indicated in red. 118

4.2 Excerpt of the geographical location tree used to compare geographical metadata. The five granularity levels and the root are labelled on the right, ordered from the world root at the top to the site level at the bottom. Nodes shown in green denote values inherited from the closest known ancestor when the corresponding granularity is missing in the original Database of Byzantine Book Epigrams (DBBE) hierarchy: the *Lazio* and *Central Macedonia* nodes at the region level inherit their values from *Lazio* and *Central Macedonia* at the part level. 125

4.3 F_1-score as a function of the α-threshold for the empirical and average weight vector configurations of the proposed measure, plotted alongside the textual baseline (Custom 1 configuration of Chapter 3 on the epigram subset). 130

4.4 Visualisation of the result of the query in Listing 4.1 using the visualisation capabilities of Neo4j Bloom. Pairs of epigrams from epigram type 1982 whose aggregated similarity meets or exceeds 0.9 and whose confidence meets or exceeds 0.5 are connected by an edge. The CONTEXT_SIMILAR_TO relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of similarity. 134

4.5 Visualisation of the result of the query in Listing 4.2 using the visualisation capabilities of Neo4j Bloom. All CONTEXT_SIMILAR_TO relationships between pairs of epigrams from epigram type 1982 with an aggregated similarity of at least 0.9 are shown. Each relationship is coloured according to its confidence score: dark red indicates low confidence, bright green indicates high confidence, and yellow indicates moderate confidence. The relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of similarity. 136

5.1 Visualisation of $\mathbb{L}$. Both components s (satisfaction) and c (confidence) range from 0 to 1. The shaded region in the upper-right quadrant represents the area of simultaneously high satisfaction and high confidence, which is generally preferred by stakeholders. 157

5.2 Example of a membership function $\mu_{\approx}$ to model proximity $\approx$ between L-grades. The function decreases linearly from 1 at $d = 0$ to 0 at $d = \delta$, and remains 0 for all distances beyond δ. 165

5.3 Fragment of the geographical hierarchy used in the DBBE, containing the branches relevant to the illustrative example. Each row corresponds to a depth level and its associated granularity, from the virtual world root at depth 0 to specific sites at depth 5. Nodes shown in green denote values inherited from the closest known ancestor when the corresponding granularity level is absent in the original hierarchy, following the filling mechanism described in Section 4.5.2. The queried site in Example 5.9, *Monastery of St. Mary*, is highlighted in red. The queried city in the illustrative example, *Messina*, is highlighted in blue. 171

6.1 Illustration of a dendrogram produced by Hierarchical Agglomerative Clustering (HAC) for eight objects. Each leaf represents an object, and the merge height indicates the inter-cluster dissimilarity at which the corresponding clusters were joined. The dashed line marks a horizontal cut yielding $k = 3$ clusters, labelled C_1, C_2, and C_3. 193

6.2 Post-hoc confidence visualisation of a selection of the smaller K-Medoids clusters, obtained by clustering the full subset on the context-aware similarity computed with the $\mathbf{w}^{\text{ctx}}_{\text{KMedoids}}$ Ordered Weighted Averaging (OWA) weight vector using K-Medoids with $k_{\text{GT}} = 21$ clusters. The relationships between each epigram and the epigram representing the medoid of its cluster are coloured according to the confidence of their assignments using a gradient from red (low confidence, $c < 0.45$) through yellow (medium confidence) to green (high confidence, $c > 0.65$). 218

6.3 Silhouette coefficient of the three clustering algorithms applied to the full subset as a function of the number of clusters k, using the context-aware similarity computed with $\mathbf{w}_{\text{avg}}$. The vertical lines mark the silhouette-optimal number of clusters for each method, $k_{\text{KMedoids}} = 15$, $k_{\text{HAC}} = 10$, and $k_{\text{FCMdd}} = 16$, and the expert-curated ground truth granularity $k_{\text{GT}} = 21$. . . 223

6.4 Dendrogram produced by HAC for the thirteen epigrams from the types with identifiers 2311 (5 epigrams) and 4155 (8 epigrams), with the epigram ids shown along the horizontal axis. The input is based on the context-aware similarity computed with $\mathbf{w}_{\text{HAC}}^{\text{ctx}}$. The merge height indicates the inter-cluster dissimilarity at which the corresponding clusters were joined. The dashed red line indicates the cut that produces the optimal partition at $k_{\text{HAC}} = 2$ clusters, which coincides with the expert-curated number of clusters $k_{\text{GT}} = 2$ for this subset. 224

6.5 Bloom visualisation of the result of Listing 6.3, for the K-Medoids clustering of the full subset on the context-aware similarity computed with the $\mathbf{w}_{\text{KMedoids}}^{\text{ctx}}$ OWA weight vector with $k_{\text{GT}} = 21$ clusters. The relationships between each epigram and the medoid of its cluster are coloured according to the confidence of their assignments, using a gradient from red (low confidence, $c < 0.45$) through yellow (medium confidence) to green (high confidence, $c > 0.65$). The SUBJECT_OF relationships connect the epigrams to their recorded subjects. 225

List of Tables

1.1 Textual content of the epigram in Figure 1.2 with id 19512 [78]. 11

1.2 Textual content of the epigram with id 22826 [78]. 12

2.1 Textual content of epigram 18306 [19] from type 2150 before and after preprocessing. 47

2.2 Textual content of epigram 32232 [19] from type 4245 after preprocessing. The recurring και ι pattern in the intermediate verses is highlighted in bold. 48

2.3 Overview of the most frequently occurring words and verses in the dataset, ranked by their in-degree in the graph. 49

3.1 Parameter values for the three configurations of the proposed similarity measure. 85

3.2 Experimental results for the verse subset using the proposed similarity measure. For each quality measure, the highest value across all parameter configurations and α values is highlighted in bold. 87

3.3 Experimental results for the epigram subset using the proposed similarity measure. For each quality measure, the highest value across all parameter configurations and α values is highlighted in bold. 88

3.4 Experimental results for the verse subset using the Two-Level string matcher. For each quality measure, the highest value across both configurations and all α values is highlighted in bold. 91

3.5 Experimental results for the epigram subset using the Two-Level string matcher. For each quality measure, the highest value across both configurations and all α values is highlighted in bold. 92

4.1 Textual content and metadata of epigram 21179 [22] after preprocessing. The dash in the subject row indicates that no value is recorded in the Database of Byzantine Book Epigrams (DBBE). 123

4.2 Availability of each metadata field across the subset of 496 epigrams. For each field, the table reports the number and proportion of epigrams for which at least one value is recorded in the DBBE. 123

4.3 Experimental results for $\mathbf{w}_{F_1}$ and $\mathbf{w}_{\text{avg}}$ across the twenty α values. For each quality measure, the highest value within each configuration is highlighted in bold. 129

5.1 Top ten context-aware similarity-confidence pairs between the reference epigram with identifier 17752 and the other epigrams. The results ordered by the lexicographic ordering $\succeq^{s}_{\text{lex}}$ are displayed in the left group of columns, while the results ordered by the non-weighted global preference $\succeq^{\text{basic}}_{\text{global}}$ are displayed in the right group. The rank column is added for reference and is not part of the query output. The similarity, confidence, and global preference values are rounded to three decimal places. 167

5.2 Top ten context-aware similarity-confidence pairs with the smallest Euclidean distance to the reference pair $(0.526, 0.667)$ between the epigrams with identifiers 17752 and 26545. The membership values are computed following Equation (5.26) with $\delta = 0.1$. The similarity, confidence, distance, and membership values are rounded to three decimal places. 169

5.3 Distribution of epigrams across L-grade groups when querying for Messina (depth 4, granularity `city`) with $u = 0$. Each row corresponds to a distinct satisfaction-confidence pair that is assigned to at least one epigram in the dataset. The number of effective endpoints at depth 4 under each location p is indicated alongside the resulting L-grade components and the number of epigrams in that group. The six groups and their associated L-grades are denoted ℓ_1 through ℓ_6 for use in the subsequent ranking demonstrations. 173

6.1 Quality measures for the clustering results obtained from the textual baseline dissimilarity matrix $M_{D_{\text{text}}}$ with $k_{\text{GT}} = 21$. The Fuzzy C-Medoids (FCMdd) clustering is performed with the optimised fuzzification exponent $q = 1.2$. The highest score for each quality measure is highlighted in bold. 208

6.2 Quality measures for the clustering results obtained from the context-aware dissimilarity matrix M_D with $k_{\text{GT}} = 21$, for each clustering algorithm and for each of the three weight vector configurations: the average weight vector $\mathbf{w}_{\text{avg}}$, the empirical weight vector $\mathbf{w}_{F_1}$ from Chapter 4, and the method-specific optimised weight vector $\mathbf{w}^{\text{ctx}}_{\text{cluster}}$. For FCMdd, the fuzzification exponent $q = 1.8$ is selected jointly with the optimised weight vector and reused across the three weight vector configurations. The highest score for each quality measure is highlighted in bold. 210

6.3 Quality measures for the clustering results obtained from the non-weighted dissimilarity matrix M'_D with $k_{\text{GT}} = 21$ for each clustering algorithm, using the method-specific optimised weight vectors $\mathbf{w}^{\text{nw}}_{\text{cluster}}$ for the context-aware similarity input. For FCMdd, the fuzzification exponent $q = 1.1$ is selected jointly with the optimised weight vector for this strategy. The highest score for each quality measure is highlighted in bold. 212

6.4 Quality measures for the clustering results obtained from the weighted dissimilarity matrix M^w_D with $k_{\text{GT}} = 21$ for each clustering algorithm and weight $w \in \{0.1, 0.2, \ldots, 1.0\}$, using the method-specific optimised weight vectors $\mathbf{w}^{\text{w}}_{\text{cluster}}$ for the context-aware similarity input. For FCMdd, the fuzzification exponent $q = 1.1$ is selected jointly with the optimised weight vector for this strategy. The highest score for each quality measure is highlighted in bold. 214

6.5 Subset of the query results from Listing 6.2. The table lists the membership degrees of a selection of epigrams to four selected clusters, rounded to four decimal places for readability. For each epigram, the highest membership degree among the four selected clusters is highlighted in bold. 220
6.6 Textual content and metadata of epigram 31962 [25] after preprocessing. The dashes in the location and subject rows indicate that no value is recorded in the DBBE. 221
6.7 Quality measures for the K-Medoids partition of the full subset of 496 epigrams for all $k \in \{10, 11, \dots, 30\}$, using the context-aware similarity computed with the $\mathbf{w}_{\text{avg}}$ Ordered Weighted Averaging (OWA) weight vector, without confidence integration. The highest score for each quality measure is highlighted in bold. 222

List of Algorithms and Listings

2.1 Main construction algorithm for the graph-based text representation. 42
2.2 Recursive procedure for constructing lower-level nodes in the graph representation. 43
2.3 Cypher query to retrieve the most prevalent words in the dataset. 49
2.4 Cypher query for retrieving verses starting with the pattern "και ι". 52
2.5 Cypher query for retrieving verses matching a combined word, Part-of-Speech (POS), and lemma pattern. 53

3.1 Hierarchical computation of pairwise similarity scores in the graph-based text representation. 81
3.2 Cypher query to retrieve similar epigram groups with a similarity score of at least 0.95. . . . 94
3.3 Cypher query to retrieve a detail view of the interconnections and similarities between two epigrams. 96

4.1 Cypher query to retrieve all epigrams from epigram type 1982 and the context-aware relationships between them that meet or exceed a similarity of 0.9 and a confidence of 0.5. . . 133
4.2 Cypher query to retrieve all epigrams from epigram type 1982 and the context-aware relationships between them that meet or exceed a similarity of 0.9, for visualisation with confidence-based colouring. 134

5.1 Cypher query to retrieve all context-aware similarity-confidence pairs between the epigram with identifier 17752 and the other epigrams, and order them by the lexicographic ordering $\succeq^{s}_{\text{lex}}$. 166
5.2 Cypher query to retrieve all context-aware similarity-confidence pairs between the epigram with identifier 17752 and the other epigrams, and order them by the non-weighted global preference $\succeq^{\text{basic}}_{\text{global}}$. 166
5.3 Cypher query to compute the Euclidean distance between the L-grade of a reference pair of epigrams and the L-grades of all other pairs involving the first epigram, ordered by increasing distance. 167
5.4 Cypher query to compute the Euclidean distance between the L-grade of a reference pair of epigrams and the L-grades of all other pairs involving the first epigram, ordered by increasing distance. The membership values for approximate equality with respect to the reference pair are also computed, following Equation (5.26) with $\delta = 0.1$. 168

6.1 Cypher query to retrieve the epigrams and their cluster assignments. The query matches each epigram node with its assigned cluster medoid through the `IN_CLUSTER` relationship and returns the epigram, the medoid, and the relationship. 217

6.2 Cypher query to retrieve epigrams and their cluster assignments to a subset of four selected medoids. The query matches each epigram node with each of the four selected medoids through the `IN_CLUSTER` relationship and returns the epigram alongside the membership degrees to those medoids. 219
6.3 Cypher query to retrieve the epigrams of a single cluster together with their subjects. The query matches each epigram assigned to the selected medoid through the `IN_CLUSTER` relationship and, where present, the `Person` nodes that are the subject of the epigram or of the medoid. 224

List of Acronyms

A

ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
AMI	Adjusted Mutual Information
ARI	Adjusted Rand Index

B

BASE	Basically Available, Soft State, Eventually Consistent
BERT	Bidirectional Encoder Representations from Transformers
BoW	Bag of Words
BSON	Binary Javascript Object Notation
BTM	Biterm Topic Model

C

CAP	Consistency, Availability, Partition Tolerance
CI	Computational Intelligence

D

DBBE	Database of Byzantine Book Epigrams
DBMS	Database Management System
DH	Digital Humanities
DL	Deep Learning
DSS	Decision Support System

E

ECM	Evidential C-Means
ER	Entity Relationship

ETL Extract, Transform, Load

F

FCM Fuzzy C-Means

FCMdd Fuzzy C-Medoids

FN False Negatives

FP False Positives

G

GCD General Conjunction Disjunction

GDS Graph Data Science

GED Graph Edit Distance

GenAI Generative Artificial Intelligence

GloVe Global Vectors for Word Representation

GSDMMM Gibbs Sampling Dirichlet Multinomial Mixture Model

GT Ground Truth

H

HAC Hierarchical Agglomerative Clustering

I

IBM International Business Machines Corporation

IDC International Data Corporation

IoT Internet of Things

J

JSON Javascript Object Notation

K

KG Knowledge Graph

KNN k-Nearest Neighbours

L

LCS Longest Common Substring

LD Linked Data

LDA Latent Dirichlet Allocation
LLM Large Language Model
LPG Labelled Property Graph
LSA Latent Semantic Analysis
LSP Logic Scoring of Preference

M

MAR Missing at Random
MCAR Missing Completely at Random
MCDM Multi-Criteria Decision Making
MI Mutual Information
ML Machine Learning
MNAR Missing Not at Random

N

NLP Natural Language Processing
NN Neural Network
NoSQL Not only SQL
NYSIIS New York State Information and Intelligence Systems

O

OWA Ordered Weighted Averaging

P

PAM Partitioning Around Medoids
PCM Possibilistic C-Means
PMI Pointwise Mutual Information
POS Part-of-Speech
PTV Possibilistic Truth Value

R

RDBMS Relational Database Management System
RDF Resource Description Framework
RI Rand Index

S

SPARQL SPARQL Protocol and Resource Description Framework Query Language

SQL Structured Query Language

T

TB Terabyte

t-conorm Triangular Conorm

TEI Text Encoding Initiative

TF-IDF Term Frequency-Inverse Document Frequency

TN True Negatives

t-norm Triangular Norm

TP True Positives

W

WM Weighted Mean

WWW World Wide Web

X

XAI eXplainable Artificial Intelligence

XML Extensible Markup Language

XPath Extensible Markup Language Path Language

XQuery Extensible Markup Language Query Language

Z

ZB Zettabyte

1

Introduction

"We are drowning in information
but starved for knowledge."

— **John Naisbitt**

As the title of this work suggests, this thesis explores the management and analysis of textual data, specifically addressing the specific challenges posed by texts within the broader landscape of big data. Textual data are generated at an unprecedented scale and speed, but the variety and trustworthiness of texts and their metadata present significant challenges for traditional data management and analysis techniques, particularly in cases where context is limited and linguistic resources are scarce. This work develops and evaluates computational methods for the management and analysis of short texts, comprising a graph-based data model, an orthographic similarity measure, a context-aware fusion of that measure with metadata-based similarities, and techniques for querying, ranking, and clustering the resulting scores. These methods are designed to extract meaningful insights from corpora on which established analytical techniques perform poorly. Short texts, such as poems, lyrics, chat messages, and social media posts, are characterised by their brevity and limited contextual information, which restrict the applicability of standard techniques. These techniques are evaluated on a corpus of short historical poems written in Byzantine Greek, a low-resource language, which serves as the primary use case of this thesis and combines orthographic variation with frequently incomplete metadata.

The remainder of this first chapter is structured as follows. First, the evolution of data, from their earliest origins to the present day is discussed in Section 1.1. In Section 1.2, the challenges and opportunities that arise when working with big data are described, with particular attention on textual data. Next, Byzantine book epigrams, a specific use case for the techniques developed in this thesis, are introduced in Section 1.3, including the particular challenges they present. The main research objectives and questions on which this thesis is based are presented in Section 1.4. An outline of the remainder of this work is given in Section 1.5. Finally, Section 1.6 provides an overview of all scientific contributions, both journal and conference publications, that have resulted from this thesis.

1.1 The Evolution of Data

The word *data* has its roots in Latin, being the plural of *datum*, meaning "(thing) given". *Datum* is the past participle of the verb *dare*, meaning "to give", which traces back to the Proto-Indo-European root *do-*, with the same meaning [1]. In classical use, the term referred to "facts given as the basis for calculation in mathematical problems" and by the late nineteenth century had evolved to denote "numerical facts collected for future reference" [1]. From their etymological roots, data were conceived as things obtained rather than constructed, serving as the raw material from which meaning could be derived. This conception remains at the heart of modern definitions. According to IBM, data are facts, numbers, words, observations, or other useful information from which organisations derive insights through processing and analysis to support decision-making [2]. Yet long before such formal definitions existed, human beings already felt a fundamental need to record and preserve information about the world around them.

The earliest evidence of this impulse dates back tens of thousands of years, to cave paintings and bone tally sticks used by prehistoric humans to track animals, seasons, and quantities [3]. These early records represent the first attempts to store information outside the human mind so that it could be retrieved and interpreted at a later time. As civilisations emerged, so did more sophisticated recording systems. In ancient Mesopotamia, around 3000 BCE, the Sumerians began inscribing transactions, inventories, and administrative records onto clay tablets [4, 5], marking a pivotal shift from data as a mnemonic aid to data as an instrument of governance and commerce. The ancient Egyptians not only used hieroglyphics for formal and religious inscriptions but also employed hieratic script to record taxation, census data, and historical events, demonstrating that the drive to collect and analyse structured information is as old as organised society itself [4]. Over the following millennia, knowledge was preserved and transmitted through hand-copied manuscripts, a laborious process that constrained the production and circulation of recorded information [6]. The invention of the printing press around 1440 marked a turning point: for the first time, information could be reproduced and distributed at scale, dramatically accelerating the accumulation and spread of recorded knowledge and transforming methods of data collection, storage, and retrieval [7]. Each of these developments reflects the same underlying human urge not merely to record information, but to organise it, retrieve it, and derive meaning from it. This urge would ultimately find its most powerful expression in the digital age.

The advent of digital computing in the mid-twentieth century fundamentally transformed how data were stored, processed, and analysed. Early computers, originally conceived as tools for numerical calculation, rapidly evolved into general-purpose machines capable of processing vast quantities of structured information at unprecedented speed [8]. The subsequent development of the internet and the World Wide Web (WWW) in the late twentieth century transformed the data landscape once more, allowing information to be generated, shared, and consumed on a truly global scale. An explosion of connected technologies, such as sensors and Internet of Things (IoT) devices, social media platforms, and digital transaction systems, gave rise to datasets of a scale and complexity that traditional data processing tools could no longer handle effectively, a phenomenon that became known as *big data* [9, 10]. Managing these vast datasets poses significant challenges in terms of storage, processing, data veracity, and analysis, while simultaneously offering immense opportunities for deriving insights and creating value across domains such as healthcare, finance, and marketing [10, 11].

Such insights are often obtained through the application of Artificial Intelligence (AI) and Machine Learning (ML) techniques, which can identify patterns and generate predictions beyond the reach of unaided human analysis [12]. However, recent developments in AI have led to the rise of Generative Artificial Intelligence (GenAI), in which generative models no longer merely analyse data but produce new data at a scale and speed that exceeds human capacity. Applications such as ChatGPT and other Large Language Model (LLM)-based applications for text, Midjourney for images, Suno for audio, and Veo for video exemplify this shift. Trained on enormous datasets of human-generated text, images, and other media, these models can synthesise content that is statistically coherent and superficially convincing, yet fabricated rather than observed or measured [13]. This surge of AI-generated content not only amplifies the volume of data but also intensifies the challenges already associated with big data. Moreover, one might question whether AI-generated content can still be considered *data* in the original sense of the word, since data produced by GenAI are not given facts but artefacts generated by statistical models. Regardless of whether such data meet the original definition of the word, this work addresses certain challenges that arise when working with textual big data, particularly where the texts and their metadata are limited, incomplete, uncertain, not fully trusted, or of varying quality.

1.2 Big Data: Challenges and Opportunities

The term *big data* broadly refers to datasets so large, fast-moving, and diverse that traditional data processing tools are unable to handle them effectively [10]. To grasp the scale of the phenomenon, consider the volume of data generated worldwide on any given day in 2026: an estimated 350 billion emails are sent, Google processes around 16.4 billion searches, and platforms such as X (formerly Twitter) and Facebook receive around 500 million and 350 million posts respectively [14]. As illustrated by Figure 1.1, the volume of data produced worldwide per year reached approximately 175 Zettabytes (ZBs) in 2025[1], according to the IDC Global DataSphere report [15], and is projected to surpass 1000 ZB by 2030 according to the Huawei Data Storage whitepaper [16]. This explosive growth is driven by the proliferation of connected devices, social media, digital transactions, and more recently, AI-generated content, as discussed in the previous section. At the same time, this wealth of data represents an immense opportunity, as organisations across all sectors can leverage these vast and diverse datasets to extract meaningful insights, enhance efficiency, drive innovation, and gain competitive advantage [17]. Yet effectively managing, processing, and exploiting data at this scale raises significant technical and organisational concerns, which are explored in the remainder of this section.

1.2.1 The Four V's of Big Data

Big data, and the challenges associated with them, are often characterised in terms of a set of V's. The concept was originally introduced using three V's: *volume*, *velocity*, and *variety* [18, 19], and has since been extended by various authors to include additional dimensions. While some literature describes big data using up to 15 V's, including visualisation, vulnerability, and volatility [20, 21], this work focuses on the four V's most commonly adopted in data management and analysis: *volume*, *velocity*, *variety*, and *veracity* [17, 22]. Each of these presents distinct challenges, which are briefly discussed below.

[1] One ZB is equal to one billion Terabytes (TBs)

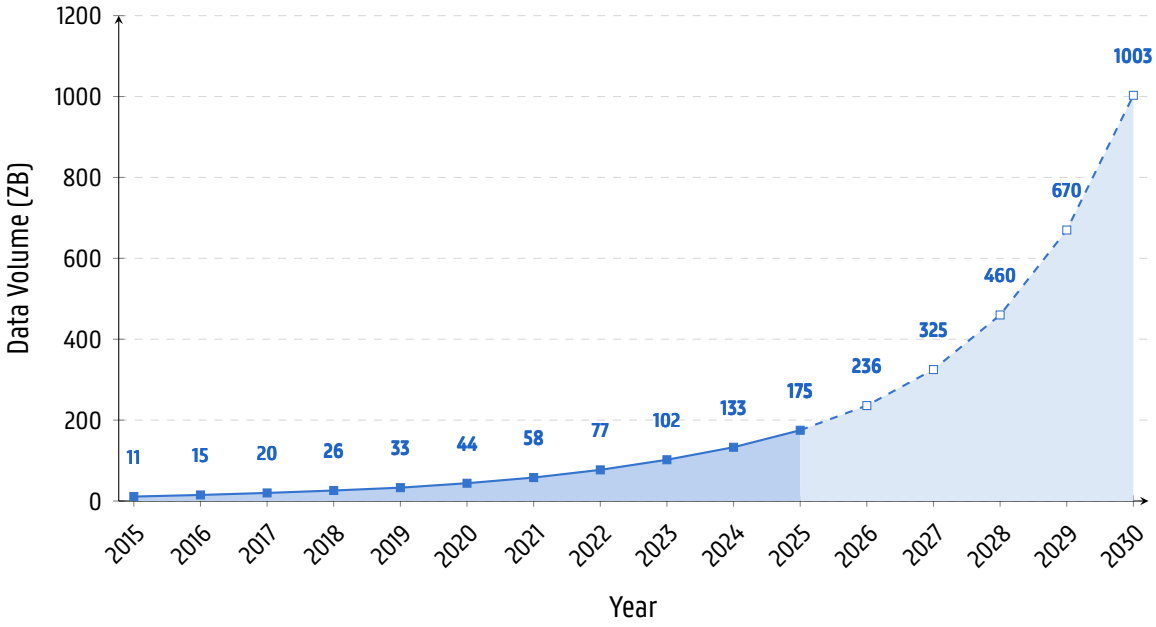


Figure 1.1: Volume of data generated per year according to the IDC Global DataSphere report [15] and Huawei Data Storage report [16], expressed in ZBs.

- **Volume**: The sheer scale of data being generated today, as illustrated in Figure 1.1, places enormous demands on storage infrastructure and data management systems. Storing these vast amounts of data requires not only sufficient physical storage capacity but also efficient strategies for organisation and indexing, so that data can be retrieved and analysed in a timely manner [10, 23, 24]. Traditional storage systems were not designed to operate at this scale, necessitating the development of new data management approaches or even entirely new database technologies.

- **Velocity**: Not only is the amount of data being generated increasing, but the speed at which they are produced and must be processed is accelerating as well. Applications such as financial trading platforms, sensor networks, and social media feeds generate data continuously and often require near-real-time processing to remain useful [22, 23]. Managing this continuous influx of data demands processing architectures that can handle high-throughput streams without introducing unacceptable delays or data loss [10].

- **Variety**: While traditional data management systems were built around data conforming to a fixed and structured schema, big data are characterised by a wide diversity of data types, formats, and sources. Data may be structured according to fixed (database) schemas, such as relational database records; may follow semi-structured description rules, such as JSON or XML documents; or may lack a formal schema altogether, as in free text, images, audio, and video [22]. Integrating and analysing data across these heterogeneous formats is a significant challenge, as traditional systems struggle to accommodate such diversity [10, 23].

- **Veracity**: Big data often originate from a wide variety of sources where quality and reliability cannot be guaranteed, resulting in datasets that are frequently incomplete, noisy, inconsistent, or biased [22]. Veracity encompasses these issues and concerns the trustworthiness, accuracy, and consistency of the

data. A lack of trust or the presence of bias or uncertainty can propagate from source data through analysis or query processing to the final outputs, ultimately undermining the reliability of any insights derived from them [25]. Handling veracity issues requires techniques that model uncertainty, improve data quality, or explicitly assess and report confidence in the analysis results, among others [26]. This is especially important when working with AI-generated data.

Together, these four characteristics define the core challenges that big data poses for data management systems. Addressing them requires not only new ways of thinking about data storage and processing, but often calls for entirely new technological paradigms. The following sections discuss how traditional relational databases have been pushed to their limits by these challenges, and how the emergence of Not only SQL (NoSQL) and polyglot persistence approaches has offered new ways to manage data at scale.

Relational Databases

Relational databases, introduced by Codd in 1970 [27], have been the dominant paradigm for data storage and management for decades. A Relational Database Management System (RDBMS) organises data into structured tables with predefined and fixed schemas, relies on Structured Query Language (SQL) for querying, and guarantees data integrity through its ACID properties: atomicity, consistency, isolation, and durability [28, 29]. These properties make relational databases particularly well-suited for applications where data are well-structured, fit naturally into a tabular representation, and where consistency and accuracy are critical, such as financial systems, inventory management, and administrative records.

However, the characteristics of big data expose several fundamental limitations of the relational model. Due to the increase in volume and velocity, relational databases struggle to scale effectively, as they were originally designed for vertical scaling, that is, improving performance by upgrading the hardware of a single server, an approach that quickly reaches physical and financial limits [30]. Distributed relational databases have been developed to address this limitation, but they introduce significant complexity, particularly in maintaining ACID guarantees across distributed nodes [30]. This complexity is formalised by the CAP theorem, which states that a distributed data store can only simultaneously guarantee two of the following three properties: consistency, availability, and partition tolerance [31]. Partition tolerance refers to the ability of a system to continue operating despite network failures that disrupt communication between nodes, and is a fundamental requirement for distributed systems, since such failures are inevitable at scale. RDBMSs traditionally prioritise consistency and availability at the cost of partition tolerance, which limits their suitability for globally distributed big data systems [32].

Furthermore, the requirement for fixed schemas makes it difficult to accommodate the variety of modern data, as unstructured and semi-structured content such as free text, multimedia content, graph-structured data, or JSON documents cannot be stored efficiently in a tabular structure [28]. While Entity Relationship (ER) modelling [33] and normalisation techniques [34, 35] offer an effective means of addressing certain veracity challenges, such as modelling potentially missing information, they assume well-structured and fixed data and can significantly decrease query performance, particularly when operating on large datasets [36]. Data cleaning and preprocessing techniques can mitigate other veracity issues, but scalable and generic solutions for

handling veracity remain an active area of research [26]. Taken together, these limitations of the relational model become prohibitive at the scale of big data, driving the need for alternative approaches to data storage and management.

NoSQL Databases

In response to the limitations of relational databases in the context of big data, a new class of database systems emerged in the late 2000s under the umbrella term NoSQL. Rather than imposing a fixed schema, NoSQL Database Management Systems (DBMSs) adopt flexible data models that can accommodate structured, semi-structured, and unstructured data [30]. They also support horizontal scaling, that is, improving performance by distributing the data and workload across multiple servers or clusters [28, 37]. To achieve this scalability, NoSQL systems have to relax the strict ACID guarantees of relational databases in favour of the BASE properties: basically available, soft state, and eventually consistent [29], consciously trading consistency for availability and partition tolerance in the context of the CAP theorem [28, 31]. In practice, this means that data may be temporarily inconsistent across nodes but are guaranteed to converge once synchronisation processes complete, a property known as eventual consistency. In doing so, NoSQL databases primarily address three challenges of big data: horizontal scaling tackles volume, flexible and schemaless designs accommodate variety, and the absence of rigid schema validation and reduced transactional overhead allow data to be ingested and processed at much higher speeds, directly addressing velocity [29, 30]. Veracity, however, remains a challenge, as the use of different, often large, data sources and the lack of enforced schemas can introduce data quality and trustworthiness concerns [26].

NoSQL DBMSs are broadly classified into four main types, each optimised for different data models and use cases [29, 30, 32]:

- **Key-value stores**, such as Redis and Amazon DynamoDB, store data as simple key-value pairs and are optimised for high-speed insertion, retrieval, and caching applications.
- **Document stores**, such as MongoDB and CouchDB, store data as self-describing documents, typically in JSON or BSON format, whose flexible schemas are both interpretable by the DBMS and well-suited to heterogeneous data.
- **Column stores**, such as Apache Cassandra and HBase, organise data by columns rather than rows, making them particularly efficient for analytical queries over very large datasets that involve a limited number of columns.
- **Graph databases**, such as Neo4j and Memgraph, represent data as graphs, often directed, consisting of nodes and edges with labels and properties, making them ideal for managing and querying highly interconnected data such as social networks or knowledge graphs, and enabling complex graph-based analytics [38].

These systems have been widely adopted in big data environments, with the choice of database type depending on the specific requirements of the application, such as the need for complex relationship analysis, flexible schemas, or high-speed access.

Polyglot Persistence

As the diversity of data types and workloads in big data environments continues to grow, it has become increasingly evident that no single database technology is optimal for every use case [39]. Relational databases excel at managing and querying structured data with well-defined relationships and strict consistency requirements, while NoSQL databases are better suited for high-volume, flexible, or rapidly changing data at the cost of complex querying facilities [29]. Polyglot persistence is an architectural strategy that addresses this reality by combining multiple database technologies within a single system, matching each data type and access pattern to the storage solution best suited to handle it [39, 40].

In practice, a polyglot persistence architecture might combine a relational database for transactional records requiring ACID guarantees, a document store for flexible semi-structured content, a graph database for interconnected data, and a key-value store for high-speed caching. This approach allows each component of a system to be independently scaled and optimised, improving overall performance and flexibility compared to relying on a single database technology.

However, a polyglot persistence architecture also introduces significant operational overhead. Managing multiple database technologies requires diverse technical expertise, and ensuring data consistency and integrity across heterogeneous systems can be challenging, particularly when transactions span multiple databases [39]. Ensuring, for instance, that updates to a relational database are reflected in a graph database modelling relationships between the same entities may require complex synchronisation and Extract, Transform, Load (ETL) processes. These trade-offs mean that such an architecture is most appropriate for complex, large-scale systems where the performance, applicability, and flexibility gains justify the added operational complexity. Regardless of the architectural approach chosen, the ultimate objective of managing big data remains the extraction of meaningful value, which is discussed next.

1.2.2 Generating Value from Big Data

While volume, velocity, variety, and veracity define the main challenges of big data, generating value from these large datasets represents its most significant opportunity. Often cited as a fifth V of big data, value captures the idea that the true potential of large datasets lies not in the data themselves but in the insights and knowledge that can be extracted from them [10]. Across domains such as healthcare, finance, and logistics, the ability to extract actionable knowledge from data has proven transformative, enabling more informed decision-making, the discovery of previously hidden patterns, and the development of predictive capabilities that would be unattainable through traditional analytical methods [41]. The outputs of such analysis often support decision-making by providing recommendations, predictions, or suggestions that inform human choices, and systems designed to support this process are referred to as Decision Support Systems (DSSs) [42]. These systems are structured frameworks that combine data, analytical models, visualisation, and user-friendly interfaces to support human decision-making [43].

The most fundamental means of extracting value from big data stored in databases is through querying and manual interpretation [44]. Traditional query languages such as SQL operate on the assumption of precise,

well-defined queries and return exact matches, which in some cases can be too rigid for real-world data analysis. An extension of this approach is the use of flexible querying techniques, which allow users to express queries in imprecise or flexible terms, returning ranked or graded answers rather than strict Boolean results [45–47]. Rather than retrieving only records that exactly match a query, these approaches can surface results that are partially relevant, providing richer and more nuanced outputs that better reflect the uncertainty and imprecision inherent in real-world big data. However, querying alone is often insufficient for deeper insight, and more advanced analytical methods are required to fully exploit the potential of large datasets.

Beyond querying, more advanced techniques to gather insights and generate value from big data include statistical analysis, data mining, and ML-based approaches [43, 48]. These methods allow analysts to summarise, aggregate, and explore large datasets, uncovering trends, correlations, and anomalies that support decision-making. Where traditional methods require explicit assumptions about the structure of data, ML algorithms can learn complex patterns directly from large datasets, making them particularly well-suited to the scale and heterogeneity of big data [12, 49]. Deep Learning (DL), a subfield of ML based on multi-layered Neural Networks (NNs), has further extended these capabilities, achieving remarkable performance in tasks such as image recognition, Natural Language Processing (NLP), and predictive modelling [49].

While these techniques have proven powerful, they also introduce new challenges, particularly in terms of interpretability and trustworthiness. As ML models have grown in complexity, a fundamental tension has emerged between predictive performance and interpretability. Many of the most powerful models, including deep NNs, operate as black boxes, producing predictions without providing any insight into the reasoning behind them [50]. Interpretability refers to the degree to which the internal mechanics or predictions of a model can be understood, while explainability concerns the ability to describe in human-understandable terms why a model arrived at a particular output [51]. This lack of transparency is particularly problematic in high-stakes domains such as healthcare, finance, and law, where decisions must be accountable and auditable. One family of approaches well-suited to addressing this concern is Computational Intelligence (CI), a subfield of AI encompassing techniques such as evolutionary algorithms, fuzzy logic, and swarm intelligence, which offer robust methods for optimisation and decision-making under imprecise or incomplete information [52]. By design, many of these methods are more interpretable than DL models, making them a natural bridge towards the broader goal of eXplainable Artificial Intelligence (XAI). XAI encompasses methods that aim to make the behaviour of AI systems more transparent and explainable to human users, ultimately improving the trustworthiness and accountability of DSSs that rely on them [50, 53].

While many of these value-generating techniques partially address the volume, velocity, and variety challenges of big data, veracity concerns often remain an open challenge when attempting to extract meaningful insights [54]. Poor data quality, whether in the form of missing values, noise, uncertainty, inconsistency, or bias, propagates through analytical pipelines and can yield misleading or erroneous insights regardless of the complexity of the methods applied [25]. When extracting meaningful insights from big data, it is therefore essential to explicitly model and manage issues related to veracity, either by improving data quality through cleaning and preprocessing techniques, by developing analytical methods that can account for uncertainty and bias in the data, or by reporting the veracity of the insights derived from the data [26].

1.2.3 Challenges of Textual Data Analysis

Since this thesis mainly focuses on the analysis of textual data, it is important to highlight some of the specific challenges that arise when working with them. Textual data, particularly in the form of semi-structured or unstructured data, accounts for a significant portion of the variety characteristic of big data [10, 55]. Texts come in many forms, including social media posts, news articles, scientific papers, song lyrics, legal documents, and historical manuscripts, varying greatly in length and style. Unlike structured data, which conforms to a predefined schema and can be directly processed by analytical systems, text has no fixed, common structure and requires dedicated preprocessing and specialised text mining techniques before meaningful information can be extracted [55]. Text mining, the process of extracting meaningful insights from large collections of text, has therefore become an essential tool for making sense of this vast and growing body of unstructured data.

A fundamental distinction in text analysis lies between its orthographic and semantic dimensions. The orthographic view treats text as a sequence of characters, words, or tokens, focusing on surface-level features such as spelling and syntactic structure. In contrast, the semantic view is concerned with the underlying meaning of individual words or texts, including the relationships between concepts and the context in which words are used. Although both perspectives operate on the same underlying text, they can produce fundamentally different results: two texts that are orthographically very different may convey the same meaning, while two texts that share many of the same words may mean entirely different things depending on context [56]. While orthographic approaches rely on similarity measures based on shared characters or tokens [57], semantic approaches require richer representations that capture meaning and context, a challenge that has driven significant developments in modern NLP [58].

NLP is the field of computer science and linguistics concerned with enabling machines to process, understand, and generate human language [58]. A central development in modern NLP is the use of vector embeddings: dense numerical representations of words, phrases, or documents in a continuous vector space. In this space, semantically related tokens or texts are assumed to be positioned closer together, enabling similarity to be measured geometrically rather than through surface-level character or token matching. These embeddings are produced by models such as Word2Vec [59], Doc2Vec [60], GloVe [61], and more recently transformer-based models such as BERT [62] and Sentence-BERT [63], which have significantly improved the ability to capture and compare semantic content in text [58]. As a result, these representations have become the foundation of a wide range of NLP applications, including sentiment analysis, machine translation, and question answering.

As the adoption of vector embeddings has grown, so has the need for dedicated storage and retrieval systems capable of handling high-dimensional vector data. This has given rise to vector databases, such as Pinecone and Weaviate, which are optimised for storing and querying such data efficiently [64]. Rather than retrieving records based on exact matches or structured queries, vector databases support similarity search, enabling the retrieval of semantically related documents based on their proximity in the embedding space. This capability is particularly valuable for tasks such as recommendation, clustering, and document ranking, where nuanced semantic similarity is more informative than exact lexical matching [65].

Despite the considerable advances brought about by NLP and vector-based representations, significant challenges remain, particularly when working with *short texts* or texts written in *low-resource languages*. Short

texts, such as social media posts, titles, poems, abstracts, or metadata fields, provide limited context for semantic analysis, making tasks such as classification, clustering, and similarity calculation considerably more difficult [66–69]. The sparsity of information in short texts means that traditional NLP methods, which rely on statistical patterns across larger bodies of text, often perform poorly. Even modern embedding-based approaches can struggle to produce reliable representations when the input is very brief [70]. Low-resource languages, on the other hand, are languages for which limited data and linguistic resources are available, and they present a related but distinct set of challenges. For a low-resource language, annotated data, linguistic resources such as lexicons and parsers, and pre-trained models are often scarce or non-existent [71–74]. Since the majority of NLP tools and pre-trained models are developed primarily for high-resource languages such as English, their performance degrades significantly when applied to low-resource settings. Such languages often exhibit linguistic structures and features that are not well captured by models trained on high-resource languages [71–74]. Together, these two dimensions define the core data conditions under which the methods proposed in this thesis are developed and evaluated. The next section introduces the specific corpus that serves as the primary use case for this work.

1.3 Byzantine Book Epigrams

As a concrete instantiation of the challenges described above, this thesis employs historical texts, specifically Byzantine book epigrams, as its primary use case. Byzantine manuscripts are hand-written books that have served as the primary medium for preserving Greek texts through the centuries. They often contain small Greek poems written in the margins, above, or below their main texts. These poems are metrical paratexts: metrical in that they are written in verse, and paratextual in that they accompany rather than form part of the main text [75]. The term *book epigrams* reflects that these poems are not only physically inscribed in the manuscripts but also concern the books themselves, their contents, and the people involved in their production [76]. They often provide valuable insights into the production history of the manuscripts, the patrons who commissioned them, the cultural context in which they were created, and the personal sentiments of the scribes who produced them.

These epigrams are characterised by a complex textual tradition. This tradition, or in other words the transmission of the texts throughout history, involved epigrams being split up, (re)combined, or otherwise reworked in the course of being copied. Consequently, many variants of the same epigram can be found across different manuscripts, resulting in an intricate network of textual interconnections between epigrams and their variants.

Beyond the challenges posed by textual variants, these epigrams also exhibit significant orthographic inconsistencies. These inconsistencies arise from multiple sources, including spelling and transcription errors, unstandardised punctuation, and irregular text wrapping. They are further compounded by the historical evolution of the Byzantine Greek language. The most significant of these linguistic developments is a phonetic shift known as itacism. Itacism refers to the merging of the classical pronunciations of the vowels ι, η, υ, and the diphthongs ει, οι into a single ι sound. As a result, these vowels and diphthongs, including their diacritic variants, can be used interchangeably throughout the corpus [77].

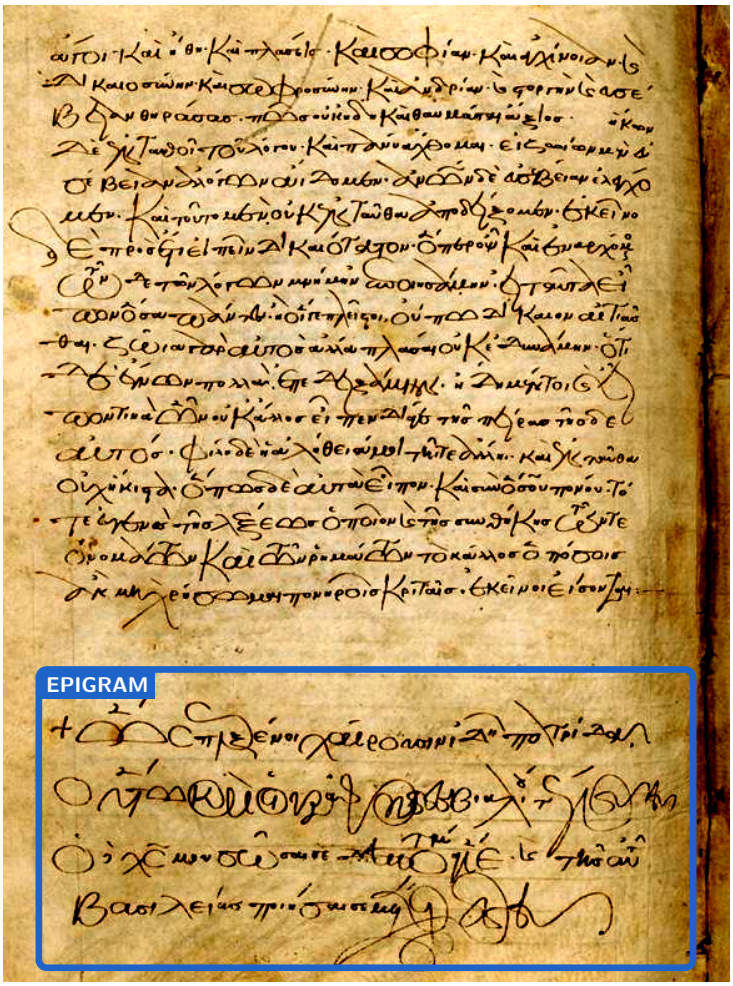


Figure 1.2: Example of an epigram with id 19512 [78] found in the margin of a Byzantine manuscript.

Together, the complex tradition of variants and the pervasive orthographic inconsistencies described above make Byzantine book epigrams a challenging but rich object of study. As an illustration, Figure 1.2 shows an epigram found in the margin of a Byzantine manuscript, the transcribed content of which is presented in Table 1.1. This epigram translates to "Just like travellers rejoice in seeing their homeland, so do writers in reaching the end of the book", and its variants are often found at the end of manuscripts as an expression of the scribe's satisfaction upon completing the copying of a manuscript.

Table 1.1: Textual content of the epigram in Figure 1.2 with id 19512 [78].

#	Epigram 19512
1.	+ ὥσπερ ξένοι χαίρουσιν ἰδ(εῖν) πατρίδα,
2.	οὕτω καὶ οἱ γράφοντες βιβλίου τέλος+

A second epigram, found in a different manuscript, is shown in Table 1.2. Although at first glance the two texts may appear substantially different, most differences are minor and the poems are related and convey the same meaning. Looking beyond the unstandardised punctuation and diacritics reveals itacism-induced inconsistencies, where ξένοι and οἱ in the first epigram are written as ξένη and ἡ in the second epigram. Additionally, the second epigram uses βλέπειν in place of ἰδεῖν, two words that are synonymous. Without

contextual information such as the time, place, or authorship of these texts, two possible causes for their interconnection can be identified. Either one epigram was, directly or indirectly, inspired by the other, or both were independently derived from a common source. This example illustrates the complexity of the textual tradition of Byzantine book epigrams. Variants can differ in subtle ways that may not be immediately apparent, and the relationships between texts can be difficult to discern without careful analysis.

Table 1.2: Textual content of the epigram with id 22826 [78].

#	Epigram 22826
1.	+ ὥσπερ ξένη χαίρου|σιν π(ατ)ρίδα βλέπειν· |
2.	οὕτως καὶ ἡ γρά|φοντες βιβλίου τέλ(ος) ⁝·

Research into these Byzantine book epigrams is an important area of study for the Ghent Institute for Byzantine Studies at the Faculty of Arts and Philosophy of Ghent University. The institute focuses on the collection, organisation, and analysis of these epigrams, with the aim of understanding their textual tradition, their cultural and historical context, and the insights they can provide into the production and reception of Byzantine manuscripts [77, 79–85]. As a means to support this work, the group has developed the Database of Byzantine Book Epigrams (DBBE), a database that collects and organises these epigrams and their variants, along with rich contextual metadata. The structure and content of this database are discussed in the following section.

1.3.1 The Database of Byzantine Book Epigrams

The Database of Byzantine Book Epigrams (DBBE)[2] [85] is a publicly accessible online repository housing just over 13 000 Byzantine book epigrams. Maintained by a group of scholars in the field of Byzantine studies, it takes the form of a relational database with a web interface and offers advanced search and browsing capabilities to support the study and analysis of the epigrams and their variants.

At the epigram level, the database organises textual content around two complementary concepts: *occurrences* and *types*. Occurrences are exact transcriptions of epigrams as they appear in the manuscripts, preserving all orthographic inconsistencies, abbreviations, irregular and unstandardised punctuation, as well as gaps resulting from missing, unclear, or damaged portions of the manuscript. Types, by contrast, are normalised versions of one or more occurrences, created and curated by scholars. A type serves as an overarching representation that groups multiple occurrences with similar or even identical textual content [85].

However, grouping occurrences into types through manual curation presents two concrete challenges. The process is inevitably shaped by the preferences, background, and judgement of the individual scholars involved, meaning that grouping decisions can vary between researchers. Furthermore, since many epigrams arose from earlier epigrams being split up or recombined over the course of their transmission, grouping only at the level of complete epigrams is often insufficient to capture the complexity of how these texts were transmitted and reworked. To address this, the database introduces a more fine-grained grouping mechanism known as *verse*

[2] https://www.dbbe.ugent.be (visited on 17/03/2026)

variants, which links similar or identical individual verses across occurrences, allowing textual relationships to be captured at a sub-epigram level [85].

In addition to organising textual content, the database records a wealth of contextual metadata about the epigrams. This includes the manuscripts in which they appear, their current physical location, the position of the epigram within the manuscript, as well as metrical, geographical, and chronological information. Where known, data on authorship are also recorded, including the names of scribes, patrons, and subjects mentioned within the epigrams.

While the DBBE represents a valuable tool for scholars to store, browse, and study Byzantine book epigrams, its curation remains entirely dependent on manual labour. Textual interconnections must be identified and recorded by hand, groupings of occurrences and verse variants are formed through individual scholarly judgement, and contextual metadata that could potentially be inferred from related epigrams must be identified and recorded by domain experts. The current system is also structurally rigid: occurrences and individual verses can each belong to only a single group, which does not accommodate the ambiguity and overlap that are inherent in a corpus with such a complex textual tradition. The need for automated tools to support, rather than replace, the manual curation process is therefore evident. Such tools could, for instance, form part of a DSS that proposes candidate groupings based on orthographic or semantic similarity and suggests missing metadata by drawing on information from related epigrams. They could also enable more flexible and exploratory analysis through clustering, visualisation, and similarity ranking. Together, these capabilities would allow domain experts to navigate the corpus more efficiently and derive richer insights from the data it contains.

1.3.2 Computational Challenges of Analysing Byzantine Book Epigrams

Although the epigrams stored in the DBBE do not meet the volume and velocity criteria of big data, they raise significant concerns in terms of variety and veracity. Their textual complexity, combined with the historically fragmented character of their transmission, means that even a relatively small corpus can exhibit the kind of analytical difficulties typically associated with much larger and more heterogeneous datasets [86]. As discussed below, these difficulties are further compounded by the nature of the language in which the epigrams are written and the brevity of the texts themselves.

The variety dimension is particularly relevant for this corpus, given the inherent diversity of the textual data and the specific characteristics of the epigrams. The DBBE stores epigrams both as complete poems and as lists of individual verses, though at these levels the text follows no fixed structure. Additional variety concerns arise from the orthographic complexity described at the start of Section 1.3, including unstandardised punctuation and spelling inconsistencies resulting from itacism and scribal variation. Since these texts rarely exceed a few verses of a few words each, they can be considered short texts, which, as discussed in Section 1.2.3, present specific challenges for analysis [66, 70]. Compounding this, Byzantine Greek is a low-resource language for which many modern NLP tools and pre-trained models are not readily available or easily adaptable, further limiting the analytical approaches that can be applied [71, 72, 74]. The aforementioned challenges of working with textual data, particularly with respect to short texts and low-resource languages, are therefore highly relevant for this corpus.

The corpus is also subject to significant veracity challenges, affecting both its textual content and its metadata. Since occurrences are intended as exact transcriptions of the epigrams as found in the manuscripts, they are not cleaned or preprocessed in any form. As a result, they contain orthographic inconsistencies, unstandardised punctuation, and gaps resulting from missing, unclear, or damaged portions of the manuscript, all of which can introduce noise into downstream analysis. The metadata associated with the epigrams are similarly affected. The most prevalent veracity concern is the extent of missing information, arising from a lack of historical records, incomplete documentation, or the uncertainty that is inherent to reconstructing historical contexts. Furthermore, geographical and chronological metadata are frequently incomplete, as the date and place of creation of many epigrams are often only approximately known. This uncertainty is reflected in the database through broad date ranges and coarse geographical locations, which limit the precision of any analysis that draws on this contextual information. Given that metadata are a valuable source of additional context for finer-grained analysis, dedicated methods to handle these veracity issues are essential for extracting meaningful insights from the corpus.

Addressing these challenges requires methods capable of handling short, orthographically inconsistent texts in a low-resource language setting, while also accounting for incomplete and uncertain metadata. A further consideration concerns the representation of the complex interconnections between epigrams and their variants, and whether a relational database alone is the appropriate tool for capturing, querying, and analysing relationships of this nature. Ultimately, the goal is to develop tools that support scholars in curating and navigating the corpus, enabling the discovery of textual interconnections, the enrichment of incomplete metadata, and a more flexible and exploratory analysis of the data.

1.3.3 The Interconnected Texts Project

Prior to presenting the specific research questions that guide this thesis, it is useful to outline the broader research context in which this work is situated. This thesis is embedded in a multi-disciplinary research project entitled “Interconnected Texts. A Graph-Based Computational Approach to Byzantine Paratexts as Nodes between Textual Transmission and Cultural and Linguistic Developments”, hereafter referred to as the Interconnected Texts project [85]. This project is funded by Ghent University under the BOF-GOA granting framework. It aims to develop new computational tools that model the complex interconnections between epigrams, their variants, and the contextual metadata associated with them, generating new insights into the textual tradition of Byzantine book epigrams as well as the cultural and linguistic developments that shaped it. As the title suggests, the project adopts a graph-based approach, given that graph databases are designed precisely to model and query such networks of relationships efficiently [38].

The project consists of four distinct but interrelated research sub-projects, each focused on a specific aspect of the overarching goal:

1. **Natural Language Processing**: development of NLP tools for semantic modelling and linguistic annotation of Byzantine book epigrams, under the constraints of short texts and low-resource languages.

2. **Graph-based and Computational Methods**: development of graph-based methods for modelling textual data and the interconnections between epigrams, as well as computational methods for analysing the epigrams and their variants and discovering new interconnections based on textual and contextual information. This sub-project is the focus of the current thesis.

3. **Linguistics**: identification and analysis of formulaic patterns in the epigrams and their evolution over time, as well as the identification of deviations from these patterns and their causes.

4. **Manuscript Culture**: investigation of the relationships between the epigrams and the primary texts alongside which they appear, as well as the broader historical and cultural context in which they were created.

While each sub-project has distinct research objectives, they are designed to be complementary and mutually reinforcing, with the outputs of one informing and enriching the work of the others. For instance, the NLP tools developed in the first sub-project can be used to enrich the textual representations and metadata of the epigrams, which the graph-based methods developed in this thesis can then use to identify new interconnections and patterns in the data. Similarly, the tools and insights generated by the graph-based, computational, and NLP methods can support the identification and interpretation of formulaic patterns, as well as the exploration of relationships between the epigrams, the manuscripts in which they appear, and the broader cultural and historical context.

1.4 Objectives and Research Questions

In the previous sections, the challenges of working with big data were outlined, particularly for short and varied textual data written in low-resource languages, and the Byzantine book epigram corpus was presented as a concrete instantiation of them. These concerns motivate the research objectives of this thesis, which focus on developing methods and tools for the analysis of short texts, addressing issues of variety, veracity, and the constraints imposed by a low-resource language setting. The objectives are organised around four concrete subjects: data management and textual representation, short text similarity, context-aware short text similarity, and the analysis of short texts. These subjects are interrelated and form a pipeline, in which each stage builds upon the results of the preceding one. Although the methods developed in this thesis are evaluated in the specific context of Byzantine book epigrams, they are designed to be broadly applicable to other corpora characterised by short texts, such as lyrics, social media posts, and emails. Such corpora may also face the additional constraints of low-resource languages or incomplete and heterogeneous metadata. Each of the following subsections formulates a specific research question corresponding to one of these objectives. Together, these research questions guide the work presented in this thesis.

1.4.1 Data Management and Textual Representation

Before similarity between texts can be computed or any analysis can be performed, the data must be stored and represented in a way that reflects the semi-structured nature of texts and their relationships with asso-

ciated metadata. Such a representation is beneficial for tasks such as identifying patterns and discovering interconnections. As discussed in Section 1.2.1, relational databases handle the variety of textual data poorly, and their tabular model requires interconnections between entities to be reconstructed through joins rather than represented directly [29, 30]. Their rigid schema requirements further make them particularly ill-suited to accommodating metadata that vary greatly in structure and availability [29, 30]. A more flexible data model is therefore required, one that can accommodate diverse textual forms, preserve the semi-structured nature of the texts, and represent the network of relationships between texts at multiple levels of granularity as well as their associated metadata.

Graph database models are natural candidates for this purpose. By design, they represent data as networks of nodes and edges, making them well-suited to capturing intricate relationships between textual and contextual data [38]. Unlike relational databases, graph databases are schemaless, meaning that each node can carry a different set of properties, which is particularly advantageous when metadata differ substantially in structure, are frequently incomplete, or are entirely absent. They further support the querying of complex networks and patterns, and enable visual navigation of the data, aiding in the exploration of a corpus and the discovery of new interconnections. Beyond storage, the choice of text representation is a foundational decision that directly impacts the performance of all downstream tasks, including querying, similarity computation, context enrichment, and clustering. As described in Section 1.2.3, short texts pose a specific representation challenge, as advanced NLP approaches can struggle with sparse input or are unavailable for low-resource languages [70, 71]. At the same time, the internal structure of texts, together with their associated metadata, could offer new possibilities for the representation and analysis of short texts that are otherwise difficult to analyse.

This motivates the first research question of this thesis:

> **RQ 1**: To what extent can graph databases provide a suitable and efficient data model for representing short texts in a semi-structured way, supporting further analysis and the identification of patterns and interconnections at multiple levels of granularity?

It is important to emphasise that a graph data model for the analysis of short texts should not be seen as a replacement for an existing (relational) database. Rather, the graph database is intended to complement an existing database within a polyglot persistence architecture, where it is used for specific analytical tasks such as similarity computation and pattern analysis. The existing database continues to serve as the primary repository, in particular for the data that are not used in the analysis. The use of a polyglot architecture, however, introduces additional complexity, requiring not only methods for synchronising the databases but also advanced ETL pipelines to extract, transform, and load data between them. Ensuring that both stores remain consistent and up to date is a non-trivial task, as discussed in Section 1.2.1.

1.4.2 Short Text Similarity

Once texts are represented in a graph-based model, an essential step for subsequent analysis is the computation of similarity between them. Similarity measures play a pivotal role in a wide range of text analysis tasks, including the identification of interconnections, text classification, clustering, information retrieval, flexible

query answering, and AI [87, 88]. The result of such a measure is typically a value between 0 and 1, indicating the degree of resemblance between two text elements, where 0 denotes complete dissimilarity and 1 denotes complete similarity.

Despite their importance, computing meaningful similarity scores for short texts is particularly challenging. Standard orthographic similarity approaches treat texts as sequences of characters or tokens, which means they are unable to capture structural nuances such as the reordering of textual units or the similarity between individual parts of the texts [89]. This loss of structural information can be especially detrimental when working with highly interconnected texts, where connections between sub-units are as meaningful as the overall similarity between them. Semantic similarity approaches could in principle capture meaning beyond surface form, but struggle with the sparsity of short texts or are simply unavailable for low-resource languages [71, 90]. An orthographic similarity measure is therefore a natural starting point for comparing short texts, particularly in low-resource language settings.

Conventional orthographic approaches, however, typically result in a single similarity score that does not capture the relationships between different parts of the texts, leading to a loss of valuable structural information. The semi-structured graph representation proposed in response to RQ 1 could offer a suitable foundation for addressing this limitation, enabling a more interpretable and explainable orthographic similarity measure to be developed, one that exploits the graph structure to compare texts at multiple levels of granularity.

These considerations lead to the second research question of this thesis:

> **RQ 2**: How can an orthographic similarity measure be designed to capture similarities between short texts effectively at multiple levels of granularity, while exploiting the graph representation to account for their structural properties?

1.4.3 Context-Aware Short Text Similarity

As established in the previous subsection, orthographic similarity measures provide a principled basis for comparing short texts. However, as discussed in Section 1.2.3, relying solely on textual content can be insufficient for short texts, where the limited amount of text provides little context for meaningful comparison [68]. In such cases, contextual metadata associated with the texts, such as temporal, geographical, or authorship information, can enrich the comparison.

Incorporating metadata into a similarity framework is, however, non-trivial, particularly when those metadata are themselves incomplete or uncertain. In many real-world scenarios, contextual information is only partially available or entirely absent for part of the data. Existing approaches for handling incomplete metadata typically either discard a metadata attribute entirely when it is not available for all records, or apply imputation techniques to fill in the gaps [91]. The former approach is overly restrictive in corpora characterised by pervasive missing metadata, as it would render large portions of the metadata unusable, resulting in a loss of potentially valuable information. The latter is heavily dependent on assumptions and training data, and the substituted values can introduce errors that propagate into the similarity computation.

A more suitable approach would incorporate all available metadata into a context-aware similarity measure, regardless of how complete the metadata record is for any given text, and without relying on imputation of the missing values. Since the similarity between two texts is directly affected by the amount of missing metadata, an additional measure that explicitly captures this uncertainty is required. Such a measure, reflecting the confidence or the degree to which the context-aware similarity can be trusted given the available metadata, can then be propagated into subsequent analysis tasks to support more informed and transparent decision-making.

The third research question of this thesis therefore asks:

> **RQ 3**: How can contextual metadata complement a textual similarity measure, even when those metadata are missing or incomplete, such that the trustworthiness of the result is made explicit alongside the similarity score for downstream analysis?

1.4.4 Analysis of Short Texts

With a flexible graph-based data model, orthographic similarity measures, and context-aware similarity scores with associated confidence scores established through the preceding research objectives, the groundwork is laid for higher-level analysis of short texts. These individual components are not goals in themselves but building blocks for analytical tools that can support domain experts in navigating, querying, interpreting, and finding new insights in a corpus of short texts with incomplete and uncertain metadata. Three concrete analytical objectives are identified, each directly building on the results of the preceding research questions.

A first analytical objective concerns the querying of the corpus to find patterns and interconnections between texts. While querying is a common operation, the capabilities of the graph-based data model proposed in response to RQ 1 for querying short texts warrant investigation, and it should be determined whether the graph database sufficiently supports the discovery of patterns and interconnections between texts, with or without considering metadata. The inclusion of similarity scores, as developed in response to RQs 2 and 3, could offer additional opportunities to explore the corpus in a more flexible way, where results are not limited to exact matches but can include partially similar or structurally related texts.

Ranking query results and candidate interconnections constitutes a second analytical objective. Ranking results based on a similarity score is a common approach to prioritise the most relevant results, but when the underlying data are uncertain or incomplete, a single similarity score can provide a partial or misleading picture. The development of novel ranking techniques, based on both the similarity score and the associated confidence score as produced in response to RQ 3, could provide a richer and more informative basis for decision-making. Such techniques would reflect the trade-off between similarity (or satisfaction of a query) and confidence.

A third objective addresses the clustering of texts into meaningful groups. Clustering short texts is inherently difficult due to the sparsity of information they contain [66], but the orthographic and context-aware similarity scores and the confidence scores resulting from the response to RQs 2 and 3 offer new possibilities for organising a corpus in an unsupervised manner. The aim is to develop a clustering approach that can exploit these scores, handle missing metadata gracefully, and make the reliability of cluster assignments explicit through confidence

scores, so that domain experts can identify potentially unreliable groupings arising from incomplete contextual information.

Together, these three analytical objectives motivate the fourth and final research question of this thesis:

> **RQ 4**: To what extent can the graph-based representation, orthographic similarity measures, and context-aware similarity scores developed in this thesis support the analysis of short texts?
>
> **RQ 4.A:** How can (flexible) query answering techniques be applied to a graph-based data model of short texts to support the discovery of patterns and interconnections, with or without considering similarity or metadata?
>
> **RQ 4.B:** How can query results and candidate interconnections be ranked based on both their similarity score and associated confidence score to provide more informative and transparent results for domain experts?
>
> **RQ 4.C:** How can short texts be clustered based on context-aware similarity scores and confidence scores, in a way that handles missing metadata gracefully and makes the reliability of cluster assignments explicit?

1.5 Outline

The remainder of this thesis is structured as follows. In Chapter 2, the design of a new graph-based data model for representing short texts and their associated metadata is presented as a proposed solution to RQ 1. Particular attention is given to the challenges of representing short texts in a way that preserves their semi-structured nature by decomposing the texts into their discourse units, laying the foundation for the methods developed in subsequent chapters. Beyond the graph-based design, illustrative examples of this representation are provided using the DBBE data, demonstrating the ability to identify patterns and interconnections at multiple levels of granularity and addressing some aspects of RQ 4.A.

Chapter 3 presents a novel orthographic similarity measure for short texts, built upon the graph-based representation introduced in the preceding chapter. This measure is designed to capture similarities between short texts at multiple levels of granularity while taking into account their structural properties, as proposed in response to RQ 2. Using the DBBE data, the performance of this similarity measure is evaluated against the expert-curated groupings of texts (epigram types and verse variants), demonstrating the potential of grouping similar texts based on their orthographic similarity while providing insights into the relationships between different parts of the texts. Additionally, examples of querying the graph database using this similarity measure are provided, further addressing RQ 4.A.

Next, in Chapter 4, the orthographic similarity measure developed in Chapter 3 is enriched with contextual information, addressing RQ 3. This results not only in a single similarity score computed based on both textual and contextual similarities, but also in an associated confidence score that reflects the extent of missing and incomplete metadata in the aggregation of the individual similarity scores. This confidence score is essential for addressing the veracity challenges associated with the metadata. As in the preceding chapter, this novel

similarity measure is evaluated against the expert-curated groupings of texts, showing that textual similarity can be meaningfully enriched with contextual information, even under veracity concerns in the metadata.

Two further analytical techniques are presented in Chapters 5 and 6. Chapter 5 introduces novel techniques for ranking pairs of satisfaction and confidence grades, building on the similarity and confidence scores introduced in Chapter 4 and addressing RQ 4.B. Chapter 6 addresses RQ 4.C, proposing novel methods for clustering short texts based on the combination of textual and contextual similarities, as well as on combinations of the similarity and confidence scores of Chapter 4.

Finally, Chapter 7 provides a summary of the main contributions of this thesis and answers to the research questions. It further examines the practical applicability of the developed techniques for the DBBE and the broader Interconnected Texts project. This chapter also reflects on the implications and limitations of this work, and considers how it is positioned within the fast-evolving landscape of GenAI and LLMs. The chapter concludes by outlining potential directions for future research.

1.6 Publications

Most of the contributions and results presented in this thesis have been published in peer-reviewed journal articles and conference papers. The following lists provide an overview of these publications; those on which a chapter is directly based are also listed at the start of that chapter. The first list presents all journal publications with “Maxime Deforche” as author or co-author, in chronological order of publication.

1. Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, **Maxime Deforche**, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout, and Colin Swaelens. *The Database of Byzantine Book Epigrams project: principles, challenges, opportunities.* Published in the Journal of Data Mining and Digital Humanities, Special Issue on the Way to the Future of Digital Manuscript Studies, 2023.
   DOI: 10.46298/jdmdh.10244

2. **Maxime Deforche**, Ilse De Vos, Antoon Bronselaer, and Guy De Tré. *A Hierarchical Orthographic Similarity Measure for Interconnected Texts Represented by Graphs.* Published in Applied Sciences, Special Issue on Data Science Methods in Big Data Era, vol 14(4), 2024.
   DOI: 10.3390/app14041529

3. Colin Swaelens, **Maxime Deforche**, Ilse De Vos, and Els Lefever. *Verse, Variation, and Meaning: Annotating Semantic Distance in Greek Marginal Texts.* Accepted for publication in the Journal of Open Humanities Data, 2026.

The second list presents all conference papers, whether or not indexed in the Web of Science, with “Maxime Deforche” as author or co-author, in chronological order of publication. Each of these publications was accompanied by a presentation at the respective conference or workshop.

4. **Maxime Deforche**, Ilse De Vos, Antoon Bronselaer, and Guy De Tré. *An Orthographic Similarity Measure for Graph-Based Text Representations.* Published in Proceedings of the 15th International Conference on Flexible Query Answering Systems (FQAS), Lecture Notes in Artificial Intelligence, vol 14113, 2023.
   DOI: 10.1007/978-3-031-42935-4_17

5. Colin Swaelens, **Maxime Deforche**, Guy De Tré, Ilse De Vos, and Els Lefever. *How Relevant is Part-Of-Speech Information to Compute Similarity between Greek Verses in a Graph Database?* Published in Proceedings of the Workshop on Data-Driven Approaches to Ancient Languages (DAAL), 2024.

6. **Maxime Deforche**, Ilse De Vos, and Guy De Tré. *A Novel Approach to Context-Aware and Responsible Short Text Clustering.* Published in Proceedings of the 16th International Conference on Flexible Query Answering Systems (FQAS), Lecture Notes in Computer Science, vol 16119, 2025.
   Best Paper Award.
   DOI: 10.1007/978-3-032-05607-8_4

7. **Maxime Deforche** and Guy De Tré. *Ranking L-grades for Veracity Handling in Logic Reasoning.* Accepted for publication in Proceedings of the Joint World Congress of the International Fuzzy Systems Association (IFSA) and the North American Fuzzy Information Processing Society (NAFIPS), 2025.

8. **Maxime Deforche**, Guy De Tré, Paolo Fosci, and Giuseppe Psaila. *On Veracity Handling in Document Stores: A Novel Technique Based on L-graded Logic Running on the J-CO Framework.* Published in Proceedings of the 21st International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU), Communications in Computer and Information Science, vol 3020, 2026.
   DOI: 10.1007/978-3-032-28997-1_23

References

[1] Online Etymology Dictionary. "Origin and History of the Word Data". url: https://www.etymonline.com/word/data (visited on 23/02/2026).

[2] IBM. "What Is Data?" url: https://www.ibm.com/think/topics/data (visited on 23/02/2026).

[3] Alexander Marshack. "The Roots of Civilization: The Cognitive Beginnings of Man's First Art, Symbol and Notation". USA: McGraw-Hill, 1971.

[4] Georges Ifrah. "The Universal History of Numbers: From Prehistory to the Invention of the Computer". UK: Harvill Press, 1998.

[5] James Gleick. "The Information: A History, a Theory, a Flood". USA: Pantheon Books, 2011.

[6] Niels Gaul. "The Manuscript Tradition". In: *A Companion to the Ancient Greek Language*. John Wiley & Sons, Ltd, 2010. Chap. 6, pp. 69–81. DOI: https://doi.org/10.1002/9781444317398.ch6.

[7] Irving Fang. "History of Mass Communication: Six Information Revolutions". USA: Focal Press, 1997.

[8] Paul E. Ceruzzi. "A History of Modern Computing". USA: MIT Press, 2003.

[9] Ibrar Yaqoob, Ibrahim Abaker Targio Hashem, Abdullah Gani, Salimah Mokhtar, Ejaz Ahmed, Nor Badrul Anuar and Athanasios V. Vasilakos. "Big Data: From Beginning to Future". In: *International Journal of Information Management* 36.6, Part B (2016), pp. 1231–1247. DOI: https://doi.org/10.1016/j.ijinfomgt.2016.07.009.

[10] Min Chen, Shiwen Mao and Yunhao Liu. "Big Data: A Survey". In: *Mobile Networks and Applications* 19.2 (2014), pp. 171–209. DOI: https://doi.org/10.1007/s11036-013-0489-0.

[11] Min Chen, Shiwen Mao, Yin Zhang, Victor CM Leung et al. "Big Data: Related Technologies, Challenges and Future Prospects". Vol. 100. Springer, 2014. DOI: https://doi.org/10.1007/978-3-319-06245-7.

[12] Lina Zhou, Shimei Pan, Jianwu Wang and Athanasios V. Vasilakos. "Machine Learning on Big Data: Opportunities and Challenges". In: *Neurocomputing* 237 (2017), pp. 350–361. issn: 0925-2312. DOI: https://doi.org/10.1016/j.neucom.2017.01.026.

[13] Fiona Fui-Hoon Nah, Ruilin Zheng, Jingyuan Cai, Keng Siau and Langtao Chen. "Generative AI and ChatGPT: Applications, Challenges, and AI-Human Collaboration". In: *Journal of Information Technology Case and Application Research* 25.3 (2023), pp. 277–304. DOI: https://doi.org/10.1080/15228053.2023.2233814.

[14] Fabio Duarte. "Amount of Data Created Daily". Exploding Topics. 2026. url: https://explodingtopics.com/blog/data-generated-per-day (visited on 02/03/2026).

[15] David Reinsel, John Gantz and John Rydning. "The Digitization of the World from Edge to Core". IDC and Seagate. 2020. url: https://www.seagate.com/files/www-content/our-story/trends/files/dataage-idc-report-final.pdf (visited on 18/02/2026).

[16] Huawei Technologies Co., Ltd. “Data Storage 2030”. 2024. url: https://www-file.huawei.com/-/media/corp2020/pdf/giv/2024/data_storage_whitepaper_2030_en.pdf (visited on 18/02/2026).

[17] Michael Chen. “What Is Big Data?” Oracle. 2024. url: https://www.oracle.com/europe/big-data/what-is-big-data/ (visited on 02/03/2026).

[18] Doug Laney. *3D Data Management: Controlling Data Volume, Velocity and Variety*. Gartner, 2001.

[19] Borko Furht and Flavio Villanustre. “Introduction to Big Data”. In: *Big Data Technologies and Applications*. Cham: Springer International Publishing, 2016, pp. 3–11. isbn: 978-3-319-44550-2. DOI: https://doi.org/10.1007/978-3-319-44550-2_1.

[20] Nawsher Khan, Mohammed Alsaqer, Habib Shah, Gran Badsha, Aftab Ahmad Abbasi and Soulmaz Salehian. “The 10 Vs, Issues and Challenges of Big Data”. In: *Proceedings of the 2018 International Conference on Big Data and Education*. ICBDE ’18. New York, NY, USA: Association for Computing Machinery, 2018, pp. 52–56. isbn: 9781450363587. DOI: https://doi.org/10.1145/3206157.3206166.

[21] Gayatri Kapil, Alka Agrawal and R. A. Khan. “A Study of Big Data Characteristics”. In: *2016 International Conference on Communication and Electronics Systems (ICCES)*. 2016, pp. 1–4. DOI: https://doi.org/10.1109/CESYS.2016.7889917.

[22] Gottfried Vossen. “Big Data as the New Enabler in Business and Other Intelligence”. In: *Vietnam Journal of Computer Science* 1.1 (2014). DOI: https://doi.org/10.1007/s40595-013-0001-6.

[23] Jinchuan Chen, Yueguo Chen, Xiaoyong Du, Cuiping Li, Jiaheng Lu, Suyun Zhao and Xuan Zhou. “Big Data Challenge: A Data Management Perspective”. In: *Frontiers of Computer Science* 7.2 (2013), pp. 157–164. DOI: https://doi.org/10.1007/s11704-013-3903-7.

[24] Abdullah Gani, Aisha Siddiqa, Shahaboddin Shamshirband and Fariza Hanum. “A Survey on Indexing Techniques for Big Data: Taxonomy and Performance Evaluation”. In: *Knowledge and Information Systems* 46.2 (2016), pp. 241–284. DOI: https://doi.org/10.1007/s10115-015-0830-y.

[25] Diego De Siqueira Braga, Marco Niemann, Bernd Hellingrath and Fernando Buarque De Lima Neto. “Survey on Computational Trust and Reputation Models”. In: *ACM Comput. Surv.* 51.5 (2018). DOI: https://doi.org/10.1145/3236008.

[26] Guy De Tré, Milan Peelman and Jozo Dujmović. “Logic Reasoning under Data Veracity Concerns”. In: *International Journal of Approximate Reasoning* 161 (2023), p. 108977. issn: 0888-613X. DOI: https://doi.org/10.1016/j.ijar.2023.108977.

[27] E. F. Codd. “A Relational Model of Data for Large Shared Data Banks”. In: *Commun. ACM* 13.6 (1970), pp. 377–387. issn: 0001-0782. DOI: https://doi.org/10.1145/362384.362685.

[28] Rick Cattell. “Scalable SQL and NoSQL Data Stores”. In: *SIGMOD Rec.* 39.4 (2011), pp. 12–27. issn: 0163-5808. DOI: https://doi.org/10.1145/1978915.1978919.

[29] Andreas Meier and Michael Kaufmann. “SQL & NoSQL Databases”. Springer, 2019. DOI: https://doi.org/10.1007/978-3-658-24549-8.

[30] Elias Dritsas and Maria Trigka. “Database Systems in the Big Data Era: Architectures, Performance, and Open Challenges”. In: *IEEE Access* 13 (2025), pp. 95068–95084. DOI: https://doi.org/10.1109/ACCESS.2025.3572059.

[31] Eric A Brewer. “Towards Robust Distributed Systems”. In: *PODC*. Vol. 7. Portland, OR. 2000, pp. 343477–343502. DOI: https://doi.org/10.1145/343477.343502.

[32] Jing Han, Haihong E, Guan Le and Jian Du. “Survey on NoSQL Database”. In: *2011 6th International Conference on Pervasive Computing and Applications*. 2011, pp. 363–366. DOI: https://doi.org/10.1109/ICPCA.2011.6106531.

[33] Peter Pin-Shan Chen. “The Entity-Relationship Model—Toward a Unified View of Data”. In: *ACM Trans. Database Syst.* 1.1 (1976), pp. 9–36. issn: 0362-5915. DOI: https://doi.org/10.1145/320434.320440.

[34] Edgar F Codd. “Further Normalization of the Data Base Relational Model”. In: *Data Base Systems* 6.1972 (1972), pp. 33–64.

[35] Edgar F Codd. “Recent Investigations in Relational Data Base Systems.” IBM Thomas J. Watson Research Division, 1974.

[36] G.L. Sanders and Seungkyoon Shin. “Denormalization Effects on Performance of RDBMS”. In: *Proceedings of the 34th Annual Hawaii International Conference on System Sciences*. 2001. DOI: https://doi.org/10.1109/HICSS.2001.926306.

[37] Jagdev Bhogal and Imran Choksi. “Handling Big Data Using NoSQL”. In: *2015 IEEE 29th International Conference on Advanced Information Networking and Applications Workshops*. 2015, pp. 393–398. DOI: https://doi.org/10.1109/WAINA.2015.19.

[38] Renzo Angles and Claudio Gutierrez. “Survey of Graph Database Models”. In: *ACM Computing Surveys (CSUR)* 40.1 (2008), pp. 1–39. DOI: https://doi.org/10.1145/1322432.1322433.

[39] Pwint Phyu Khine and Zhaoshun Wang. “A Review of Polyglot Persistence in the Big Data World”. In: *Information* 10.4 (2019). issn: 2078-2489. DOI: https://doi.org/10.3390/info10040141.

[40] Felix Kiehn, Mareike Schmidt, Daniel Glake, Fabian Panse, Wolfram Wingerath, Benjamin Wollmer, Martin Poppinga and Norbert Ritter. “Polyglot Data Management: State of the Art & Open Challenges”. In: *Proc. VLDB Endow.* 15.12 (2022), pp. 3750–3753. issn: 2150-8097. DOI: https://doi.org/10.14778/3554821.3554891.

[41] Jean-Louis Monino. “Data Value, Big Data Analytics, and Decision-Making”. In: *Journal of the Knowledge Economy* 12.1 (2021), pp. 256–267. issn: 1868-7873. DOI: https://doi.org/10.1007/s13132-016-0396-2.

[42] Gustavo Grander, Luciano Ferreira da Silva and Ernesto Del Rosário Santibañez Gonzalez. “Big Data as a Value Generator in Decision Support Systems: A Literature Review”. In: *Revista de Gestão* 28.3 (2021), pp. 205–222. issn: 1809-2276. DOI: https://doi.org/10.1108/REGE-03-2020-0014.

[43] Manoj Kumar Gupta and Pravin Chandra. “A Comprehensive Survey of Data Mining”. In: *International Journal of Information Technology* 12.4 (2020), pp. 1243–1257. issn: 2511-2112. DOI: https://doi.org/10.1007/s41870-020-00427-7.

[44] Manjula Ramannavar and Nandini S. Sidnal. “Big Data and Analytics—A Journey through Basic Concepts to Research Issues”. In: *Proceedings of the International Conference on Soft Computing Systems*. New Delhi: Springer India, 2016, pp. 291–306. isbn: 978-81-322-2674-1. DOI: https://doi.org/10.1007/978-81-322-2674-1_29.

[45] L.A. Zadeh. “Fuzzy Logic = Computing with Words”. In: *IEEE Transactions on Fuzzy Systems* 4.2 (1996), pp. 103–111. DOI: https://doi.org/10.1109/91.493904.

[46] Patrick Bosc and Olivier Pivert. “Some Approaches for Relational Databases Flexible Querying”. In: *Journal of Intelligent Information Systems* 1.3 (1992), pp. 323–354. issn: 1573-7675. DOI: https://doi.org/10.1007/BF00962923.

[47] Troels Andreasen, Gloria Bordogna, Guy De Tré, Janusz Kacprzyk, Henrik Legind Larsen and Sławomir Zadrożny. “The Power and Potentials of Flexible Query Answering Systems: A Critical and Comprehensive Analysis”. In: *Data & Knowledge Engineering* 149 (2024), p. 102246. issn: 0169-023X. DOI: https://doi.org/10.1016/j.datak.2023.102246.

[48] Indranil Bose and Radha K. Mahapatra. “Business Data Mining — a Machine Learning Perspective”. In: *Information & Management* 39.3 (2001), pp. 211–225. issn: 0378-7206. DOI: https://doi.org/https://doi.org/10.1016/S0378-7206(01)00091-X.

[49] Rajiv Chooramun. “Evolution of Data Mining: From Statistical Foundations to Big Data and Deep Learning”. In: *International Journal of Computer Applications* 187.4 (2025), pp. 12–20. issn: 0975-8887. DOI: https://doi.org/10.5120/ijca2025924837.

[50] Dang Minh, H. Xiang Wang, Y. Fen Li and Tan N. Nguyen. “Explainable Artificial Intelligence: A Comprehensive Review”. In: *Artificial Intelligence Review* 55.5 (2022), pp. 3503–3568. issn: 1573-7462. DOI: https://doi.org/10.1007/s10462-021-10088-y.

[51] Emrullah Şahin, Naciye Nur Arslan and Durmuş Özdemir. “Unlocking the Black Box: An in-Depth Review on Interpretability, Explainability, and Reliability in Deep Learning”. In: *Neural Computing and Applications* 37.2 (2025), pp. 859–965. issn: 1433-3058. DOI: https://doi.org/10.1007/s00521-024-10437-2.

[52] Andries P Engelbrecht et al. “Computational Intelligence: An Introduction”. Vol. 2. Wiley Online Library, 2007. DOI: https://doi.org/10.1002/9780470512517.

[53] Alberto Fernandez, Francisco Herrera, Oscar Cordon, Maria Jose del Jesus and Francesco Marcelloni. “Evolutionary Fuzzy Systems for Explainable Artificial Intelligence: Why, When, What for, and Where To?” In: *IEEE Computational Intelligence Magazine* 14.1 (2019), pp. 69–81. DOI: https://doi.org/10.1109/MCI.2018.2881645.

[54] Vishnu Pendyala. “Veracity of Big Data”. In: *Machine Learning and Other Approaches to Verifying Truthfulness* (2018). DOI: https://doi.org/10.1007/978-1-4842-3633-8.

[55] Hossein Hassani, Christina Beneki, Stephan Unger, Maedeh Taj Mazinani and Mohammad Reza Yeganegi. "Text Mining in Big Data Analytics". In: *Big Data and Cognitive Computing* 4.1 (2020). issn: 2504-2289. DOI: https://doi.org/10.3390/bdcc4010001.

[56] Jiapeng Wang and Yihong Dong. "Measurement of Text Similarity: A Survey". In: *Information* 11.9 (2020). issn: 2078-2489. DOI: https://doi.org/10.3390/info11090421.

[57] Najlah Gali, Radu Mariescu-Istodor and Pasi Fränti. "Similarity Measures for Title Matching". In: *2016 23rd International Conference on Pattern Recognition (ICPR)*. IEEE. 2016, pp. 1548–1553. DOI: https://doi.org/10.1109/ICPR.2016.7899857.

[58] Rajvardhan Patil, Sorio Boit, Venkat Gudivada and Jagadeesh Nandigam. "A Survey of Text Representation and Embedding Techniques in NLP". In: *IEEE Access* 11 (2023), pp. 36120–36146. DOI: https://doi.org/10.1109/ACCESS.2023.3266377.

[59] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado and Jeff Dean. "Distributed Representations of Words and Phrases and Their Compositionality". In: *Advances in Neural Information Processing Systems*. Vol. 26. Curran Associates, Inc., 2013. DOI: https://doi.org/10.48550/arXiv.1310.4546.

[60] Quoc V. Le and Tomas Mikolov. "Distributed Representations of Sentences and Documents". In: *arXiv Preprint* (2014). DOI: https://doi.org/10.48550/arXiv.1405.4053.

[61] Jeffrey Pennington, Richard Socher and Christopher D Manning. "Glove: Global Vectors for Word Representation". In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1532–1543. DOI: https://doi.org/10.3115/v1/D14-1162.

[62] Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: https://doi.org/10.18653/v1/N19-1423.

[63] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992. DOI: https://doi.org/10.18653/v1/D19-1410.

[64] James Jie Pan, Jianguo Wang and Guoliang Li. "Vector Database Management Techniques and Systems". In: *Companion of the 2024 International Conference on Management of Data*. SIGMOD '24. Santiago AA, Chile: Association for Computing Machinery, 2024, pp. 597–604. isbn: 9798400704222. DOI: https://doi.org/10.1145/3626246.3654691.

[65] Dhivya Chandrasekaran and Vijay Mago. "Evolution of Semantic Similarity—A Survey". In: *ACM Comput. Surv.* 54.2 (2021). issn: 0360-0300. DOI: https://doi.org/10.1145/3440755.

[66] Majid Hameed Ahmed, Sabrina Tiun, Nazlia Omar and Nor Samsiah Sani. “Short Text Clustering Algorithms, Application and Challenges: A Survey”. In: *Applied Sciences* 13.1 (2023). issn: 2076-3417. DOI: https://doi.org/10.3390/app13010342.

[67] Rania Albalawi, Tet Hin Yeap and Morad Benyoucef. “Using Topic Modeling Methods for Short-Text Data: A Comparative Analysis”. In: *Frontiers in Artificial Intelligence* Volume 3 (2020). DOI: https://doi.org/10.3389/frai.2020.00042.

[68] Dimas Wibisono Prakoso, Asad Abdi and Chintan Amrit. “Short Text Similarity Measurement Methods: A Review”. In: *Soft Computing* 25.6 (2021), pp. 4699–4723. issn: 1433-7479. DOI: https://doi.org/10.1007/s00500-020-05479-2.

[69] Ge Song, Yunming Ye, Xiaolin Du, Xiaohui Huang and Shifu Bie. “Short Text Classification: A Survey”. In: *Journal of Multimedia* 9.5 (2014). DOI: https://doi.org/10.4304/jmm.9.5.635-643.

[70] Junyang Chen, Zhiguo Gong, Wei Wang, Xiao Dong, Wei Wang, Weiwen Liu, Cong Wang and Xian Chen. “Inductive Document Representation Learning for Short Text Clustering”. In: *Machine Learning and Knowledge Discovery in Databases*. Cham: Springer International Publishing, 2021, pp. 600–616. isbn: 978-3-030-67664-3. DOI: https://doi.org/10.1007/978-3-030-67664-3_36.

[71] Partha Pakray, Alexander Gelbukh and Sivaji Bandyopadhyay. “Natural Language Processing Applications for Low-Resource Languages”. In: *Natural Language Processing* 31.2 (2025), pp. 183–197. DOI: https://doi.org/10.1017/nlp.2024.33.

[72] Alexandre Magueresse, Vincent Carles and Evan Heetderks. *Low-Resource Languages: A Review of Past Work and Future Challenges*. 2020. DOI: https://doi.org/10.48550/arXiv.2006.07264.

[73] Nathan Stringham and Mike Izbicki. “Evaluating Word Embeddings on Low-Resource Languages”. In: *Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems*. Online: Association for Computational Linguistics, 2020, pp. 176–186. DOI: https://doi.org/10.18653/v1/2020.eval4nlp-1.17.

[74] Colin Swaelens, Ilse De Vos and Els Lefever. “Evaluating Existing Lemmatisers on Unedited Byzantine Greek Poetry”. In: *Proceedings of the Ancient Language Processing Workshop*. Varna, Bulgaria, 2023, pp. 111–116. DOI: https://doi.org/10.5281/zenodo.8337363.

[75] Gérard Genette. “Paratexts: Thresholds of Interpretation”. Vol. 20. Cambridge University Press, 1997. DOI: https://doi.org/10.1017/CBO9780511549373.

[76] Marc D. Lauxtermann. “Byzantine Poetry from Pisides to Geometres: Texts and Contexts”. Vol. 1. Wiener Byzantinistische Studien. Vienna: Austrian Academy of Sciences Press, 2003.

[77] Colin Swaelens, Ilse De Vos and Els Lefever. “Linguistic Annotation of Byzantine Book Epigrams”. In: *Language Resources and Evaluation* 59.1 (2025), pp. 109–134. DOI: https://doi.org/10.1007/s10579-023-09703-x.

[78] Kristoffel Demoen, Gilbert Bentein, Klaas Bentein, Floris Bernard, Julián Bértola, Julie Boeten, Mathijs Clement, Cristina Cocola, Eline Daveloose, Sien De Groot, Pieterjan De Potter, Ilse De Vos, Krystina Kubina, Hanne Lauwers, Paulien Lemay, Renaat Meesters, Marjolein Morbé, Delphine Nachtergaele, Marthe

Nemegeer, Joachim Nielandt, Mace Ojala, Lisa-Lou Péchillon, Raf Praet, Rachele Ricceri, Anne-Sophie Rouckhout, Jeroen Schepens, Febe Schollaert, Lev Shadrin, Nina Sietis, Dimitrios Skrekas, Colin Swaelens, Maria Tomadaki, Sarah-Helena Van den Brande, Merel Van Nieuwerburgh, Lotte Van Olmen, Noor Vanhoe and Nina Vanhoutte. *Database of Byzantine Book Epigrams (DBBE)*. Version 1.0.0. Zenodo, Feb. 2023. DOI: https://doi.org/10.5281/zenodo.7682523.

[79] Floris Bernard and Kristoffel Demoen. "Byzantine Book Epigrams from Manuscripts to a Digital Database". In: *From Ancient Manuscripts to the Digital Era : Readings and Literacies, Proceedings*. Lausanne, Switzerland: PPUR, 2012, p. 8.

[80] Kristoffel Demoen. "Epigrams on Authors and Books as Text and Paratext". In: *Greek Epigram from the Hellenistic to the Early Byzantine Era*. Oxford University Press, 2019, pp. 66–82. isbn: 9780198836827. DOI: https://doi.org/10.1093/oso/9780198836827.003.0005.

[81] Julie Boeten and Sien De Groot. "Byzantine Book Epigrams: The Case of ωσπερ ξενοι..." In: *Proceedings of the Byzantine International Congress*. Belgrade, 2016.

[82] Julie Boeten and Sien De Groot. "The Byzantine Antiquarian : A Case Study of a Compiled Colophon". In: *BYZANTINISCHE ZEITSCHRIFT* 112.1 (2019), pp. 31–46. issn: 0007-7704. DOI: https://doi.org/10.1515/bz-2019-0003.

[83] Kyriaki Giannikou. "Formulaic Language in Byzantine Book Epigrams : A Typology Grounded in Ritually-Charged Pragmatic Functions". In: *GREEK ROMAN AND BYZANTINE STUDIES* 66.1 (2026), pp. 1–41. issn: 0017-3916.

[84] Eleonora Lauro. "Alongside the Text : Byzantine Metrical Paratexts in Gospel Manuscripts from Medieval Southern Italy". In: *Intersection : Interdisciplinary Approaches to New Testament Text and Manuscript Studies*. Hendrickson, 2025, pp. 175–189. isbn: 9798400508349.

[85] Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, Maxime Deforche, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout and Colin Swaelens. "The Database of Byzantine Book Epigrams Project : Principles, Challenges, Opportunities". eng. In: *Journal of Data Mining and Digital Humanities* On the Way to the Future of Digital Manuscript Studies (2023), p. 41. issn: 2416-5999. DOI: https://doi.org/10.46298/jdmdh.10244.

[86] Rob Kitchin and Tracey P. Lauriault. "Small Data in the Era of Big Data". In: *GeoJournal* 80.4 (2015), pp. 463–475. issn: 1572-9893. DOI: https://doi.org/10.1007/s10708-014-9601-7.

[87] Sonali Vijay Gaikwad, Archana Chaugule and Pramod Patil. "Text Mining Methods and Techniques". In: *International Journal of Computer Applications* 85.17 (2014). DOI: https://doi.org/10.5120/14937-3507.

[88] Andreas Hotho, Andreas Nürnberger and Gerhard Paaß. "A Brief Survey of Text Mining". In: *Journal for Language Technology and Computational Linguistics* 20.1 (2005), pp. 19–62. DOI: https://doi.org/10.21248/jlcl.20.2005.68.

[89] Najlah Gali, Radu Mariescu-Istodor, Damien Hostettler and Pasi Fränti. "Framework for Syntactic String Similarity Measures". In: *Expert Systems with Applications* 129 (2019), pp. 169–185. issn: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2019.03.048.

[90] Wael H Gomaa, Aly A Fahmy et al. "A Survey of Text Similarity Approaches". In: *International Journal of Computer Applications* 68.13 (2013), pp. 13–18. DOI: https://doi.org/10.5120/11638-7118.

[91] Ludmila Himmelspach and Stefan Conrad. "Clustering Approaches for Data with Missing Values: Comparison and Evaluation". In: *2010 Fifth International Conference on Digital Information Management (ICDIM)*. 2010, pp. 19–28. DOI: https://doi.org/10.1109/ICDIM.2010.5664691.

2

A Graph Data Model for Short Interconnected Texts

"Structure is more important than content
in the transmission of information."

— **Abbie Hoffman**

This chapter presents the design of a graph-based data model for representing short texts and their associated metadata. Within a Labelled Property Graph (LPG), the model decomposes text documents into hierarchically organised discourse units, where the smallest units are represented as individual nodes and larger units as subgraphs composed of their constituent elements. Each text is further connected to contextual metadata such as geographical regions, authors, and genres, while additional textual properties, where available, are stored on the textual nodes themselves. The resulting graph structure preserves the semi-structured nature of the texts, captures the numerous interconnections between them at multiple levels of granularity, and supports (flexible) querying and visual exploration of the corpus to identify patterns and interconnections between the texts.

The design of this graph-based text representation, alongside the modelling of associated metadata, addresses the first research question of this thesis:

> **RQ 1:** To what extent can graph databases provide a suitable and efficient data model for representing short texts in a semi-structured way, supporting further analysis and the identification of patterns and interconnections at multiple levels of granularity?

In addition, the querying and exploration capabilities demonstrated in this chapter partially address:

> **RQ 4.A:** How can (flexible) query answering techniques be applied to a graph-based data model of short texts to support the discovery of patterns and interconnections, with or without considering similarity or metadata?

The content of this chapter is based on parts of the following publications:

- **Maxime Deforche**, Ilse De Vos, Antoon Bronselaer, and Guy De Tré. *An Orthographic Similarity Measure for Graph-Based Text Representations.* Published in Proceedings of the 15th International Conference on Flexible Query Answering Systems (FQAS), Lecture Notes in Artificial Intelligence, vol 14113, 2023.
  DOI: 10.1007/978-3-031-42935-4_17

- **Maxime Deforche**, Ilse De Vos, Antoon Bronselaer, and Guy De Tré. *A Hierarchical Orthographic Similarity Measure for Interconnected Texts Represented by Graphs.* Published in Applied Sciences, Special Issue on Data Science Methods in Big Data Era, vol 14(4), 2024.
  DOI: 10.3390/app14041529

The remainder of this second chapter is structured as follows. Section 2.1 introduces the problem of representing interconnected short texts for computational analysis and outlines the novel graph-based representation proposed in this chapter. Related work on text representation for computational analysis is discussed in Section 2.2. Section 2.3 provides the necessary preliminaries on text, graphs, and the LPG model. The proposed graph-based text representation is presented in Section 2.4, followed by the modelling of metadata and additional textual properties in Section 2.5. Section 2.6 demonstrates the querying and exploration capabilities of the graph database through illustrative examples based on the Database of Byzantine Book Epigrams (DBBE) dataset. Finally, Section 2.7 summarises the contributions of this chapter, discusses limitations, and outlines directions for future work.

2.1 Introduction

Textual data, from which a wide range of insights can be extracted, constitute one of the most prevalent forms of information in the modern big data landscape [1]. In their raw form, however, texts are sequences of characters with no fixed structure, which do not readily lend themselves to advanced analysis [2]. To enable such analysis, texts must first be transformed into a suitable representation that captures their content, structure, context, and semantics in a way that supports downstream tasks. These tasks include text mining operations such as classification, clustering, and information retrieval, as well as the computation of similarity scores, visualisation, and the discovery of patterns [3–7]. In some corpora, individual texts do not exist in isolation but are deeply interconnected: they may share words, sentences, or larger structural units, and entire texts may be partial or reworked variants of one another. Capturing these interconnections within the representation, alongside the contextual metadata associated with each text, such as authorship, date, or geographical origin, is essential for a comprehensive analysis of these corpora.

The representation of texts for computational analysis is, however, far from trivial. Text usually has no fixed structure and exhibits considerable variety in form, length, and content, making it difficult to represent with a one-size-fits-all approach [2]. Although vector embeddings derived from modern Natural Language Processing (NLP) techniques are a typical solution [8], such approaches often fail when applied to short texts, which offer fewer discriminative features and less context [9–11]. This limitation is compounded in low-resource language settings, where NLP tools are often unavailable or insufficient [12–15]. The limited applicability of these techniques motivates us to explore a structurally oriented representation of raw textual content, rather

than one rooted in statistical or semantic modelling. At the same time, the modelling of interconnections between texts poses a distinct challenge: relational databases are not designed to efficiently store and traverse the dense networks of relationships that emerge when texts share content at multiple levels of granularity [16]. Similarly, contextual metadata, which can vary in structure and are frequently incomplete or entirely absent, are difficult to accommodate within a fixed relational schema [16]. Taken together, these challenges call for novel representation techniques that can more adequately handle unstructured, short, and potentially noisy texts in a flexible and semi-structured manner, while simultaneously capturing the interconnections between them and accommodating heterogeneous metadata.

In this chapter, we address these challenges by proposing a graph-based data model. We decompose each text into its discourse units, such as words, sentences, paragraphs, and complete texts, and represent each unit as a node in the graph. These nodes are connected by directed edges that encode the hierarchical containment relationships between the different levels of the decomposition, so that the original text can be reconstructed by traversing the graph from higher-level to lower-level nodes. Identical textual elements between different texts are represented by the same node in the graph, so that shared content is captured directly by its structure. Additional textual properties can be stored on the textual nodes themselves, and contextual metadata can be modelled as separate nodes connected to the corresponding text nodes, allowing metadata of varying structure and availability to coexist within the same graph without requiring a fixed schema. The resulting graph can be implemented in a graph database system, where it serves both as a representation for further analysis, including similarity computation, ranking, and clustering, as developed in subsequent chapters, and as a tool for flexible querying and visual exploration of the corpus.

The principal novelty of the proposed model lies in the hierarchical, graph-based decomposition of texts into their discourse units as a general-purpose representation for short, interconnected texts. Unlike flat text representations, the proposed model preserves the internal structure of each text in a semi-structured manner, and, through the sharing of nodes for identical content, makes the interconnections between texts directly visible in the graph rather than leaving them to be computed or discovered through separate analysis. This structural representation enables analysis at multiple levels of granularity, from individual words to complete texts, and provides a natural foundation for the development of similarity measures that exploit textual structure. The integration of textual properties and contextual metadata within the same graph further enriches the representation, supporting enhanced retrieval and pattern analysis that combines textual and contextual criteria. The links between textual and metadata nodes, combined with the visual and interactive capabilities of modern graph database systems such as Neo4j [17], facilitate the exploration of relationships that might otherwise remain hidden in a tabular or record-based data store. Because the model operates on the orthographic surface form of the texts rather than on their semantics, it requires no language-specific resources and can be applied to corpora in any language, including low-resource ones. Together with the integrated contextual metadata, this structural representation forms the foundation for the similarity measures and other analytical techniques developed in subsequent chapters.

We illustrate the proposed data model through its application to the DBBE corpus of Byzantine book epigrams [18, 19], introduced in Section 1.3: a collection of typically short poems written in a low-resource language. The epigrams are decomposed into three hierarchical levels of discourse units, namely words, verses, and complete

texts, and enriched with the available contextual metadata, which are often incomplete. Using this representation implemented in a graph database, we demonstrate the discovery of important words and verses shared across multiple texts, the identification and analysis of interconnections between related epigrams, and the ability to query the corpus for structural and textual patterns.

2.2 Related Work

A wide range of techniques have been developed for representing texts in a form suitable for computational analysis. In this section, we review the most prominent approaches and discuss their strengths and limitations in the context of short, interconnected texts.

The most basic form of text representation treats a text as a raw sequence of characters or tokens. Operations on such representations include full-text search, substring matching, and the application of regular expressions to identify patterns [20]. Full-text search engines such as Apache Lucene [21] and systems built upon it, such as Elasticsearch [22], extend plain text storage with inverted indices that map each token to the documents in which it appears, enabling efficient retrieval through approximate matching, stemming, and relevance ranking. While these systems are effective for document retrieval, they treat each text as an independent, flat collection of tokens, without preserving word order, internal structure, or any notion of shared content across texts.

Tries and suffix trees introduce a degree of structural organisation into string-based text representations [23, 24]. A trie stores a set of strings as a tree in which each path from the root to a leaf corresponds to a string, with shared prefixes represented by shared branches [25]. Suffix trees build on a similar principle by storing and indexing all suffixes of a given string, enabling efficient substring search, Longest Common Substring (LCS) computation, and repeated pattern detection [26, 27]. Both structures operate at the character level and do not distinguish between different levels of discourse such as words, sentences, or paragraphs. They are optimised for efficient string operations rather than for representing the semantic or structural organisation of a text, and they do not naturally capture the connections between distinct texts beyond shared substrings.

To enable quantitative comparison and computational processing of texts, a well-established family of methods transforms texts into numerical vectors, which can then serve as features for Machine Learning (ML) tasks, similarity computation, or text mining [8, 28]. Broadly, two types of vector representations are commonly used: frequency-based representations and embedding-based representations [28]. Frequency-based representations, such as the Bag of Words (BoW) model, represent each text as a vector whose values indicate the frequency of each token from the vocabulary [29, 30]. The Term Frequency-Inverse Document Frequency (TF-IDF) weighting scheme refines this approach by down-weighting tokens that appear frequently across the corpus, thereby emphasising more discriminative terms [31]. While simple to compute and language-independent, these representations produce sparse, high-dimensional vectors, especially for short texts, and discard word order, hierarchical structure, and the relationships between sub-elements of the texts.

Embedding-based representations, by contrast, produce dense, low-dimensional vectors learned from large corpora, in which tokens appearing in similar contexts receive similar representations [8]. Figure 2.1a illustrates a simple example of a word embedding, in which the words *cat*, *dog*, and *bird* are represented as vectors in a three-

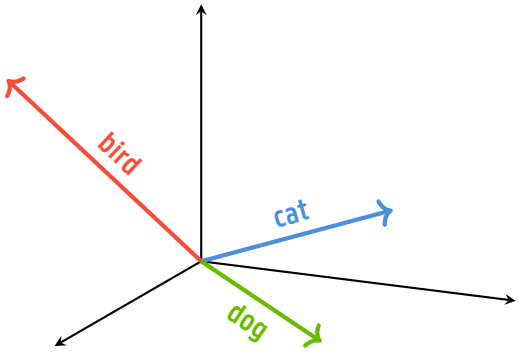
dimensional space. The resulting positions show that *cat* and *dog* are closer to each other than to *bird*, reflecting their greater semantic similarity. Prominent techniques include Word2Vec [32], which learns embeddings from local context, GloVe [33], which derives them from global co-occurrence statistics, and FastText [34], which incorporates sub-word information for robustness to morphological variation. These word-level representations can be extended to entire texts by averaging the constituent word embeddings [35] or by training dedicated document-level models such as Doc2Vec [36], though both strategies lose effectiveness for very short texts [11]. The state of the art in this category is formed by contextualised embeddings from transformer-based models such as Bidirectional Encoder Representations from Transformers (BERT) [37], and sentence-level variants such as Sentence-BERT [38]. Despite their effectiveness for many tasks, all embedding-based approaches require large, representative training corpora, which are often unavailable for low-resource languages [12–14]. Moreover, the resulting numerical vectors do not explicitly encode structural relationships between texts or their sub-elements, limiting their suitability for tasks that require the exploration of interconnections at multiple levels of granularity.

Beyond vector representations, topic models, such as Latent Semantic Analysis (LSA) [39] and Latent Dirichlet Allocation (LDA) [40, 41], constitute another class of textual representations. These models represent texts in a lower-dimensional space of latent, or hidden, topics that capture the underlying themes in the corpus. For instance, the words *computer*, *smartphone*, and *television* could be associated with a latent topic *technology*. In this way, a text can be characterised by a small list of topics and their associated weights. These representations also require sufficiently large documents and corpora to produce meaningful topic distributions, making them less suitable for short texts [9].

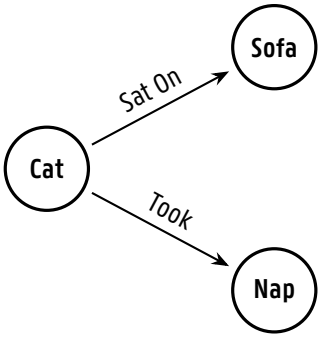
Textual data can also be encoded in structured formats designed to preserve editorial and structural detail. In the field of Digital Humanities (DH), the Text Encoding Initiative (TEI) has established itself as the standard for encoding textual data in a structured XML-based format [42]. TEI is used, among other purposes, to annotate texts with structural, editorial, and contextual information and to associate metadata with specific text elements [43]. While TEI is widely adopted for scholarly editing and archival purposes, it is primarily a storage format rather than a representation optimised for computational analysis. Querying a TEI document is possible through XML query languages such as XPath [44] or XQuery [45], but is limited and ill-suited to complex retrieval tasks.

TEI encodes textual structure within individual documents, but it does not naturally represent relationships across documents or between their discourse units. Graph-based models offer a different paradigm, representing textual elements and their relationships as nodes and edges, and providing a natural framework for capturing both the structural and relational properties of a corpus [46]. A common variant is the co-occurrence network, in which word nodes are connected whenever the corresponding words appear in close proximity within the text. Co-occurrence networks have been applied to authorship attribution, classification, and sentiment analysis [47–50], but they do not preserve the structure of the text. Hierarchical graph-based decompositions instead retain the structure of the text: wiki revisions, for instance, are modelled as graphs composed of tokens, sentences, and paragraphs for authorship tracking [51]. This approach shares the principle of hierarchical decomposition with the model proposed in this chapter, but it constructs independent graphs for each document rather than merging shared elements across documents into a single graph. Graphs representing texts have also been constructed using semantic information from NLP tools, such as graphs based on Part-of-Speech (POS) tags [52]

or Knowledge Graphs (KGs) based on NLP-derived information [46] that model entities as nodes and their relationships as edges. Figure 2.1b illustrates such a KG for the sentence "*The cat sat on the sofa and took a nap*", where entities are represented as nodes and edges represent semantic relationships such as actions or properties. While such enrichments improve expressiveness, they depend on language-specific resources that are often unavailable for low-resource languages.



(a) Example of a simple word embedding.



(b) Example of a simple NLP-based KG representation.

Figure 2.1: Examples of text representations: (a) an illustrative vector embedding for the words *cat*, *dog*, and *bird*, indicating that the words *cat* and *dog* are semantically closer to each other than to the word *bird*, and (b) a KG representation of the sentence "*The cat sat on the sofa and took a nap*" based on NLP-derived information, where entities are represented as nodes and edges represent semantic relationships such as actions or properties.

The graph-based data model proposed in this chapter differs from the approaches discussed above in several respects. Texts are represented through a hierarchical decomposition into discourse units, where identical textual elements across different texts are merged into the same node, making interconnections an inherent structural property of the graph. By operating on the orthographic surface form of the text, the model does not depend on language-specific NLP tools or external semantic resources, making it applicable to low-resource languages. Contextual metadata are integrated within the same graph structure, enabling queries and analyses that combine textual and contextual criteria to be performed efficiently within a single system rather than across disjoint data stores. Unlike frequency-based and embedding-based representations, the proposed model is not a numerical feature vector designed as input for ML algorithms, but a structural data model designed for exploration, querying, and the identification of interconnections. The model can, however, serve as the foundation from which features, such as similarity scores at multiple levels of granularity, are derived for downstream analytical tasks including ranking and clustering, as developed in subsequent chapters of this thesis.

2.3 Preliminaries

In this section, we introduce the foundational concepts on which the graph-based data model proposed in this chapter is built. We first briefly discuss the notion of text and its decomposition into smaller units, then introduce the basic mathematical concept of a graph, and finally present graph databases and the LPG model that underpins them.

2.3.1 Text and Textual Decomposition

At the most fundamental level, a text is a finite, ordered sequence of characters, of no fixed length and with no imposed structure, drawn from an alphabet (or character set) $\mathcal{A}$. A text t can thus be represented as a character sequence (or string)

$$t = c_1 c_2 \dots c_n, \tag{2.1}$$

where $c_i \in \mathcal{A}$ for $1 \leq i \leq n$ and n denotes the length of the text. The set of all finite character sequences over $\mathcal{A}$, including the empty sequence, is denoted by $\mathcal{A}^*$, so that $t \in \mathcal{A}^*$ for every text t. While this representation captures the full content of a text, it treats the text as a flat, undifferentiated sequence and does not reflect the structural organisation inherent in natural language.

In practice, texts are not typically processed as raw character sequences but are first decomposed into meaningful units through operations such as *tokenisation* and *segmentation* [53, 54]. Tokenisation is the process of splitting a character sequence into individual tokens, typically words or sub-word units, based on delimiters such as whitespace or punctuation. Segmentation, on the other hand, refers to the identification of larger structural boundaries within the text, such as sentences, paragraphs, verses, or chapters. Although these operations differ in the granularity of the units they produce, both can be formalised as instances of a split function that decomposes a text t into an ordered list of non-overlapping, non-empty substrings

$$\text{split}(t) = [t_0, t_1, \dots, t_{m-1}], \tag{2.2}$$

where each t_i, with $0 \leq i \leq m-1$, is a contiguous, non-empty subsequence of characters from t, referred to as a *substring* and denoted $t_i \sqsubset t$, and m corresponds to the number of resulting substrings. Together with the delimiters that define the split boundaries, these substrings must fully account for the content of t, so that the original text can be reconstructed as $t = t_0 \oplus d_0 \oplus t_1 \oplus \cdots \oplus d_{m-2} \oplus t_{m-1}$. Here, $\oplus$ denotes string concatenation and each d_i, with $0 \leq i \leq m-2$, denotes the (possibly empty) delimiter separating t_i from t_{i+1} in t. Once produced, these substrings serve as the basic units on which further processing and analysis tasks, such as similarity computation, embedding generation, pattern discovery, and classification, can be performed.

2.3.2 Graphs and Graph Databases

A graph G is a mathematical structure consisting of a pair (N, E), where N is a finite set of nodes (or vertices) and $E \subseteq N \times N$ is a set of edges (or relationships) connecting pairs of nodes [55]. When the pairs in E are ordered, the graph is said to be directed, so that each edge has a source node and a target node. When the pairs are unordered, the graph is undirected. Graphs provide a natural way to represent entities and the relationships between them, making them well-suited for modelling data in which the relationships between entities are as important as the entities themselves, such as social networks, transportation systems, and, as we propose in this chapter, interconnected textual corpora.

Graph databases are Database Management Systems (DBMSs) that use graph structures to manage, store, and query data [56]. Unlike relational databases, where data are modelled by means of tables with a fixed

schema, graph databases represent data as nodes connected by edges, mirroring the mathematical graph structure introduced above. Two properties make graph databases particularly effective for the representation and analysis of interconnected data. First, they have no fixed schema: each node and edge can carry its own set of properties, allowing entities with varying or evolving data structures to coexist in the same database without requiring schema modifications [16]. Second, graph databases are optimised for traversing relationships between entities, making them efficient for queries that involve following chains of connections, a task that is expensive in relational databases due to the need for multiple join operations [57].

Several graph data models have been proposed for use in graph database systems. Two of the most prominent are the Resource Description Framework (RDF) model and the Labelled Property Graph (LPG) model.

RDF represents all information as a set of triples, each consisting of a subject, a predicate, and an object [58, 59]. The subject and object of each triple correspond to nodes, while the predicate corresponds to a typed, directed edge between them. As the foundation of the Linked Data (LD) paradigm, it is widely used for knowledge representation and interoperability, particularly in combination with domain-specific ontologies [60]. Data stored in RDF can be queried using the SPARQL query language [61, 62]. While RDF excels at expressing typed relationships between entities and at enabling data integration across heterogeneous sources, its triple-based structure can be verbose, does not natively support properties on edges, and is less suited for directly attaching contextual information to nodes and edges.

The LPG model provides an alternative in which both nodes and edges can carry labels as well as key-value properties directly [63]. This model has been adopted by several widely used graph database systems, including Neo4j, Memgraph, Amazon Neptune, and JanusGraph. Such systems are typically queried through declarative pattern-matching languages, of which Cypher [64] is the most widely adopted. The ability to store properties on both nodes and edges, combined with the use of labels to distinguish different types of entities and relationships, makes the LPG model well-suited for representing structured and semi-structured data in which entities may carry different sets of attributes. A formal definition of the LPG model is provided in Definition 2.1.

Definition 2.1 (Labelled Property Graph [63])**.** Let P be a set of property names, V a set of atomic values, and L a set of labels. An LPG G is a tuple $(N, E, \rho, \lambda_N, \lambda_E, \sigma)$, where

- N is a finite set of nodes;
- E is a finite set of edges;
- $\rho : E \to N \times N$ is a total function that associates each edge in E with an ordered pair of nodes, indicating its direction;
- $\lambda_N : N \to \mathcal{P}(L)$ is a total function that assigns a subset of labels to each node, where $\mathcal{P}$ denotes the powerset operator;
- $\lambda_E : E \to L$ is a total function that assigns a label to each edge;
- $\sigma : (N \cup E) \times P \to \mathcal{P}(V)$ is a partial function that assigns a set of property values to nodes and edges.

In what follows, we adopt the shorthand notations

$$N_\ell = \{n \mid n \in N, \ell \in \lambda_N(n)\} \tag{2.3}$$

and

$$E_\ell = \{e \mid e \in E, \lambda_E(e) = \ell\} \tag{2.4}$$

to denote the set of nodes and the set of edges with a specific label ℓ, respectively. For example, in a graph database storing textual data, $N_{\texttt{Word}}$ would denote all nodes labelled by `Word`, representing individual words, while $E_{\texttt{CONTAINS}}$ would denote all edges labelled by `CONTAINS`, representing containment relationships between nodes.

In this chapter, the LPG model serves as the formal foundation on which the graph-based text representation is built. The choice of this model over RDF is motivated by its native support for properties on both nodes and edges, its ability to distinguish different textual levels through node labels and to express their containment relationships through directed, labelled edges, and its alignment with the query and visualisation capabilities of modern graph database systems such as Neo4j [17]. Building on this foundation, the following section defines the proposed graph-based representation for short, interconnected texts.

2.4 Representing a Textual Corpus as a Graph

The foundational step in the design of our graph-based data model is the representation of the textual corpus as a graph. As established in Section 2.3, textual documents can be decomposed into smaller units through split functions, and labelled property graphs provide a flexible data model for representing entities that may carry different sets of attributes and numerous relationships. In this section, we bring these two concepts together by proposing a graph model in which text documents are partitioned into hierarchically organised discourse units, each represented as a node in the graph. A key property of this model is that identical textual elements occurring in different text documents are represented by the same node at every level of the hierarchy except the top level, where each text document retains its own node to remain uniquely identifiable. As a result, interconnections between text documents become an inherent structural feature of the graph rather than something that must be discovered through subsequent analysis.

We first describe how text documents can be structured as a hierarchy of textual elements in Section 2.4.1. Section 2.4.2 then presents the formal graph model and its properties, and Section 2.4.3 explains the process of constructing the graph from a given corpus.

2.4.1 Hierarchical Decomposition of Textual Documents

Textual documents typically exhibit an inherent hierarchical structure. A thesis, for instance, is composed of chapters, which consist of sections, which in turn consist of paragraphs, sentences, and ultimately individual tokens. Similarly, a poem can be decomposed into verses, each consisting of individual words. These components, into which a text can be decomposed, each correspond to a specific unit of discourse, such as words, tokens,

sentences, half-verses, verses, paragraphs, sections, chapters, or full documents. As introduced in Section 2.3.1, such a decomposition can be achieved through the application of split functions, as defined by Equation (2.2), where each application targets a different level of granularity. We denote the split function associated with each level of the decomposition as split_i, where i identifies the targeted level. For instance, a first split may divide a document into chapters, a second may divide those chapters into sections, and so on down to individual tokens.

We formalise the hierarchical decomposition of a text, or of all the texts in a corpus, as an ordered list of discourse units

$$H = [h_0, h_1, \dots, h_n], \tag{2.5}$$

where each h_i, $0 \leq i \leq n$, denotes a unit of discourse. Since these units correspond to different hierarchical levels of the text, they serve as natural labels for the nodes in the graph representation, as described in Section 2.4.2. For this reason, we also refer to them as labels for textual elements throughout this work. The size of the list, $|H|$, equals the number of hierarchical levels in the decomposition. Within this ordered list, h_{i+1} denotes the label of textual components that are hierarchically above those with label h_i, for $0 \leq i < n$. The top-level elements of the hierarchy have label $h_{|H|-1} = h_n$, and the lowest-level elements have label h_0. Hereafter, we refer to text elements with label h_i as higher-level relative to text elements with label h_j when $i > j$, and as lower-level when $i < j$. Returning to the examples above, a simplified decomposition of this thesis is $H = [\texttt{Token}, \texttt{Sentence}, \texttt{Paragraph}, \texttt{Section}, \texttt{Chapter}, \texttt{Document}]$, and that of a poem is $H = [\texttt{Token}, \texttt{Verse}, \texttt{Document}]$.

There is no single correct hierarchical decomposition for every type of text or corpus, as levels can be added or removed at any position in the hierarchy. The most suitable decomposition depends on the nature of the corpus and the analytical tasks at hand, and can be determined through expert knowledge, empirical analysis, or visual inspection of the texts. For interconnected texts, a suitable decomposition is one that targets the discourse units at which the most connections between texts occur.

2.4.2 Formal Description of the Graph-Based Text Model

We propose a graph model to represent a set of text documents, in which the content of each text is partitioned into smaller textual components. The granularity of these components is determined by a given hierarchical decomposition of the texts, as described in Section 2.4.1. All text documents within a given corpus must share the same hierarchical decomposition, as the model relies on a uniform set of labels to enable the merging of identical textual elements across different texts.

Each text element is represented by a node with the corresponding label, connected by directed edges to the higher-level elements in which it occurs and to the lower-level elements of which it is composed, where applicable. For example, in a text with decomposition $H = [\texttt{Token}, \texttt{Verse}, \texttt{Document}]$, a `Verse` node is connected to the `Document` nodes in which this verse occurs and to the `Token` nodes that make up this verse. Each edge additionally carries a `rank` property indicating the position of the lower-level element within its containing higher-level element, such as the position of a token within a verse or a verse within a document. These ranks are necessary to reconstruct the original text from its graph representation, as they preserve the

order of the textual components within their containing elements. Only the nodes that represent the lowest level of the hierarchy store the actual textual content. The content of higher-level text elements can be reconstructed from their subgraph by traversing down to the lowest-level components.

Textual components with the exact same content and at the same hierarchical level are represented by the same node in the graph. As a result, when two or more texts share a common sub-element, this shared content is captured directly by the structure of the graph, and a basic notion of similarity between texts and their sub-elements can be deduced from the graph without requiring any additional computation. Top-level elements, on the other hand, are always represented by distinct nodes, so that each text in the graph remains uniquely identifiable. A formal definition of this graph model is provided in Definition 2.2.

Definition 2.2 (Graph-Based Text Model). Let T be a set of text documents, and let H be a hierarchical decomposition of the texts in T, as described in Section 2.4.1. The graph model for representing texts is an LPG $G(N, E, \rho, \lambda_N, \lambda_E, \sigma)$ defined by the following:

- The label set L, which is ordered to reflect the hierarchy, consists of the labels of the hierarchical decomposition of the texts H, such that $L = \left[h_0, h_1, \dots, h_{|H|-1}\right]$.
- The set N consists of all nodes representing a textual element of a text in T and is the union of the subsets representing textual components with a specific label, i.e., $N = \bigcup_{h \in L} N_h$. The subsets N_h are pairwise disjoint.
- The set of edges $E = E_{\texttt{CONTAINS}}$ represents the containment relationships between textual elements.
- The containment relationships are mapped to the corresponding nodes by the ρ function: $\rho(e) \in N_{h_{i+1}} \times N_{h_i}$, with $e \in E_{\texttt{CONTAINS}}$ and $h_{i+1}, h_i \in L$. This mapping indicates which nodes with a lower-level label h_i are contained by a node with label h_{i+1}.
- The `rank` property, indicating the position of a lower-level text element within its containing higher-level element, is assigned by $\sigma(e, \texttt{rank}) \in \mathbb{N}$, $e \in E_{\texttt{CONTAINS}}$. This property is required to reconstruct any text from its graph representation.
- Top-level nodes $n \in N_{h_{|H|-1}}$, which represent entire textual documents, are assigned a unique `id` by $\sigma(n, \texttt{id}) \in \mathbb{N}$.
- The lowest-level nodes $n \in N_{h_0}$ store the actual textual content in a `text` property by $\sigma(n, \texttt{text}) \in T_{h_0}$, where T_{h_0} denotes the set of all unique text elements from texts in T at hierarchical level h_0.

Furthermore, textual elements that contain the exact same content and share the same hierarchical level are represented by a single node, which may consequently have multiple containment relationships to distinct higher-level nodes. Top-level elements are the sole exception to this rule and are always represented by distinct nodes, each assigned a unique identifier, ensuring that every text in the graph is uniquely identifiable.

In summary, the graph-based text model captures the hierarchical structure of a corpus through labelled nodes and directed containment edges, while the merging of identical sub-elements ensures that shared content across texts is reflected directly in the topology of the graph. The construction of this graph from a given set of texts is described in the following subsection.

2.4.3 Constructing the Graph Representation from a Textual Corpus

Given a set of textual documents T and a hierarchical decomposition H, the graph-based text representation defined in Definition 2.2 can be constructed using Algorithm 2.1. The algorithm starts from an empty LPG and adds each text document individually to the graph.

Before a text is inserted into the graph, an optional preprocessing step allows the text to be altered, normalised, or simplified. Common preprocessing operations include case standardisation, the removal of punctuation, and the removal of redundant spacing. Since exact text matches are represented by the same node, the choice of preprocessing directly affects which textual elements are considered identical and thus influences the topology of the resulting graph. After preprocessing, a node is created for each text with the label of the top-level element ($h_{|H|-1}$). This node serves as the root node for the corresponding text and is assigned a unique identifier.

Algorithm 2.1 Main construction algorithm for the graph-based text representation.

Input: A set of textual documents T and a hierarchical list of units $H = [h_0, h_1, \dots, h_n]$
Output: An LPG $G(N, E, \rho, \lambda_N, \lambda_E, \sigma)$ representing the documents in T

1: Create an empty LPG: $G(N, E, \rho, \lambda_N, \lambda_E, \sigma)$
2: **for all** $t \in T$ **do**
3: $t \leftarrow \text{Preprocess}(t)$
4: $i \leftarrow |H| - 1$
5: Create node n
6: $\sigma(n, \texttt{id}) \leftarrow \text{UniqueID}()$
7: $\lambda_N(n) \leftarrow \{h_i\}$
8: $\text{ComputeChildNodes}(t, i-1, n)$
9: $N \leftarrow N \cup \{n\}$
10: **end for**
11: **return** $G(N, E, \rho, \lambda_N, \lambda_E, \sigma)$

From each root node, the lower levels of the text are constructed recursively by the `ComputeChildNodes` procedure, given in Algorithm 2.2. At each level of the hierarchical decomposition, the current text element is split into smaller components using split_i, an instance of the split function introduced in Equation (2.2). The specific splitting strategy depends on the current unit of discourse h_i and must be specified for each level of the decomposition. When the target discourse unit is words, for instance, the text can be split on whitespace, whereas for tokens or sentences, tokenisation [53] or sentence segmentation [54] methods can be used.

For each resulting component $s_j \in S$, the algorithm checks whether a node with the same label and textual content already exists in the graph. At the lowest level, this check compares the `text` property directly, whereas at higher levels it compares the content reconstructed from the subgraph of each candidate node, as described in Section 2.4.2. If such a node exists, it is reused, and no further recursive steps are required for that component. Otherwise, a new node with the corresponding label is created. If the recursion has reached the lowest level of the hierarchy (i.e., when $i = 0$), the node is associated with the actual textual content and the recursion terminates. In all other cases, the `ComputeChildNodes` procedure is called recursively to construct the

Algorithm 2.2 Recursive procedure for constructing lower-level nodes in the graph representation.

```
1:  procedure ComputeChildNodes(t, i, p)
2:      S ← split_i(t)                          ▷ Text splitting based on current discourse unit
3:      for j ← 0 to |S| − 1 do
4:          if a node already exists on level i that represents s_j then
5:              Define that existing node as n
6:          else
7:              Create node n
8:              λ_N(n) ← {h_i}
9:              if i > 0 then
10:                 ComputeChildNodes(s_j, i − 1, n)
11:             else
12:                 σ(n, text) ← s_j
13:             end if
14:             N ← N ∪ {n}
15:         end if
16:         Create edge e
17:         ρ(e) ← (p, n)
18:         σ(e, rank) ← j
19:         λ_E(e) ← CONTAINS
20:         E ← E ∪ {e}
21:     end for
22: end procedure
```

lower-level text elements at level $i - 1$. Regardless of whether the node already existed or was newly created, an edge labelled CONTAINS is created from the parent node to the child node, with the rank property set to j to record the position of the child within its containing higher-level element. This ensures that containment relationships are recorded even when the content is already present from a previous text document. Any newly created node is then added to the graph.

Once the recursive construction of all child nodes is complete, the root node is added to the graph. This process repeats for every text document in T, after which the fully constructed graph is returned. To illustrate the graph model and its construction, Example 2.1 provides an example based on a small corpus of two texts.

Example 2.1. Consider a set of two slightly different text documents

$$T = \{\text{“What is this? A } \textbf{first} \text{ text to } \textbf{analyse}\text{.”, “what is this? A } \textbf{second} \text{ text to } \textbf{analyze}\text{.”}\}$$

and a list of textual units $H = [\texttt{Word}, \texttt{Sentence}, \texttt{Text}]$, where elements with the label Text represent a text document as a whole. Executing Algorithm 2.1 on this example results in the graph-based text representation displayed in Figure 2.2. The text documents undergo preprocessing involving case standardisation and the removal of punctuation. In this example, the text documents are segmented into sentences and, subsequently, the sentences are split into words based on whitespace.

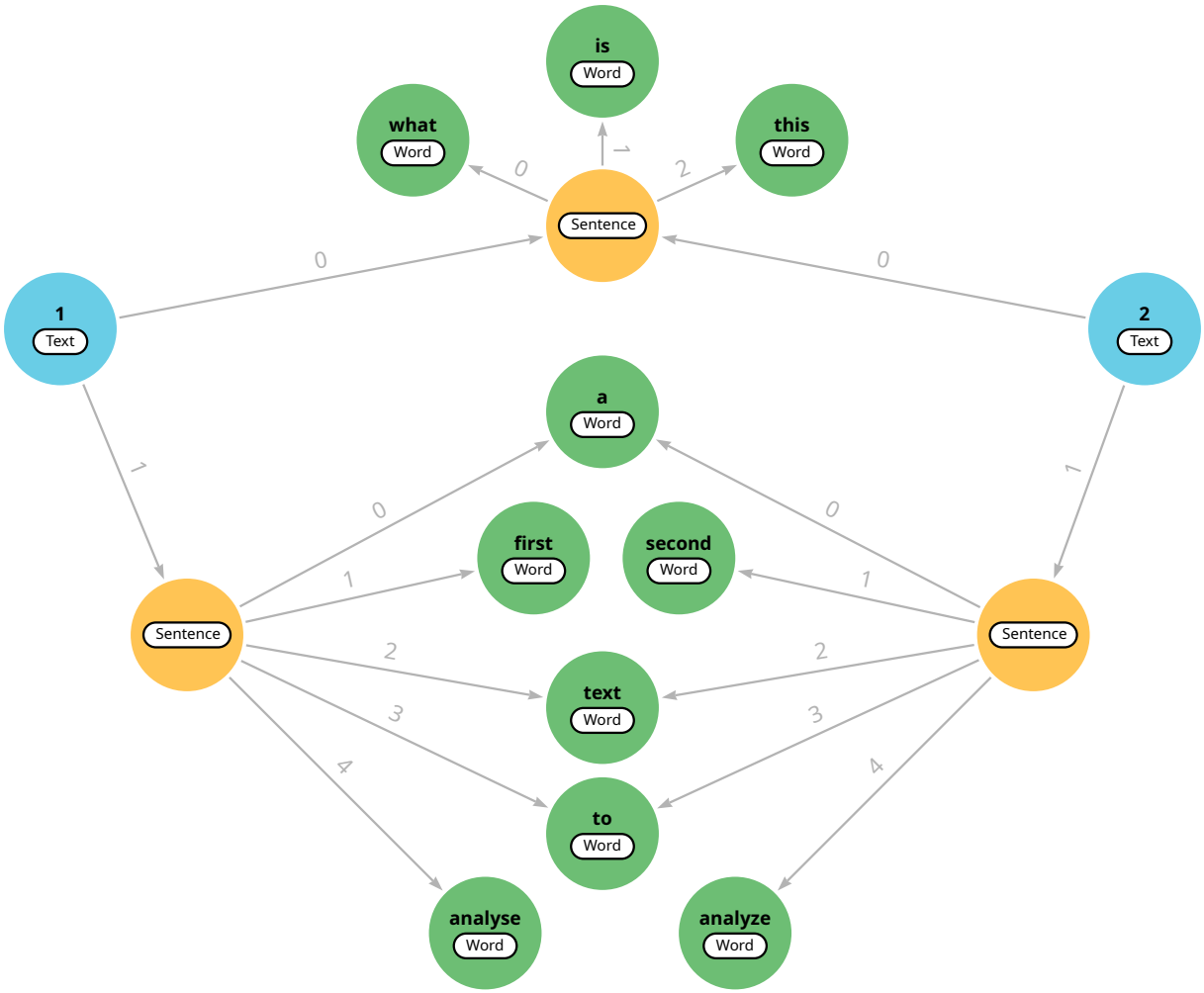


Figure 2.2: A graph-based text representation for the texts in Example 2.1. `Word` nodes are indicated in green and display their `text` property, whereas `Sentence` nodes are indicated in yellow, and `Text` nodes, including their numeric `id` property, are indicated in blue. Values of the `rank` property are displayed on the edges.

> The first sentence of both text documents (left) is represented by a single `Sentence` node, signifying that both preprocessed sentences are orthographically identical. The second sentence, on the other hand, differs by two words, resulting in a distinct `Sentence` node for each text document. When text documents share orthographically identical elements, the structure of the graph provides a direct indication of the similarity between them.

The graph is not limited to a single construction pass. Additional text documents can be added to the graph after its initial construction by re-executing Algorithm 2.1 with the set T consisting of one or more new text documents. Rather than creating an empty graph, the algorithm operates on the already constructed graph G, and the node-merging mechanism ensures that shared content between new and existing text documents is captured automatically.

The graph-based text model, as constructed by the algorithms above, represents the textual content of a corpus but does not yet account for the contextual metadata associated with each text document. The following section describes how such metadata can be incorporated into the same graph structure.

2.5 Modelling Metadata and Additional Textual Properties

The graph-based text model of the previous section captures only the hierarchical structure and textual content of a corpus, whereas texts are in practice often accompanied by a wide range of supplementary information such as contextual metadata and textual properties. In this section, we describe how both types of information, when available, can be incorporated into the graph.

The incorporation of contextual metadata and textual properties is optional and depends on the availability and relevance of such information for the intended analysis; the graph-based text model defined in Section 2.4 can be constructed and used without such supplementary information. When such data are available and useful, integrating them within the same graph structure allows textual content, structural organisation, and contextual or linguistic attributes to be queried and explored jointly, without the need to consult multiple data sources. The integration of these additional data, however, introduces new challenges. The graph structure becomes more complex, and additional data have to be ingested, requiring a synchronisation mechanism to ensure that updates to the original sources are reflected in the graph.

2.5.1 Contextual Metadata

Contextual metadata describe the circumstances under which a text or one of its elements was produced or the attributes associated with it, rather than its linguistic content. Examples include the geographical region where a text was written, the author who produced it, the subject it discusses, or the source from which it originated. Such metadata are often heterogeneous in structure and can be incomplete, with some attributes available for certain texts but absent for others.

We model contextual metadata as separate nodes in the graph, connected to the corresponding text nodes by labelled, directed edges. Each metadata type is represented by its own node label, and each distinct metadata value is represented by a single node. The edge label encodes the specific relationship between a text and a metadata node, so that the same entity, such as a person, can be connected to different texts in different roles, for instance as the author of one text and as the subject of another. When two or more texts share the same metadata value, they are connected to the same metadata node, mirroring the node-merging principle from Section 2.4.2 and making shared metadata a structural property of the graph. Since graph databases do not impose a fixed schema, the set of metadata fields is not necessarily uniform across all texts: a text for which a particular field is unavailable simply has no edge connecting it to a node of that type.

Formally, the incorporation of metadata extends the LPG G from Definition 2.2. Let $M = \{m_1, m_2, \dots, m_k\}$ be the set of labels for k metadata types, such as `Location`, `Author`, or `Date`. The label list L of Definition 2.2 is extended with the metadata labels, resulting in $L = \left[h_0, h_1, \dots, h_{|H|-1}, m_1, m_2, \dots, m_k\right]$, where only the first $|H|$ entries, the labels for the hierarchical decomposition of the texts, reflect the hierarchical ordering. The node set N of G is extended with new subsets N_m, where $m \in M$ and each N_m contains distinct metadata nodes n_m with label $\lambda_N(n_m) = \{m\}$. Metadata values are modelled as properties of the corresponding nodes using the σ function, where the specific properties depend on the metadata type and the available information. Text nodes $n_t \in N_h$, with $h \in H$, are then connected to corresponding metadata

nodes n_m by a new edge e, added to E, such that $\rho(e) = (n_t, n_m)$ and $\lambda_E(e)$ specifies the type of the relationship, such as `HAS_SUBJECT` or `WRITTEN_BY`. Metadata are most often associated with the nodes representing entire texts, but the same mechanism applies at any level of the hierarchy. The direction and naming of these edges are a modelling choice: an implementation may equally orient them from the metadata node towards the text node, as will be illustrated in the later chapters of this thesis. When required, properties can also be assigned to these edges using the σ function, for example to record attributes that pertain to the relationship itself rather than to either of the connected nodes.

2.5.2 Textual Properties

In addition to contextual metadata, the graph can be enriched with textual properties stored directly on the textual nodes using the σ function of the LPG. Such properties include POS tags, lemmatised forms, or static word embeddings, stored as key-value pairs on the relevant nodes. A word node $n \in N_{h_0}$, for instance, could carry a property $\sigma(n, \texttt{pos})$ storing its POS tag or a property $\sigma(n, \texttt{lemma})$ storing its lemmatised form. These properties enable pattern-based queries that combine textual, structural, and linguistic criteria, such as retrieving all sentences that begin with a word tagged as a noun followed by a specific verb identified by its lemma. The availability of such properties depends on the NLP resources that exist for the language of the corpus, and, as with contextual metadata, they do not be present on every node.

The enrichment of the graph with metadata and textual properties completes the data model proposed in this chapter. The following section demonstrates the querying and exploration capabilities of the resulting graph through illustrative examples.

2.6 Practical Applicability

The graph-based data model proposed in this chapter is primarily designed as a structural foundation for the methods developed in the subsequent chapters. The graph representation itself, however, already offers useful capabilities for querying, exploration, and visualisation of textual corpora. All examples presented in this section operate only on the structural properties of the graph, namely the nodes, edges, and labels that result from the hierarchical decomposition and the node-merging mechanism, without relying on any computed similarity scores. Only the final example of Section 2.6.4 additionally assumes textual properties on the word nodes. For the examples in this section, we use a Neo4j graph database system [17], which provides a declarative query language, Cypher [64], and an interactive visualisation interface. We first describe the example dataset and its preprocessing, then demonstrate the capabilities of the graph through importance measures, interconnection analysis, and pattern matching.

2.6.1 Example Dataset and Database Setup

To demonstrate the capabilities of the proposed graph model, we apply it to the Database of Byzantine Book Epigrams (DBBE) corpus of Byzantine book epigrams [18, 19]. As introduced in Section 1.3, the DBBE houses a collection of short Greek poems found in the margins of medieval manuscripts. These texts are characterised by

their brevity, orthographic inconsistencies, and a complex tradition of textual reuse and variation, making them a suitable test case for a graph-based representation designed for handling short, interconnected texts.

Using the hierarchical decomposition described in Section 2.4.1, we decompose the epigrams into three levels of discourse units, yielding the decomposition $H = [\texttt{Word}, \texttt{Verse}, \texttt{Epigram}]$. Since the DBBE stores epigrams as lists of verses, the segmentation of epigrams into verses is extracted directly from the database, while words are obtained by splitting each verse on whitespace. Before importing the texts into the graph, we apply a preprocessing step to reduce orthographic noise and aid the merging of identical textual elements, as described in Section 2.4.3. The preprocessing consists of:

(i.) replacing capital letters with their lowercase counterparts,
(ii.) removing characters that are not part of the Greek alphabet,
(iii.) omitting accents and other diacritical marks,
(iv.) replacing additional spacing between words with a single space, and
(v.) systematically replacing the vowels η, and υ, the last only where it does not form part of the diphthongs αυ, ευ, and ου, as well as the diphthongs ει and οι, with the character ι to address the phenomenon of itacism, as introduced in Section 1.3.

As an illustration, Table 2.1 shows the textual content of the epigram with identifier 18306 [19] before and after the application of these preprocessing steps. The result is a text with reduced orthographic variation, which increases the likelihood of merging identical textual elements across different epigrams during graph construction.

Table 2.1: Textual content of epigram 18306 [19] from type 2150 before and after preprocessing.

#	Before preprocessing	After preprocessing
1.	Ὥσπερ ξένη χαῖροντες ἰδεῖν π(ατ)ρίδα,	ωσπερ ξενι χαιροντες ιδιν πατριδα
2.	καὶ ἡ θαλατεύοντες εὑρὴν λοιμένα	και ι θαλατευοντες ευριν λιμενα
3.	οὕτως καὶ ἡ γράφοντες βιβλίου τέλος.	ουτως και ι γραφοντες βιβλιου τελος

The examples presented in this section are based on epigrams belonging to three groups (epigram types) in the DBBE, with identifiers 2148, 2150, and 4245 [18, 19]. These three epigram types were selected because they form a compact yet representative subset of the corpus, exhibiting extensive textual reuse across epigrams of varying length, from nearly identical two-verse epigrams to more loosely related variants of up to six verses. The epigram in Table 2.1 belongs to type 2150, which groups three-verse epigrams that share similar first, second, and last verses but exhibit orthographic variation or minor textual differences. The remaining two types also contain epigrams with similar first and last verses, but differ in length. Type 2148 groups two-verse epigrams, such as the examples in Tables 1.1 and 1.2, whose verses correspond to the opening and closing verses of the longer types. Type 4245 groups epigrams of three to six verses, again with similar first and last verses. Their intermediate verses follow the structural pattern of the second verse of type 2150, each beginning with variants of και οι (or και ι after preprocessing) but differing in the remaining words. Table 2.2 shows an example from type 4245, namely epigram 32232 after preprocessing, in which the recurring και ι pattern across the

intermediate verses is highlighted. Together, these three types comprise 154 epigrams, containing a total of 410 verses and 2204 words.

Table 2.2: Textual content of epigram 32232 [19] from type 4245 after preprocessing. The recurring και ι pattern in the intermediate verses is highlighted in bold.

#	Epigram 32232 after preprocessing
1.	ωσπερ ξενι χαιρουσιν ιδιν πατριδα
2.	**και ι** θαλατεβονταις ιδιν λιμωνα
3.	**και ι** πραγματευταις ιδιν το κιρδως
4.	**και ι** στρατευομενι ιδιν το νικος
5.	ουτος και ι γραφονταις ιδιν βιβλιου τελος

The two-verse epigrams in type 2148, as noted in Section 1.3, can generally be translated as "Just like travellers rejoice in seeing their homeland, so do writers in reaching the end of the book". The three-verse epigram in Table 2.1 can be translated as "Just like travellers rejoice in seeing their homeland, and the sailors rejoice to find a harbour, so do writers in reaching the end of the book". In epigrams with three or more verses from types 2150 and 4245, the intermediate verses are structured similarly to "and the sailors rejoice to find a harbour", but can differ in the specific words used. Examples include "and the merchants rejoice to find profit" and "and the soldiers rejoice to find victory".

After preprocessing and graph construction, the resulting graph contains 154 epigram nodes, 185 unique verse nodes, and 190 unique word nodes. The difference between the total number of verses (410) and unique verse nodes (185) illustrates the extent of textual reuse within this subset of the corpus. It underlines both the role of preprocessing in enabling the merging of identical textual elements and the value of studying the interconnections between epigrams.

2.6.2 Identifying Important Textual Elements

A first application demonstrating the added value of the graph model is the identification of textual elements that occur frequently across the corpus. In the proposed representation, the *in-degree* of a node with respect to `CONTAINS` edges, that is, the number of containment edges directed towards it, directly reflects how many higher-level text elements contain that node. A `Word` node with a high in-degree, for instance, appears in many different verses, while a `Verse` node with a high in-degree occurs in many different epigrams. Identifying these highly connected nodes reveals the most frequently shared building blocks of the corpus, or a subset thereof, which can be of interest for linguistic or philological analysis. Conversely, textual nodes with a low in-degree are less commonly shared and may be more specific to individual texts, which can also be of interest for certain analyses.

Listing 2.3 shows a Cypher query that finds the `Word` nodes with the highest in-degree. First, the query matches all `Word` nodes that are connected to a `Verse` node through a `CONTAINS` relationship. Then, for each unique `Word` node, the number of such relationships is counted and returned alongside the textual content of the node.

Finally, the results are ordered from highest to lowest in-degree. A similar query can be constructed to find the most frequently occurring verses in the corpus.

```
MATCH (w:Word)<-[c:CONTAINS]-(v:Verse)
RETURN w.text AS word, count(c) AS in_degree
ORDER BY in_degree DESC
```

Listing 2.3: Cypher query to retrieve the most prevalent words in the dataset.

Table 2.3 lists the results of the query in Listing 2.3 alongside the results of a similar query for the most frequently occurring verses. Given the composition of the dataset described above, the results are as expected. At the word level, the most prevalent words are the conjunction και ("and") and ι, the preprocessed form of the article οἱ ("the") after itacism replacement. Both appear in the recurring formula of the last verse as well as in the middle verses of the longer epigrams. Other frequently occurring words include ιδιν ("to see"), βιβλιου ("book"), and τελος ("end"), all of which belong to the recurring formula of the first or last verse and therefore appear in nearly all epigrams of the selected dataset. At the verse level, the most frequently shared verses are the first and last verses, which occur in almost every epigram of the dataset, albeit in varying orthographic forms. However, the verse-level results are more affected by orthographic variation, such as differences in the spelling of ουτος and ουτως ("thus") or the different forms χαιρουσιν and χαιροντες ("they rejoice"). Since these differences are not fully resolved by the preprocessing steps, the in-degree values at the verse level are less concentrated than at the word level, with shared content spread across a larger number of distinct verse nodes.

Table 2.3: Overview of the most frequently occurring words and verses in the dataset, ranked by their in-degree in the graph.

#	Most frequent words		Most frequent verses	
	word	(in_degree)	verse	(in_degree)
1.	και	(113)	ωσπερ ξενι χαιρουσιν ιδιν πατριδα	(45)
2.	ι	(105)	ουτω και ι γραφοντες βιβλιου τελος	(19)
3.	ιδιν	(81)	ωσπερ ξενι χαιροντες ιδιν πατριδα	(18)
4.	βιβλιου	(62)	ουτος και ι γραφοντες βιβλιου τελος	(17)
5.	τελος	(61)	ουτως και ι γραφοντες βιβλιου τελος	(15)
⋮	⋮		⋮	

The preceding analysis identifies the most frequently shared words and verses across the entire dataset. Since Cypher is an expressive query language, more fine-grained analyses are possible by filtering on specific subsets, metadata, textual properties, or combinations thereof. Beyond simple in-degree counts, more sophisticated importance measures such as PageRank [65] can be applied to the graph to identify textual elements whose importance derives not only from their frequency but also from the prominence of the elements that contain them. Such analyses, along with many other graph algorithms, can be performed using the built-in Graph Data Science (GDS) library [66] of Neo4j. Furthermore, the identification of highly connected textual elements can serve as a starting point for more advanced analyses, such as the extraction of formulaic expressions from

frequently shared verses. These results can also guide domain experts towards potential new groups of related texts based on shared building blocks or lead to the identification of new interconnections between texts that were previously considered unrelated.

2.6.3 Interconnection Identification and Visualisation

A second application, beyond identifying individual elements of interest, is the visual exploration of interconnections between texts that the graph model enables. Since texts that share identical words or verses are connected through the same nodes, the topology of the graph can directly reveal links or even groups of related texts without requiring any similarity computation.

As a first example, we query the database to retrieve all `Verse` nodes and their contained `Word` nodes that correspond to the first verse of the epigrams with ids 17053, 17574, 17604, 18306, 19735, and 20553 [19]. Figure 2.3 displays the resulting subgraph using Neo4j Bloom [67], the interactive visualisation tool provided by Neo4j. A first observation is that only five `Verse` nodes are returned, even though six epigrams were queried. This is a direct consequence of the node-merging mechanism: two of the epigrams contain an orthographically identical first verse after preprocessing, and are therefore connected to the same `Verse` node in the graph. Furthermore, the verses can be visually grouped based on their shared `Word` nodes, revealing that the three verses at the top differ only in the spelling of the word at rank two. At the bottom of the image, the two verses exhibit more substantial differences. The verse on the left contains a different word, βλεπιν, which is synonymous with ιδιν, while the verse on the right diverges further, differing in both the number and choice of words. These observations illustrate how the graph structure can support the comparative analysis of textual variants: the degree to which verse nodes share word nodes provides an inherent, visual indication of their relatedness, without the need for any computed similarity score.

Using similar techniques, Figure 2.4 shows the full structure of epigrams with ids 18540 and 18542, both belonging to type 2150 [19]. This visualisation allows us to inspect the internal structure of both epigrams simultaneously, revealing that they share the exact same first and last verses, while their middle verses differ. The shared verses are each represented by a single node connected to both epigram nodes, a direct consequence of identical content being merged during graph construction. Upon closer inspection, the middle verses differ in their fourth word: ιδιν (“to see”) in epigram 18540 and ευριν (“to find”) in epigram 18542. Such fine-grained differences, which might be difficult to detect through manual comparison in a large corpus, become immediately visible when the hierarchical structure of the texts is laid out in the graph.

The interactive nature of the visualisation tool additionally supports dynamic exploration of the graph. By expanding nodes, a researcher can inspect the underlying structure and content of the texts, and collapsing nodes shifts the focus to higher-level connections by hiding lower-level detail. Contextual metadata can further enrich this exploration by allowing a researcher to identify not only which texts share content but also whether those texts originate from the same region, were copied by the same scribe, or share a common subject. Although the examples above focus on textual structure, the same visualisation mechanism applies to metadata nodes, which can be included in such visualisations without any further adaptation.

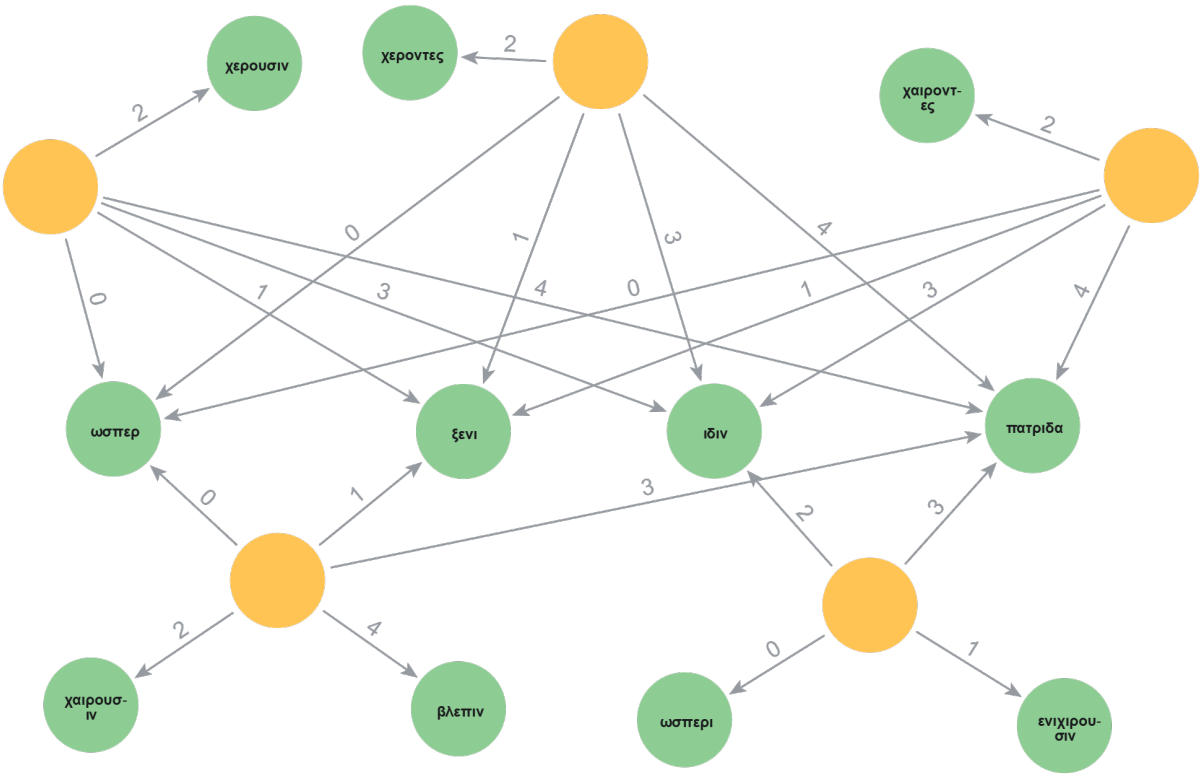


Figure 2.3: Neo4j Bloom visualisation of the subgraph containing the first verses of a subset of epigrams. `Verse` nodes are indicated in yellow, `Word` nodes are indicated in green and display their `text` property, and the `CONTAINS` relationships are shown as edges including their `rank` property.

The visualisations presented in this subsection demonstrate how the graph structure makes shared content between texts immediately visible, and how the hierarchical decomposition enables inspection at multiple levels of granularity. Such visual exploration can serve as a starting point for more advanced investigations, such as identifying new groups of related texts, discovering previously unknown interconnections, or supplementing incomplete metadata based on shared content. Beyond visual exploration, the graph structure also supports declarative pattern matching, as demonstrated in the following subsection.

2.6.4 Pattern Matching

A third application of the graph-based text model is the ability to retrieve specific structural patterns through declarative queries. Cypher, the query language of Neo4j, employs graph pattern matching through its `MATCH` clause, which allows users to specify structural templates of nodes and relationships that the database engine matches against the stored graph [64]. This capability is particularly valuable in the context of the proposed graph model, as it allows researchers to query for specific configurations of shared content across texts.

As a first example, Listing 2.4 shows a Cypher query retrieving all verses that start with the word $\kappa\alpha\iota$, followed by the word ι. As described in Section 2.6.1, the middle verses of the longer epigrams of types 2150 and 4245 follow a similar formula, starting with these two words but with possibly different remaining content. The query matches a verse node v that is connected to a `Word` node with text $\kappa\alpha\iota$ at rank 0 and to a `Word` node

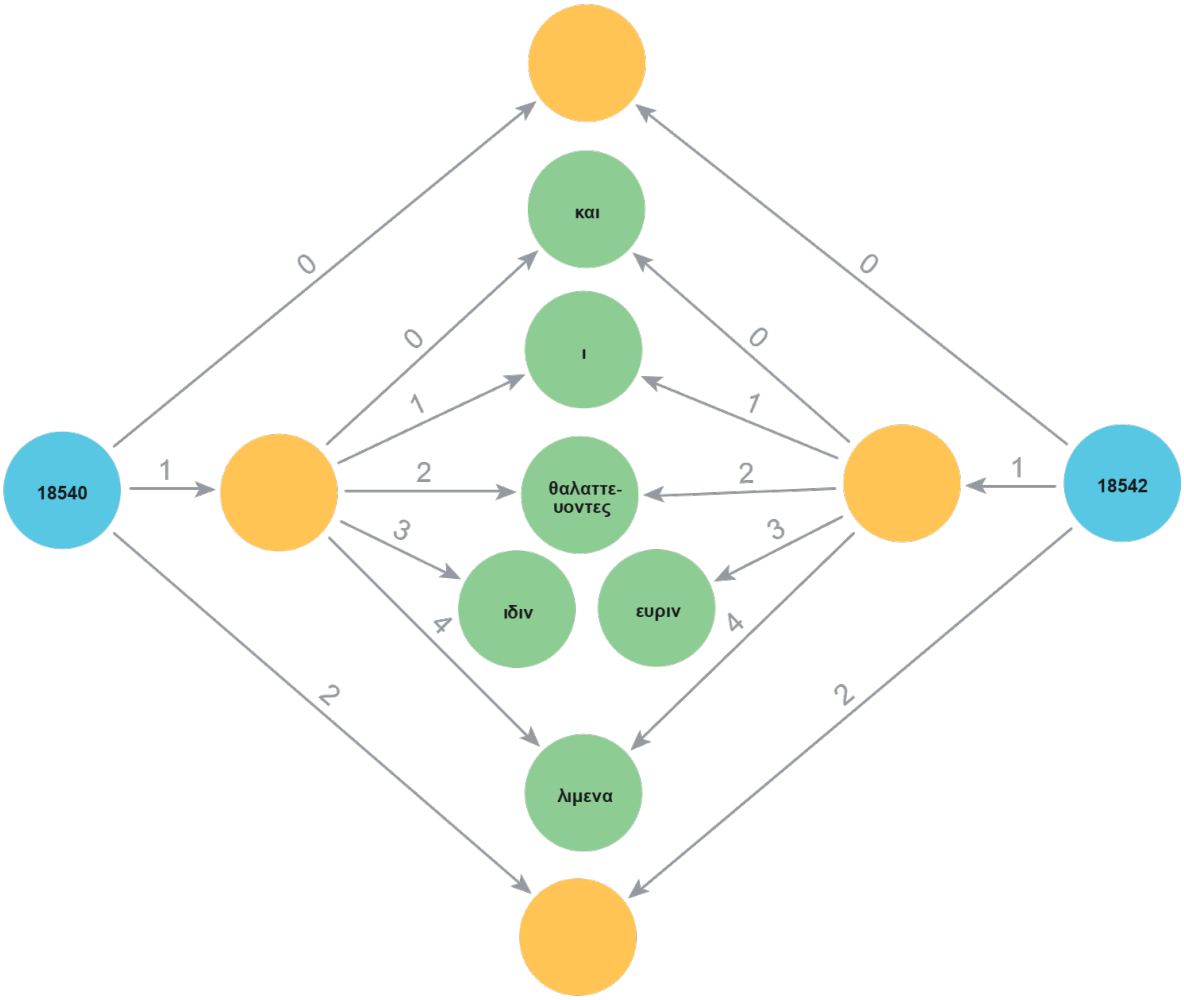


Figure 2.4: Neo4j Bloom visualisation of the subgraph containing the epigrams with ids 18540 and 18542, both belonging to type 2150. `Epigram` nodes are indicated in blue and display their `id` property, `Verse` nodes are indicated in yellow, `Word` nodes are indicated in green and display their `text` property, and the `CONTAINS` relationships are shown as edges including their `rank` property.

with text ι at rank 1. It thereby retrieves those middle verses of the longer epigrams of types 2150 and 4245, as well as any other verse that begins with the same two words. Executing this query on the dataset returns 44 unique `Verse` nodes representing 93 verse occurrences, confirming that the formulaic pattern is widespread across the longer epigrams.

```
MATCH (v:Verse)
MATCH (v)-[:CONTAINS {rank: 0}]->(:Word {text: "και"})
MATCH (v)-[:CONTAINS {rank: 1}]->(:Word {text: "ι"})
RETURN v
```

Listing 2.4: Cypher query for retrieving verses starting with the pattern “και ι”.

The utility of pattern matching increases substantially when textual properties are available on the word nodes, as patterns can then combine exact word matches with linguistic or other criteria. To illustrate this, we assume the availability of Part-of-Speech (POS) tags [68, 69] and lemmatised forms [15, 70] on the word nodes. Such annotations are an outcome of the NLP sub-project of the Interconnected Texts project and can be stored as

textual node properties, as described in Section 2.5.2. Listing 2.5 queries for verses whose first word exactly matches ωσπερ, whose second word has a noun POS tag, and whose third word has the lemma χαιρω ("to rejoice"). This query retrieves all verses that follow the specified pattern, which includes the first verse of nearly all epigrams in the dataset, as well as any other verse that matches the same combination of lexical and linguistic criteria.

```
MATCH (v:Verse)
MATCH (v)-[:CONTAINS {rank: 0}]->(:Word {text: "ωσπερ"})
MATCH (v)-[:CONTAINS {rank: 1}]->(:Word {pos: "noun"})
MATCH (v)-[:CONTAINS {rank: 2}]->(:Word {lemma: "χαιρω"})
RETURN v
```

Listing 2.5: Cypher query for retrieving verses matching a combined word, POS, and lemma pattern.

The examples above illustrate only a fraction of the pattern matching possibilities offered by the graph model, particularly when combined with textual properties. The addition of metadata nodes and relationships broadens the scope of such queries, enabling combinations of structural, textual, and contextual criteria, such as retrieving all verses matching a given pattern within epigrams that share a specific subject or scribe. More broadly, the ability to retrieve structural and textual patterns through declarative queries enables researchers to explore the corpus in a targeted manner and to identify patterns of shared content relevant to their analyses.

2.7 Conclusions and Future Work

In this chapter, we presented a graph-based data model for representing short texts, their associated contextual metadata, and additional textual properties. We first introduced a hierarchical decomposition of texts into discourse units, formalised as an ordered list of such units, and proposed a graph model based on an LPG in which each text element is represented as a node connected to its containing and contained elements through directed edges. A key property of this model is that identical textual elements across different texts are represented by the same node, except at the top level, where each text retains its own node to remain uniquely identifiable, making interconnections between texts a direct feature of the graph structure. We described the construction algorithm that transforms a corpus into this representation, including an optional, corpus-specific preprocessing step that normalises the texts before import. The model was extended with contextual metadata, modelled as separate nodes connected to text nodes, and with textual properties stored directly on the nodes representing a part of the text. We demonstrated the practical applicability of the model through illustrative examples based on the DBBE corpus of Byzantine book epigrams implemented in a Neo4j graph database system, showing how the graph model supports the identification of frequently shared textual elements, the visual exploration of interconnections between texts, and declarative pattern matching through graph queries.

The principal contribution of this chapter is a general-purpose, language-independent graph-based text representation that preserves the hierarchical structure of short texts while capturing the interconnections between them at multiple levels of granularity. Where flat text representations reduce a text to a single sequence, the proposed model retains its internal organisation in a semi-structured manner, enabling analysis at the level of individual words, verses, sentences, or any other discourse unit appropriate for the corpus at hand.

The node-merging mechanism, whereby textual components with equal content are represented by the same node, transforms shared content from something that must be discovered through pairwise comparison into an inherent property of the graph topology, which supports both visual exploration and analytical querying. The integration of contextual metadata within the same graph structure enables queries and analyses that combine textual and contextual criteria, while the schemaless nature of graph databases accommodates metadata of varying structure and availability without requiring a fixed schema. Although this chapter demonstrates the model on a corpus of historical short poems, it is not tied to this specific domain. It applies equally to other types of short, interconnected texts, such as social media posts, song lyrics, news headlines, or short product reviews, provided that suitable preprocessing steps and textual properties are selected for the corpus at hand. Together, these properties make the proposed model a flexible foundation for a range of downstream tasks.

While we have shown that the model can handle a real-world corpus such as that of the DBBE, we must also acknowledge several limitations. The node-merging mechanism relies on exact string equality after preprocessing, which means that two textual elements differing by even a single character are represented by separate nodes and are not recognised as related. In corpora characterised by extensive orthographic variation, such as the corpus of Byzantine book epigrams, this strict criterion can cause closely related verses or words to appear unconnected in the graph, despite being near-identical in content. The preprocessing steps introduced in this chapter mitigate this limitation to some extent, but they are rigid and corpus-specific, and cannot account for all forms of variation. Furthermore, the model operates entirely on the orthographic features of the text and does not capture semantic relationships between textual elements, such as synonyms, which would require language-specific NLP techniques. Finally, the model is demonstrated on a single corpus, and the extent to which its behaviour transfers to corpora with different structural characteristics remains to be assessed.

Several directions for future work arise from these limitations and from the practical deployment of the model. Enriching the graph with linguistic annotations stored as textual properties could support more advanced pattern matching and capture relationships beyond the orthographic level, although this would require language-specific resources that are not generally available for low-resource languages. On a practical level, deploying the graph model alongside existing data stores requires Extract, Transform, Load (ETL) pipelines to keep both systems consistent as new data are added or existing records are modified. Aside from these open directions, the graph-based data model developed in this chapter underpins the analytical techniques introduced in the remainder of this thesis. In Chapter 3, a hierarchical orthographic similarity measure that operates on the graph representation is proposed. This measure directly addresses the strict string-equality limitation of the node-merging mechanism by quantifying, at every level of the hierarchy, the resemblance between textual elements that are not identical. In Chapter 4, these textual similarities are combined with similarities computed over the metadata nodes, producing a context-aware similarity measure paired with a confidence score that reflects the completeness of the available metadata, thereby accommodating missing or incomplete metadata. Chapter 5 then develops ranking techniques that integrate these similarity-confidence pairs, while Chapter 6 investigates clustering methods that exploit these scores to group short texts in an unsupervised manner. Taken together, the chapters that follow transform the graph-based data model proposed here into the backbone of a framework for the structured, context-aware analysis of short, interconnected texts.

References

[1] Min Chen, Shiwen Mao and Yunhao Liu. "Big Data: A Survey". In: *Mobile Networks and Applications* 19.2 (2014), pp. 171–209. DOI: https://doi.org/10.1007/s11036-013-0489-0.

[2] Hossein Hassani, Christina Beneki, Stephan Unger, Maedeh Taj Mazinani and Mohammad Reza Yeganegi. "Text Mining in Big Data Analytics". In: *Big Data and Cognitive Computing* 4.1 (2020). issn: 2504-2289. DOI: https://doi.org/10.3390/bdcc4010001.

[3] Andreas Hotho, Andreas Nürnberger and Gerhard Paaß. "A Brief Survey of Text Mining". In: *Journal for Language Technology and Computational Linguistics* 20.1 (2005), pp. 19–62. DOI: https://doi.org/10.21248/jlcl.20.2005.68.

[4] Sonali Vijay Gaikwad, Archana Chaugule and Pramod Patil. "Text Mining Methods and Techniques". In: *International Journal of Computer Applications* 85.17 (2014). DOI: https://doi.org/10.5120/14937-3507.

[5] Taeho Jo. "Text Mining". In: *Studies in Big Data* 45 (2019). DOI: https://doi.org/10.1007/978-3-319-91815-0.

[6] Sholom M Weiss, Nitin Indurkhya, Tong Zhang and Fred Damerau. "Text Mining: Predictive Methods for Analyzing Unstructured Information". Springer Science & Business Media, 2010. DOI: https://doi.org/10.1007/978-0-387-34555-0.

[7] Ronen Feldman and James Sanger. "The Text Mining Handbook: Advanced Approaches in Analyzing Unstructured Data". Cambridge university press, 2007. DOI: https://doi.org/10.1017/CBO9780511546914.

[8] Rajvardhan Patil, Sorio Boit, Venkat Gudivada and Jagadeesh Nandigam. "A Survey of Text Representation and Embedding Techniques in NLP". In: *IEEE Access* 11 (2023), pp. 36120–36146. DOI: https://doi.org/10.1109/ACCESS.2023.3266377.

[9] Rania Albalawi, Tet Hin Yeap and Morad Benyoucef. "Using Topic Modeling Methods for Short-Text Data: A Comparative Analysis". In: *Frontiers in Artificial Intelligence* Volume 3 (2020). DOI: https://doi.org/10.3389/frai.2020.00042.

[10] Dimas Wibisono Prakoso, Asad Abdi and Chintan Amrit. "Short Text Similarity Measurement Methods: A Review". In: *Soft Computing* 25.6 (2021), pp. 4699–4723. issn: 1433-7479. DOI: https://doi.org/10.1007/s00500-020-05479-2.

[11] Junyang Chen, Zhiguo Gong, Wei Wang, Xiao Dong, Wei Wang, Weiwen Liu, Cong Wang and Xian Chen. "Inductive Document Representation Learning for Short Text Clustering". In: *Machine Learning and Knowledge Discovery in Databases*. Cham: Springer International Publishing, 2021, pp. 600–616. isbn: 978-3-030-67664-3. DOI: https://doi.org/10.1007/978-3-030-67664-3_36.

[12] Partha Pakray, Alexander Gelbukh and Sivaji Bandyopadhyay. "Natural Language Processing Applications for Low-Resource Languages". In: *Natural Language Processing* 31.2 (2025), pp. 183–197. DOI: https://doi.org/10.1017/nlp.2024.33.

[13] Alexandre Magueresse, Vincent Carles and Evan Heetderks. *Low-Resource Languages: A Review of Past Work and Future Challenges*. 2020. DOI: https://doi.org/10.48550/arXiv.2006.07264.

[14] Nathan Stringham and Mike Izbicki. "Evaluating Word Embeddings on Low-Resource Languages". In: *Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems*. Online: Association for Computational Linguistics, 2020, pp. 176–186. DOI: https://doi.org/10.18653/v1/2020.eval4nlp-1.17.

[15] Colin Swaelens, Ilse De Vos and Els Lefever. "Evaluating Existing Lemmatisers on Unedited Byzantine Greek Poetry". In: *Proceedings of the Ancient Language Processing Workshop*. Varna, Bulgaria, 2023, pp. 111–116. DOI: https://doi.org/10.5281/zenodo.8337363.

[16] Andreas Meier and Michael Kaufmann. "SQL & NoSQL Databases". Springer, 2019. DOI: https://doi.org/10.1007/978-3-658-24549-8.

[17] Neo4j. "Neo4j Graph Database". url: https://neo4j.com/ (visited on 27/03/2026).

[18] Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, Maxime Deforche, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout and Colin Swaelens. "The Database of Byzantine Book Epigrams Project : Principles, Challenges, Opportunities". eng. In: *Journal of Data Mining and Digital Humanities* On the Way to the Future of Digital Manuscript Studies (2023), p. 41. issn: 2416-5999. DOI: https://doi.org/10.46298/jdmdh.10244.

[19] Kristoffel Demoen, Gilbert Bentein, Klaas Bentein, Floris Bernard, Julián Bértola, Julie Boeten, Mathijs Clement, Cristina Cocola, Eline Daveloose, Sien De Groot, Pieterjan De Potter, Ilse De Vos, Krystina Kubina, Hanne Lauwers, Paulien Lemay, Renaat Meesters, Marjolein Morbé, Delphine Nachtergaele, Marthe Nemegeer, Joachim Nielandt, Mace Ojala, Lisa-Lou Péchillon, Raf Praet, Rachele Ricceri, Anne-Sophie Rouckhout, Jeroen Schepens, Febe Schollaert, Lev Shadrin, Nina Sietis, Dimitrios Skrekas, Colin Swaelens, Maria Tomadaki, Sarah-Helena Van den Brande, Merel Van Nieuwerburgh, Lotte Van Olmen, Noor Vanhoe and Nina Vanhoutte. *Database of Byzantine Book Epigrams (DBBE)*. Version 1.0.0. Zenodo, Feb. 2023. DOI: https://doi.org/10.5281/zenodo.7682523.

[20] Gonzalo Navarro. "A Guided Tour to Approximate String Matching". In: *ACM Computing Surveys (CSUR)* 33.1 (2001), pp. 31–88. DOI: https://doi.org/10.1145/375360.375365.

[21] Andrzej Białecki, Robert Muir and Grant Ingersoll. "Apache Lucene 4". In: *Proceedings of the SIGIR 2012 Workshop on Open Source Information Retrieval (OSIR)*. 2012, p. 17.

[22] Clinton Gormley and Zachary Tong. "Elasticsearch: The Definitive Guide: A Distributed Real-Time Search and Analytics Engine". O'Reilly Media, 2015.

[23] Peter Weiner. "Linear Pattern Matching Algorithms". In: *14th Annual Symposium on Switching and Automata Theory (SWAT)*. 1973, pp. 1–11. DOI: https://doi.org/10.1109/SWAT.1973.13.

[24] Edward M. McCreight. "A Space-Economical Suffix Tree Construction Algorithm". In: *J. ACM* 23.2 (1976), pp. 262–272. issn: 0004-5411. DOI: https://doi.org/10.1145/321941.321946.

[25] Edward Fredkin. "Trie Memory". In: *Commun. ACM* 3.9 (1960), pp. 490–499. issn: 0001-0782. DOI: https://doi.org/10.1145/367390.367400.

[26] Dan Gusfield. “Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology”. Cambridge University Press, 1997. isbn: 9780511574931. DOI: https://doi.org/10.1017/CBO9780511574931.

[27] Esko Ukkonen. “On-Line Construction of Suffix Trees”. In: *Algorithmica* 14.3 (1995), pp. 249–260. issn: 1432-0541. DOI: https://doi.org/10.1007/BF01206331.

[28] Felipe Almeida and Geraldo Xexéo. “Word Embeddings: A Survey”. In: *arXiv Preprint* (2023). DOI: https://doi.org/10.48550/arXiv.1901.09069.

[29] Wisam A. Qader, Musa M. Ameen and Bilal I. Ahmed. “An Overview of Bag of Words;Importance, Implementation, Applications, and Challenges”. In: *2019 International Engineering Conference (IEC)*. 2019, pp. 200–204. DOI: https://doi.org/10.1109/IEC47844.2019.8950616.

[30] Yin Zhang, Rong Jin and Zhi-Hua Zhou. “Understanding Bag-of-Words Model: A Statistical Framework”. In: *International Journal of Machine Learning and Cybernetics* 1.1 (2010), pp. 43–52. issn: 1868-808X. DOI: https://doi.org/10.1007/s13042-010-0001-0.

[31] Gerard Salton and Christopher Buckley. “Term-Weighting Approaches in Automatic Text Retrieval”. In: *Information Processing & Management* 24.5 (1988), pp. 513–523. issn: 0306-4573. DOI: https://doi.org/https://doi.org/10.1016/0306-4573(88)90021-0.

[32] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado and Jeff Dean. “Distributed Representations of Words and Phrases and Their Compositionality”. In: *Advances in Neural Information Processing Systems*. Vol. 26. Curran Associates, Inc., 2013. DOI: https://doi.org/10.48550/arXiv.1310.4546.

[33] Jeffrey Pennington, Richard Socher and Christopher D Manning. “Glove: Global Vectors for Word Representation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1532–1543. DOI: https://doi.org/10.3115/v1/D14-1162.

[34] Piotr Bojanowski, Edouard Grave, Armand Joulin and Tomas Mikolov. “Enriching Word Vectors with Subword Information”. In: *Transactions of the Association for Computational Linguistics* 5 (2017), pp. 135–146. issn: 2307-387X. DOI: https://doi.org/10.1162/tacl_a_00051.

[35] Tom Kenter and Maarten de Rijke. “Short Text Similarity with Word Embeddings”. In: *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*. CIKM ’15. Melbourne, Australia: Association for Computing Machinery, 2015, pp. 1411–1420. isbn: 9781450337946. DOI: https://doi.org/10.1145/2806416.2806475.

[36] Quoc V. Le and Tomas Mikolov. “Distributed Representations of Sentences and Documents”. In: *arXiv Preprint* (2014). DOI: https://doi.org/10.48550/arXiv.1405.4053.

[37] Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova. “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: https://doi.org/10.18653/v1/N19-1423.

[38] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992. DOI: https://doi.org/10.18653/v1/D19-1410.

[39] Thomas K Landauer and Susan T Dumais. “A Solution to Plato’s Problem: The Latent Semantic Analysis Theory of Acquisition, Induction, and Representation of Knowledge”. In: *Psychological Review* 104.2 (1997), p. 211. DOI: https://doi.org/10.1037/0033-295X.104.2.211.

[40] David M. Blei, Andrew Y. Ng and Michael I. Jordan. “Latent Dirichlet Allocation”. In: *J. Mach. Learn. Res.* 3 (2003), pp. 993–1022. issn: 1532-4435.

[41] Hamed Jelodar, Yongli Wang, Chi Yuan, Xia Feng, Xiahui Jiang, Yanchao Li and Liang Zhao. “Latent Dirichlet Allocation (LDA) and Topic Modeling: Models, Applications, a Survey”. In: *Multimedia Tools and Applications* 78.11 (2019), pp. 15169–15211. issn: 1573-7721. DOI: https://doi.org/10.1007/s11042-018-6894-4.

[42] TEI Consortium. “Text Encoding Initiative Guidelines”. url: https://tei-c.org/guidelines/ (visited on 28/03/2026).

[43] Lou Burnard. “What Is the Text Encoding Initiative?” OpenEdition Books, 2014. DOI: https://doi.org/10.4000/books.oep.426.

[44] James Clarck and Steve DeRose. “XML Path Language (XPath)”. W3C. 1999. url: https://www.w3.org/TR/1999/REC-xpath-19991116/ (visited on 28/03/2026).

[45] Don Chamberlin. “XQuery: An XML Query Language”. In: *IBM Systems Journal* 41.4 (2002), pp. 597–615. DOI: https://doi.org/10.1147/sj.414.0597.

[46] Ahmed Hamza Osman and Omar Mohammed Barukub. “Graph-Based Text Representation and Matching: A Review of the State of the Art and Future Challenges”. In: *IEEE Access* 8 (2020), pp. 87562–87583. DOI: https://doi.org/10.1109/ACCESS.2020.2993191.

[47] Laura V.C. Quispe, Jorge A.V. Tohalino and Diego R. Amancio. “Using Virtual Edges to Improve the Discriminability of Co-Occurrence Text Networks”. In: *Physica A: Statistical Mechanics and its Applications* 562 (2021), p. 125344. issn: 0378-4371. DOI: https://doi.org/10.1016/j.physa.2020.125344.

[48] Mikaela Irene Fudolig, Thayer Alshaabi, Michael V. Arnold, Christopher M. Danforth and Peter Sheridan Dodds. “Sentiment and Structure in Word Co-Occurrence Networks on Twitter”. In: *Applied Network Science* 7.1 (2022), p. 9. issn: 2364-8228. DOI: https://doi.org/10.1007/s41109-022-00446-2.

[49] Camilo Akimushkin, Diego R. Amancio and Osvaldo N. Oliveira. “On the Role of Words in the Network Structure of Texts: Application to Authorship Attribution”. In: *Physica A: Statistical Mechanics and its Applications* 495 (2018), pp. 49–58. issn: 0378-4371. DOI: https://doi.org/10.1016/j.physa.2017.12.054.

[50] Emilio Sulis, Llio Humphreys, Fabiana Vernero, Ilaria Angela Amantea, Davide Audrito and Luigi Di Caro. “Exploiting Co-Occurrence Networks for Classification of Implicit Inter-Relationships in Legal Texts”. In:

Information Systems 106 (2022), p. 101821. issn: 0306-4379. DOI: https://doi.org/10.1016/j.is.2021.101821.

[51] Fabian Flöck and Maribel Acosta. "WikiWho: Precise and Efficient Attribution of Authorship of Revisioned Content". In: *Proceedings of the 23rd International Conference on World Wide Web*. 2014, pp. 843–854. DOI: https://doi.org/10.1145/2566486.2568026.

[52] Antoon Bronselaer and Gabriella Pasi. "An Approach to Graph-Based Analysis of Textual Documents". In: *8th European Society for Fuzzy Logic and Technology (EUSFLAT-2013)*. Atlantis Press. 2013, pp. 634–641. DOI: https://doi.org/10.2991/eusflat.2013.96.

[53] Jonathan J Webster and Chunyu Kit. "Tokenization as the Initial Phase in NLP". In: *COLING 1992 Volume 4: The 14th International Conference on Computational Linguistics*. 1992. DOI: https://doi.org/10.3115/992424.992434.

[54] David D Palmer. "Tokenisation and Sentence Segmentation". In: *Handbook of Natural Language Processing*. New York: Marcel Dekker, Inc, 2000, pp. 11–35.

[55] Allan Bickle. "Fundamentals of Graph Theory". American Mathematical Soc., 2020. isbn: 978-1-4704-5342-8.

[56] Renzo Angles and Claudio Gutierrez. "Survey of Graph Database Models". In: *ACM Computing Surveys (CSUR)* 40.1 (2008), pp. 1–39. DOI: https://doi.org/10.1145/1322432.1322433.

[57] Shalini Batra and Charu Tyagi. "Comparative Analysis of Relational and Graph Databases". In: *International Journal of Soft Computing and Engineering (IJSCE)* 2.2 (2012), pp. 509–512.

[58] Frank Manola, Eric Miller et al. "RDF Primer". W3C. 2004. url: https://www.w3.org/TR/rdf-primer/ (visited on 28/03/2026).

[59] Stefan Decker, Sergey Melnik, Frank Van Harmelen, Dieter Fensel, Michel Klein, Jeen Broekstra, Michael Erdmann and Ian Horrocks. "The Semantic Web: The Roles of XML and RDF". In: *IEEE Internet Computing* 4.5 (2000), pp. 63–73. DOI: https://doi.org/10.1109/4236.877487.

[60] Lihua Zhao and Ryutaro Ichise. "Ontology Integration for Linked Data". In: *Journal on Data Semantics* 3.4 (2014), pp. 237–254. issn: 1861-2040. DOI: https://doi.org/10.1007/s13740-014-0041-9.

[61] Jorge Pérez, Marcelo Arenas and Claudio Gutierrez. "Semantics and Complexity of SPARQL". In: *ACM Trans. Database Syst.* 34.3 (2009). issn: 0362-5915. DOI: https://doi.org/10.1145/1567274.1567278.

[62] Eric Prud'hommeaux, Andy Seaborne and Steve Harris. "SPARQL 1.1 Query Language". W3C. url: https://www.w3.org/TR/sparql11-query/ (visited on 28/03/2026).

[63] Renzo Angles. "The Property Graph Database Model". In: *Proceedings of the 12th Alberto Mendelzon International Workshop on Foundations of Data Management (AMW)*. Vol. 2100. CEUR Workshop Proceedings. CEUR-WS.org, 2018, pp. 1–10.

[64] Neo4j. "Cypher Query Language". url: https://neo4j.com/docs/cypher-manual/current/ (visited on 01/04/2026).

[65] Lawrence Page, Sergey Brin, Rajeev Motwani and Terry Winograd. *The PageRank Citation Ranking: Bringing Order to the Web*. Tech. rep. Stanford InfoLab, 1999.

[66] Neo4j. "Neo4j Graph Data Science". url: https://neo4j.com/product/graph-data-science/ (visited on 10/05/2026).

[67] Neo4j. "Neo4j Bloom". url: https://neo4j.com/bloom/ (visited on 03/04/2026).

[68] Colin Swaelens, Ilse De Vos and Els Lefever. "Lemmatisation & Morphological Analysis of Unedited Greek : Do Simple Tasks Need Complex Solutions?" In: *Findings of the Association for Computational Linguistics : ACL 2025*. Ed. by Wanxiang Che, Joyce Nabende, Ekaterina Shutova and Mohammad Taher Pilehvar. Vienna, Austria: Association for Computational Linguistics (ACL), 2025, pp. 7681–7689. isbn: 9798891762565.

[69] Colin Swaelens, Ilse De Vos and Els Lefever. "DBBErt : Part-of-Speech Tagging of Pre-Modern Greek Text". eng. In: *CLARIN Annual Conference Proceedings*. Leuven, Belgium, 2023, pp. 155–158.

[70] Colin Swaelens, Pranaydeep Singh, Ilse De Vos and Els Lefever. "Lemmatisation of Medieval Greek: Against the Limits of Transformer's Capabilities?" In: *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. Turin, Italy: ELRA, 2024, pp. 10293–10302. isbn: 9782493814104. DOI: https://doi.org/10.63317/35evzb2ctsc3.

3

A Hierarchical Orthographic Similarity Measure for Short Texts

"Nothing is particularly hard
if you divide it into small jobs."

— **Henry Ford**

In this chapter, a novel hierarchical orthographic similarity measure for short texts is described, operating on the graph-based data model introduced in Chapter 2. The measure exploits the hierarchical structure of the graph to compute similarity in a bottom-up fashion, first computing scores for the lowest-level textual elements and then using these scores to determine the similarity between higher-level textual elements and, ultimately, between the full texts. By adapting the concept of edit distances to operate on graph nodes rather than character sequences alone, the proposed approach preserves structural information that conventional flat similarity measures discard. The resulting computation yields similarity scores between all pairs of elements at every level of the textual hierarchy instead of a single score for the texts as a whole, enhancing the interpretability and explainability of the result. Once computed, these similarity scores are stored as edges in the graph, extending the data model from Chapter 2 and enabling flexible querying and exploration of the corpus based on degrees of textual resemblance.

The development and evaluation of this hierarchical orthographic similarity measure address the second research question of this thesis:

> **RQ 2:** How can an orthographic similarity measure be designed to capture similarities between short texts effectively at multiple levels of granularity, while exploiting the graph representation to account for their structural properties?

In addition, by extending the querying capabilities already demonstrated in Chapter 2 with the computed similarity scores, this chapter further addresses:

> **RQ 4.A:** How can (flexible) query answering techniques be applied to a graph-based data model of short texts to support the discovery of patterns and interconnections, with or without considering similarity or metadata?

The content of this chapter is based on parts of the following publications:

- **Maxime Deforche**, Ilse De Vos, Antoon Bronselaer, and Guy De Tré. *An Orthographic Similarity Measure for Graph-Based Text Representations.* Published in Proceedings of the 15th International Conference on Flexible Query Answering Systems (FQAS), Lecture Notes in Artificial Intelligence, vol 14113, 2023.
  DOI: 10.1007/978-3-031-42935-4_17
- **Maxime Deforche**, Ilse De Vos, Antoon Bronselaer, and Guy De Tré. *A Hierarchical Orthographic Similarity Measure for Interconnected Texts Represented by Graphs.* Published in Applied Sciences, Special Issue on Data Science Methods in Big Data Era, vol 14(4), 2024.
  DOI: 10.3390/app14041529

The remainder of this third chapter is structured as follows. Section 3.1 introduces the problem of measuring similarity between short texts and outlines the proposed approach. Related work on textual similarity measures, including orthographic and semantic approaches, is discussed in Section 3.2. Section 3.3 provides the necessary preliminaries on similarity measures, fuzzy sets, and fuzzy graphs, and extends the Labelled Property Graph (LPG) model from Chapter 2 into a fuzzy LPG. The proposed hierarchical orthographic similarity measure is presented in Section 3.4. This section covers both a string-based component, which computes similarity at the lowest level of the graph between elements such as words or tokens, and a node-based component, which computes similarity at higher levels of the hierarchy between elements such as sentences, verses, and paragraphs. Section 3.5 reports on the experimental evaluation of the proposed measure using the Database of Byzantine Book Epigrams (DBBE) dataset, including a comparison with an existing baseline method. This section also demonstrates the practical applicability of the similarity-enriched graph through querying and visualisation examples. Finally, Section 3.6 summarises the contributions of this chapter, discusses limitations, and outlines directions for future work.

3.1 Introduction

With the continued growth of data, and textual data in particular, the need for effective techniques that can analyse and extract information from large volumes of text has increased substantially [1, 2]. Text mining, the process of transforming semi-structured or unstructured text into structured representations suitable for analysis, encompasses a wide range of such tasks, from classification and clustering to summarisation, information extraction, and flexible query answering [3–5]. A crucial component of most of these techniques is the measurement of similarity between textual documents or parts thereof. The result of such a similarity measure is typically a value between 0 and 1, where 0 indicates complete dissimilarity and 1 indicates that the two elements are considered fully similar according to the measure at hand. In duplicate or co-referent record detection in database systems, for instance, similarity scores help determine whether two records refer to the

same entity [6–8]. In information retrieval and flexible query answering, these measures enable the comparison of documents against user queries to rank and retrieve relevant results [9]. More broadly, the effectiveness of any downstream analytical task, be it clustering, classification, or ranking, strongly depends on the quality of the similarity measure on which it is built. This dependency motivates the need for tailored approaches that capture the specific characteristics of the texts under consideration.

This need is particularly acute for corpora that exhibit dense interconnections between their texts, such as shared phrases, common structural elements, or entire texts that are variants of one another. For such corpora, identifying and analysing these connections is essential for understanding the relationships within and between the texts. Consider, for instance, two texts that differ only in the ordering of two consecutive sentences. Conventional orthographic similarity measures compare texts as sequences or sets of characters or tokens [10, 11]. Such measures either overstate this structural difference or overlook it entirely, in both cases resulting in an unrepresentative similarity score. A second shortcoming is that a single score offers only a global view and no insight into which parts of the texts are similar and which differ, leading to the loss of potentially valuable information and limiting both the interpretability and the explainability of the result. Semantic similarity approaches, which rely on embeddings or language models to represent meaning-level relationships [11, 12], often cannot capture these structural differences either, since they typically represent texts or parts thereof as single vectors without explicitly modelling internal structure. Especially when texts are short, written in low-resource languages, or both, semantic approaches often lack the effectiveness required for meaningful comparison [13–16]. For such texts, orthographic approaches therefore remain the most viable option, but the structural limitations described above persist and must be addressed by the measure itself. Taken together, these limitations call for a novel similarity measure that operates on the structural representation of texts, that produces scores at multiple levels of granularity, and that does not depend on language-specific resources.

In this chapter, we address these limitations by proposing a novel hierarchical orthographic similarity measure that operates on the graph-based text representation developed in Chapter 2. Following the hierarchical decomposition of texts into discourse units, from the smallest textual elements up to complete texts, the measure computes similarity in a bottom-up fashion. At the lowest level of the hierarchy, a string-based similarity measure is applied to pairs of textual elements. This measure is based on an edit distance, normalised into a similarity score, with customisable cost parameters for each edit operation. At each subsequent level, a node-based similarity measure determines the resemblance between higher-level textual elements by computing an edit distance over their ordered lists of child nodes, where the costs of certain edit operations are informed by the similarity scores obtained at the previous level. This incorporation of lower-level similarity scores into the computation at each successive level is the central mechanism of the proposed approach, ensuring that the structure of the texts plays a defining role in the resulting similarity scores between complete texts. All computed scores are stored as property values on edges in the graph, extending the Labelled Property Graph (LPG) from Chapter 2 into a fuzzy LPG in which the pairwise similarity between all comparable textual elements is directly accessible as part of the graph structure.

The principal novelty of the proposed approach lies in the hierarchical application of edit distances to graph-structured text representations. Unlike conventional orthographic measures, which yield a single score for an entire text, the proposed measure produces scores at every level of the textual hierarchy, each informed by the

scores computed at lower levels. Because not only the overall similarity but also the individual lower-level scores from which it is derived can be inspected, the multi-level nature of the proposed measure enhances both the interpretability of the final result and the explainability of downstream analysis techniques. The proposed method can be understood as a generalisation of soft similarity measures to entire texts, extending the traditional approach of integrating character-based similarities into token-based measures [17]. Furthermore, the parameters defining the cost of each edit operation can be adjusted at every level of the hierarchy, allowing the measure to be fine-tuned to the characteristics of a specific corpus through expert knowledge or empirical analysis, which is particularly valuable for corpora characterised by extensive orthographic variation. Because all similarity scores reside in the graph itself, downstream applications, including querying, ranking, and clustering, can access precomputed similarity information directly within the graph database, supporting both flexible retrieval and graph-based analytics.

To validate these contributions, we evaluate the proposed measure through experiments on the Database of Byzantine Book Epigrams (DBBE) corpus [18, 19], a collection of short Greek poems characterised by extensive orthographic variation and dense interconnections, previously introduced in Chapter 1. A representative subset of the corpus is used to compute similarity scores between all pairs of comparable textual elements at all levels of the hierarchy. The resulting scores are filtered using a threshold, and the retained pairs are compared against the pairwise relationships obtained from the expert-curated groupings of similar epigrams (epigram types) and similar verses (verse variants). This comparison is performed at both the epigram and verse level across a range of thresholds and parameter configurations to assess the quality of the measure. We further compare the results of the hierarchical approach with an existing two-level string-matching baseline [20, 21] to situate its relative performance. Finally, we demonstrate the practical applicability of the similarity-enriched graph through querying and visualisation examples that illustrate how the computed scores support the exploration and analysis of the corpus.

3.2 Related Work

The measurement of similarity between texts has been studied extensively, and a broad spectrum of approaches has been proposed, ranging from purely orthographic methods that compare surface-level character and token sequences to semantic methods that rely on the meaning of the textual content [11, 22]. In this section, we review the most prominent approaches and discuss their strengths and limitations in the context of measuring similarity between short, interconnected texts.

Orthographic similarity is based on the resemblance between individual characters or tokens, without taking into account the meaning of the textual content and, for most such methods, without making assumptions about the language in which the texts are written [10]. Two broad families can be distinguished: character-based and token-based methods [11]. Character-based methods determine the similarity between two strings by comparing character sequences, with well-known examples including the Levenshtein [23] and Damerau-Levenshtein [24] edit distances, Jaro-Winkler similarity [25, 26], and N-gram-based approaches [27]. Token-based methods operate at the level of entire tokens and include set-based measures such as the Jaccard similarity [28], the Dice coefficient [29], and the overlap coefficient [11], as well as token-level adaptations of character-based edit

distances [17]. Some of these techniques produce a distance, possibly greater than 1, rather than a similarity score between 0 and 1; however, since a normalised distance can be converted into a similarity score and vice versa, the two formulations can be used interchangeably.

A further line of work relies on *phonetic* encoding, in which each string is mapped onto a code approximating its pronunciation, so that strings sharing a code are treated as matching and the codes themselves can serve as index keys. Soundex, NYSIIS, and Metaphone and its Double Metaphone successor are the best-known encoders, widely used in name matching and record linkage [30]. Strictly speaking, such encoders yield a binary decision rather than a graded score, which is obtained only when the resulting codes are compared by a character-based measure [31]. More importantly, their encoding rules derive from the phonology of one specific language, which makes them inapplicable to historical and low-resource languages, for which dedicated encoders are rarely available.

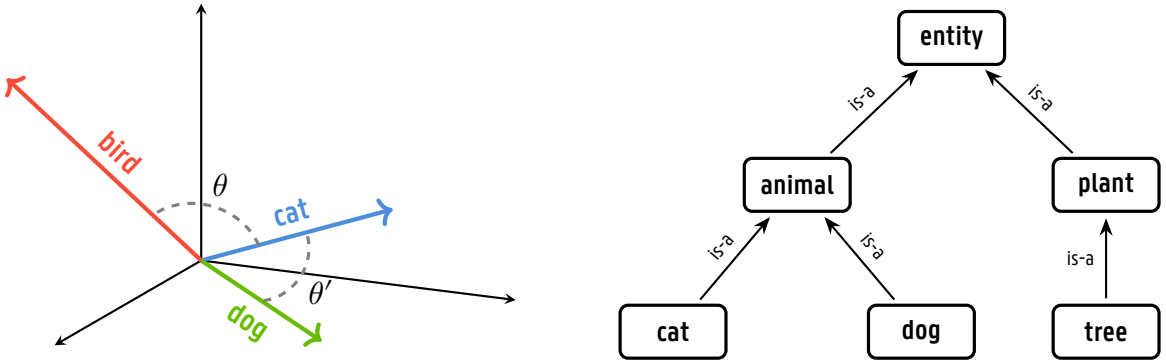
Character- and token-based measures each compute similarity based on one specific type of textual element, either characters or tokens. Few techniques have been proposed that combine character-level and token-level information into a single measure. Such composite techniques, commonly referred to as *soft* similarity measures, consider two tokens to be equal not solely in the case of an exact match but also when the character-based similarity between them meets or exceeds a predefined threshold [10, 17, 20, 21]. In effect, soft measures replace the crisp matching criterion used in traditional token-based methods with an approximate one, allowing tokens that are nearly identical at the character level to be treated as matching. The soft cosine similarity, for example, extends the cosine similarity by taking the character-based similarity between tokens into account, and such soft measures have been shown to outperform their crisp counterparts on text comparison tasks [32, 33]. Similarly, using the character-level dissimilarity between two tokens as the replacement cost in a token-based edit distance yields strong performance in record matching for data cleaning and search applications [34]. While orthographic approaches, including their soft variants, have proven their effectiveness, they typically yield a single similarity score for an entire text and capture its internal structure only to a limited extent, if at all.

Whereas orthographic measures operate on the surface form of texts, semantic similarity measures attempt to capture the meaning of the textual content, relying on assumptions about the language in which the texts are written [12]. Semantic measures fall into two broad categories: corpus-based and knowledge-based methods [11, 35].

Corpus-based measures derive representations of tokens or documents from patterns observed in large text corpora, using techniques such as Latent Semantic Analysis (LSA) [36] and Word2Vec [37], or co-occurrence statistics such as Pointwise Mutual Information (PMI) [38]. The similarity between two such representations is then typically quantified by the cosine similarity, which measures the cosine of the angle between two vectors in the embedding space, as illustrated in Figure 3.1a. More recent embedding-based approaches, including transformer-based models such as BERT [39] and Sentence-BERT [40] (see also Section 2.2), have achieved state-of-the-art results on many similarity benchmarks. However, they tend to perform poorly on short texts and depend on large, representative training corpora that are often unavailable for low-resource languages [13–15].

Knowledge-based methods take a different approach by exploiting information from semantic networks, which organise concepts through the semantic relations that hold between them. Among these, WordNet [41], a

lexical database for the English language, is the most prominent example. Such a network structures concepts hierarchically, ranging from concrete concepts such as *cat*, *dog*, and *tree* to broader categories such as *animal*, *plant*, and *entity*, as illustrated in Figure 3.1b. The edges represent semantic relations such as *is-a*, indicating, for instance, that a *cat* is an *animal* and an *animal* is an *entity*. The similarity between two concepts can then be derived from their position in the network, for instance through the path lengths connecting them to their lowest common ancestor [42]. Alternatively, similarity can be based on the specificity of that ancestor, quantified as its information content [43], so that concepts sharing *animal* are considered more similar than concepts sharing only *entity*. Since both corpus-based and knowledge-based methods depend on the availability of large corpora or semantic networks for a particular language, their applicability is more restricted than that of orthographic measures, particularly for languages lacking sufficient semantic resources.



(a) Similarity between vector embeddings. **(b)** Fragment of a semantic network.

Figure 3.1: Illustrations of semantic similarity approaches: (a) an illustrative vector embedding for the words *cat*, *dog*, and *bird*, where the angle θ between the vectors for *cat* and *bird* indicates their degree of similarity, with the smaller angle θ' between *cat* and *dog* indicating greater similarity, and (b) a fragment of an illustrative semantic network organised as a taxonomy, showing the concepts *cat*, *dog*, and *tree* and their relations to higher-level categories.

Several hybrid approaches have been proposed that combine orthographic and semantic features into a single similarity measure, typically by computing both scores independently and aggregating them through a weighted combination [44]. Others use semantic information to inform the comparison of tokens, for instance by deriving the cost of matching two tokens from the distance between their word embeddings [45]. While such methods can improve performance when both surface form and meaning are relevant, they inherit the language dependence of their semantic components, limiting their use to languages for which sufficient semantic resources exist.

Beyond pairwise textual similarity measures, graph-based approaches have been explored extensively for text analysis [46], though the majority pursue objectives other than the computation of similarity scores. As discussed in Section 2.2, co-occurrence networks, hierarchical graph decompositions, and semantically enriched text graphs each capture certain aspects of textual structure or meaning but are not designed to produce pairwise similarity scores between texts or their constituent parts. A related line of research concerns the computation of similarity between general graph structures through the Graph Edit Distance (GED), defined as the cost of the least expensive sequence of edit operations required to transform one graph into another [47]. Proposed approaches to compute the GED include suboptimal approximation algorithms [48], graph kernels [49], and subgraph matching [50], alongside probabilistic models that treat the edit process as a stochastic transformation of one

graph into another [47]. However, these methods operate on arbitrary graph structures and do not exploit the specific hierarchical textual decomposition that characterises the graph-based text representations considered in this work.

The similarity measure proposed in this chapter differs from the approaches discussed above in several respects. Like traditional orthographic measures, it operates on the surface form of the texts and does not require language-specific resources, preserving applicability to texts written in any language, including low-resource languages. Unlike these measures, however, it does not reduce texts to flat sequences of characters or tokens; it instead exploits the graph-based text representation introduced in Chapter 2, producing similarity scores at every level of the textual hierarchy rather than a single aggregated score. The hierarchical computation shares with soft similarity measures the principle of combining character-level and token-level information, but generalises it to an arbitrary number of hierarchical levels, where the similarity at each level is informed by the scores computed at the level below. Unlike GED methods, the proposed measure is designed specifically for hierarchically decomposed text representations and exploits this structure to compute similarity in a bottom-up manner. The resulting scores are stored directly in the graph, so that downstream analyses can retrieve them without recomputing similarity at query time. Because the proposed measure does not rely on semantic information, it is particularly well-suited for short texts, for texts written in low-resource languages, and for corpora characterised by dense interconnections.

3.3 Preliminaries

This section introduces the formal concepts on which the proposed similarity measure is built. It first defines the notion of a similarity measure, then introduces the basic principles of fuzzy sets, and demonstrates how pairwise similarity scores give rise to a fuzzy similarity relation. Finally, the Labelled Property Graph (LPG) model from Chapter 2 is extended with fuzzy edges to obtain a fuzzy LPG, and the graph-based text model from that chapter is shown to fit within this extended model.

3.3.1 Similarity Measures

Consider an alphabet $\mathcal{A}$ and the set $\mathcal{A}^*$ of all finite character sequences over it, as introduced in Section 2.3.1. A corpus is then a finite set of texts $T \subseteq \mathcal{A}^*$, where the texts under consideration may be complete documents or any of the smaller textual elements into which they are decomposed. A textual similarity measure is a function

$$\mathrm{sim} : T \times T \to [0, 1] \tag{3.1}$$

that quantifies the degree of similarity between two texts. The resulting score reflects, according to the specific measure, how similar two texts are: a score of 1 indicates that the two texts are fully similar, whereas a score of 0 indicates full dissimilarity.

In this work, we consider only similarity measures that are reflexive,

$$\mathrm{sim}(x, x) = 1, \quad \forall x \in T, \tag{3.2}$$

and symmetric,

$$\mathrm{sim}(x, y) = \mathrm{sim}(y, x), \quad \forall x, y \in T. \tag{3.3}$$

The reflexivity condition ensures that every text is considered fully similar to itself, and the symmetry condition ensures that the order in which two texts are compared does not affect the result. Both are natural requirements for a similarity measure between texts. We deliberately do not require that two different texts have a similarity score strictly less than 1, since the measure may be configured to disregard certain differences and thus consider such texts fully similar.

Transitivity, by contrast, is a property that a similarity measure may or may not possess. One common graded formulation requires that

$$\mathrm{sim}(x, z) \geq \mathrm{sim}(x, y) + \mathrm{sim}(y, z) - 1, \quad \forall x, y, z \in T, \tag{3.4}$$

which is precisely the triangle inequality for the associated distance $d(x, y) = 1 - \mathrm{sim}(x, y)$ [51, 52]. Transitivity is therefore guaranteed for any similarity measure derived from a distance that is a metric, and fails as soon as that distance violates the triangle inequality. Since transitivity is not required for the techniques developed in this work, the measure introduced in Section 3.4 is not constrained to satisfy it, and its metric properties are examined alongside the definitions of the underlying distances.

3.3.2 Fuzzy Sets

In classical set theory, an element either belongs to a set or it does not. Fuzzy set theory, introduced by Zadeh [53], generalises this notion by allowing elements to belong to a set to a certain degree. Given a finite universe of elements U, a fuzzy set $\tilde{S}$ is characterised by a membership function

$$\mu_{\tilde{S}} : U \to [0, 1], \tag{3.5}$$

that assigns a degree of membership to each element. A fuzzy set $\tilde{S}$ can then be described by

$$\tilde{S} = \left\{ \left(u, \mu_{\tilde{S}}(u)\right) \mid u \in U \wedge 0 < \mu_{\tilde{S}}(u) \leq 1 \right\}. \tag{3.6}$$

Only elements of U with non-zero membership are included, each paired with its grade $\mu_{\tilde{S}}(u)$, where a value close to 0 indicates weak membership and a value of 1 indicates full membership.

The meaning of the membership grades is context-dependent, and three broad interpretations have been distinguished [54]. The membership grade may represent a degree of similarity to a prototype or category, a degree of truth or preference reflecting how well an element satisfies a given property, or a degree of uncertainty reflecting the belief that an element belongs to the set.

We use three concepts derived from fuzzy set theory throughout this chapter. The support of a fuzzy set $\tilde{S}$ is the crisp set of all elements with a non-zero degree of membership,

$$\text{supp}\big(\tilde{S}\big) = \big\{u \mid u \in U \wedge \mu_{\tilde{S}}(u) > 0\big\}. \tag{3.7}$$

The core of $\tilde{S}$, by contrast, is the crisp set of all elements with full membership,

$$\text{core}\big(\tilde{S}\big) = \big\{u \mid u \in U \wedge \mu_{\tilde{S}}(u) = 1\big\}. \tag{3.8}$$

The α-cut of a fuzzy set $\tilde{S}$, for a given threshold $\alpha \in \,]0, 1]$, is the crisp set of all elements $u \in U$ whose degree of membership meets or exceeds α,

$$\tilde{S}_\alpha = \big\{u \mid u \in U \wedge \mu_{\tilde{S}}(u) \geq \alpha\big\}. \tag{3.9}$$

The α-cut provides a mechanism for converting a fuzzy set into a crisp set by retaining only those elements that satisfy a minimum degree of membership.

3.3.3 Fuzzy Similarity Relations

The similarity measure introduced in Section 3.3.1 quantifies the resemblance between a single pair of texts. To model the pairwise similarity between all texts in a set T, we define a fuzzy relation $\tilde{S}$ over T [55], where the membership grade of each pair $(x, y) \in T \times T$ is given by the similarity score between x and y,

$$\mu_{\tilde{S}}(x, y) = \text{sim}(x, y). \tag{3.10}$$

Since the underlying similarity measure is reflexive and symmetric, the membership function inherits both properties. It is reflexive,

$$\mu_{\tilde{S}}(x, x) = 1, \quad \forall x \in T, \tag{3.11}$$

and symmetric,

$$\mu_{\tilde{S}}(x, y) = \mu_{\tilde{S}}(y, x), \quad \forall x, y \in T. \tag{3.12}$$

As with the textual similarity measure, transitivity is not assumed for the fuzzy similarity relation. In the context of textual similarity, the membership grades of this fuzzy relation correspond to the first of the three interpretations introduced in Section 3.3.2: each grade reflects the extent to which two texts resemble one another.

By representing the pairwise similarity between texts as a fuzzy relation, several useful operations, such as the support, core, and α-cut defined in Section 3.3.2, become available. The support of the fuzzy relation identifies all pairs of texts that share at least some degree of similarity, the core identifies all pairs that are fully similar, and the α-cut retains only those pairs whose similarity meets or exceeds a specified threshold. The α-cut is particularly relevant for the evaluation of a similarity measure, as it allows the computed similarity scores

to be compared against a crisp ground truth for varying levels of strictness. We demonstrate this evaluation procedure in Section 3.5.

3.3.4 Fuzzy Graphs and Fuzzy Labelled Property Graphs

In Chapter 2, we introduced the LPG model as the formal foundation for the graphs and graph databases used throughout this work. In this subsection, we extend this structure with the notion of fuzzy edges to accommodate the storage of similarity scores within the graph.

A fuzzy graph, as proposed by Rosenfeld [56], is a graph $\tilde{G}\big(N, \tilde{E}\big)$ where N is a set of nodes and $\tilde{E}$ is a fuzzy relation over $N \times N$ representing the edges between nodes, with a symmetric membership function $\mu_{\tilde{E}}$ [53, 55]. The connectedness of a fuzzy graph is defined in terms of its underlying crisp graph $G(N, E)$, where

$$E = \text{supp}\big(\tilde{E}\big) = \big\{(n_1, n_2) \mid (n_1, n_2) \in N \times N \wedge \mu_{\tilde{E}}(n_1, n_2) > 0\big\}. \tag{3.13}$$

The α-cut on the edges of a fuzzy graph retains all edges with a membership grade that meets or exceeds the α-threshold,

$$\tilde{E}_\alpha = \big\{(n_1, n_2) \mid (n_1, n_2) \in E \wedge \mu_{\tilde{E}}(n_1, n_2) \geq \alpha\big\}. \tag{3.14}$$

Incorporating fuzzy edges into the LPG model defined in Definition 2.1 requires extending the edge set to a fuzzy set. A formal definition of the resulting fuzzy LPG is provided in Definition 3.1.

> **Definition 3.1** (Fuzzy Labelled Property Graph [57]). A fuzzy Labelled Property Graph is an extension of the LPG model from Definition 2.1, denoted $\tilde{G}\big(N, \tilde{E}, \rho, \lambda_N, \lambda_E, \sigma\big)$, with the following additional properties:
>
> - $\tilde{E}$ is a fuzzy set over the edge set E of the underlying LPG, with membership function $\mu_{\tilde{E}}(e)$ assigning a grade to each edge $e \in E$, so that $E = \text{supp}\big(\tilde{E}\big)$.
> - For a given $\alpha \in \,]0, 1]$, the α-cut over $\tilde{G}$ yields a crisp LPG $G\big(N, \tilde{E}_\alpha, \rho, \lambda_N, \lambda_E, \sigma\big)$, where $\tilde{E}_\alpha = \big\{e \mid e \in E \wedge \mu_{\tilde{E}}(e) \geq \alpha\big\}$.

In practice, the membership grade of a fuzzy edge can be stored as a property on the edge in a graph database system. Crisp edges, for which the membership grade is implicitly 1, do not require this additional property.

The graph-based text representation introduced in Definition 2.2 is a special case of a fuzzy LPG in which all edges are crisp, that is, $\mu_{\tilde{E}}(e) = 1$ for all $e \in E_{\texttt{CONTAINS}}$. The same holds for other crisp edges in the graph, such as those representing relationships between texts and metadata nodes. This observation confirms that the text model from Chapter 2 is fully compatible with the fuzzy LPG framework, allowing the model to be extended with fuzzy edges representing the similarity scores between text elements, as proposed in Section 3.4.

3.4 Hierarchical Orthographic Similarity Measure

With the graph-based text representation from Chapter 2 in place, we now turn to the computation of similarity scores between the textual elements in the graph. The proposed similarity measure operates in a bottom-up

manner, starting from the lowest-level textual elements and progressing upward through the hierarchy, in a process analogous to the construction of the textual graph.

In Section 3.4.1, we first present a string-based similarity measure that computes pairwise similarity scores between all nodes representing the smallest textual components in the graph. Next, in Section 3.4.2, we describe a node-based similarity measure that hierarchically determines the similarity between all nodes at the same level by incorporating the similarity scores computed at lower levels in a preceding step. As a result, the graph-based text model is extended with edges that encode the similarity between all comparable textual elements, from the lowest level up to the complete texts. Finally, Section 3.4.3 presents the full hierarchical computation and revisits the small textual corpus example from Chapter 2. A graphical overview of the entire process is provided in Figure 3.2.

3.4.1 String-Based Similarity Measure

The first step in the hierarchical computation is the calculation of pairwise similarity between all nodes representing the smallest textual components in the graph. Since these nodes store the actual textual content, we employ a character-based similarity measure founded on the edit distance between two strings. Given the hierarchical decomposition H as defined in Section 2.4.1, this step operates on all node pairs at the lowest level h_0.

For each pair of nodes $(n_1, n_2) \in N_{h_0}^2$, the edit distance between the two character strings they store is computed. A formal definition of this string-based edit distance, which is inspired by the Damerau-Levenshtein distance [24], is provided in Definition 3.2.

Definition 3.2 (String-Based Edit Distance). The string-based edit distance between two character strings a and b is recursively defined by the function $d^s_{a,b}(i,j)$, whose value at (i,j) is the minimum cumulative cost of transforming the first i characters of a into the first j characters of b. The supported edit operations are the deletion or insertion of a single character, the replacement of one character by another, and the transposition of two consecutive characters.

$$d^s_{a,b}(i,j) = \begin{cases} i \cdot c_D & \text{, if } j = 0 \\ j \cdot c_I & \text{, if } i = 0 \\ \min \begin{pmatrix} d^s_{a,b}(i-1,j) + c_D, \\ d^s_{a,b}(i,j-1) + c_I, \\ d^s_{a,b}(i-1,j-1) + \mathbb{1}_{(a_i \neq b_j)} c_R, \\ d^s_{a,b}(i-2,j-2) + c_T \end{pmatrix} & \text{if } (i>1) \wedge (j>1) \\ & \wedge\ (a_i = b_{j-1}) \\ & \wedge\ (a_{i-1} = b_j) \\ \min \begin{pmatrix} d^s_{a,b}(i-1,j) + c_D, \\ d^s_{a,b}(i,j-1) + c_I, \\ d^s_{a,b}(i-1,j-1) + \mathbb{1}_{(a_i \neq b_j)} c_R \end{pmatrix} & \text{, otherwise} \end{cases} \tag{3.15}$$

The first and second cases are the base cases of the recursion, where one of the strings is empty and the edit distance equals the length of the other string multiplied by the cost of a deletion or an insertion, respectively.

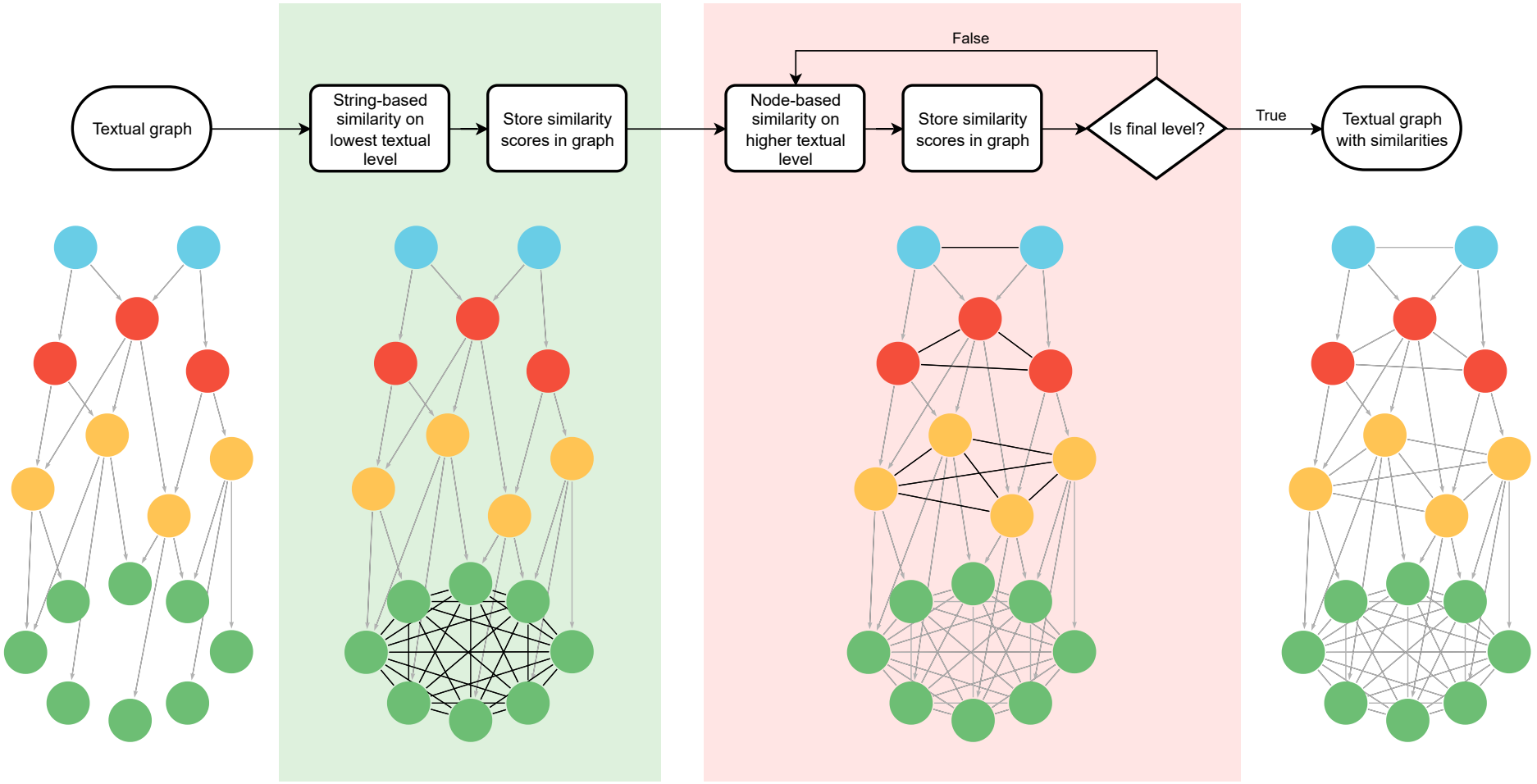


Figure 3.2: A graphical overview of the steps involved in computing the hierarchical similarity measure between textual components in the graph model. Starting from a textual corpus represented by the graph model from Chapter 2, a string-based score is calculated between the text values stored in the nodes representing the smallest textual components (green section). The pairwise scores between all other nodes at the same level are then computed hierarchically using a node-based similarity measure (red section). The result is a textual graph extended with similarity scores between all textual components at every level of the hierarchy.

The four terms in the third case correspond to a deletion, an insertion, a replacement, and a transposition. The fourth case omits the transposition term, as the condition for a transposition between two consecutive characters is not met. The cost of each edit operation is denoted by $c_* \in [0, 1]$, where $*$ stands for one of $\{D, I, R, T\}$ and $c_D = c_I$ to ensure symmetry. The indicator function $\mathbb{1}_{(a_i \neq b_j)}$ equals 1 when $a_i \neq b_j$ and 0 otherwise, ensuring that no replacement cost is incurred when the characters at positions i and j are identical.

The cost of the insertion and deletion operations must be equal to ensure a symmetric edit distance [58]. When all costs are equal to 1, this edit distance corresponds to the restricted variant of the Damerau-Levenshtein edit distance, in which no substring is edited more than once [24, 59]. It is this restriction that permits the recursive formulation above, since the transposition term returns to $d^s_{a,b}(i-2, j-2)$ and thereby excludes any further operation on the two transposed characters. The unrestricted variant lifts this condition, at the price of a considerably less straightforward computation. It is also known that the restricted variant does not satisfy the triangle inequality and is therefore not a metric [59]. The restricted variant is nonetheless adopted here, since the techniques developed in this work do not rely on the metric properties of the distance, as discussed in Section 3.3.1. The two variants can moreover only diverge from the unrestricted variant when a transposition is followed by a further edit of the same characters, which requires at least three edit operations concentrated on the same short sequence and is uncommon in practice. A cost of 0 allows the distance between two distinct strings to be 0, so that such strings are considered fully similar [60]. The costs of the edit operations can be set according to the specific requirements of the application, based on expert knowledge or empirical analysis of the data at hand.

Let $s_1 = \sigma(n_1, \texttt{text})$ and $s_2 = \sigma(n_2, \texttt{text})$ denote the strings stored in the nodes n_1 and n_2. The edit distance between these strings is found by the recursive application of Equation (3.15) and results in a non-negative real number between 0 and $\max(|s_1|, |s_2|)$, the maximal length of the two strings. The resulting value depends on the assigned costs for each supported edit operation. As edit distances indicate dissimilarity rather than similarity between two strings, Equation (3.16) is used to convert this dissimilarity into a similarity score within the required bounds $[0, 1]$. Since the edit distance between a string and itself is 0, the resulting similarity measure is reflexive, and since both the edit distance and the conversion to a similarity score are symmetric, it is symmetric as well. Transitivity, however, is not guaranteed, as the underlying edit distance does not satisfy the triangle inequality.

$$\mathrm{sim}^s(s_1, s_2) = 1 - \frac{d^s_{s_1,s_2}(|s_1|, |s_2|)}{\max(|s_1|, |s_2|)} \tag{3.16}$$

The calculation of this similarity measure is illustrated in Example 3.1.

Example 3.1. Consider two strings $s_1 = \texttt{RELEVANT}$ and (conveniently misspelled) $s_2 = \texttt{ELEPHATN}$. For the sake of the example, all costs are set to $c_I = c_D = c_R = c_T = 1$. The minimal edit distance between these two strings requires exactly one application of each of the four supported edit operations, as

shown in Figure 3.3, and is $d^s_{s_1,s_2}(|s_1|,|s_2|) = 4$. The similarity score, determined by Equation (3.16), is $\text{sim}^s(s_1, s_2) = 1 - \frac{4}{8} = 0.5$.

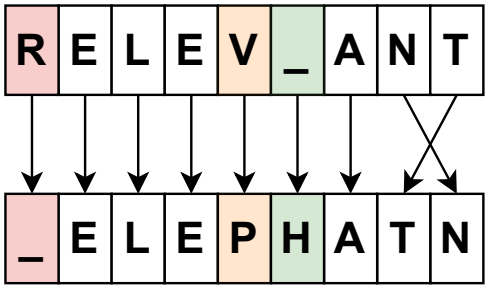


Figure 3.3: A visual representation of the edit operations required to transform the string RELEVANT into the string ELEPHATN. Deletions are indicated in red, replacements in orange, insertions in green, and transpositions by a pair of crossing arrows.

The time complexity of a single edit distance calculation between two strings is $\mathcal{O}(|s_1| \cdot |s_2|)$. Since the string-based edit distance is computed on the lowest-level textual elements, the strings under consideration are often short, resulting in manageable execution times. The time complexity of calculating the edit distance on all nodes in N_{h_0} is $\mathcal{O}(m^2)$, with $m = \left|N_{h_0}\right|$, which can result in performance issues for a very large corpus. However, since exact string matches are stored in the same nodes through the node-merging process described in Section 2.4.2, the number of nodes at the lowest level of the hierarchy is often significantly smaller than the total number of lowest-level strings across all texts in the corpus, reducing the computational burden.

Computing the edit distance between two strings s_1 and s_2 typically requires a dynamic programming matrix of dimensions $(|s_1|+1) \times (|s_2|+1)$, which stores the intermediate distances between all pairs of prefixes. The implementation can, however, be optimised to store the intermediate results using only three rows of length $\min(|s_1|,|s_2|)+1$, since we only ever have to look back two steps in case of a transposition. As a result, the space complexity of a single edit distance calculation is $\mathcal{O}(\min(|s_1|,|s_2|))$. The space required to store all calculated similarities between the nodes in N_{h_0} is $\mathcal{O}(m^2)$, where m is equal to the number of nodes at that base level of the hierarchy.

Finally, after calculating the similarity scores between all node pairs $(n_1, n_2) \in N^2_{h_0}$, these similarities are incorporated into the graph. In accordance with Definition 3.1, a new edge e is created with label $\lambda_E(e) =$ SIMILAR_TO for each computed similarity score. The membership grade for this edge is set to the obtained similarity score:

$$\mu_{\tilde{E}}(e) = \text{sim}^s(\sigma(n_1, \texttt{text}), \sigma(n_2, \texttt{text})). \tag{3.17}$$

This edge is then associated with the related nodes $\rho(e) = (n_1, n_2)$. Since this similarity measure is symmetric, this edge can be interpreted as an undirected edge and is only stored once.

3.4.2 Node-Based Similarity Measure

The next step in the hierarchical computation is the calculation of pairwise similarity scores between higher-level text elements. The computation follows the order of the decomposition H defined in Section 2.4.1, starting with

all pairs $(n_1, n_2) \in N^2_{h_1}$. This process is repeated for every subsequent level until the similarities between all node pairs $(n_1, n_2) \in N^2_{h_{|H|-1}}$ representing complete texts are determined. The node-based similarity measure used at these levels is a novel contribution of this work. It is also based on the edit distance principle but diverges from the string-based edit distance defined in Definition 3.2 in three respects, each described in the subsections below.

Child Node Sequences

A first difference compared to the string-based edit distance is that the node-based edit distance operates on the ordered list of child nodes of a given node rather than on the characters in a string. Consider a node $n \in N_{h_i}, 0 < i < |H|$. The edit distance is calculated on the ordered list of child nodes of n,

$$\text{child}(n) = K_n = [k_0, k_1, \dots, k_m], \tag{3.18}$$

where $k_j \in N_{h_{i-1}}$ and $\rho(e_j) = (n, k_j)$, with $e_j \in \text{supp}\left(\tilde{E}_{\texttt{CONTAINS}}\right)$ and $j = \sigma(e_j, \texttt{rank})$. Additionally, the number of children of n, denoted $|\text{child}(n)| = m + 1$, is equal to the number of relationships in $\tilde{E}_{\texttt{CONTAINS}}$ between the node n and individual elements of $N_{h_{i-1}}$.

Fuzzy Transposition Matching

A second difference concerns how we determine whether a transposition between two consecutive elements occurs. In contrast to the string-based edit distance, where a transposition is detected only when two consecutive characters match exactly, the node-based edit distance employs a fuzzy match according to a fixed threshold value $t \in [0, 1]$. Let $n_1, n_2 \in N_{h_i}, 0 < i < |H|$, be two nodes. A fuzzy match between these nodes occurs when

$$\text{match}(n_1, n_2) \equiv \mu_{\tilde{E}}(e) \geq t, \tag{3.19}$$

where $e \in \text{supp}\left(\tilde{E}_{\texttt{SIMILAR_TO}}\right)$ is the edge between n_1 and n_2, with $\rho(e) = (n_1, n_2)$.

This fuzzy transposition matching allows for the detection of transpositions even when the child nodes involved are not fully similar, as long as their similarity meets or exceeds the specified threshold t. This is particularly useful when the textual elements represented by the child nodes exhibit minor differences, such as typographical errors or variations in formatting, but remain similar enough to be treated as a transposition. The threshold is a parameter of the level at which the comparison takes place, so that a distinct value may be chosen for every level of the hierarchy while remaining fixed for all comparisons within a given level.

Adaptive Replacement Costs

A third and final distinction from the string-based edit distance lies in how the cost of a replacement is determined. In the string-based edit distance, a replacement between two characters has a fixed cost c_R. In our node-based edit distance, the replacement cost is instead adaptive: it equals the dissimilarity between the two child nodes involved in the replacement, computed as 1 minus their similarity. Additionally, a similarity threshold

$r \in [0, 1]$ is introduced to accommodate orthographic inconsistencies in a corpus. When the similarity between two child nodes meets or exceeds r, the nodes are considered fully similar, resulting in a replacement cost of 0.

Let $n_1, n_2 \in N_{h_i}, 0 < i < |H|$, be two nodes. The cost of a replacement between these two nodes is determined by:

$$c_R(n_1, n_2) = 1 - (r \Rightarrow_{Go} \mu_{\tilde{E}}(e)), \tag{3.20}$$

where $\rho(e) = (n_1, n_2)$ and $e \in \text{supp}\left(\tilde{E}_{\texttt{SIMILAR_TO}}\right)$. The similarity threshold r is enforced by means of the Gödel R-implication [61], which is defined by

$$p \Rightarrow_{Go} q = \begin{cases} 1 & \text{if } p \leq q \\ q & \text{otherwise.} \end{cases} \tag{3.21}$$

In effect, when the similarity $\mu_{\tilde{E}}(e)$ meets or exceeds the threshold r, the Gödel implication evaluates to 1, yielding a replacement cost of 0; otherwise, the replacement cost equals $1 - \mu_{\tilde{E}}(e)$. Like the fuzzy matching threshold, r is a parameter of the level at which the comparison takes place, and may therefore be chosen independently for every level while remaining fixed within it.

Node-Based Similarity Calculation

Building on the string-based edit distance from Section 3.4.1 and the three adaptations introduced above, we now define the node-based edit distance in Definition 3.3. This edit distance is computed as the minimum cumulative cost of the edit operations required to transform a text element into another, where each text element is represented by its ordered list of child nodes.

Definition 3.3 (Node-Based Edit Distance). The node-based edit distance between two ordered lists of nodes a and b is recursively defined by the function $d^n_{a,b}(i, j)$, whose value at (i, j) is the minimum cumulative cost of transforming the first i nodes of a into the first j nodes of b using the following edit operations: the deletion or insertion of a single node, the replacement of one node by another, and the transposition of two consecutive nodes.

$$d^n_{a,b}(i,j) = \begin{cases} i \cdot c_D & \text{, if } j = 0 \\ j \cdot c_I & \text{, if } i = 0 \\ \min \begin{pmatrix} d^n_{a,b}(i-1,j) + c_D, \\ d^n_{a,b}(i,j-1) + c_I, \\ d^n_{a,b}(i-1,j-1) + \mathbb{1}_{(a_i \neq b_j)} \cdot c_R(a_i, b_j), \\ d^n_{a,b}(i-2,j-2) + c_T \end{pmatrix} & \text{if } (i > 1) \wedge (j > 1) \\ & \wedge \text{match}(a_i, b_{j-1}) \\ & \wedge \text{match}(a_{i-1}, b_j) \\ \min \begin{pmatrix} d^n_{a,b}(i-1,j) + c_D, \\ d^n_{a,b}(i,j-1) + c_I, \\ d^n_{a,b}(i-1,j-1) + \mathbb{1}_{(a_i \neq b_j)} \cdot c_R(a_i, b_j) \end{pmatrix} & \text{, otherwise} \end{cases} \tag{3.22}$$

The first and second cases are the base cases of the recursion, where one of the lists is empty and the edit distance equals the length of the other list multiplied by the cost of a deletion or an insertion, respectively. The four terms in the third case correspond to a deletion, an insertion, a replacement, and a transposition. The fourth case omits the transposition term, as the condition for a transposition between two consecutive nodes is not met. The predefined costs for deletion, insertion, and transposition are denoted by $c_D, c_I, c_T \in [0, 1]$, respectively, with $c_D = c_I$ to ensure symmetry. The similarity threshold $r \in [0, 1]$ is used to determine the cost of a replacement between two nodes as defined by Equation (3.20), and the fuzzy matching threshold $t \in [0, 1]$ determines whether a transposition between two consecutive nodes is recognised, as defined by Equation (3.19).

This node-based edit distance inherits the restricted formulation of its string-based counterpart, since the transposition term likewise returns to $d^n_{a,b}(i-2, j-2)$, and is therefore not a metric either. A replacement cost of 0 moreover arises whenever the similarity between two distinct child nodes meets the threshold r, so that two distinct lists of nodes can be at distance 0 and are consequently considered fully similar. This is intended rather than incidental, as the threshold is what accommodates the orthographic inconsistencies discussed in Section 3.4.2. The two thresholds, like the edit operation costs, can be set according to the specific requirements of the task at hand.

Let $n_1, n_2 \in N_{h_i}, 0 < i < |H|$, be two nodes, and let $K_{n_1} = \text{child}(n_1)$ and $K_{n_2} = \text{child}(n_2)$ denote their respective ordered lists of child nodes. The pairwise similarity scores between these child nodes have been precomputed at the preceding level. As with the string-based edit distance, the node-based edit distance is a measure of dissimilarity and must be converted into a similarity measure. This conversion is achieved by applying Equation (3.23), which, like its string-based counterpart, is symmetric.

$$\text{sim}^n(n_1, n_2) = 1 - \frac{d^n_{K_{n_1}, K_{n_2}}\big(\big|K_{n_1}\big|, \big|K_{n_2}\big|\big)}{\max\big(\big|K_{n_1}\big|, \big|K_{n_2}\big|\big)} \tag{3.23}$$

We now establish that the node-based similarity measure satisfies the reflexivity and symmetry properties specified by Equations (3.2) and (3.3).

Proposition 3.1. The node-based similarity in Equation (3.23) is reflexive:

$$\text{sim}^n(n, n) = 1, \forall n \in N_{h_i}, 0 < i < |H|$$

Proof. Let $K_n = \text{child}(n)$ denote the ordered sequence of children of the node n. The node-based edit distance in Equation (3.22) results in $d^n_{K_n, K_n}(|K_n|, |K_n|) = 0$ since no transformations are required to transform K_n into itself. It follows that Equation (3.23) results in a similarity score of 1. □

Proposition 3.2. The node-based similarity in Equation (3.23) is symmetric:

$$\text{sim}^n(n_1, n_2) = \text{sim}^n(n_2, n_1), \forall n_1, n_2 \in N_{h_i}, 0 < i < |H|$$

Proof. Proof by induction on i, which corresponds to the level in the hierarchy.

Base step.
At level $i = 0$, similarity is determined by the string-based measure (Equation (3.16)), whose underlying edit distance is symmetric by design [58]. Since the max operator is also symmetric, it follows that the string-based similarity is symmetric, which settles the base case.

Inductive step.
Assume that the similarity measure at level $i \geq 0$ is symmetric. We prove that the node-based similarity at level $i + 1$ is also symmetric. Since the max operator is symmetric, it suffices to prove that the underlying node-based edit distance defined by Equation (3.22) is symmetric. An edit distance is symmetric when it has a reverse operation for every edit operation with the same cost [58]. Insertions and deletions are reverse operations of each other and have the same fixed cost, $c_I = c_D$, as stated in Definition 3.3. Replacements, on the other hand, are their own reverse operations. The cost of replacements, as shown by Equation (3.20), depends on the similarity scores between nodes at level i. Since the similarity measure at level i is symmetric, it follows that the costs of reversed replacement operations for the same child nodes are equal. Finally, transpositions always have a fixed cost and are the reverse operations of themselves if their fuzzy match operator is symmetric. Because this operator, as stated in Equation (3.19), is fully determined by the similarity scores at level i, it is also symmetric. Since each of the four edit operations has a reverse operation with equal cost, the node-based edit distance is symmetric, and consequently the node-based similarity is also symmetric. □

Much like its string-based counterpart, the node-based similarity measure is not guaranteed to be transitive, as the underlying edit distance does not satisfy the triangle inequality. Example 3.2 now illustrates the calculation of the measure.

Example 3.2. Consider two texts that have been preprocessed by transforming all characters to lowercase and removing punctuation, resulting in the set

$$T = \{\text{“this is a } \textbf{test} \text{ sentence too”}, \text{“is this a } \textbf{tset} \text{ sentence”}\},$$

with a hierarchical decomposition $H = [\texttt{Word}, \texttt{Sentence}]$. The texts differ in three respects: the first two words are reordered, the word “test” is intentionally misspelled to illustrate the effect of orthographic variation, and the first text ends with an additional word. The graph-based text representation for this small corpus is displayed in Figure 3.4, where the similarity scores between the `Word` nodes have already been computed using the string-based measure from Section 3.4.1. All edit operation costs and both thresholds (r and t) are set to 1 at every level of the hierarchy.
To calculate the similarity between the two `Sentence` nodes, we first determine the edit distance between their ordered lists of child nodes. Based on the edit operation costs, the minimum edit distance is obtained by three operations, corresponding to the three differences noted above:

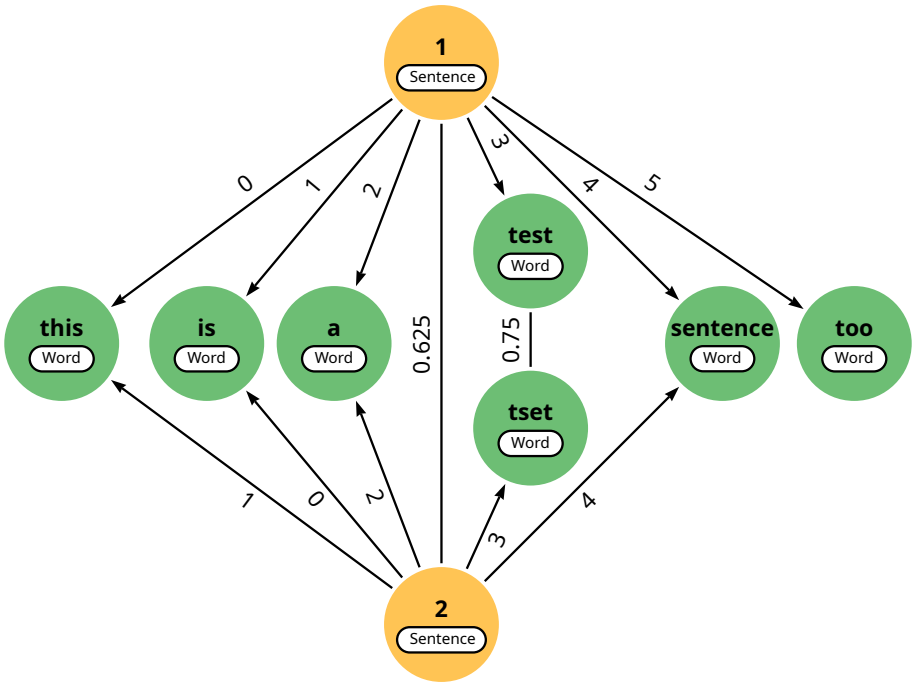


Figure 3.4: The graph-based representation of the two texts in Example 3.2. The yellow nodes represent `Sentence` nodes and show their `id` property, whereas the green nodes represent the `Word` nodes and show their `text` property. The values of the `rank` property are shown on the directed edges between different levels of the graph, and the `similarity` values are shown on the undirected edges between word nodes at the same level. Only the relevant similarity scores are displayed: the similarity between the words "test" and "tset", and the similarity between the sentence nodes for the case $r = 1$.

(i.) a transposition between the first two nodes with a cost of 1,

(ii.) a replacement of the word "test" by the word "tset" with a cost equal to the dissimilarity between those words (i.e., $1 - 0.75 = 0.25$), and

(iii.) an insertion or deletion of the last node, depending on the direction of the transformation, with a cost of 1.

As a result, the edit distance equals 2.25 and the similarity, determined by Equation (3.23), equals $1 - \frac{2.25}{6} = 0.625$.

Consider an alternative scenario where the similarity threshold r is 0.7, leading to a replacement cost of 0 between the words "test" and "tset" as their similarity 0.75 exceeds r. In this case, the edit distance equals 2 and the similarity between the two sentences equals $1 - \frac{2}{6} \approx 0.667$. The graph-based text representation for this second scenario is displayed in Figure 3.5.

The time complexity of a single node-based edit distance calculation between nodes n_1 and n_2 depends on the lengths of both lists of child nodes and equals $\mathcal{O}(|\text{child}(n_1)| \cdot |\text{child}(n_2)|)$. In practice, choosing a hierarchical decomposition H that produces child-node lists of limited length keeps the cost of individual edit distance computations low. As with the string-based similarity measure, the time complexity of computing all pairwise similarities between nodes in N_{h_i}, with $0 < i < |H|$ and the number of nodes $m = \left|N_{h_i}\right|$, is $\mathcal{O}(m^2)$.

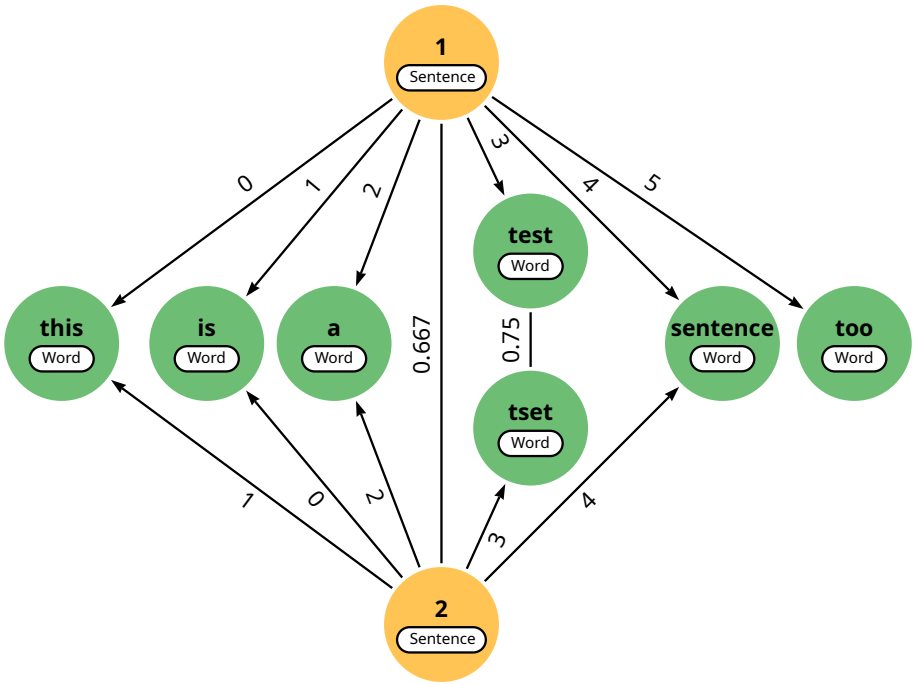


Figure 3.5: The same representation as in Figure 3.4, for the alternative scenario in which the similarity threshold r equals 0.7. The replacement cost between the words "test" and "tset" is now 0, which raises the similarity between the sentence nodes.

The space complexity of the node-based edit distance follows the same pattern as that of the string-based edit distance, as described in Section 3.4.1, except that the dimensions of the intermediate matrix correspond to the lengths of two child-node lists rather than the lengths of two strings.

3.4.3 Full Hierarchical Similarity Calculation

With the string-based and node-based components defined, we now present the full hierarchical similarity computation. The procedure for computing pairwise similarity scores between all textual elements, up to and including the full texts, is provided in Algorithm 3.1.

The algorithm hierarchically computes pairwise similarity scores for all node pairs sharing the same label, using the string-based measure from Section 3.4.1 at the lowest level and the node-based measure from Section 3.4.2 at all subsequent levels. The parameter values are supplied as input, namely c_D, c_I, c_R, and c_T for the string-based similarity at the lowest level, and c_D, c_I, c_T, r, and t for the node-based similarity at all subsequent levels. They may vary between levels of the hierarchy, but remain fixed throughout the calculations within a specific level, so every parameter at every level can be tuned independently. Tuning may rely on expert knowledge of the corpus, or on empirical analysis in which the parameters are optimised against the performance of the similarity measure in a downstream task. Each computed similarity score is stored as a new edge in the graph, as described in Sections 3.4.1 and 3.4.2. Since the measure is symmetric, only a single undirected edge is created between each pair of nodes. On completion of the procedure, the graph-based text representation is returned, extended with similarity scores between all comparable textual elements at every

Algorithm 3.1 Hierarchical computation of pairwise similarity scores in the graph-based text representation.

Input: A fuzzy graph $\tilde{G}\big(N, \tilde{E}, \rho, \lambda_N, \lambda_E, \sigma\big)$ representing the texts in a corpus
Input: The parameter values $c_D, c_I, c_R,$ and c_T for level h_0
Input: The parameter values $c_D, c_I, c_T, r,$ and t for every level $h_i, 0 < i < |H|$
Output: The fuzzy graph $\tilde{G}$ with the calculated similarity scores

for $i \leftarrow 0$ **to** $|H| - 1$ **do**
  Load the parameter values for level h_i
  for all $(n_a, n_b) \in N_{h_i}^2$ with $a < b$ **do**
    if $i = 0$ **then**
      $s = \mathsf{sim}^s(\sigma(n_a, \texttt{text}), \sigma(n_b, \texttt{text}))$
    else
      $s = \mathsf{sim}^n(n_a, n_b)$
    end if
    Create edge e
    $\rho(e) \leftarrow (n_a, n_b)$
    $\lambda_E(e) \leftarrow \texttt{SIMILAR_TO}$
    $\tilde{E} \leftarrow \tilde{E} \cup \{e\}$
    $\mu_{\tilde{E}}(e) \leftarrow s$
  end for
end for
return $\tilde{G}\big(N, \tilde{E}, \rho, \lambda_N, \lambda_E, \sigma\big)$

level of the hierarchy. As an illustration, the result of the hierarchical computation between two texts is provided in Example 3.3.

> **Example 3.3.** In Figure 3.6, we revisit the textual graph from Example 2.1 and display the similarity scores between all textual elements that are required to compute the similarity between the two full texts. For this example, all parameters c_D, c_I, c_R, c_T, r, and t are set to 1 at every level. The similarity between the two sentences with `rank` 1 is determined by two replacements: one between the words “first” and “second”, with a dissimilarity of $1 - 0 = 1$, and one between the words “analyse” and “analyze”, with a dissimilarity of $1 - 0.86 = 0.14$. This yields a sentence similarity of $1 - \frac{1.14}{5} \approx 0.77$. The similarity between the two texts, computed hierarchically from the word-level scores through the sentence level, is obtained from a single replacement between the second sentences of the two texts, with a cost equal to their dissimilarity, $1 - 0.77 = 0.23$. This yields a final text-level similarity of $1 - \frac{0.23}{2} \approx 0.89$.

The total time required to compute similarities across all levels of the hierarchy is the sum of the computation times at each individual level. In Sections 3.4.1 and 3.4.2, we showed that the time required at each level i is $\mathcal{O}(n_i^2)$, where $n_i = \left|N_{h_i}\right|$ and $0 \leq i < |H|$. Treating the number of hierarchical levels $|H|$ as a constant, the overall time complexity is $\mathcal{O}(\max_i n_i^2)$.

The space complexity of the complete algorithm is determined by the largest space requirement among the different hierarchical levels. As a result, the space complexity is also $\mathcal{O}(\max_i n_i^2)$.

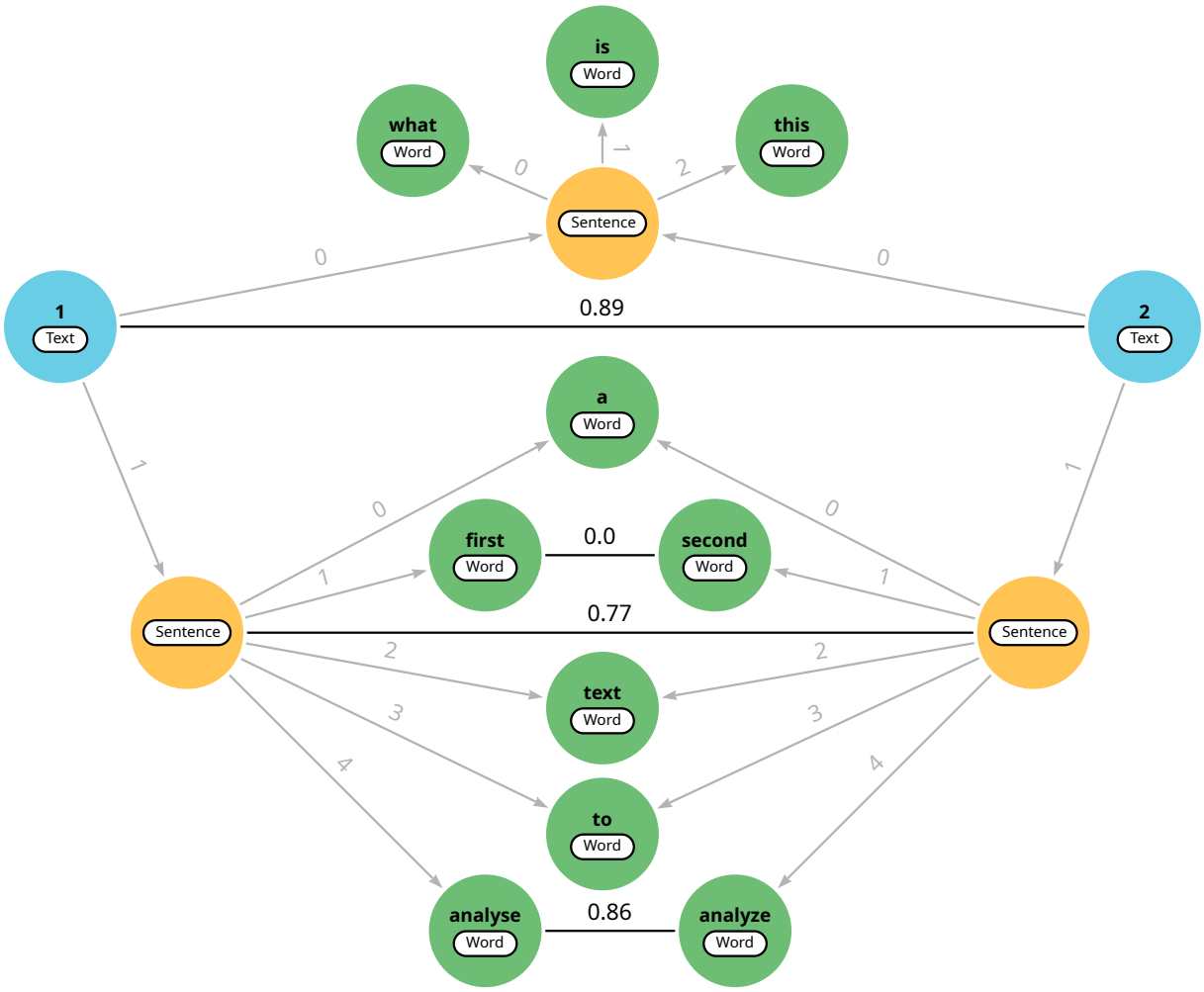


Figure 3.6: Subset of the graph-based text representation for the texts in Example 3.3 after computing the similarities between all elements. `Word` nodes are shown in green and display their `text` property; `Sentence` nodes are shown in yellow; `Text` nodes are shown in blue and display their numeric `id` property. The values of the `rank` property are shown on the directed edges (grey) between different levels of the graph, and the `similarity` values are shown on the undirected edges (black) between same-level textual elements. For clarity, only the similarities relevant to the minimum edit distance computation between the two texts are displayed, namely those between nodes from different texts whose similarity score is below 1.

After the initial computation of similarity scores, additional texts can be added to the graph. The new texts are first incorporated into the graph as described in Section 2.4. The similarity scores then need to be computed only for pairs involving at least one newly added node: pairs between two new nodes, and pairs between a new node and an existing node at the same hierarchical level. Existing pairs do not need to be recomputed, which makes incremental updates considerably cheaper than the initial construction. With the hierarchical similarity measure now fully specified, we turn to its experimental evaluation in Section 3.5.

3.5 Experiments

To evaluate the effectiveness of the proposed hierarchical similarity measure, we conduct experiments using the Database of Byzantine Book Epigrams (DBBE) corpus [18, 19]. The computed similarity scores are used to identify pairs of similar texts, which are then compared against the pairwise relationships established from expert-curated groupings of similar epigrams and verses. This comparison serves to assess the quality of the measure across a range of parameter configurations and thresholds. The results are further situated against an existing two-level string-matching baseline. Section 3.5.1 describes the dataset used in the experiments. Section 3.5.2 presents the experimental setup, including the methods under comparison, the parameter configurations, and the evaluation methodology. Section 3.5.3 reports and discusses the results. Finally, Section 3.5.4 demonstrates the practical applicability of the similarity-enriched graph in a visual graph database environment.

3.5.1 Dataset

The experiments in this section are conducted on the DBBE corpus of Byzantine book epigrams [18, 19], introduced in Section 1.3: a collection of short poems characterised by extensive orthographic variation and dense interconnections between texts. Following Section 2.6, the epigrams are decomposed according to the hierarchical decomposition $H = [\texttt{Word}, \texttt{Verse}, \texttt{Epigram}]$. They subsequently undergo the preprocessing steps described in Section 2.6.1, including case standardisation, removal of non-Greek characters and diacritical marks, whitespace normalisation, and the systematic replacement of vowels and diphthongs affected by itacism.

The DBBE contains two categories of expert-curated groupings that serve as ground truth for evaluating the proposed similarity measure. Epigram types group similar epigrams, while verse variants group similar individual verses across different epigrams [18]. Because these groupings are manually composed by domain experts, they may include texts that are only partially similar or that differ at the semantic level, for instance through the use of synonyms. The ground truth therefore does not exclusively reflect orthographic similarity. It is also defined as a crisp partition: each text belongs to at most one group, with no indication of the degree of similarity between texts within a group.

For the experiments, a representative subset of each category is selected in collaboration with domain experts and exported from the DBBE corpus. The subset of verses consists of the verses contained in 32 verse variant groups with identifiers 445, 2520, 3949, 4636, 5354, 5549, 5840, 6342, 6353, 9033, 10671, 10869, 11233, 12121, 12122, 12251, 12290, 13035, 14667, 14802, 14852, 14866, 14872, 14932, 15261, 15278, 15327, 15377, 15470, 15652, 15681, and 15904, resulting in a subset of 750 verses. The subset of epigrams consists of the epigrams contained

in 21 epigram type groups with identifiers 1862, 1953, 1982, 2054, 2148, 2150, 2225, 2311, 2326, 3147, 3436, 3762, 3987, 4152, 4155, 4245, 5030, 5248, 6473, 6475, and 31190, resulting in a subset of 496 epigrams when only the epigrams with known text are considered. These identifiers correspond to the unique identifiers of the verse variant and epigram type groups in the DBBE relational database. With the dataset and ground truth defined, we now turn to the experimental setup.

3.5.2 Experimental Setup

In the experiments, two methods for computing textual similarity are compared. The first is the hierarchical orthographic similarity measure proposed in this chapter. The second is the Two-Level string matcher, a soft similarity measure that combines token-based and character-based similarity [20, 21]. Both methods are evaluated on the verse and epigram subsets described in Section 3.5.1.

Proposed Method

Before the similarity scores can be computed, the corpus must be transformed into a graph-based representation. For the epigram-level experiment, this representation is constructed according to Algorithm 2.1 with the hierarchical decomposition $H_{\texttt{Epigram}} = [\texttt{Word}, \texttt{Verse}, \texttt{Epigram}]$. For the verse-level experiment, the decomposition $H_{\texttt{Verse}} = [\texttt{Word}, \texttt{Verse}]$ is used instead. Both representations are constructed using the segmentation and preprocessing steps described in Section 3.5.1. Similarity scores are then computed using Algorithm 3.1. The resulting graphs are each stored in a separate Neo4j [62] database, and the similarity computation is performed using a Neo4j plugin[1] that accepts the parameters for each hierarchical level as input.

For each of the two subsets, three distinct similarity computations are performed, each with a different set of parameter values, as listed in Table 3.1. These parameter values are informed by the characteristics of the corpus and the expected degree of orthographic variation between similar texts, and are refined through preliminary tests on the subsets described above. Parameters at the `Word` level correspond to those of the string-based edit distance in Equation (3.15), while parameters at the `Verse` and `Epigram` levels correspond to those of the node-based edit distance in Equation (3.22). The first configuration uses default values of 1 for all parameters, whereas the second and third configurations incrementally reduce the costs for certain edit operations and lower the thresholds for similarity and transposition matching. These adjustments increase the tolerance for minor orthographic differences, which are prevalent in this corpus, allowing the measure to be tailored to its specific characteristics.

Two-Level String Matcher

The Two-Level string matcher computes the similarity between two texts by first transforming each text into a bag of tokens [20, 21]. Each token from the first bag is then compared with each token from the second bag by means of a token matcher, and the resulting pairwise token similarities are stored in a matrix. This matrix is used to compute a leximax-optimal assignment between tokens from both bags, that is, an assignment that

[1] The Java code for this Neo4j plugin can be found at https://github.com/MaximeDeforche/DBBESimilarity.

Table 3.1: Parameter values for the three configurations of the proposed similarity measure.

Level	Parameter	Default	Custom 1	Custom 2
`Word`	c_I	1.0	0.9	0.8
	c_D	1.0	0.9	0.8
	c_T	1.0	0.9	0.0
	c_R	1.0	1.0	1.0
`Verse`	c_I	1.0	0.9	0.8
	c_D	1.0	0.9	0.8
	c_T	1.0	0.0	0.0
	r	1.0	0.9	0.8
	t	1.0	0.8	0.7
`Epigram`	c_I	1.0	0.95	0.85
	c_D	1.0	0.95	0.85
	c_T	1.0	0.0	0.0
	r	1.0	0.9	0.8
	t	1.0	0.8	0.7

maximises the sequence of individual token similarities in lexicographic order. The resulting sequence of values is then aggregated into a final similarity score using a weighted minimum, where the weights are determined by a parameterised quantifier function. This two-step methodology, in which similarities are first computed at the token level and then at the bag level, makes the Two-Level string matcher a soft multi-level similarity measure as well.

Several properties of the Two-Level string matcher are relevant in the context of these experiments. First, the bag model does not preserve the order of tokens, so two texts differing only in token order receive a perfect similarity score. Second, the token matcher can account for spelling errors at the token level, for instance by using edit distances, though the default token matcher proposed by the authors is designed for speed and produces sparse similarity matrices [21]. Third, when two bags of tokens differ in cardinality, the quantifier function governs how this difference affects the final score; its choice can influence the effectiveness of the method significantly [20].

In the experiments, the Two-Level string matcher is applied to both subsets using two different token matchers. An implementation is available in Java as part of the ledc-framework[2] and can be found on GitLab[3]. The first token matcher is the one proposed by the original authors [21], which produces sparse similarity matrices and enables faster computation. In the context of Byzantine book epigrams, however, this sparsity may cause token pairs with non-negligible similarity to receive a score of zero. To address this limitation, the second token matcher uses the Damerau-Levenshtein distance [24], the same distance on which the string-based edit distance at the `Word` level in the proposed approach is founded. Distances exceeding 3 are mapped to a similarity of

[2] https://ledc.ugent.be/
[3] https://gitlab.com/ledc/ledc-match

0 to limit the computational cost. Preliminary tests indicated that the default parameters of the quantifier function are too restrictive for this corpus; setting the main quantifier parameter α to 0.9 in all configurations yielded improved results.

Evaluation Methodology

The results of both methods are expressed as a fuzzy similarity relation $\tilde{S}$, representing the pairwise similarity between all verses or all epigrams in the respective subsets. To compare the computed similarities against the crisp groupings in the ground truth, an α-cut is performed on $\tilde{S}$, as defined in Equation (3.9). Each pair of verses or epigrams retained after the α-cut is checked against the ground truth: if both elements belong to the same expert-curated group, the pair counts towards the True Positives (TP); otherwise, it counts towards the False Positives (FP). Similarly, pairs pruned by the α-cut count towards the False Negatives (FN) if they belong to the same group in the ground truth, and towards the True Negatives (TN) otherwise.

Based on these four counts, three quality measures are computed. Recall measures the fraction of similar pairs in the ground truth that are successfully identified,

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{3.24}$$

Precision quantifies the proportion of identified pairs that are indeed similar according to the ground truth,

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \tag{3.25}$$

The F_1-score is the harmonic mean of precision and recall and provides a single summary statistic that balances both,

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \tag{3.26}$$

These quality measures are computed for twenty distinct α values, ranging from 0.05 to 1 at intervals of 0.05, for each parameter configuration and subset. This results in a total of $20 \cdot 3 \cdot 2 = 120$ evaluations for the proposed method and $20 \cdot 2 \cdot 2 = 80$ for the Two-Level string matcher.

3.5.3 Results

This section presents and discusses the experimental results. We first analyse the results of the proposed similarity measure across its three parameter configurations, then discuss those of the Two-Level string matcher, and finally provide a comparative analysis of both methods.

Proposed Similarity Measure

Tables 3.2 and 3.3 present the precision, recall, and F_1-score for all three parameter configurations and α values of the proposed similarity measure on the verse and epigram subsets, respectively. The F_1-scores across all configurations are further visualised in Figure 3.7, and precision-recall curves for the default parameter configuration are shown in Figure 3.8.

Table 3.2: Experimental results for the verse subset using the proposed similarity measure. For each quality measure, the highest value across all parameter configurations and α values is highlighted in bold.

α	Default			Custom 1			Custom 2		
	Precision	Recall	$\mathbf{F_1}$-score	Precision	Recall	$\mathbf{F_1}$-score	Precision	Recall	$\mathbf{F_1}$-score
0.05	0.1458	**1.0000**	0.2544	0.1447	**1.0000**	0.2528	0.1447	**1.0000**	0.2528
0.10	0.1618	0.9990	0.2785	0.1455	**1.0000**	0.2540	0.1447	**1.0000**	0.2529
0.15	0.2339	0.9948	0.3787	0.1536	0.9999	0.2663	0.1455	**1.0000**	0.2540
0.20	0.4175	0.9932	0.5878	0.2112	0.9951	0.3485	0.1502	**1.0000**	0.2611
0.25	0.6867	0.9900	0.8109	0.3816	0.9939	0.5515	0.1861	0.9997	0.3138
0.30	0.8823	0.9831	0.9300	0.6899	0.9910	0.8135	0.3028	0.9948	0.4642
0.35	0.9477	0.9761	0.9617	0.8874	0.9872	0.9346	0.6256	0.9917	0.7672
0.40	0.9609	0.9715	0.9661	0.9518	0.9833	0.9673	0.8616	0.9895	0.9211
0.45	0.9688	0.9574	0.9631	0.9626	0.9755	0.9690	0.9516	0.9845	0.9678
0.50	0.9782	0.9417	0.9596	0.9718	0.9621	0.9669	0.9633	0.9777	**0.9704**
0.55	0.9952	0.9102	0.9508	0.9822	0.9402	0.9608	0.9726	0.9643	0.9684
0.60	0.9986	0.8688	0.9292	0.9978	0.8980	0.9453	0.9854	0.9343	0.9592
0.65	0.9996	0.8129	0.8966	0.9991	0.8543	0.9211	0.9988	0.9013	0.9475
0.70	**1.0000**	0.7452	0.8540	0.9999	0.7885	0.8817	0.9995	0.8576	0.9231
0.75	**1.0000**	0.6838	0.8122	**1.0000**	0.7307	0.8444	**1.0000**	0.7796	0.8762
0.80	**1.0000**	0.6072	0.7556	**1.0000**	0.6442	0.7836	**1.0000**	0.7065	0.8280
0.85	**1.0000**	0.4686	0.6382	**1.0000**	0.5128	0.6780	**1.0000**	0.5952	0.7462
0.90	**1.0000**	0.3405	0.5080	**1.0000**	0.3643	0.5340	**1.0000**	0.4253	0.5968
0.95	**1.0000**	0.2092	0.3460	**1.0000**	0.2216	0.3628	**1.0000**	0.2595	0.4121
1.00	**1.0000**	0.1069	0.1932	**1.0000**	0.1179	0.2110	**1.0000**	0.2329	0.3778

In the case of verses, a consistent trend is observed across all three parameter configurations. At low α values, recall approaches 1 while precision remains below 0.25, indicating that nearly all genuinely similar pairs are identified but at the cost of a large number of FPs. As α increases, precision rises sharply while recall decreases more gradually, leading to a region between $\alpha = 0.35$ and $\alpha = 0.60$ where both precision and recall are high and the F_1-score reaches its peak. This behaviour is visible in the left plot of Figure 3.7. The highest F_1-score of 0.9704 is achieved with the most tolerant parameter configuration (Custom 2) at $\alpha = 0.50$, where precision and recall are 0.9633 and 0.9777, respectively. At the same threshold, the Default configuration achieves an F_1-score of 0.9596 with a precision of 0.9782 and a recall of 0.9417. This result demonstrates that the measure performs well even without corpus-specific parameter tuning. Beyond $\alpha = 0.55$, precision rises to 1 at $\alpha = 0.75$ for all configurations, but recall declines steadily, resulting in progressively lower F_1-scores. The precision-recall curve for the verse subset in Figure 3.8 further illustrates this behaviour: precision remains high across a wide range of recall values and only begins to decrease as recall approaches 1. This pattern indicates strong discriminative performance on this subset.

Table 3.3: Experimental results for the epigram subset using the proposed similarity measure. For each quality measure, the highest value across all parameter configurations and α values is highlighted in bold.

α	Default			Custom 1			Custom 2		
	Precision	Recall	$\mathbf{F_1}$-score	Precision	Recall	$\mathbf{F_1}$-score	Precision	Recall	$\mathbf{F_1}$-score
0.05	0.0901	0.9984	0.1653	0.0798	**1.0000**	0.1478	0.0798	**1.0000**	0.1478
0.10	0.1639	0.9903	0.2812	0.0889	0.9998	0.1632	0.0798	**1.0000**	0.1478
0.15	0.3811	0.9850	0.5496	0.1530	0.9906	0.2650	0.0798	**1.0000**	0.1478
0.20	0.5444	0.9740	0.6984	0.3515	0.9829	0.5178	0.0998	0.9998	0.1815
0.25	0.5669	0.9596	0.7127	0.5404	0.9701	0.6942	0.2402	0.9902	0.3866
0.30	0.5809	0.9498	0.7209	0.5636	0.9574	0.7095	0.5155	0.9800	0.6756
0.35	0.5967	0.9324	0.7277	0.5837	0.9480	0.7226	0.5586	0.9680	0.7084
0.40	0.6129	0.9146	0.7340	0.5992	0.9318	0.7294	0.5727	0.9562	0.7163
0.45	0.6424	0.8959	0.7482	0.6161	0.9125	0.7356	0.5889	0.9413	0.7245
0.50	0.6794	0.8587	0.7586	0.6492	0.8929	0.7518	0.6041	0.9236	0.7305
0.55	0.7285	0.8224	0.7726	0.6873	0.8536	0.7615	0.6275	0.9076	0.7420
0.60	0.7895	0.7783	0.7839	0.7424	0.8123	0.7758	0.6555	0.8634	0.7452
0.65	0.8610	0.7279	0.7889	0.8271	0.7638	**0.7942**	0.7223	0.8300	0.7724
0.70	0.8797	0.6787	0.7662	0.8759	0.7187	0.7896	0.7255	0.7954	0.7588
0.75	0.8856	0.6186	0.7284	0.8830	0.6632	0.7574	0.8687	0.7289	0.7927
0.80	0.9071	0.5583	0.6912	0.8895	0.5977	0.7150	0.8834	0.6882	0.7736
0.85	0.9529	0.4854	0.6432	0.9353	0.5282	0.6751	0.8874	0.6290	0.7361
0.90	0.9895	0.3756	0.5446	0.9731	0.4354	0.6016	0.8873	0.5352	0.6677
0.95	0.9975	0.2883	0.4474	0.9925	0.3363	0.5024	0.9510	0.4973	0.6531
1.00	**1.0000**	0.1502	0.2612	0.9971	0.2805	0.4378	0.9507	0.4939	0.6500

For the epigram subset, the same general trend is observed, though the scores are lower than those for verses. This difference is attributable to two factors. First, epigrams exhibit greater structural variation than individual verses, as they may differ in the number, ordering, or composition of their constituent verses. Second, the longer texts increase the potential for orthographic inconsistencies to accumulate across multiple levels of the hierarchy, which might suggest that the similarity measure should be more tolerant at higher levels of the hierarchy. In practice, however, the results indicate that the intermediate tolerance of Custom 1, rather than the greater tolerance of Custom 2, is best suited to this level. The highest F_1-score of 0.7942 is achieved with this configuration at $\alpha = 0.65$, where precision is 0.8271 and recall is 0.7638. As shown in the right plot of Figure 3.7, the F_1-score curves for the epigram subset are flatter and peak at lower values than those for verses, reflecting the increased challenge of handling this subset. Compared to the verse subset, the peak F_1-score is reached at a higher α value, suggesting that a stricter threshold is required before the accumulated orthographic variation is filtered out. At lower α values, precision remains considerably lower than for verses. This is consistent with the greater difficulty of distinguishing structurally similar but non-matching epigrams from genuinely related ones. In Figure 3.8, the epigram curve confirms this pattern: while its overall shape is

similar to that of the verse subset, precision declines more rapidly as recall increases, reflecting the reduced discriminative power at this level of the hierarchy.

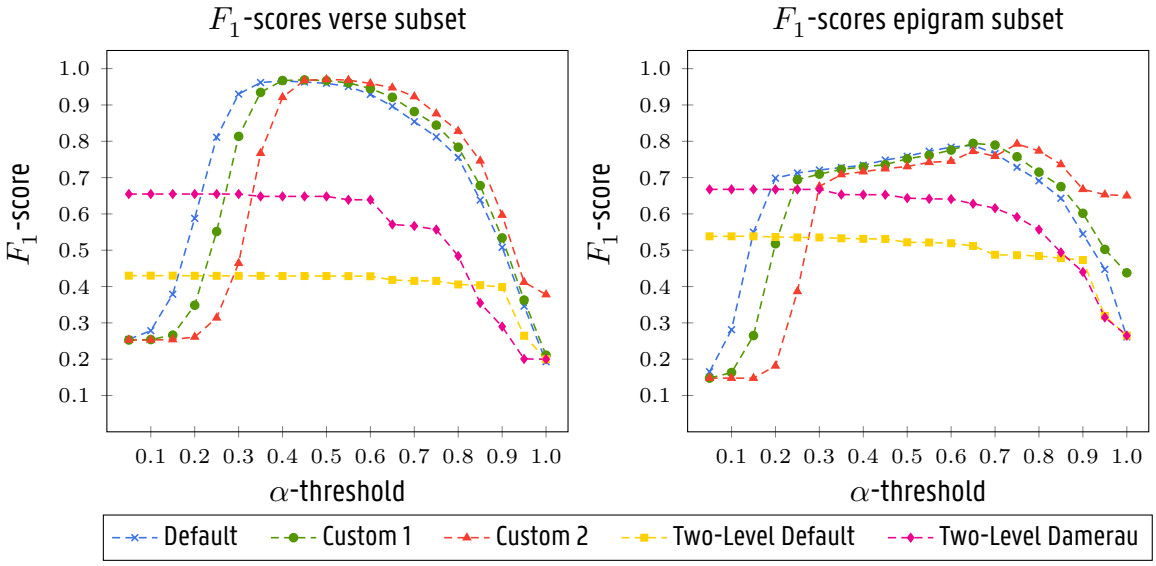


Figure 3.7: F_1-scores as a function of the α-threshold for all parameter configurations of the proposed similarity measure and both configurations of the Two-Level string matcher. The left plot corresponds to the verse subset and the right plot to the epigram subset.

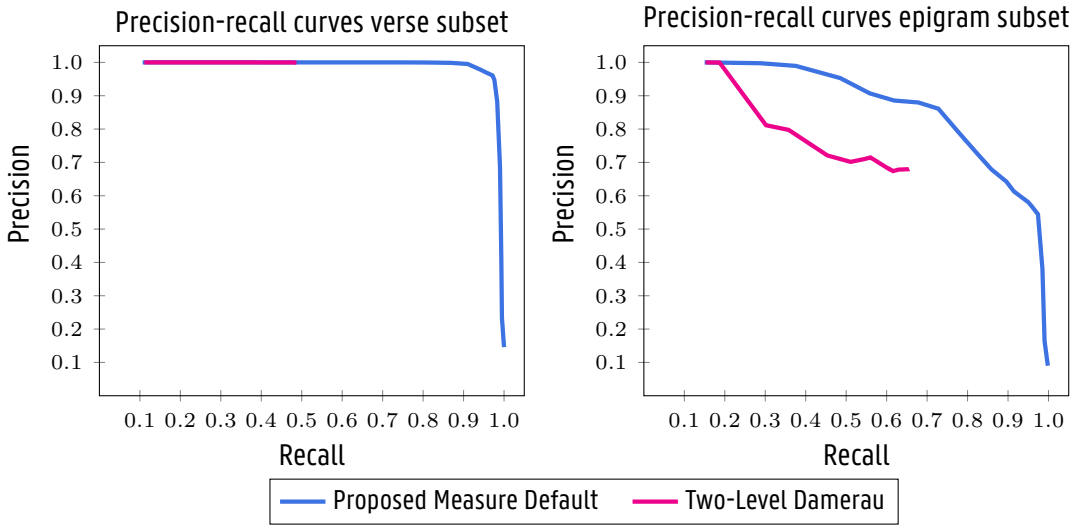


Figure 3.8: Precision-recall curves for the default configuration of the proposed similarity measure and the Damerau configuration of the Two-Level string matcher. The verse subset is displayed on the left and the epigram subset on the right. The remaining configurations are omitted for clarity, as they exhibit similar patterns.

Several observations can be drawn from these results. The choice of parameters at each level of the hierarchical decomposition significantly influences the performance of the measure. More tolerant configurations, which reduce the cost of certain edit operations and lower the thresholds for fuzzy matching, improve performance to a certain extent by accommodating the orthographic variation characteristic of this corpus. When these adjustments become too permissive, however, the measure begins to treat genuinely different texts as similar,

leading to decreased precision without a proportional gain in recall. This effect is visible in the Custom 2 configuration on the epigram subset, where the peak F_1-score of 0.7927 at $\alpha = 0.75$ does not surpass the 0.7942 achieved by Custom 1 at $\alpha = 0.65$, despite the lower edit operation costs. The α threshold itself provides a mechanism for controlling the trade-off between precision and recall: a low threshold favours recall by identifying as many similar pairs as possible, while a high threshold favours precision by retaining only those pairs with higher similarity scores. Two properties of the data bound the scores that can be achieved. Not all orthographic inconsistencies in the corpus can be resolved through preprocessing. Moreover, the expert-curated groupings serving as ground truth include texts that are incomplete, only partially similar, or that differ at the semantic level. Given these limitations, the results demonstrate the viability of the proposed measure for identifying related short texts.

Two-Level String Matcher

The precision, recall, and F_1-scores for both configurations of the Two-Level string matcher are listed in Table 3.4 for the verse subset and in Table 3.5 for the epigram subset. The corresponding F_1-scores are included in Figure 3.7 alongside those of the proposed measure, and Figure 3.8 displays precision-recall curves for the Damerau configuration.

For the verse subset, precision is perfect or near-perfect across the full range of α values in both configurations: the default configuration maintains a precision of 1.0 at every threshold, while the Damerau configuration yields values between 0.9990 and 1.0. Recall, by contrast, remains low throughout. The default configuration of the Two-Level string matcher reaches a maximum recall of only 0.2739 at the lowest α values, resulting in a peak F_1-score of 0.4300. The Damerau configuration improves recall to 0.4869 with only a marginal decrease in precision, yielding a peak F_1-score of 0.6547 at $\alpha \leq 0.30$. In both configurations, the scores remain largely stable across the lower α values, as shown in the left plot of Figure 3.7, before declining gradually at higher thresholds. The precision-recall curve for the verse subset in Figure 3.8 makes the limited recall of the Two-Level string matcher particularly apparent: the curve does not extend beyond a recall of approximately 0.49, even at the lowest thresholds. This indicates that a substantial portion of genuinely similar verse pairs is never identified.

For the epigram subset, precision is lower than for verses but still relatively high, ranging from 0.7270 to 1.0 for the default configuration and from 0.6739 to 1.0 for the Damerau configuration. The peak F_1-score for the default configuration is 0.5383 at $\alpha = 0.05$, with a precision of 0.7381 and a recall of 0.4236. The Damerau configuration again yields improved results, achieving a peak F_1-score of 0.6676 at the same threshold, with a recall of 0.6559 and a precision of 0.6798. As with the verse subset, the scores are relatively insensitive to the choice of α across the lower range, as shown in the right plot of Figure 3.7, and decrease at higher thresholds. In Figure 3.8, recall reaches at most approximately 0.66 for the Damerau configuration, and the trend otherwise resembles that of the verse subset. Unlike the verse subset, however, precision also declines steadily as recall increases.

The consistently high precision and limited recall observed across both subsets and configurations are a direct consequence of the aggregation strategy employed by the Two-Level string matcher. The weighted minimum

Table 3.4: Experimental results for the verse subset using the Two-Level string matcher. For each quality measure, the highest value across both configurations and all α values is highlighted in bold.

α	Default			Damerau		
	Precision	Recall	$\mathbf{F_1}$-score	Precision	Recall	$\mathbf{F_1}$-score
0.05	**1.0000**	0.2739	0.4300	0.9990	**0.4869**	**0.6547**
0.10	**1.0000**	0.2739	0.4300	0.9990	**0.4869**	**0.6547**
0.15	**1.0000**	0.2739	0.4300	0.9990	**0.4869**	**0.6547**
0.20	**1.0000**	0.2735	0.4296	0.9990	**0.4869**	**0.6547**
0.25	**1.0000**	0.2734	0.4295	0.9990	**0.4869**	**0.6547**
0.30	**1.0000**	0.2734	0.4295	0.9990	**0.4869**	**0.6547**
0.35	**1.0000**	0.2734	0.4295	0.9998	0.4796	0.6482
0.40	**1.0000**	0.2730	0.4289	0.9998	0.4796	0.6482
0.45	**1.0000**	0.2730	0.4289	0.9998	0.4796	0.6482
0.50	**1.0000**	0.2730	0.4289	0.9998	0.4796	0.6482
0.55	**1.0000**	0.2728	0.4287	0.9999	0.4695	0.6389
0.60	**1.0000**	0.2725	0.4283	0.9999	0.4695	0.6389
0.65	**1.0000**	0.2645	0.4183	0.9999	0.3996	0.5710
0.70	**1.0000**	0.2622	0.4155	**1.0000**	0.3955	0.5668
0.75	**1.0000**	0.2622	0.4155	**1.0000**	0.3859	0.5569
0.80	**1.0000**	0.2546	0.4058	**1.0000**	0.3192	0.4840
0.85	**1.0000**	0.2530	0.4038	**1.0000**	0.2156	0.3547
0.90	**1.0000**	0.2489	0.3986	**1.0000**	0.1694	0.2898
0.95	**1.0000**	0.1525	0.2646	**1.0000**	0.1115	0.2006
1.00	**1.0000**	0.1111	0.2000	**1.0000**	0.1111	0.2000

used in the bag-level similarity computation assigns high similarity scores only when a sufficient proportion of tokens are well-matched, which strongly favours precision but causes the method to miss pairs that contain even a moderate number of poorly matched tokens. This effect is amplified in a corpus characterised by pervasive orthographic inconsistencies, where many genuinely similar token pairs receive low character-level similarity scores, particularly under the default token matcher, which produces sparse similarity matrices. The Damerau configuration partially mitigates this by employing an edit-distance-based token matcher that assigns non-zero similarity scores to a wider range of token pairs, thereby improving recall without substantially compromising precision. However, even with this improvement, the bag-of-tokens approach does not preserve the ordering of tokens within a text, limiting its ability to capture structural differences. As a result, recall values remain well below those achieved by the proposed hierarchical similarity measure.

Table 3.5: Experimental results for the epigram subset using the Two-Level string matcher. For each quality measure, the highest value across both configurations and all α values is highlighted in bold.

α	Default			Damerau		
	Precision	Recall	$\mathbf{F_1}$-score	Precision	Recall	$\mathbf{F_1}$-score
0.05	0.7381	0.4236	0.5383	0.6798	**0.6559**	**0.6676**
0.10	0.7383	0.4236	0.5383	0.6798	**0.6559**	**0.6676**
0.15	0.7383	0.4236	0.5383	0.6798	**0.6559**	**0.6676**
0.20	0.7394	0.4210	0.5365	0.6797	0.6557	0.6675
0.25	0.7391	0.4194	0.5352	0.6797	0.6556	0.6674
0.30	0.7391	0.4193	0.5351	0.6797	0.6556	0.6674
0.35	0.7380	0.4170	0.5329	0.6784	0.6298	0.6532
0.40	0.7372	0.4154	0.5313	0.6783	0.6294	0.6529
0.45	0.7372	0.4151	0.5311	0.6783	0.6294	0.6529
0.50	0.7328	0.4051	0.5218	0.6739	0.6158	0.6435
0.55	0.7325	0.4046	0.5213	0.6802	0.6070	0.6415
0.60	0.7316	0.4025	0.5193	0.6799	0.6062	0.6409
0.65	0.7347	0.3927	0.5118	0.7148	0.5600	0.6280
0.70	0.7276	0.3666	0.4875	0.7097	0.5438	0.6158
0.75	0.7270	0.3654	0.4864	0.7016	0.5110	0.5913
0.80	0.7488	0.3576	0.4840	0.7208	0.4536	0.5568
0.85	0.7457	0.3517	0.4780	0.7976	0.3577	0.4939
0.90	0.7476	0.3460	0.4731	0.8115	0.3019	0.4401
0.95	0.9995	0.1900	0.3193	0.9995	0.1871	0.3151
1.00	**1.0000**	0.1526	0.2649	**1.0000**	0.1526	0.2649

Comparative Analysis

A comparison of the F_1-scores across both methods, as visualised in Figure 3.7, reveals a substantial performance difference in favour of the proposed hierarchical similarity measure on both subsets. On the verse subset, the proposed measure achieves peak F_1-scores above 0.96 across all three parameter configurations, compared to 0.6547 for the best-performing Two-Level string matcher configuration. On the epigram subset, the difference is smaller but still pronounced, with the proposed measure peaking at 0.7942 compared to 0.6676. The precision-recall curves in Figure 3.8 further illustrate this contrast: the proposed measure covers a wide range of recall values while maintaining high precision, whereas the Two-Level string matcher, despite its high precision, is unable to exceed a recall of approximately 0.49 on verses and 0.66 on epigrams.

The superior performance of the proposed measure on this corpus can be attributed to two factors. First, the proposed measure explicitly preserves textual structure. By hierarchically decomposing texts into their constituent discourse units and computing an edit distance at every level of the hierarchy, it captures structural

similarities that a bag-of-tokens approach, which disregards token ordering, cannot represent. Second, separate parameters can be configured at each level of the hierarchy, allowing researchers to tune the measure to the orthographic variation that characterises a specific corpus. By contrast, the Two-Level string matcher offers fewer parameters for adaptation and, through its use of a weighted minimum for aggregation, penalises texts with even a moderate number of poorly matched tokens. This behaviour is particularly limiting in corpora characterised by pervasive orthographic inconsistencies.

Taken together, these results demonstrate that the proposed hierarchical orthographic similarity measure substantially outperforms the Two-Level string matcher on both the verse and epigram subsets. The combination of structure-preserving computation, multi-level parameterisation, and the integration of lower-level similarity scores into higher-level comparisons yields a measure that achieves strong performance across both precision and recall. These properties make the proposed approach particularly well-suited for corpora of short, interconnected texts with significant orthographic variation. Having established the quantitative performance of the proposed measure, we now demonstrate its practical applicability through querying and visualisation in a graph database environment.

3.5.4 Practical Applicability

In addition to the quantitative evaluation, and in line with Section 2.6, we demonstrate the practical applicability of the proposed measure as a hands-on research tool for corpus exploration and analysis. Starting from the same subset of epigrams described in Section 2.6.1, we compute the similarity scores at every level of the hierarchy, from words through verses to complete epigrams, using the proposed similarity measure with the Custom 1 parameter configuration. In practice, the Neo4j graph database from Chapter 2 is extended with `SIMILAR_TO` edges. Each edge stores its membership grade, as defined in Definition 3.1, as an edge property called `similarity`. The following examples illustrate how the similarity-enriched graph can be explored and analysed through the query and visualisation capabilities of the graph database system.

Recall from Section 2.6 that the epigrams in this subset are grouped into three types of similar epigrams, all of which share variants of the same first and last verses. Type 2148 consists only of variants comprising those two verses. In type 2150, an intermediate verse following a specific pattern is inserted between the shared first and last verses, resulting in three-verse epigrams. Type 4245 again shares the same first and last verses but contains one to four intermediate verses following the same pattern as those in type 2150. Using these three types, we demonstrate two approaches for exploring the corpus using the computed similarity scores: grouping textual elements through database queries based on their degree of similarity, and extending the identification and visualisation of interconnections between texts with similarity information.

Query-Based Grouping of Similar Text Nodes

A first approach to exploring the similarity-enriched graph is to retrieve and visualise groups of similar texts directly through database queries using the Neo4j Cypher query language [63]. By performing an α-cut on the fuzzy LPG, only those `SIMILAR_TO` edges whose similarity score meets or exceeds the chosen threshold are retained, revealing clusters of similar nodes based on their remaining connections in the graph. As an example,

Listing 3.2 shows a Cypher query that retrieves all pairs of epigrams connected by a SIMILAR_TO edge with a similarity score of at least 0.95, corresponding to an α-cut with $\alpha = 0.95$ on the fuzzy LPG. Although edges in an LPG and in the Neo4j Database Management System (DBMS) are always directed, a Cypher query can match edges regardless of their direction, allowing pairs of nodes to be retrieved without specifying the edge orientation.

```
MATCH (e1:Epigram)-[s:SIMILAR_TO]-(e2:Epigram)
WHERE s.similarity >= 0.95
RETURN e1, e2, s
```

Listing 3.2: Cypher query to retrieve similar epigram groups with a similarity score of at least 0.95.

The result of this query is visualised in Figure 3.9 using Neo4j Bloom [64], the interactive visualisation environment of Neo4j. Four distinct groups of epigram nodes can be identified, and by inspecting the identifiers of the epigrams in each group, each can be mapped to an expert-based epigram type or a subgroup thereof. The large, closely connected group at the bottom of the figure consists of the two-verse epigrams from type 2148, whose structural simplicity and shared verses result in consistently high pairwise similarity scores. On the right, a smaller but equally closely connected group contains a subset of four-verse epigrams from type 4245, which share the same first and last verses as type 2148 but include additional intermediate verses following the same pattern as those in type 2150. The three-verse epigrams from type 2150 form the large group on the left, connected more loosely than the two-verse group, reflecting the greater structural variability introduced by the intermediate verse and the orthographic variation across these epigrams. A smaller, sparsely connected group at the top of the figure also consists of epigrams associated with type 2148. These epigrams, however, share a common variation that is substantial enough to cause their similarity scores with the other two-verse epigrams to fall below the threshold of 0.95. The differences between this fourth group and the remaining two-verse epigrams are examined in more detail below.

Epigram nodes with no remaining connections and small groups of two or three connected nodes are omitted from the visualisation for clarity. These outliers correspond to epigrams for which the text is only partially known, or to extreme variants within a type, such as epigrams with substantially different intermediate verses, whose similarity with the remaining epigrams does not meet the α-cut threshold.

The emergence of these visually distinct groups from a single query, without any prior knowledge of the expert-curated types, demonstrates that the proposed similarity measure captures meaningful structural and orthographic relationships between the texts. Starting from these groups, more targeted investigations can be performed within the interactive graph database environment, for instance by adjusting the similarity threshold, expanding the epigram nodes into their hierarchical components, or exploring the interconnections and metadata of the underlying texts.

Interconnection Identification and Visualisation

In Chapter 2, we showed that the graph-based text representation supports the identification and visualisation of interconnections between texts, for instance by revealing shared nodes that represent identical textual elements. The addition of similarity scores to the graph extends these capabilities by making the degree of

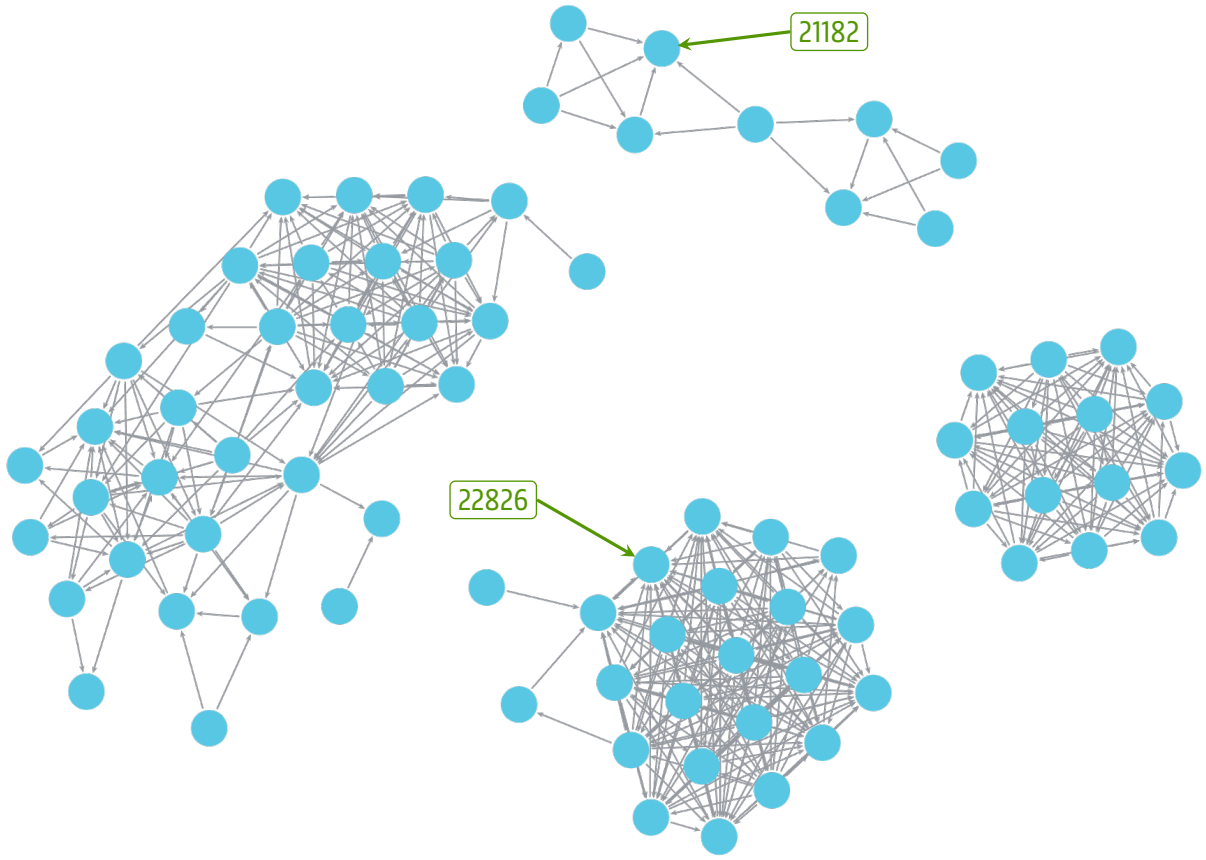


Figure 3.9: Visualisation of the result of the query in Listing 3.2 using Neo4j Bloom. All `SIMILAR_TO` relationships between `Epigram` nodes with a similarity score of at least 0.95 are visualised. For clarity, small groups of two or three connected epigram nodes, or nodes with no remaining connections, are omitted from the visualisation. The `SIMILAR_TO` relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of this similarity. More fine-grained analysis can be performed by zooming into specific areas of the visualisation as well as by interacting with the graph and by expanding nodes into their hierarchical components. The green labels indicate the identifiers of two epigrams that belong to different groups in the visualisation, which are further examined in the following section.

resemblance between textual elements at every level of the hierarchy directly accessible and queryable. Rather than relying solely on exact matches, which are captured through the node-merging mechanism described in Section 2.4.2, researchers can now explore approximate relationships and quantify the extent to which two texts or their components differ.

To illustrate this, we examine two epigrams from type 2148 that belong to different groups in the visualisation of Figure 3.9: epigram 22826 from the large, closely connected group at the bottom, and epigram 21182 from the smaller, sparsely connected group at the top. Listing 3.3 shows the Cypher query used to retrieve a detailed view of both epigrams, including their hierarchical components, their shared nodes, and the similarity scores between some of their textual elements. The result of this query is visualised in Figure 3.10.

```
MATCH (e1:Epigram {id:22826})-[:CONTAINS]->(v1:Verse)-[:CONTAINS]->(w1:Word)
MATCH (e2:Epigram {id:21182})-[:CONTAINS]->(v2:Verse)-[:CONTAINS]->(w2:Word)
MATCH (e1)-[ep_sim:SIMILAR_TO]-(e2)
MATCH (e1)-[:CONTAINS {rank:0}]->(:Verse)-[verse_sim:SIMILAR_TO]-(:Verse)
 ↪ <-[:CONTAINS {rank:0}]-(e2)
RETURN e1, e2, v1, v2, w1, w2, ep_sim, verse_sim
```

Listing 3.3: Cypher query to retrieve a detail view of the interconnections and similarities between two epigrams.

This visualisation reveals both the structural similarities and the specific differences between the two epigrams. The second verse of both epigrams is represented by the same node in the graph, a direct consequence of the node-merging mechanism, indicating that these verses are textually identical. The first verses, however, differ in two respects. In epigram 22826, the word βλεπιν is used where epigram 21182 uses ιδιν, which are synonymous. This lexical substitution is accompanied by a positional change: ιδιν occupies the fifth position in epigram 21182, whereas βλεπιν occupies the fourth position in epigram 22826, with the word πατριδα shifting accordingly. These combined differences, a lexical substitution and a transposition, result in a similarity score of approximately 0.67 between the first verses of both epigrams.

Since the second verses are identical, the overall epigram-level similarity is determined primarily by the reduced similarity at the first verse, yielding a score of approximately 0.84 using the Custom 1 parameter configuration. This score falls below the threshold of 0.95 used in the grouping query of Listing 3.2, which explains why both epigrams appear in separate groups in Figure 3.9. Further investigation reveals that all epigrams in the smaller group at the top of Figure 3.9 share this same first-verse variation, which explains why they form a separate group in the graph. At a lower threshold, for instance $\alpha = 0.80$, the epigrams from both groups would be connected and the groups would merge, illustrating how the choice of α influences the granularity of the resulting groupings.

This example demonstrates how the similarity-enriched graph extends the interconnection identification and exploration capabilities introduced in Chapter 2 and supports researchers with rich exploration facilities. Where the original graph model reveals shared textual elements through node merging, the addition of similarity scores allows researchers to determine not only which texts share components but also to what extent their non-shared components resemble one another. By combining the query and visualisation capabilities of the graph database with the hierarchical similarity scores, domain experts can interactively trace the origins of a

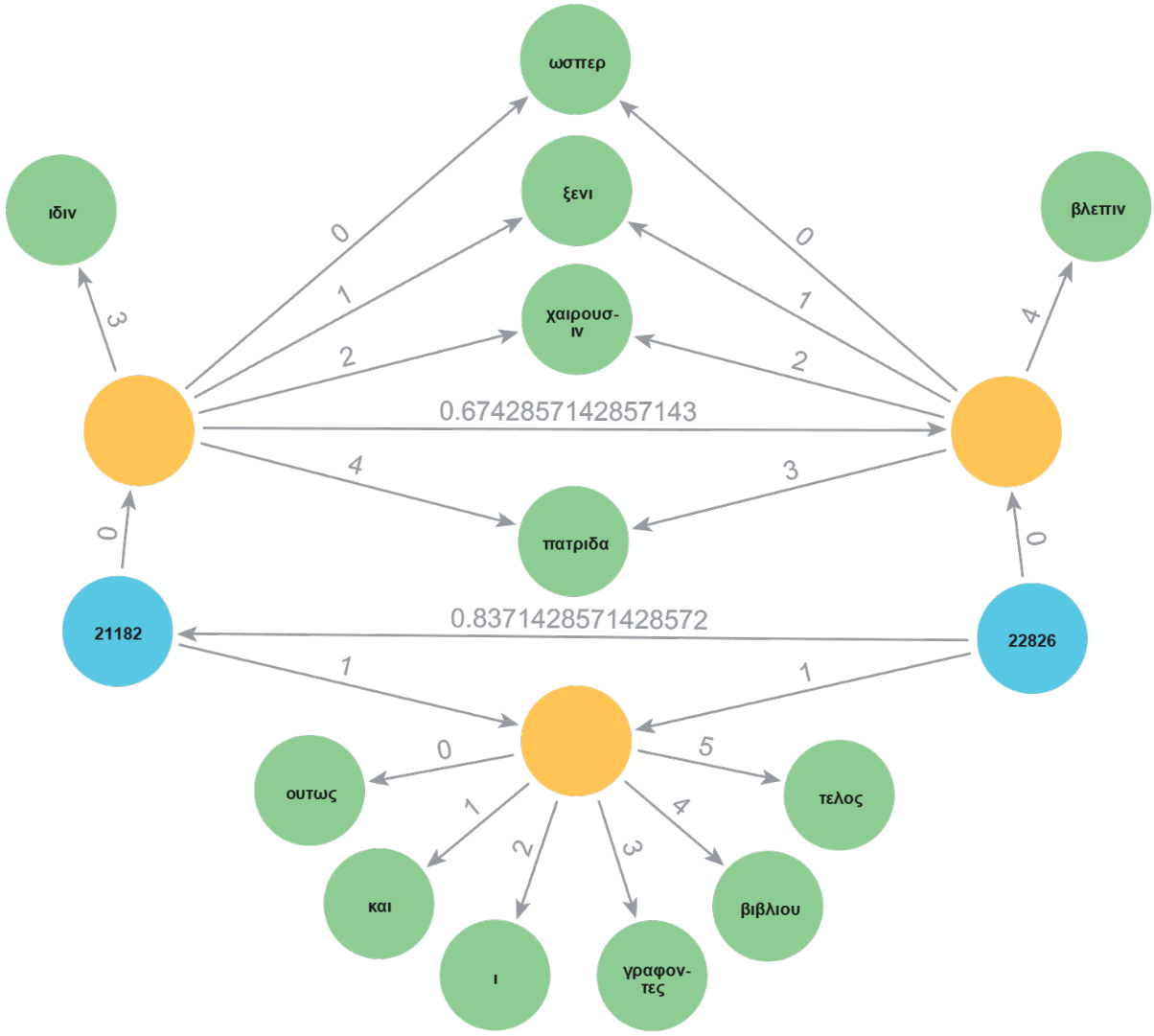


Figure 3.10: Visualisation of the result of the query in Listing 3.3 using Neo4j Bloom. The hierarchical decompositions of epigrams 22826 and 21182 into their constituent verses and words are shown. `Word` nodes are indicated in green, `Verse` nodes in yellow, and `Epigram` nodes in blue. The edges between same-level textual elements represent the `SIMILAR_TO` relationships and display the computed similarity scores. Other edges, which connect nodes at different levels, represent the `CONTAINS` relationships that capture the hierarchical decomposition of the epigrams. The `SIMILAR_TO` relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of this similarity.

similarity score through the levels of the textual hierarchy, inspect which components contribute to or reduce the overall similarity, and thereby gain a deeper understanding of the relationships between texts. This capacity to trace results back to their constituent scores demonstrates the interpretability of the proposed approach and supports the explainability of downstream applications.

3.6 Conclusions and Future Work

In this chapter, we presented a novel hierarchical orthographic similarity measure for short texts represented by the graph-based data model introduced in Chapter 2. The measure operates in a bottom-up fashion, starting from a string-based edit distance between the lowest-level textual elements and progressing upward through the hierarchy using a node-based edit distance that incorporates the similarity scores computed at each preceding level. The central mechanism of this approach is the integration of lower-level similarity scores into the computation at each successive level, ensuring that the structure of the texts is properly reflected in the resulting similarity between complete texts. All computed similarity scores are stored as property values on edges in the graph, extending the LPG from Chapter 2 into a fuzzy LPG in which the pairwise similarity between all comparable textual elements is directly accessible as part of the graph structure.

The principal contribution of this chapter is the hierarchical application of edit distances to graph-structured text representations, producing similarity scores at every level of the textual hierarchy rather than a single aggregated score. The availability of similarity scores at every level of the hierarchy contributes to the interpretability of the overall similarity between two texts and to the explainability of downstream analysis techniques. Not only the final score but also the lower-level scores from which it is derived can be inspected and traced through the graph. Unlike conventional orthographic measures, which reduce texts to flat sequences of characters or tokens, the proposed measure exploits the hierarchical decomposition of texts and is therefore able to capture structural differences, such as the reordering of sentences or the substitution of individual components, that flat measures cannot distinguish. Furthermore, the cost parameters can be adjusted independently at each level, enabling corpus-specific tuning through expert knowledge or empirical analysis. Because the measure relies solely on the written form and structure of the texts, it requires no language-specific resources and can therefore be applied to corpora in low-resource languages.

We evaluated the proposed measure through experiments on the DBBE corpus of Byzantine book epigrams, which is characterised by extensive orthographic variation and dense interconnections between texts. The results demonstrate a peak F_1-score of 0.9704 on the verse subset and of 0.7942 on the more structurally variable epigram subset. A comparison with an existing two-level string-matching baseline shows that the proposed measure substantially outperforms this baseline on both subsets, with the difference attributable to the explicit preservation of textual structure and the flexibility of the hierarchical parameterisation. Beyond the quantitative evaluation, we demonstrated the practical applicability of the similarity-enriched graph through querying and visualisation examples in a Neo4j graph database. These examples illustrate how the computed scores support the interactive exploration of interconnections and the identification of groups of similar texts.

The proposed measure is subject to a number of limitations. First, the measure is purely orthographic and does not account for semantic relationships between texts, which limits its ability to capture meaning-level resemblance between textual elements. Second, the measure computes pairwise similarity scores between all nodes at each level of the hierarchy, resulting in a quadratic number of comparisons that may present scalability challenges for very large corpora. Although the node-merging mechanism reduces the number of nodes by consolidating identical textual elements, the quadratic growth remains a potential bottleneck. Third, the evaluation relies on a single corpus with specific characteristics, and the expert-curated groupings used as ground truth are themselves imperfect. As discussed in Sections 1.3 and 3.5.1, these groupings may contain incomplete texts and semantic variants, and reflect the preferences, background, and judgement of the domain experts who composed them. While the measure has proven effective in its current form for the DBBE corpus, evaluating it on other corpora of short texts, with different languages, text lengths, and structural properties, would strengthen the evidence for its generalisability.

These limitations, together with the contributions of this chapter, suggest several directions for future work. On the methodological side, the hierarchical framework is not restricted to the specific edit distances proposed here. Other orthographic similarity measures that operate on ordered or unordered sequences could be adopted at individual levels of the hierarchy, and applying different measures at different levels could further enhance the adaptability of the framework. Beyond orthographic measures, the integration of semantic similarity into the hierarchical computation could further enrich the analysis, though the effectiveness of such approaches remains limited for short texts and low-resource languages, as discussed in Section 3.2. Another avenue for future investigation is the scalability limitation identified above, which could be addressed through more efficient similarity measures or through approximate methods that identify candidate pairs without exhaustive comparison.

Looking beyond the scope of this chapter, the similarity scores computed here provide a foundation for several extensions explored in the subsequent chapters of this thesis. In Chapter 4, the orthographic similarity measure is enriched with contextual metadata, resulting in a context-aware similarity measure with an associated confidence score that quantifies the reliability of that similarity given the available metadata. Building on the orthographic and context-aware similarity scores, Chapter 5 develops ranking techniques that account for both similarity and confidence. Chapter 6 exploits these scores in clustering methods that group short texts in an unsupervised manner. Each of these extensions relies on the hierarchical similarity measure proposed in this chapter to support increasingly sophisticated forms of textual analysis, and the similarity scores computed here could equally serve as input for other downstream analytical techniques beyond those explored in this thesis.

References

[1] Min Chen, Shiwen Mao and Yunhao Liu. "Big Data: A Survey". In: *Mobile Networks and Applications* 19.2 (2014), pp. 171–209. DOI: https://doi.org/10.1007/s11036-013-0489-0.

[2] Hossein Hassani, Christina Beneki, Stephan Unger, Maedeh Taj Mazinani and Mohammad Reza Yeganegi. "Text Mining in Big Data Analytics". In: *Big Data and Cognitive Computing* 4.1 (2020). issn: 2504-2289. DOI: https://doi.org/10.3390/bdcc4010001.

[3] Ah-Hwee Tan et al. "Text Mining: The State of the Art and the Challenges". In: *Proceedings of the PAKDD 1999 Workshop on Knowledge Discovery from Advanced Databases*. Vol. 8. 1999, pp. 65–70.

[4] Andreas Hotho, Andreas Nürnberger and Gerhard Paaß. "A Brief Survey of Text Mining". In: *Journal for Language Technology and Computational Linguistics* 20.1 (2005), pp. 19–62. DOI: https://doi.org/10.21248/jlcl.20.2005.68.

[5] Sonali Vijay Gaikwad, Archana Chaugule and Pramod Patil. "Text Mining Methods and Techniques". In: *International Journal of Computer Applications* 85.17 (2014). DOI: https://doi.org/10.5120/14937-3507.

[6] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis and Vassilios S. Verykios. "Duplicate Record Detection: A Survey". In: *IEEE Transactions on Knowledge and Data Engineering* 19.1 (2007), pp. 1–16. DOI: https://doi.org/10.1109/TKDE.2007.250581.

[7] Antoon Bronselaer and Guy De Tré. "Semantical Evaluators". In: *Proceedings of the Joint International Fuzzy Systems Association (IFSA) World Congress and European Society for Fuzzy Logic and Technology (EUSFLAT) Conference*. 2009, pp. 663–668.

[8] Antoon Bronselaer, Axel Hallez and Guy De Tré. "Evaluation in the Possibilistic Framework for Object Matching". In: *Proceedings of the 12th International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU)*. 2008, pp. 1701–1708.

[9] Christopher D Manning. "Introduction to Information Retrieval". Syngress Publishing, 2008. DOI: https://doi.org/10.1017/CBO9780511809071.

[10] Najlah Gali, Radu Mariescu-Istodor, Damien Hostettler and Pasi Fränti. "Framework for Syntactic String Similarity Measures". In: *Expert Systems with Applications* 129 (2019), pp. 169–185. issn: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2019.03.048.

[11] Wael H Gomaa, Aly A Fahmy et al. "A Survey of Text Similarity Approaches". In: *International Journal of Computer Applications* 68.13 (2013), pp. 13–18. DOI: https://doi.org/10.5120/11638-7118.

[12] Dhivya Chandrasekaran and Vijay Mago. "Evolution of Semantic Similarity—A Survey". In: *ACM Comput. Surv.* 54.2 (2021). issn: 0360-0300. DOI: https://doi.org/10.1145/3440755.

[13] Junyang Chen, Zhiguo Gong, Wei Wang, Xiao Dong, Wei Wang, Weiwen Liu, Cong Wang and Xian Chen. "Inductive Document Representation Learning for Short Text Clustering". In: *Machine Learning and Knowledge Discovery in Databases*. Cham: Springer International Publishing, 2021, pp. 600–616. isbn: 978-3-030-67664-3. DOI: https://doi.org/10.1007/978-3-030-67664-3_36.

[14] Alexandre Magueresse, Vincent Carles and Evan Heetderks. *Low-Resource Languages: A Review of Past Work and Future Challenges*. 2020. DOI: https://doi.org/10.48550/arXiv.2006.07264.

[15] Partha Pakray, Alexander Gelbukh and Sivaji Bandyopadhyay. "Natural Language Processing Applications for Low-Resource Languages". In: *Natural Language Processing* 31.2 (2025), pp. 183–197. DOI: https://doi.org/10.1017/nlp.2024.33.

[16] Nathan Stringham and Mike Izbicki. "Evaluating Word Embeddings on Low-Resource Languages". In: *Proceedings of the First Workshop on Evaluation and Comparison of NLP Systems*. Online: Association for Computational Linguistics, 2020, pp. 176–186. DOI: https://doi.org/10.18653/v1/2020.eval4nlp-1.17.

[17] William Cohen, Pradeep Ravikumar and Stephen Fienberg. "A Comparison of String Metrics for Matching Names and Records". In: *KDD Workshop on Data Cleaning and Object Consolidation*. Vol. 3. 2003, pp. 73–78.

[18] Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, Maxime Deforche, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout and Colin Swaelens. "The Database of Byzantine Book Epigrams Project : Principles, Challenges, Opportunities". eng. In: *Journal of Data Mining and Digital Humanities* On the Way to the Future of Digital Manuscript Studies (2023), p. 41. issn: 2416-5999. DOI: https://doi.org/10.46298/jdmdh.10244.

[19] Kristoffel Demoen, Gilbert Bentein, Klaas Bentein, Floris Bernard, Julián Bértola, Julie Boeten, Mathijs Clement, Cristina Cocola, Eline Daveloose, Sien De Groot, Pieterjan De Potter, Ilse De Vos, Krystina Kubina, Hanne Lauwers, Paulien Lemay, Renaat Meesters, Marjolein Morbé, Delphine Nachtergaele, Marthe Nemegeer, Joachim Nielandt, Mace Ojala, Lisa-Lou Péchillon, Raf Praet, Rachele Ricceri, Anne-Sophie Rouckhout, Jeroen Schepens, Febe Schollaert, Lev Shadrin, Nina Sietis, Dimitrios Skrekas, Colin Swaelens, Maria Tomadaki, Sarah-Helena Van den Brande, Merel Van Nieuwerburgh, Lotte Van Olmen, Noor Vanhoe and Nina Vanhoutte. *Database of Byzantine Book Epigrams (DBBE)*. Version 1.0.0. Zenodo, Feb. 2023. DOI: https://doi.org/10.5281/zenodo.7682523.

[20] Antoon Bronselaer and Guy De Tré. "Properties of Possibilistic String Comparison". In: *IEEE Transactions on Fuzzy Systems* 18.2 (2010), pp. 312–325. DOI: https://doi.org/10.1109/TFUZZ.2010.2041353.

[21] Antoon Bronselaer and Guy De Tré. "A Possibilistic Approach to String Comparison". In: *IEEE Transactions on Fuzzy Systems* 17.1 (2009), pp. 208–223. DOI: https://doi.org/10.1109/TFUZZ.2008.2008025.

[22] Jiapeng Wang and Yihong Dong. "Measurement of Text Similarity: A Survey". In: *Information* 11.9 (2020). issn: 2078-2489. DOI: https://doi.org/10.3390/info11090421.

[23] Vladimir I Levenshtein. "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals". In: *Soviet Physics Doklady*. Vol. 10. 8. 1966, pp. 707–710.

[24] Fred J Damerau. "A Technique for Computer Detection and Correction of Spelling Errors". In: *Communications of the ACM* 7.3 (1964), pp. 171–176. DOI: https://doi.org/10.1145/363958.363994.

[25] Matthew A Jaro. "Probabilistic Linkage of Large Public Health Data Files". In: *Statistics in Medicine* 14.5-7 (1995), pp. 491–498. DOI: https://doi.org/10.1002/sim.4780140510.

[26] William E Winkler. "String Comparator Metrics and Enhanced Decision Rules in the Fellegi-Sunter Model of Record Linkage". ERIC, 1990.

[27] Grzegorz Kondrak. "N-Gram Similarity and Distance". In: *International Symposium on String Processing and Information Retrieval*. Springer. 2005, pp. 115–126. DOI: https://doi.org/10.1007/11575832_13.

[28] Paul Jaccard. "Étude Comparative de la Distribution Florale dans une Portion des Alpes et des Jura". In: *Bulletin de la Société Vaudoise des Sciences Naturelles* 37 (1901), pp. 547–579.

[29] Lee R Dice. "Measures of the Amount of Ecologic Association Between Species". In: *Ecology* 26.3 (1945), pp. 297–302. DOI: https://doi.org/10.2307/1932409.

[30] Peter Christen. "A Comparison of Personal Name Matching: Techniques and Practical Issues". In: *Sixth IEEE International Conference on Data Mining Workshops (ICDMW'06)*. IEEE, 2006, pp. 290–294. DOI: https://doi.org/10.1109/ICDMW.2006.2.

[31] Justin Zobel and Philip Dart. "Phonetic String Matching: Lessons from Information Retrieval". In: *Proceedings of the 19th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '96)*. ACM, 1996, pp. 166–172. DOI: https://doi.org/10.1145/243199.243258.

[32] Sergio Jimenez, Fabio Gonzalez and Alexander Gelbukh. "Text Comparison Using Soft Cardinality". In: *International Symposium on String Processing and Information Retrieval*. Springer. 2010, pp. 297–302. DOI: https://doi.org/10.1007/978-3-642-16321-0_31.

[33] Grigori Sidorov, Alexander Gelbukh, Helena Gómez-Adorno and David Pinto. "Soft Similarity and Soft Cosine Measure: Similarity of Features in Vector Space Model". In: *Computación y Sistemas* 18.3 (2014), pp. 491–504. DOI: https://doi.org/10.13053/CyS-18-3-2043.

[34] Surajit Chaudhuri, Kris Ganjam, Venkatesh Ganti and Rajeev Motwani. "Robust and Efficient Fuzzy Match for Online Data Cleaning". In: *Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data*. SIGMOD '03. San Diego, California: Association for Computing Machinery, 2003, pp. 313–324. isbn: 158113634X. DOI: https://doi.org/10.1145/872757.872796.

[35] Rada Mihalcea, Courtney Corley, Carlo Strapparava et al. "Corpus-Based and Knowledge-Based Measures of Text Semantic Similarity". In: *Proceedings of the 21st National Conference on Artificial Intelligence (AAAI)*. 2006, pp. 775–780.

[36] Thomas K Landauer and Susan T Dumais. "A Solution to Plato's Problem: The Latent Semantic Analysis Theory of Acquisition, Induction, and Representation of Knowledge". In: *Psychological Review* 104.2 (1997), p. 211. DOI: https://doi.org/10.1037/0033-295X.104.2.211.

[37] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado and Jeff Dean. "Distributed Representations of Words and Phrases and Their Compositionality". In: *Advances in Neural Information Processing Systems*. Vol. 26. Curran Associates, Inc., 2013. DOI: https://doi.org/10.48550/arXiv.1310.4546.

[38] Peter D Turney. "Mining the Web for Synonyms: PMI-IR Versus LSA on TOEFL". In: *European Conference on Machine Learning*. Springer. 2001, pp. 491–502. DOI: https://doi.org/10.1007/3-540-44795-4_42.

[39] Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: https://doi.org/10.18653/v1/N19-1423.

[40] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992. DOI: https://doi.org/10.18653/v1/D19-1410.

[41] George A Miller. "WordNet: A Lexical Database for English". In: *Communications of the ACM* 38.11 (1995), pp. 39–41. DOI: https://doi.org/10.1145/219717.219748.

[42] Zhibiao Wu and Martha Palmer. "Verb Semantics and Lexical Selection". In: *Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 1994, pp. 133–138. DOI: https://doi.org/10.48550/arXiv.cmp-lg/9406033.

[43] Philip Resnik. "Using Information Content to Evaluate Semantic Similarity in a Taxonomy". In: *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. 1995, pp. 448–453. DOI: https://doi.org/10.48550/arXiv.cmp-lg/9511007.

[44] Aminul Islam and Diana Inkpen. "Semantic Text Similarity Using Corpus-Based Word Similarity and String Similarity". In: *ACM Transactions on Knowledge Discovery from Data (TKDD)* 2.2 (2008), pp. 1–25. DOI: https://doi.org/10.1145/1376815.1376819.

[45] Matt J. Kusner, Yu Sun, Nicholas I. Kolkin and Kilian Q. Weinberger. "From Word Embeddings to Document Distances". In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by Francis Bach and David Blei. Vol. 37. Proceedings of Machine Learning Research. PMLR, 2015, pp. 957–966.

[46] Ahmed Hamza Osman and Omar Mohammed Barukub. "Graph-Based Text Representation and Matching: A Review of the State of the Art and Future Challenges". In: *IEEE Access* 8 (2020), pp. 87562–87583. DOI: https://doi.org/10.1109/ACCESS.2020.2993191.

[47] Xinbo Gao, Bing Xiao, Dacheng Tao and Xuelong Li. "A Survey of Graph Edit Distance". In: *Pattern Analysis and Applications* 13.1 (2010), pp. 113–129. issn: 1433-755X. DOI: https://doi.org/10.1007/s10044-008-0141-y.

[48] Michel Neuhaus, Kaspar Riesen and Horst Bunke. "Fast Suboptimal Algorithms for the Computation of Graph Edit Distance". In: *Structural, Syntactic, and Statistical Pattern Recognition*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 163–172. isbn: 978-3-540-37241-7. DOI: https://doi.org/10.1007/11815921_17.

[49] M. Neuhaus and H. Bunke. "A Convolution Edit Kernel for Error-Tolerant Graph Matching". In: *18th International Conference on Pattern Recognition (ICPR'06)*. Vol. 4. 2006, pp. 220–223. DOI: https://doi.org/10.1109/ICPR.2006.57.

[50] Horst Bunke. "Graph Matching: Theoretical Foundations, Algorithms, and Applications". In: *Proc. Vision Interface*. Vol. 2000. 2000, pp. 82–88.

[51] Llorenç Valverde. "On the Structure of F-Indistinguishability Operators". In: *Fuzzy Sets and Systems* 17.3 (1985), pp. 313–328. issn: 0165-0114. DOI: https://doi.org/10.1016/0165-0114(85)90096-X.

[52] Bernard De Baets and Radko Mesiar. "Metrics and T-Equalities". In: *Journal of Mathematical Analysis and Applications* 267.2 (2002), pp. 531–547. issn: 0022-247X. DOI: https://doi.org/10.1006/jmaa.2001.7786.

[53] Lotfi A Zadeh. "Fuzzy Sets". In: *Information and Control* 8.3 (1965), pp. 338–353. DOI: https://doi.org/10.1016/0165-0114(78)90029-5.

[54] Didier Dubois and Henri Prade. "The Three Semantics of Fuzzy Sets". In: *Fuzzy Sets and Systems* 90.2 (1997). Fuzzy Sets: Where Do We Stand? Where Do We Go?, pp. 141–150. issn: 0165-0114. DOI: https://doi.org/10.1016/S0165-0114(97)00080-8.

[55] Lotfi A Zadeh. "Similarity Relations and Fuzzy Orderings". In: *Information Sciences* 3.2 (1971), pp. 177–200. DOI: https://doi.org/10.1016/S0020-0255(71)80005-1.

[56] Azriel Rosenfeld. "Fuzzy Graphs". In: *Fuzzy Sets and Their Applications to Cognitive and Decision Processes*. Elsevier, 1975, pp. 77–95. DOI: https://doi.org/10.1016/B978-0-12-775260-0.50008-6.

[57] Olivier Pivert, Virginie Thion, Hélène Jaudoin and Grégory Smits. "On a Fuzzy Algebra for Querying Graph Databases". In: *2014 IEEE 26th International Conference on Tools with Artificial Intelligence*. IEEE. 2014, pp. 748–755. DOI: https://doi.org/10.1109/ICTAI.2014.116.

[58] Gonzalo Navarro. "A Guided Tour to Approximate String Matching". In: *ACM Computing Surveys (CSUR)* 33.1 (2001), pp. 31–88. DOI: https://doi.org/10.1145/375360.375365.

[59] Leonid Boytsov. "Indexing Methods for Approximate Dictionary Searching: Comparative Analysis". In: *ACM Journal of Experimental Algorithmics* 16 (2011), pp. 1.1–1.91. DOI: https://doi.org/10.1145/1963190.1963191.

[60] Gregory V. Bard. *Spelling-Error Tolerant, Order-Independent Pass-Phrases via the Damerau-Levenshtein String-Edit Distance Metric*. Tech. rep. 2006/364. Cryptology ePrint Archive, IACR, 2006.

[61] George J. Klir and Bo Yuan. "Fuzzy Sets and Fuzzy Logic: Theory and Applications". Kluwer Academic Publishers, 1995.

[62] Neo4j. "Neo4j Graph Database". url: https://neo4j.com/ (visited on 27/03/2026).

[63] Neo4j. "Cypher Query Language". url: https://neo4j.com/docs/cypher-manual/current/ (visited on 01/04/2026).

[64] Neo4j. “Neo4j Bloom”. url: https://neo4j.com/bloom/ (visited on 03/04/2026).

4

A Context-Aware and Responsible Similarity Measure for Short Texts

"The whole is greater
than the sum of its parts."

— **Aristotle**

In this chapter, a context-aware and responsible similarity measure for short texts is presented by enriching the textual similarity measure introduced in Chapter 3 with contextual information drawn from associated metadata. The measure is responsible in the sense that the veracity issues affecting those metadata, such as incomplete, uncertain, or entirely missing values, are reported alongside the resulting score rather than absorbed into it. Of these issues, this chapter focuses on incomplete and missing metadata. The textual similarity and the metadata similarities are fused into a single combined score for each pair of texts through an Ordered Weighted Averaging (OWA)-based aggregation mechanism, adapted to operate solely on the information available for that pair. This avoids the need for imputation of missing values or exclusion of incomplete metadata fields. A confidence score, derived from the total weight of the inputs available during the aggregation, is introduced alongside the aggregated similarity to indicate how reliable that score is. Together, they form a similarity-confidence pair, reporting the resemblance between two texts and the reliability of that score side by side.

The development and evaluation of this context-aware and responsible similarity measure address the third research question of this thesis:

> **RQ 3:** How can contextual metadata complement a textual similarity measure, even when those metadata are missing or incomplete, such that the trustworthiness of the result is made explicit alongside the similarity score for downstream analysis?

In addition, by extending the querying capabilities already demonstrated in Chapters 2 and 3 with the context-aware similarity scores and associated confidence scores, this chapter further addresses:

> **RQ 4.A:** How can (flexible) query answering techniques be applied to a graph-based data model of short texts to support the discovery of patterns and interconnections, with or without considering similarity or metadata?

The content of this chapter is based on parts of the following publication:

- **Maxime Deforche**, Ilse De Vos, and Guy De Tré. *A Novel Approach to Context-Aware and Responsible Short Text Clustering.* Published in Proceedings of the 16th International Conference on Flexible Query Answering Systems (FQAS), Lecture Notes in Computer Science, vol 16119, 2025.
  DOI: 10.1007/978-3-032-05607-8_4

The remainder of this fourth chapter is structured as follows. Section 4.1 motivates the integration of contextual metadata into textual similarity measures, discusses the veracity issues that frequently affect such metadata in practice, and outlines the approach proposed in this chapter. Related work on the composition of multiple criteria into a single similarity score, on aggregation operators and context-aware similarity measures, and on the handling of missing and incomplete metadata is discussed in Section 4.2. Section 4.3 provides the necessary preliminaries on aggregation operators in general, on the OWA operator in particular, and on set-based similarity measures. The proposed context-aware and responsible similarity measure is presented in Section 4.4, along with the procedure for computing it over an entire corpus of short texts. Section 4.5 reports on the experimental evaluation using the Database of Byzantine Book Epigrams (DBBE) dataset, with the textual similarity measure from Chapter 3 serving as a baseline. This section also demonstrates the practical applicability of the resulting similarity-confidence pairs through querying and visualisation examples. Finally, Section 4.6 summarises the contributions of this chapter, discusses limitations, and outlines directions for future work.

4.1 Introduction

Similarity or distance measures, such as cosine similarity [1], Jaccard similarity [2], or Levenshtein distance [3], are a cornerstone of textual analysis, providing the numerical foundation on which many analysis tasks are built. Tasks such as clustering, classification, information retrieval, and duplicate record detection, as well as applications such as recommender systems, rely heavily on the ability to quantify the similarity or distance between texts [1, 4–7]. The choice of the underlying measure is crucial, as it significantly impacts both the quality and the interpretability of the resulting outcomes. However, such measures typically capture only the textual content of the documents under comparison and disregard the contextual information that often accompanies texts in real-world datasets. Two text documents may share considerable textual content while originating from different periods, regions, or authors, and conversely, two textually dissimilar texts may be tightly connected through a shared context. Incorporating such contextual information into similarity measures can therefore provide a more complete and nuanced view of the relationships between texts, which can in turn improve performance in subsequent analytical tasks. This integration is particularly valuable for short texts, whose limited length restricts the discriminative power of textual measures [8–11], and for texts written in low-resource languages, where language-specific resources for semantic analysis are scarce or absent [12, 13].

In both settings, metadata offer an additional source of information that can complement the textual content and reveal connections that would otherwise remain invisible [14].

Contextual metadata can be used to enrich analysis in various ways. In its simplest form, metadata can be used to filter specific texts from a corpus through queries that target particular metadata values, such as selecting all texts from a given period or region. Using these query results, more advanced analysis can be performed, for instance by clustering the selected texts based on their textual similarity. When metadata need to be integrated into the analysis itself, however, two paths are available. Either techniques that operate directly on the metadata are required, such as clustering based on metadata values, or the metadata must be incorporated into the similarity measure itself, so that any downstream technique that relies on a similarity or distance measure can benefit from the contextual information. The latter approach goes beyond the capabilities of query-based filtering and is the one pursued in this chapter. Integrating contextual metadata into a similarity measure raises two difficulties, however. Since many downstream techniques compare texts on one similarity value per pair rather than on a collection of attribute-level scores, a fusion mechanism is required to combine textual similarity with the similarity scores computed over the metadata into a single aggregated score [15]. Moreover, metadata are frequently affected by veracity issues, including values that are incomplete, uncertain, or entirely missing [16, 17]. Missing metadata are especially common in real-world corpora, particularly in historical datasets, where records have been compiled over long periods or from varied sources, leaving certain fields unpopulated for a substantial share of the data. This chapter focuses on incomplete and missing metadata, for which two responses are traditionally employed [18, 19]. The first, known as marginalisation, discards the affected metadata fields entirely from every record, even when the values are available for some records. The second, imputation, estimates the missing values from the available data but relies on assumptions that may not hold in practice and can introduce errors that propagate into the similarity computation and any further analysis. A suitable approach should therefore incorporate all available metadata without requiring either strategy, and should make the impact of missing information on the resulting similarity score explicit, so that subsequent analysis can distinguish high-confidence matches from those inferred from limited evidence.

In this chapter, we address these challenges by proposing a context-aware and responsible similarity measure for short texts. The measure is *context-aware* because it compares texts on the metadata associated with them in addition to their textual content. It is *responsible* because the veracity issues affecting those metadata are neither ignored nor silently absorbed into the resulting score, but are quantified and reported alongside it, leaving the reliability of every comparison open to inspection. Concretely, the measure combines a textual similarity score with similarity scores computed over the available metadata of both texts through an Ordered Weighted Averaging (OWA)-based aggregation mechanism [20], producing a single aggregated score for each pair of texts. To accommodate incomplete and missing metadata, the aggregation adapts to the metadata fields that are available for a given pair of texts. The weight vector is truncated to the number of available inputs and renormalised, so that the aggregated score remains meaningful regardless of how much information is missing. Alongside the aggregated similarity, a confidence score is introduced that reflects the reliability of the resulting value and is derived directly from the total weight of the inputs available during the aggregation. Together, the aggregated similarity and its confidence score form a similarity-confidence pair, capturing both the extent of

resemblance between two texts and the trustworthiness of that assessment. These similarity-confidence pairs can be used for further tasks such as querying, visualisation, ranking, and clustering.

The principal novelty of the proposed approach lies in the combination of an adapted OWA-based fusion mechanism with an explicit confidence score that quantifies the reliability of each aggregated value. Unlike marginalisation, which excludes entire metadata fields, and imputation, which fills in missing values under potentially unreliable assumptions, the proposed fusion mechanism uses all metadata that are available for a given pair of texts. Because the accompanying confidence score follows from the weight vector rather than from the data, it requires no assumption about the mechanism behind the missing values. The confidence score is not a new similarity criterion, but complementary information about the aggregated similarity, which remains a single value. Because the aggregation is defined over generic similarity inputs, the framework is independent of any particular textual or metadata similarity measure and can be applied to any combination of measures suited to a given corpus. The resulting similarity-confidence pairs provide a natural foundation for analytical techniques that reason over both similarity and reliability, enhancing the interpretability of the results and the explainability of the subsequent analytical tasks.

We evaluate the proposed measure through experiments on the Database of Byzantine Book Epigrams (DBBE) corpus [21, 22], the collection of short Byzantine poems introduced in Chapter 1, for which a wide range of metadata fields are available but whose values are often incomplete or absent. The hierarchical orthographic similarity measure developed in Chapter 3 is used as the textual input to the fusion, and is complemented by similarity scores computed over the location, genre, metre, scribe, and subject metadata of the epigrams. Two weight vector configurations are considered to examine how the balance between textual and contextual information influences the aggregated scores, and the resulting pairs are filtered using a threshold to retain the pairs of highly similar epigrams. These pairs are then compared against the pairwise relationships derived from the expert-curated epigram types, to quantify the effect of incorporating metadata. As a baseline, the purely textual similarity measure from Chapter 3 is used, enabling a direct assessment of the added value of the contextual information in the presence of veracity concerns. Finally, we demonstrate the practical applicability of the similarity-confidence pairs in a graph database through querying and visualisation, illustrating how the similarity and confidence scores together support the exploration of the corpus at varying levels of reliability.

4.2 Related Work

Combining multiple pieces of information into a composite similarity score is a recurring problem whenever the comparison between two records rests on more than a single criterion. Accommodating gaps and uncertainties in such information is a long-standing concern, particularly for historical datasets, which are typically compiled from heterogeneous and incomplete sources. The approach proposed in this chapter is informed by both, and in this section we review the techniques most directly relevant to it.

Computing similarity as a combination of multiple criteria has a long tradition in record linkage and duplicate record detection, where two records are compared by first computing attribute-level similarities and then combining these into an overall similarity or match score [7, 23]. The foundational formulation by Fellegi and

Sunter [24] frames record linkage as a probabilistic decision problem over pairs of records, in which per-attribute agreement is combined through probabilistic weights into a single score, and each pair is classified as a match, a non-match, or a possible match requiring manual review. Subsequent work has extended this foundation with rule-based combinations, supervised classifiers trained on labelled pairs, and unsupervised methods that exploit the structure of the data [7, 23]. A closely related line of research concerns soft or possibilistic approaches to matching, in which the combination mechanism is designed to produce graded similarity scores rather than discrete match decisions [25–27]. What these techniques share with the approach proposed in this chapter is the premise that the similarity between two records is better captured by composing several attribute-level similarities through an explicit combination mechanism than by relying on a single criterion.

These combination mechanisms fall under the general theory of aggregation operators [28, 29]. The simplest aggregation strategies, such as weighted arithmetic and geometric means, combine inputs through fixed additive or multiplicative rules but can express only a limited range of behaviours [28]. Introduced by Yager [20], the OWA operator generalises weighted averages by applying weights to the sorted inputs rather than to fixed sources, recovering the minimum, maximum, and arithmetic mean as special cases. Adjusting the weight vector tunes the sensitivity of the aggregation to high or low values among the ordered inputs, providing a flexible mechanism for controlling the behaviour of the operator. A broader family of aggregators is provided by the General Conjunction Disjunction (GCD) operator central to the Logic Scoring of Preference (LSP) method [30–32]. The GCD supports a continuous range of aggregators spanning from strict conjunction to strict disjunction through parameters known as andness and orness, offering finer control over how partial satisfaction is traded off between multiple (weighted) criteria. More expressive techniques include fuzzy integrals, such as the Choquet [33] and Sugeno [34] integrals, which model interactions between criteria through a fuzzy measure that assigns weights to all subsets of inputs rather than to individual inputs [35, 36]. Operators of this kind offer a natural means of incorporating contextual criteria alongside the textual ones, an approach that recurs across a range of application domains. In context-aware recommender systems, for instance, user preferences are modelled as a function not only of the items themselves but also of surrounding factors such as time, location, device, or mood [14]. Similar mechanisms have been used to re-rank information retrieval results based on session or location information in search personalisation [37], and to incorporate metadata as additional features during clustering [38]. Among these operators, the OWA operator is adopted in this chapter because of its simplicity, its explicit tuning through a weight vector, and its sorting-based semantics, which make it particularly well-suited for aggregating similarity scores in a source-independent manner.

A common challenge for these matching frameworks and for the aggregation operators that underpin them is that they assume every input to be available for each record, which is often not the case in practice, particularly for real-world datasets drawn from multiple sources or compiled over long periods of time [16–19]. How such gaps are best treated depends on the mechanism that produced them [17, 39]. The mechanisms are distinguished by how the cause of a missing value relates to the data themselves. Values are Missing Completely at Random (MCAR) when their absence is purely accidental and unrelated to any characteristics of the data, such as physical damage to a document affecting a collection at random. Values are Missing at Random (MAR) when missingness in one metadata field can be explained by information recorded in another field: a language field might frequently be missing in older entries, but because the publication date itself is known, the pattern

of missingness is systematic and observable. Values are Missing Not at Random (MNAR) when a value is missing precisely because of what that missing value is, such as an origin field remaining empty specifically because the document originates from an undocumented or unknown region. The distinction matters because imputation and related reconstructions are valid only under the first two mechanisms, whereas under the third the absence of a value is itself informative and that value cannot be recovered from the observed data alone. In practice, the mechanism is rarely known, and in historical corpora the absence of a value cannot generally be assumed to be accidental. The most widespread responses treat missing inputs as a preprocessing problem. Marginalisation discards the affected attributes from the computation and therefore cannot exploit the values that are available for some records [18]. In contrast, imputation fills in the missing values from the available data using strategies that range from simple mean imputation to more elaborate model-based methods, including k-Nearest Neighbours (KNN) imputation, multiple imputation, and random-forest-based approaches [40–42]. Although imputation produces a fully populated dataset and thereby enables the use of standard computations, the estimated values often rest on assumptions about the distribution of the data and about the mechanism behind the missing values, which can rarely be verified in practice. Errors introduced in this way propagate silently into the similarity computation and into every analysis built on it. Record-linkage-specific strategies attempt to mitigate these limitations by treating missing values as a distinct category during attribute comparison, for instance by assigning a neutral similarity or a weight proportional to the amount of missing data [23, 43], but do not explicitly model or report the uncertainty introduced by the missing values. An alternative is to adapt the aggregation mechanism itself rather than the data feeding into it. The simplest instance of this idea is available-case analysis [17], in which averaging operators are applied to whichever criteria are present and the weights are adjusted accordingly. A more elaborate mechanism is found in the LSP decision method [44], in which mandatory and optional criteria are distinguished and the GCD aggregation adapts to the subset of criteria available for a given record, preserving the intended semantics of the aggregation without requiring imputation. The approach proposed in this chapter falls into the same family: the OWA operator is adapted to operate on the similarity scores that are available for a given pair of texts, with the weight vector renormalised accordingly. Unlike the LSP method, which absorbs the effect of missing inputs into the aggregated value itself, the proposed approach pairs the aggregated similarity with an explicit confidence score derived from the weight left unused when inputs are missing. The impact of the missing information is thereby retained as a distinct indicator rather than being silently compensated for. This confidence score is available for downstream analysis, so that the reliability of each aggregated similarity can be considered alongside the similarity itself. Because the proposed aggregation neither estimates missing values nor discards fields, it requires no assumption about the mechanism behind the missing values.

The work surveyed in this section addresses the two main themes on which this chapter builds: the composition of multiple criteria into a single score, and the handling of missing inputs during that composition. Existing frameworks for combining multiple criteria demonstrate how several attribute-level similarities can be aggregated into a single value, but they assume that all inputs are available for all records. The extensions that address missing inputs, whether through imputation, marginalisation, or adapted aggregation, absorb the effect of the missing information into the score itself without preserving it as a separate, interpretable value. The method proposed in this chapter builds on these two themes by adapting the OWA operator to operate on the available similarity scores with renormalised weights, while introducing an explicit confidence score that

quantifies the reliability of the aggregated similarity based on the extent of missing information. The following section introduces the formal concepts on which this fusion mechanism is built.

4.3 Preliminaries

This section provides the formal background needed to present the context-aware and responsible similarity measure proposed in this chapter. Two concepts are central to the proposed method: an aggregation operator that combines multiple similarity scores into a single value, and the similarity measures that produce the individual scores over which the aggregation operates. The aggregation makes no assumption about how those scores are obtained, so the measures introduced here reflect the choice made for this chapter rather than a requirement of the method. Section 4.3.1 gives a general introduction to aggregation operators, including the properties they are expected to satisfy and the most common instances. The OWA operator, on which the proposed method is built, is then discussed in detail in Section 4.3.2. Finally, Section 4.3.3 introduces three widely used set-based similarity measures: the Jaccard, Dice, and overlap coefficients. These are applied in the experiments to compute a similarity score for each metadata field, thereby providing the metadata inputs to the OWA aggregation.

4.3.1 Aggregation Operators

An aggregation operator combines multiple input values, such as the similarity scores computed over individual metadata fields, into a single value [28, 29]. Formally, it is a function

$$\text{aggregator} : [0,1]^n \to [0,1] \tag{4.1}$$

that maps a tuple of n inputs from the unit interval to a single value in the same interval. Aggregation operators are generally expected to satisfy two basic properties: boundary conditions, requiring that the operator returns 0 when all inputs are 0 and 1 when all inputs are 1, and monotonicity, requiring that increasing any input does not decrease the aggregated value.

Several well-known functions satisfy this definition. The simplest and most widely used aggregation operators are the minimum, the maximum, and the arithmetic mean [28]. The minimum and the maximum represent the two extremes: the minimum returns the smallest input and the maximum returns the largest, whereas the arithmetic mean is an intermediate operator that treats all inputs equally and returns their average. Weighted aggregation operators such as the weighted arithmetic or geometric mean further allow the contribution of each input to be tuned through a weight vector [28]. More expressive operators, including the GCD operator and fuzzy integrals, mentioned in Section 4.2, can express a wider range of behaviours at the cost of increased complexity. The OWA operator, on which the method proposed in this chapter is based, is introduced in the next subsection.

4.3.2 The Ordered Weighted Averaging Operator

The OWA operator, introduced by Yager [20], encompasses a wide range of aggregation behaviours, including the minimum, maximum, and arithmetic mean as special cases. The formal description of the OWA operator is provided in Definition 4.1.

Definition 4.1 (Ordered Weighted Averaging Operator [20])**.** The Ordered Weighted Averaging (OWA) operator maps a tuple of n inputs $\mathbf{a} = (a_1, a_2, \dots, a_n) \in [0,1]^n$ to a single value in the unit interval $[0,1]$, based on a weight vector $\mathbf{w} = (w_1, w_2, \dots, w_n) \in [0,1]^n$, satisfying the condition $\sum_{i=1}^{n} w_i = 1$. The operator is defined by

$$\mathrm{OWA}(\mathbf{a}, \mathbf{w}) = \sum_{i=1}^{n} a_{\rho(i)} \cdot w_i, \tag{4.2}$$

where $\rho : \{1, 2, \dots, n\} \to \{1, 2, \dots, n\}$ is a permutation that orders the elements in decreasing order, so that $a_{\rho(1)} \geq a_{\rho(2)} \geq \dots \geq a_{\rho(n)}$.

The defining feature of the OWA operator is that the weights are applied to the sorted inputs rather than to the inputs at fixed positions. Each weight w_i therefore expresses how much the i-th largest input should contribute to the aggregated value, rather than how much the input in position i should contribute, so that the weight vector is applied to the ordered values rather than to fixed sources. This source-independent character makes the OWA operator well-suited for aggregating heterogeneous criteria such as similarity scores.

Several well-known aggregation operators arise as special cases of the OWA, obtained through specific choices of the weight vector. When $\mathbf{w} = (1, 0, \dots, 0)$ or $\mathbf{w} = (0, \dots, 0, 1)$, only the largest or smallest input contributes to the aggregated value, yielding the maximum and the minimum operators, respectively. When $\mathbf{w} = \left(\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n}\right)$, all inputs contribute equally and the OWA reduces to the arithmetic mean.

The choice of weight vector is central to the behaviour of the OWA operator, and three main approaches for determining it can be distinguished [45]. Expert-based methods rely on a domain expert specifying the weights directly, based on the relative importance of the sorted positions. Linguistic quantifiers such as “most”, “at least half”, or “as many as possible” offer an alternative, allowing preferences to be expressed in natural language and then mapped to numerical weights through a function that captures their semantics [46]. Finally, data-driven methods derive the weight vector from training data, for instance by finding the weight vector that minimises an error measure on a training set [45]. In this chapter, the weight vector is determined empirically, following a data-driven approach, as discussed in Section 4.5.

4.3.3 Set-Based Similarity Measures

Set-based similarity measures compare two sets through the elements they share, taking into account only whether two elements are equal or distinct, and no further structure such as an ordering or a distance between them. They are introduced here because the metadata fields of the corpus used in the experiments of this chapter are naturally represented as sets, and the scores they produce are the metadata similarity scores that enter the OWA aggregation. Analogously to the textual similarity measure defined by Equation (3.1), a set-based

similarity measure is a function that quantifies the resemblance between two sets, A and B, by returning a value in the unit interval $[0, 1]$. Three classical members of this family of similarity measures are used in this chapter, each taking a different view of how the degree of overlap should be normalised.

The Jaccard coefficient [2],

$$\text{jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}, \tag{4.3}$$

measures the proportion of elements shared by A and B relative to the total number of distinct elements in their union. It returns 1 when A and B are identical and non-empty, and 0 when they are disjoint. When both A and B are empty, we define $\text{jaccard}(A, B) = 0$, since no shared information is available to support a non-zero similarity.

The Dice coefficient [47],

$$\text{dice}(A, B) = \frac{2 \cdot |A \cap B|}{|A| + |B|}, \tag{4.4}$$

normalises the size of the intersection by the mean size of the two sets rather than by the size of their union. Whenever two non-empty sets A and B overlap without being identical, the Dice coefficient returns a value strictly greater than the Jaccard coefficient on the same pair. As with the Jaccard coefficient, $\text{dice}(A, B) = 0$ when both sets are empty.

The overlap coefficient [6],

$$\text{overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}, \tag{4.5}$$

normalises the size of the intersection by the size of the smaller of the two sets. It reaches 1 whenever one set is a subset of the other, even when the larger set contains many elements that are not shared. When one or both of the two sets are empty, the overlap coefficient is defined to be 0, since there is no overlap between an empty set and any other set.

Together, these three coefficients span a spectrum of strictness. At the strict end, the Jaccard coefficient penalises any element that is not shared by both sets, so that high similarity requires the two sets to be nearly identical. The Dice coefficient occupies an intermediate position: by normalising against the mean set size rather than the union, it is less sensitive to elements present in only one of the two sets than the Jaccard coefficient. At the permissive end, the overlap coefficient yields high similarity for any form of containment, treating subset relationships as full matches. These measures serve as the metadata similarity inputs to the fusion mechanism presented in the next section, with the choice between them guided by the semantics of each metadata field and discussed in Section 4.5.2.

4.4 Context-Aware and Responsible Fusion of Similarity Scores

This section presents the proposed method for aggregating the similarity scores computed between pairs of short texts into a single value, while explicitly addressing the veracity issues that arise when metadata are missing. The measure rests on two components that together produce a pair of scores for each pair of texts. The

first component, presented in Section 4.4.1, computes an aggregated similarity score using the OWA operator, with a tailored mechanism to account for potentially missing information. The second component, introduced in Section 4.4.2, is a confidence score that quantifies the reliability of each aggregated similarity based on the total weight of the available inputs. Finally, Section 4.4.3 extends the pairwise computation to an entire corpus of short texts, producing a similarity-confidence pair for every pair of texts, which can then be stored in a database and used for further analysis.

4.4.1 OWA-Based Fusion of Similarity Scores

Let T be a corpus of short texts, and let $t_1, t_2 \in T$ be two of its texts. Several *similarity criteria* can be evaluated for such a pair of texts, each assessing their similarity with respect to a single aspect of the information available about them. Some criteria draw on the textual content, for instance through an orthographic or semantic measure, and others on the metadata fields associated with each text. From the two texts, a similarity tuple $\mathbf{s} = (s_0, s_1, \dots, s_m)$ is derived, containing the score returned by each criterion. The fusion mechanism presented below operates on these components, provided that at least one of them is available for every pair. In this chapter, and in the remainder of this thesis, the tuple contains a single textual similarity score s_0, computed by the hierarchical orthographic similarity measure introduced in Chapter 3, and m metadata similarity scores s_1 to s_m. Each metadata score is computed by a similarity measure chosen for the field in question, such as one of the set-based measures introduced in Section 4.3.3 when the values of a field can be represented as sets. When a criterion cannot be evaluated for a given pair, the corresponding similarity score is undefined and denoted by the symbol $\diamond$, representing a `NULL` value [48], so that $s_i \in [0,1] \cup \{\diamond\}$ for every $0 \leq i \leq m$. In the present setting, this arises when a metadata field is not recorded for t_1, t_2, or both. In this work, the textual similarity score is assumed to be available for every pair, so that $s_0 \neq \diamond$. The similarity tuple is accompanied by a weight vector of equal length, $\mathbf{w} = (w_0, w_1, \dots, w_m) \in [0,1]^{m+1}$, whose components sum to one, as required by the OWA operator. The weight vector can be determined through expert-based or empirical approaches, as discussed in Section 4.3.2.

Since the OWA operator is not defined for missing inputs, the proposed method restricts the similarity tuple to its available components and adapts the weight vector accordingly. The restricted similarity tuple $\bar{\mathbf{s}}$ is obtained by omitting the undefined components of $\mathbf{s}$ while preserving the relative order of those that remain,

$$\bar{\mathbf{s}} = (s_i)_{i \in I}, \qquad I = \{i \in \{0, 1, \dots, m\} \mid s_i \neq \diamond\}, \tag{4.6}$$

where the indices in I are taken in increasing order. Since at least one criterion is available for every pair, in this case a textual similarity score, $\bar{\mathbf{s}}$ always contains at least one component. To compute the OWA operator over $\bar{\mathbf{s}}$, the weight vector $\mathbf{w}$ must be adapted to match the reduced number of inputs. The corresponding restricted weight vector $\overline{\mathbf{w}}$ retains the first $|\bar{\mathbf{s}}|$ components of $\mathbf{w}$ and renormalises them so that they sum to one,

$$\overline{\mathbf{w}} = \left(\overline{w}_0, \overline{w}_1, \dots, \overline{w}_{|\bar{\mathbf{s}}|-1}\right), \qquad \overline{w}_i = \frac{w_i}{\sum_{j=0}^{|\bar{\mathbf{s}}|-1} w_j}, \tag{4.7}$$

with $0 \leq i < |\bar{\mathbf{s}}|$. Equation (4.7) is undefined in the degenerate case in which the original weight vector assigns no weight to the rank positions that remain after the restriction, so that the retained weights sum to zero. In that case the restricted weight vector is defined as the uniform vector $\overline{w}_i = \frac{1}{|\bar{\mathbf{s}}|}$, so that the OWA reduces to the arithmetic mean of the available similarities and no preference is imposed among them. The confidence score introduced in Section 4.4.2 is zero for such a pair, marking the aggregated value as resting on none of the intended weight.

The aggregated similarity s between t_1 and t_2 is then obtained by applying the OWA operator to the restricted similarity tuple and the restricted weight vector,

$$s = \mathrm{OWA}(\bar{\mathbf{s}}, \overline{\mathbf{w}}). \tag{4.8}$$

When no similarities are missing, $\bar{\mathbf{s}} = \mathbf{s}$ and $\overline{\mathbf{w}} = \mathbf{w}$, so that the restricted computation reduces to the standard OWA aggregation $s = \mathrm{OWA}(\mathbf{s}, \mathbf{w})$. The adapted mechanism thereby coincides with the standard OWA in the absence of missing information and extends it consistently to cases in which one or more metadata similarities are undefined. This fusion strategy uses all available information without requiring the imputation of missing values or the exclusion of incomplete metadata fields for the entire corpus. Example 4.1 illustrates the mechanism.

> **Example 4.1.** Consider three similarity tuples $\mathbf{s}_1 = (0.9, 0.7, 0.6, 0.8)$, $\mathbf{s}_2 = (0.6, 1.0, \diamond, 0.7)$, and $\mathbf{s}_3 = (0.8, \diamond, \diamond, \diamond)$, along with a common weight vector $\mathbf{w} = (0.3, 0.2, 0.3, 0.2)$. For $\mathbf{s}_1$, no similarities are missing, and the restricted tuple and weight vector coincide with the original inputs, so that $\bar{\mathbf{s}}_1 = \mathbf{s}_1$ and $\overline{\mathbf{w}}_1 = \mathbf{w}$. For $\mathbf{s}_2$, one similarity is missing, leading to the restricted tuple $\bar{\mathbf{s}}_2 = (0.6, 1.0, 0.7)$ with retained weights $(0.3, 0.2, 0.3)$, renormalised to $\overline{\mathbf{w}}_2 = (0.375, 0.25, 0.375)$. For $\mathbf{s}_3$, only the textual similarity is available, resulting in the restricted tuple $\bar{\mathbf{s}}_3 = (0.8)$ and the restricted weight vector $\overline{\mathbf{w}}_3 = (1.0)$. Applying the OWA operator to each restricted tuple and its corresponding weight vector yields the aggregated similarities $s_1 = \mathrm{OWA}(\bar{\mathbf{s}}_1, \overline{\mathbf{w}}_1) = 0.76$, $s_2 = \mathrm{OWA}(\bar{\mathbf{s}}_2, \overline{\mathbf{w}}_2) = 0.775$, and $s_3 = \mathrm{OWA}(\bar{\mathbf{s}}_3, \overline{\mathbf{w}}_3) = 0.8$. A visual overview of the mechanism for $\mathbf{s}_2$ is provided in Figure 4.1, where the restriction of the similarity tuple is indicated in green and the restriction of the weight vector is indicated in blue.

The similarity tuple is not limited in general to one textual criterion and a set of metadata criteria. A pair of texts may also be compared on several textual criteria, for instance an orthographic and a semantic measure, or on several orthographic measures with different parameters, alongside any number of metadata criteria. All criteria are aggregated in a single operation, which matches the source-independent character of the OWA operator, whose weights are associated with rank positions rather than with individual criteria. The relative contribution of each similarity score then follows from the weight vector rather than from the structure of the aggregation. A single normalised weight vector moreover remains available for the confidence score introduced in the next subsection.

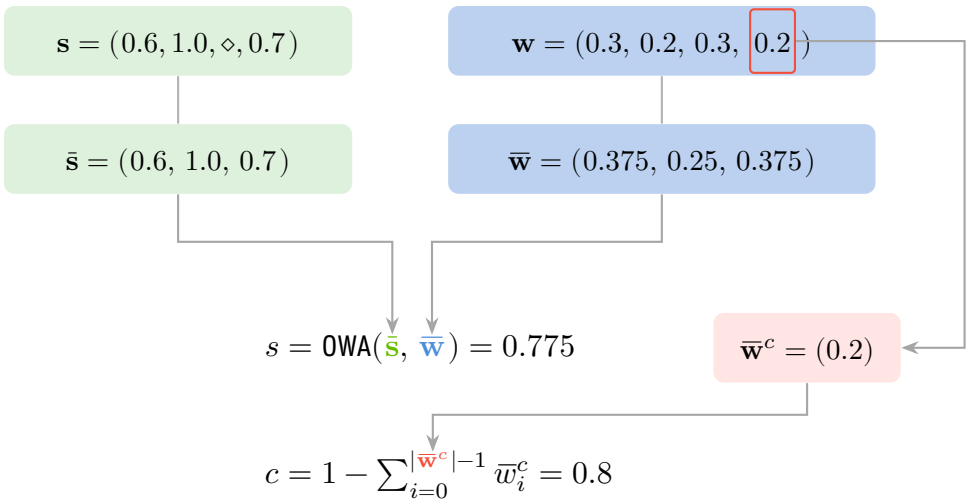


Figure 4.1: Overview of the computation of the aggregated similarity and confidence score for the similarity tuple $\mathbf{s} = (0.6, 1.0, \diamond, 0.7)$ and weight vector $\mathbf{w} = (0.3, 0.2, 0.3, 0.2)$ from Example 4.1. The restriction of the similarity tuple $\bar{\mathbf{s}}$ is indicated in green, the restriction of the weight vector $\overline{\mathbf{w}}$ is indicated in blue, and the residual weight vector $\overline{\mathbf{w}}^c$ is indicated in red.

4.4.2 Confidence Score Computation

By restricting the similarity tuple to the available components and renormalising the restricted weight vector, the fusion mechanism produces a meaningful aggregated similarity even when some metadata are missing. In doing so, however, all information about the extent of missing metadata is discarded, and the aggregated similarity alone offers no indication of how much of the original information contributed to its computation. Since the extent of missing information is central to assessing the trustworthiness of the aggregated similarity, a second score is introduced alongside it to retain this information. The confidence score $c \in [0, 1]$ reflects the total weight retained by the restriction and thereby indicates how representative the aggregated similarity s is of the complete set of criteria. Together, the similarity-confidence pair (s, c) captures both the degree of resemblance between two texts and a representation of the reliability of that assessment.

Let $\mathbf{s}$ and $\mathbf{w}$ denote the original similarity tuple and weight vector containing $m + 1$ components, and let $\bar{\mathbf{s}}$ and $\overline{\mathbf{w}}$ denote their restricted counterparts, as defined in Section 4.4.1. The residual weight vector $\overline{\mathbf{w}}^c$ collects the weights of $\mathbf{w}$ that are dropped during the restriction and forms the complement of the restricted weight vector $\overline{\mathbf{w}}$ with respect to $\mathbf{w}$,

$$\overline{\mathbf{w}}^c = \left(\overline{w}_0^c, \overline{w}_1^c, \ldots, \overline{w}_{m-|\bar{\mathbf{s}}|}^c\right) = \left(w_{|\bar{\mathbf{s}}|}, w_{|\bar{\mathbf{s}}|+1}, \ldots, w_m\right). \tag{4.9}$$

When no similarities are missing, so that $\bar{\mathbf{s}} = \mathbf{s}$, $\overline{\mathbf{w}}^c$ is empty. The confidence score c is then defined by one minus the total residual weight,

$$c = 1 - \sum_{i=0}^{|\overline{\mathbf{w}}^c|-1} \overline{w}_i^c. \tag{4.10}$$

A confidence score of 1 indicates that all similarity scores contributed to the aggregation, whereas lower values indicate that part of the information was unavailable. Because the residual weights are components of $\mathbf{w}$ and

therefore sum to at most one, the confidence score lies in the unit interval and decreases monotonically with the total residual weight. It reaches zero in the degenerate case described in Section 4.4.1, where the retained weights sum to zero and the aggregated similarity rests on none of the intended weight.

The extent of the missing information also bounds the value the aggregation would have produced had all criteria been recorded. Had the missing metadata been recorded, every component of $\mathbf{s}$ would have been available and the OWA operator would have returned a single value over the completed tuple. That value is at least the result of substituting 0 for each missing component and at most the result of substituting 1, so it lies within an interval determined by the components that are available. This interval narrows as fewer components are missing, and collapses to the aggregated similarity itself when the similarity tuple is complete. Example 4.2 illustrates the confidence score computation and the resulting bounds.

Example 4.2. Revisiting Example 4.1, the similarity tuple $\mathbf{s}_1$ contains no missing values, so the residual weight vector $\overline{\mathbf{w}}_1^c$ is empty and the associated confidence score is $c_1 = 1$. The aggregated similarity $s_1 = 0.76$ is therefore fully reliable in this context. For $\mathbf{s}_2$, one similarity is missing, resulting in the residual weight vector $\overline{\mathbf{w}}_2^c = (0.2)$, which corresponds to the last component of $\mathbf{w}$, and a confidence score of $c_2 = 1 - 0.2 = 0.8$. For $\mathbf{s}_3$, three similarities are missing, leading to the residual weight vector $\overline{\mathbf{w}}_3^c = (0.2, 0.3, 0.2)$, corresponding to the last three components of $\mathbf{w}$, and a confidence score of $c_3 = 1 - (0.2 + 0.3 + 0.2) = 0.3$. The aggregated similarities $s_2 = 0.775$ and $s_3 = 0.8$ are therefore less reliable than s_1, with s_3 being the least reliable of the three due to the larger amount of missing information, even though it has the highest similarity score. Substituting 0 and 1 for the missing components and aggregating with the original weight vector $\mathbf{w}$ shows how the aggregated similarity could still vary within bounds determined by the available information. For $\mathbf{s}_1$ there is nothing to substitute, so the bounds coincide with $s_1 = 0.76$, whereas for $\mathbf{s}_2$ they are 0.62 and 0.83. For $\mathbf{s}_3$ they are 0.24 and 0.96, leaving the aggregated similarity almost unconstrained. Figure 4.1 additionally shows the residual weight vector $\overline{\mathbf{w}}_2^c$ in red.

The confidence score does not influence the aggregated similarity itself, which is determined solely by the available similarity scores and their corresponding renormalised weights. It instead provides complementary information about that similarity, to be propagated alongside it into subsequent tasks such as querying, visualisation, ranking, or clustering. In principle, a confidence score can incorporate a variety of veracity-related concepts, but in this chapter we restrict its scope to the extent of missing information. Within that scope, availability is a binary property, since a criterion either contributes a similarity score or it does not. A metadata value that is only partially known, such as a date recorded as a range or a value given as a distribution over several possibilities, therefore remains part of the aggregation, provided that the measure chosen for that field returns a similarity score for such a value. The criterion then counts its full weight towards c, so that the confidence score reflects which criteria contributed to the aggregation rather than how precisely their underlying values are recorded. Whichever veracity concepts the confidence score encodes, the (s, c)-pairs produced by the method are examples of L-grades, satisfaction-confidence pairs on which specific L-grade logic can be directly applied [49]. This connection is further explored in Chapter 5.

With both components of the pairwise computation defined, the next subsection describes how the resulting similarity-confidence pairs can be computed for each pair of texts in a corpus.

4.4.3 Integration into the Graph-Based Text Model

The pairwise similarity and confidence computation described in the preceding subsections can be applied to every pair of texts in a corpus, producing a context-aware and responsible view of how those texts relate to one another. The resulting similarity-confidence pairs can be used directly for subsequent analysis or stored for later retrieval. In the context of this thesis, we incorporate them into the fuzzy Labelled Property Graph (LPG) that represents the corpus, extending the graph-based text model of Chapters 2 and 3 with context-aware and responsible similarity information.

We assume that the metadata required for these computations are already part of the graph-based text model introduced in Chapter 2. As discussed in Section 2.5.1, each text node is connected to its associated metadata through dedicated labelled edges and metadata nodes. For a given pair of text nodes and a set of metadata fields, the corresponding metadata values are retrieved by traversing the labelled edges from each text node to the connected metadata nodes. For each metadata field, a similarity score is then computed between the retrieved values through a specific similarity measure, such as one of the set-based measures introduced in Section 4.3.3. When no metadata node of the requested field is connected to one or both of the text nodes, no similarity can be computed for that field and the corresponding score is set to $\diamond$. The graph thereby provides all the information needed to construct the similarity tuple for any pair of text nodes.

Consider a level i of the hierarchical decomposition H, where $0 \leq i < |H|$, and let N_{h_i} denote the set of nodes representing the textual elements at that level. Consider each pair of nodes $(n_a, n_b) \in N_{h_i}^2$, where $a < b$, so that the similarity is computed only once per pair. The textual similarity already stored on the existing `SIMILAR_TO` edge from Chapter 3 is combined with the metadata similarities retrieved from the graph as described above, yielding the similarity tuple $\mathbf{s}_{a,b}$ for that pair. This tuple is aggregated through the adapted OWA into a similarity $s_{a,b}$ and accompanied by a confidence score $c_{a,b}$ derived from the residual weights of any missing metadata. By convention, and consistent with the reflexivity property of similarity measures from Equation (3.2), every node is maximally similar and fully reliable with respect to itself, so that $s_{a,a} = 1$ and $c_{a,a} = 1$; these self-pairs require no edge in the graph. A new fuzzy edge e with label $\lambda_E(e) =$ `CONTEXT_SIMILAR_TO` and $\rho(e) = (n_a, n_b)$ is added to the graph, distinct from the textual `SIMILAR_TO` edge so that both kinds of similarity remain available for analysis. In accordance with Definition 3.1, the aggregated similarity defines the membership grade of this edge,

$$\mu_{\tilde{E}}(e) = s_{a,b}, \tag{4.11}$$

and the confidence score is stored as an additional edge property through the property function σ,

$$\sigma(e, \texttt{confidence}) = c_{a,b}, \tag{4.12}$$

so that each CONTEXT_SIMILAR_TO edge carries both the extent of resemblance between the two textual elements and the reliability of that assessment.

The full procedure for enriching the graph at level i can be summarised as follows:

1. identify the set of textual element nodes N_{h_i} at the level of interest i, $0 \leq i < |H|$;
2. for each pair $(n_a, n_b) \in N_{h_i}^2$ with $a < b$, retrieve the textual similarity score from the existing SIMILAR_TO edge and compute the metadata similarities, setting the similarity of missing metadata to $\diamond$;
3. construct the similarity tuple $\mathbf{s}_{a,b}$ from these scores;
4. apply the adapted OWA on $\bar{\mathbf{s}}_{a,b}$ and the predefined weight vector $\mathbf{w}$ to obtain the aggregated similarity $s_{a,b}$ and compute the confidence score $c_{a,b}$ from the residual weights;
5. add a new CONTEXT_SIMILAR_TO edge e between n_a and n_b, i.e. $\rho(e) = (n_a, n_b)$, with $\mu_{\tilde{E}}(e) = s_{a,b}$ and $\sigma(e, \texttt{confidence}) = c_{a,b}$.

Because the underlying similarity measures are symmetric, so that $s_{a,b} = s_{b,a}$ and $c_{a,b} = c_{b,a}$, each edge is interpreted as undirected and stored only once per node pair.

In the resulting graph, the output of the method is directly accessible through the query and visualisation capabilities of the graph database, and the (s, c)-pairs can be propagated into subsequent analytical tasks. Section 4.5 evaluates the performance of the aggregated similarity against the textual similarity from Chapter 3 and demonstrates the practical applicability of the resulting graph through querying and visualisation examples.

4.5 Experiments

The proposed context-aware and responsible similarity measure is evaluated on the Database of Byzantine Book Epigrams (DBBE) corpus [21, 22], whose rich but unevenly populated metadata provide a realistic setting in which to assess how the measure handles missing information. Pairs of epigrams identified as similar by the aggregated score are checked against the expert-curated epigram types, and the resulting precision, recall, and F_1-score are reported for two weight vector configurations across twenty similarity thresholds. The textual similarity from Chapter 3, which does not incorporate any metadata, provides a natural baseline against which the contribution of the contextual metadata is assessed. The accompanying confidence scores are not included in the quantitative comparison in this chapter, but their behaviour and the analytical opportunities they offer are illustrated separately in Section 4.5.5 and further explored in Chapters 5 and 6.

Section 4.5.1 describes the dataset and the metadata fields used in the experiments. The similarity measures applied to the textual content and to each metadata field are introduced in Section 4.5.2. Section 4.5.3 then presents the experimental setup, including the weight vector configurations and the evaluation methodology, after which the results are reported and discussed in Section 4.5.4. Finally, the practical applicability of the resulting similarity-confidence pairs is demonstrated in Section 4.5.5 through querying and visualisation in a graph database environment.

4.5.1 Dataset

The experiments are conducted on the same subset of 496 epigrams introduced in Section 3.5.1, drawn from 21 expert-curated epigram types and preprocessed as described in Section 2.6.1. Since most metadata fields in the DBBE relate to complete epigrams rather than to individual verses or words, the experiments are conducted at the epigram level only. The graph-based representation from Chapters 2 and 3 is extended accordingly, with each metadata field linked to its associated epigram nodes through dedicated metadata nodes and labelled edges, in line with Section 2.5.1.

The five metadata fields considered in the experiments are summarised below.

- **Geographical location**: The geographical location of writing is captured through a geographical hierarchy in the DBBE, structured across five granularities. These range from a `country` at the broadest level, through a `part` of a country, a `region` within that part, and a `city`, down to a fixed `site` such as a monastery at the narrowest. The granularity at which an epigram is associated with a geographical location varies from one epigram to another.
- **Genre**: The genre of an epigram is determined by domain experts and describes its type, indicating, for instance, whether the epigram concerns the text or image it accompanies, or its scribe, patron, or reader. An epigram is typically assigned a single genre, though multiple genres occasionally occur.
- **Metre**: The metre describes the metrical structure of an epigram's verses, expressed through the number of syllables per verse, and is likewise determined by domain experts. A single metre is recorded in most cases, with multiple metres arising in the rare cases where the metrical structure changes within a single epigram.
- **Scribe**: Each epigram may be linked to one or, occasionally, more persons in the role of scribe, indicating those responsible for composing or copying the epigram.
- **Subject**: Each epigram may also be linked to one or more persons in the role of subject, indicating those referenced within the epigram. Only persons are recorded as subjects in the DBBE.

As a concrete illustration, Table 4.1 shows the textual content and metadata values for epigram 21179. The textual content consists of two verses and translates to "Just like travellers rejoice in seeing their homeland, so do writers in reaching the end of the book", a variant of the epigram in Table 1.1. Geographically, it is annotated at the `city` granularity with the value *Corfinio*, which the geographical hierarchy locates within the `region` *Abruzzo*, the `part` *Southern Italy*, and the `country` *Italy*. Its genre is recorded as *Scribe-related* and its metre as *Dodecasyllable*, indicating that each verse consists of twelve syllables. The recorded scribe is *Leontios*, and no subject is recorded.

The five metadata fields are unevenly populated across the subset, and the extent of missing information varies considerably between fields. Table 4.2 reports, for each field, the number and proportion of epigrams for which at least one value is available. The genre and metre fields are almost fully populated, which is unsurprising given that they are systematically annotated by domain experts. The remaining three fields are far sparser, with the

Table 4.1: Textual content and metadata of epigram 21179 [22] after preprocessing. The dash in the subject row indicates that no value is recorded in the DBBE.

Metadata	Values for epigram 21179
Text	1. ωσπερ ξενι χαιρουσιν ιδιν πατριδα 2. ουτως και ι γραφοντες βιβλιου τελος
Location	Corfinio
Genre	Scribe-related
Metre	Dodecasyllable
Scribe	Leontios
Subject	—

location field being the least complete: fewer than one in six epigrams carries a recorded location. This uneven availability makes the dataset well-suited to assessing the proposed measure under realistic conditions of metadata sparsity. The same trends are observed across the entire DBBE corpus, although the exact percentages differ.

Table 4.2: Availability of each metadata field across the subset of 496 epigrams. For each field, the table reports the number and proportion of epigrams for which at least one value is recorded in the DBBE.

Metadata	Amount available	Availability percentage %
Location	72	14.5 %
Genre	495	99.8 %
Metre	494	99.6 %
Scribe	144	29.0 %
Subject	193	38.9 %

Values are absent for different reasons across the five fields. Location and scribe are missing when the origin of a manuscript or the identity of an author is not known, and subject is missing mainly for epigrams that are incomplete, so that the annotation could not be determined. The absence of a value is in these cases related to the value itself, which corresponds to MNAR data, whereas the few gaps in the otherwise systematic genre and metre annotation are accidental and correspond to MCAR data. The subject field also covers epigrams that reference no person, for which no value applies. The proposed method requires no assumption about these mechanisms, since it neither estimates missing values nor discards the affected fields. Before the proposed method can be applied, similarity measures for the textual content and for each of the five metadata fields must be defined; these are introduced in Section 4.5.2.

4.5.2 Similarity Measures

Computing the aggregated similarity introduced in Section 4.4.1 requires individual similarity measures for the textual content and for each of the five metadata fields described in Section 4.5.1. The choice of measure for

each metadata field is informed by the properties of the values it stores and by domain expert knowledge. For the genre, metre, scribe, and subject fields, whose values can be treated as flat sets of categorical elements, a set-based measure from Section 4.3.3 is applied directly. The location field stores a richer geographical hierarchy that is first transformed into a set representation, after which one of the same set-based similarity measures is used. The remainder of this subsection introduces the measure used for each field along with the rationale behind its choice.

Textual Similarity

For the textual content, the hierarchical orthographic similarity measure of Chapter 3 is reused without modification, applied to the same graph-based representation of the corpus introduced in Chapter 2. We use the Custom 1 parameter configuration of Table 3.1, which yielded the highest peak F_1-score on the epigram subset in Section 3.5.3 and therefore provides a strong textual baseline. Pairwise epigram-level scores are taken directly from the `SIMILAR_TO` edges, ensuring that every comparison in this chapter rests on the same textual evidence as in the previous one.

Location Similarity

The geographical metadata in the DBBE are organised across five granularity levels, ordered from the broadest to the narrowest: `country`, `part`, `region`, `city`, and `site`. Epigrams can be associated with a single location at one of these levels, but the level used varies from one epigram to another. When no geographical annotation is recorded for an epigram, no location similarity is computed and the corresponding component of the similarity tuple is set to $\diamond$. The geographical information held by the DBBE is restricted to places connected to the corpus, namely those where epigrams were known to have been written or copied, rather than to a complete world geography. To enable comparison at any granularity, all known geographical locations from the DBBE are represented as a tree, an excerpt of which is shown in Figure 4.2. A virtual root represents the world, the five granularity levels appear as successive depths beneath it, and every location is placed at the depth corresponding to its granularity. Whenever an intermediate level along the path from the root to a location is missing in the DBBE, that level is filled with the value of the closest known ancestor. For instance, the city *Serres* is recorded directly under the part *Central Macedonia* without an intermediate `region` value, so an intermediate `region` node is inserted with the inherited value *Central Macedonia*. This filling guarantees that every geographical location lies at the end of a complete path from the root to itself. When new locations are added to the DBBE and associated with epigrams, or when existing epigrams are linked by its curators to locations that are already known, the tree is updated accordingly, with new nodes added at the appropriate depth and missing intermediate levels filled as needed.

For a chosen granularity g on which to compute the similarity, the location of an epigram is represented by a set L_g derived from the geographical tree. Three cases arise. When g matches the granularity of the recorded location for an epigram, L_g is the singleton containing that recorded location. When g is above the recorded granularity, L_g contains the unique ancestor of the recorded location at level g. When g is below the recorded granularity, every branch beneath the recorded location is descended towards level g. On branches that reach level g in the tree, the corresponding node is added to L_g directly. On branches that terminate before reaching

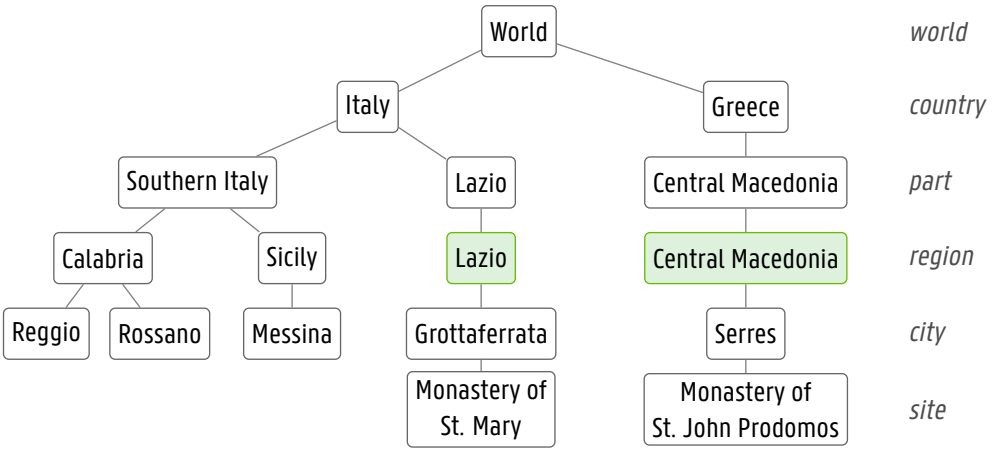


Figure 4.2: Excerpt of the geographical location tree used to compare geographical metadata. The five granularity levels and the root are labelled on the right, ordered from the `world` root at the top to the `site` level at the bottom. Nodes shown in green denote values inherited from the closest known ancestor when the corresponding granularity is missing in the original DBBE hierarchy: the *Lazio* and *Central Macedonia* nodes at the `region` level inherit their values from *Lazio* and *Central Macedonia* at the `part` level.

level g, the deepest available descendants on those branches are added to L_g instead, mimicking the filling of missing ancestors during tree construction and ensuring that the comparison is not affected by the absence of deeper levels in the hierarchy. The set L_g thereby contains an entry for every branch beneath the recorded location, so that the comparison reflects the full breadth of what lies beneath it rather than only those branches that happen to reach the chosen level g.

The similarity between two locations at granularity g is then computed by the Dice coefficient defined by Equation (4.4), applied to the corresponding pair of sets. The Dice coefficient is preferred over the more permissive overlap coefficient, since an epigram associated with a specific city is not necessarily equivalent to another epigram associated with the broader region that contains it. At the same time, the Dice coefficient remains less strict than the Jaccard coefficient, returning a moderate similarity whenever there is partial overlap. An epigram without a geographical annotation could in principle be placed at the world root and compared in the same way, rather than being assigned $\diamond$. This is not done here, since the criterion would then be treated as available and the confidence score would no longer record that no location is known. The resulting similarity would moreover say little about the pair, as a root annotation expands to every node at the chosen granularity, so that the Dice coefficient approaches zero for any epigram it is compared with. Example 4.3 illustrates the similarity computation at three granularities, including the case where the requested granularity lies below the deepest available level.

> **Example 4.3.** Consider two epigrams, the first associated with the `region` *Calabria* and the second with the `city` *Rossano*, both drawn from the illustrated tree in Figure 4.2. At `part` granularity, both annotations resolve upward to the same ancestor *Southern Italy*, yielding $L_{\texttt{part},1} = L_{\texttt{part},2} = \{\textit{Southern Italy}\}$ and a Dice similarity of 1. At `city` granularity, the first annotation expands downward to its descendants $L_{\texttt{city},1} = \{\textit{Reggio}, \textit{Rossano}\}$, while the second remains at the level at which it is annotated, $L_{\texttt{city},2} = \{\textit{Rossano}\}$, yielding a Dice similarity of $\frac{2\cdot 1}{2+1} = \frac{2}{3}$. At `site` granularity, neither *Calabria* nor *Rossano* has

> any descendants of the `site` granularity in the geographical hierarchy, and the propagation rule applies to both annotations. The branches beneath *Calabria* terminate at *Reggio* and *Rossano*, both at the `city` level, so their values are included, yielding $L_{\texttt{site},1} = \{\textit{Reggio}, \textit{Rossano}\}$. The branch beneath *Rossano* consists of *Rossano* itself, which is included likewise, so $L_{\texttt{site},2} = \{\textit{Rossano}\}$. The Dice similarity at `site` granularity is therefore $\frac{2 \cdot 1}{2+1} = \frac{2}{3}$, the same value as at `city` granularity, since the data offer no further refinement at the deeper level.

An alternative would be to compare each pair at the most specific level that the two locations share, rather than at a granularity fixed in advance for the whole corpus. Both locations resolve to a single node at that level, so the comparison returns 0 or 1 and treats a location as fully matching any broader location that contains it, discarding the graded distinctions the measure is intended to capture. A granularity fixed for the whole corpus preserves those distinctions, and leaves the level as a single parameter that is determined empirically, as in Section 4.5.3, or chosen by a domain expert to suit a particular analytical task.

Genre, Metre, Scribe, and Subject Similarity

The four remaining metadata fields are genre, metre, scribe, and subject. Genres and metres are selected from a fixed list maintained by domain experts, whereas scribe and subject values refer to curated person records. Where a value is recorded, most epigrams carry a single value per field. Assignments of two or three values arise occasionally, for instance when the metrical structure changes within an epigram or when several persons are recorded as subjects. The recorded values are all treated as sets of curated values and used directly without further preprocessing. When no value is recorded for a field, no similarity is computed for that pair on that field, the corresponding component of the similarity tuple is set to $\diamond$, and the aggregation mechanism of Section 4.4.1 restricts the tuple accordingly. The similarity between two epigrams on any of these four fields is computed as the Jaccard coefficient defined by Equation (4.3), applied to the corresponding pair of sets. The Jaccard coefficient is the strictest of the three set-based measures introduced in Section 4.3.3. This strictness is appropriate for fields whose values denote categorical distinctions without an internal hierarchy and where partial overlap should not contribute as strongly to the similarity as it does for location. In the dominant case of two singleton sets, the Jaccard coefficient reduces to an equality check, returning 1 for matching values and 0 otherwise. Example 4.4 illustrates this calculation on the genre metadata.

> **Example 4.4.** Consider three epigrams whose genre metadata are $C_1 = \{\textit{Scribe-related}\}$, $C_2 = \{\textit{Scribe-related}, \textit{Patron-related}\}$, and $C_3 = \{\textit{Patron-related}, \textit{Text-related}\}$. Computing the Jaccard similarity for each pair yields $\text{jaccard}(C_1, C_2) = \frac{1}{2}$, $\text{jaccard}(C_1, C_3) = 0$, and $\text{jaccard}(C_2, C_3) = \frac{1}{3}$.

The textual and metadata similarity measures introduced above are combined into a single aggregated score through the OWA-based mechanism of Section 4.4.1. The weight vectors and threshold configurations governing this aggregation are specified next, in Section 4.5.3.

4.5.3 Experimental Setup

Two OWA weight vector configurations for the proposed method are examined in the experiments, one determined empirically and one defined without tuning. To assess the impact of adding the chosen metadata fields on the performance of the textual similarity measure, the aggregated similarity is evaluated for each of these two configurations through the α-cut procedure of Section 3.5.2, again with twenty α values ranging from 0.05 to 1 in steps of 0.05, yielding $20 \cdot 2 = 40$ evaluations in total. The textual similarity measure from Chapter 3 using the Custom 1 parameter configuration serves as the baseline for comparison. The evaluation reports precision, recall, and F_1-score, and the empirical weight vector is determined by maximising the F_1-score. Confidence scores accompany every aggregated similarity by construction, but they are not part of the present α-cut evaluation, since the ground truth records only which epigrams belong together and therefore supports no quality measure that assesses reliability rather than grouping. Section 4.5.4 returns to this point in more detail. The confidence score is therefore examined qualitatively in Section 4.5.5 and quantitatively in Chapter 6, where it enters the clustering input and the resulting partitions are compared against those obtained from the similarity alone.

Evaluation Methodology

The evaluation methodology of Section 3.5.2 is applied directly to the aggregated similarities instead of to the purely textual scores from Chapter 3. An α-cut is performed on the fuzzy similarity relation $\tilde{S}$ formed by the aggregated scores, and the resulting partition of pairs is matched against the expert-curated epigram types through the True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN) counts defined in Section 3.5.2. From these counts, recall, precision, and F_1-score are computed for each α value using Equations (3.24), (3.25) and (3.26).

Weight Vector Configurations

The first weight vector for the OWA aggregation is obtained empirically, by maximising the F_1-score, and the second corresponds to the arithmetic mean over all six similarities. Together, these two configurations allow the contribution of the contextual metadata and the impact of empirical weight tuning to be assessed independently.

The empirical weight vector is determined through a two-stage grid search over the space of possible weight vectors, guided by the quality measures obtained from the α-cut procedure. The search is conducted independently at each of the five location granularities, and the granularity that yields the highest F_1-score is selected. In the first stage, a coarse grid search with steps of 0.05 is performed over all weight vectors where each component is drawn from $\{0, 0.05, 0.10, \dots, 1.00\}$ and where all components sum to 1. The best-performing vector over all α values is retained. In the second stage, a finer grid search with steps of 0.005 is performed within 0.05 of each component of the retained vector. Concretely, all valid weight vectors are considered whose i-th component, with $0 \leq i < 6$, lies within 0.05 of the i-th component of the retained vector. This local refinement allows for more precise tuning while keeping the computational cost manageable. Whichever valid vector maximises the F_1-score over all α values in this refined grid is retained as the final empirical weight

vector, denoted $\mathbf{w}_{F_1}$. The granularity at which it is obtained is selected as the granularity for the location similarity in all further experiments.

The average weight vector, which causes the OWA to reduce to the arithmetic mean, is constructed without tuning and serves as a simple baseline for comparison against the empirical vector. This weight vector is denoted $\mathbf{w}_{\text{avg}}$ with all weights equal to $\frac{1}{6}$, so that the weight is distributed equally over all six rank positions. Comparing $\mathbf{w}_{\text{avg}}$ against the empirically determined vector shows whether informed weighting is necessary, or whether an unweighted aggregation already performs well.

Because $\mathbf{w}_{F_1}$ is optimised directly on the data against which it is then evaluated, the scores reported for it should be read as a best case for this corpus rather than as an estimate of performance on unseen data. This framing is deliberate and consistent with the intended application of the measure. The weight vector is not a fixed constant of the method but an adjustable parameter through which the aggregation can be calibrated towards the analytical priorities of the experts using it. In the decision-support setting for which the measure is designed, moreover, every derived result remains subject to expert review rather than being applied automatically. The empirical search therefore demonstrates what the measure can achieve when it is aligned with the expert-curated ground truth, while $\mathbf{w}_{\text{avg}}$ shows what an entirely untuned configuration yields. Held-out protocols such as cross-validation are not employed here, since the reported scores are not intended as estimates of performance on unseen data.

The expert-curated types against which the F_1-score is maximised are themselves compiled primarily on textual grounds, with metadata consulted only where the text is ambiguous or incomplete. The margin by which contextual metadata can improve on a strong textual baseline is therefore limited by how the reference partition was assembled, a point taken up again in Section 4.5.4.

4.5.4 Results

This subsection presents and discusses the results of the experiments described above. The resulting weight vectors and the selected granularity for the location similarity are reported first. The precision, recall, and F_1-score obtained under each configuration are then presented across the twenty α values, after which a discussion situates the findings against the textual baseline of Chapter 3 and against the broader scope of the work.

Selected Weight Vectors and Location Granularity

The empirical search for $\mathbf{w}_{F_1}$ achieved its highest F_1-score at the `city` granularity for the location similarity measure. Accordingly, the `city` granularity is fixed for both weight vector configurations and for any subsequent experiments. The weight vector that yielded the best F_1-score across all α values for this subset of the corpus is $\mathbf{w}_{F_1} = (0.005, 0.200, 0.770, 0.000, 0.005, 0.020)$, which places most of the weight on the second and third rank positions. The low weights at the last three rank positions are consistent with the sparsity of the metadata: the lowest-ranked similarities are frequently absent from the restricted tuple, making it natural for the search to assign them little weight. The average weight vector is $\mathbf{w}_{\text{avg}} = \left(\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}\right)$, which

distributes the weight equally across all six rank positions. These two weight vectors are compared next in terms of quality measures across the twenty α values, alongside the textual baseline of Chapter 3.

Quality Measure Results

Table 4.3 reports the precision, recall, and F_1-score obtained by $\mathbf{w}_{F_1}$ and $\mathbf{w}_{\text{avg}}$ across the twenty α values. The corresponding F_1-score curves for both configurations, alongside the textual baseline of Chapter 3, are plotted in Figure 4.3.

Table 4.3: Experimental results for $\mathbf{w}_{F_1}$ and $\mathbf{w}_{\text{avg}}$ across the twenty α values. For each quality measure, the highest value within each configuration is highlighted in bold.

α	$\mathbf{w}_{F_1}$			$\mathbf{w}_{\text{avg}}$		
	Precision	**Recall**	**$\mathbf{F_1}$-score**	**Precision**	**Recall**	**$\mathbf{F_1}$-score**
0.05	0.2974	**0.9992**	0.4583	0.1122	**1.0000**	0.2018
0.10	0.3112	0.9985	0.4745	0.1442	**1.0000**	0.2520
0.15	0.3134	0.9971	0.4769	0.1487	**1.0000**	0.2589
0.20	0.3168	0.9901	0.4800	0.1585	**1.0000**	0.2737
0.25	0.3569	0.9796	0.5232	0.1665	**1.0000**	0.2855
0.30	0.3670	0.9781	0.5338	0.1769	**1.0000**	0.3006
0.35	0.4379	0.9669	0.6028	0.1786	**1.0000**	0.3031
0.40	0.5180	0.9545	0.6715	0.2874	0.9993	0.4465
0.45	0.5250	0.9362	0.6727	0.3505	0.9985	0.5189
0.50	0.5437	0.9217	0.6840	0.3514	0.9974	0.5198
0.55	0.5619	0.8977	0.6912	0.3793	0.9930	0.5490
0.60	0.5893	0.8585	0.6989	0.3967	0.9803	0.5648
0.65	0.6214	0.7959	0.6979	0.4040	0.9550	0.5678
0.70	0.6700	0.7445	0.7053	0.3997	0.9034	0.5542
0.75	0.7530	0.6883	**0.7192**	0.5355	0.8297	0.6509
0.80	0.7477	0.6280	0.6826	0.6070	0.7679	**0.6780**
0.85	0.7453	0.5566	0.6372	0.6823	0.6667	0.6744
0.90	**0.7607**	0.4772	0.5865	0.8657	0.5534	0.6752
0.95	0.7508	0.3913	0.5144	0.9298	0.4181	0.5768
1.00	0.7403	0.3161	0.4430	**0.9986**	0.2261	0.3687

The two configurations differ substantially across the threshold range. $\mathbf{w}_{\text{avg}}$ peaks at an F_1-score of 0.6780 at $\alpha = 0.80$, with a precision of 0.6070 and recall of 0.7679, whereas $\mathbf{w}_{F_1}$ peaks at an F_1-score of 0.7192 at $\alpha = 0.75$, with a precision of 0.7530 and recall of 0.6883. The empirical configuration therefore outperforms the average configuration by approximately four percentage points on the F_1-score, and the gap between the two curves is visible across most of the α-threshold range in Figure 4.3. The arithmetic mean smooths out the differences between the six similarities, treating ranked similarities as equally important. This smoothing translates into an F_1-score that consistently lies below that of $\mathbf{w}_{F_1}$ across most of the threshold range. The

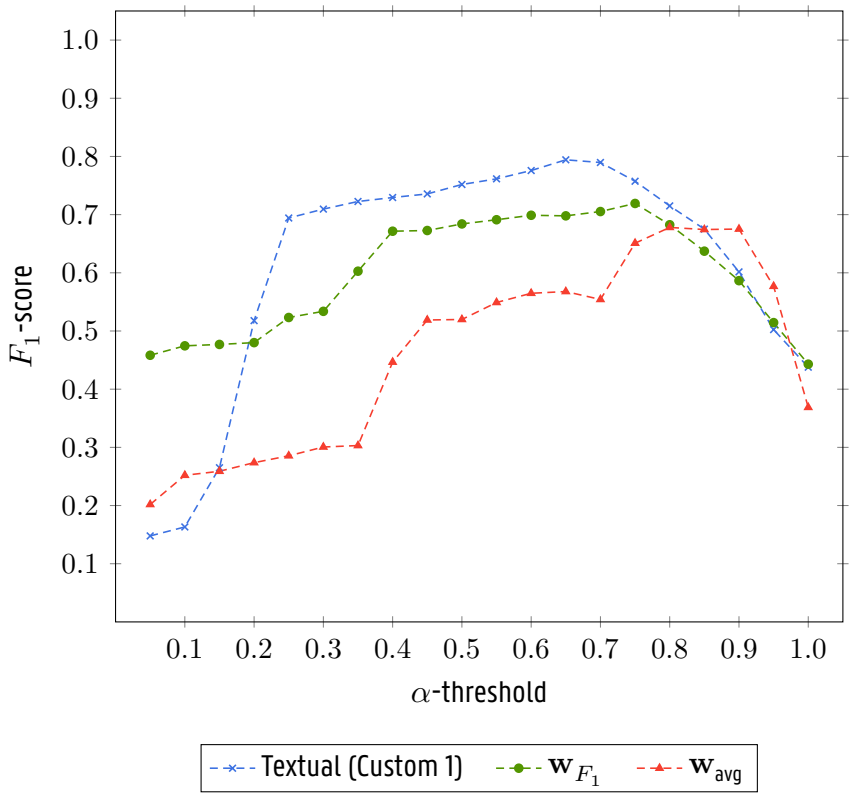


Figure 4.3: F_1-score as a function of the α-threshold for the empirical and average weight vector configurations of the proposed measure, plotted alongside the textual baseline (Custom 1 configuration of Chapter 3 on the epigram subset).

gap narrows only at $\alpha \geq 0.85$, where the average configuration's higher precision begins to compensate for its lower recall, and the average configuration briefly overtakes the empirical one between $\alpha = 0.85$ and $\alpha = 0.95$. The contrast between these two weight vectors highlights the importance of informed weighting in the aggregation process, as the empirical weight vector yields a substantial improvement over the average weight vector when considering the maximum F_1-score over all α values. Beyond the peak performance, $\mathbf{w}_{F_1}$ also demonstrates greater robustness to the threshold selection, with a much flatter F_1-score curve across the α-threshold range than $\mathbf{w}_{\text{avg}}$, whose F_1-score drops more sharply as α moves away from its optimal value of 0.80.

The peak F_1-score of $\mathbf{w}_{F_1}$ falls below the corresponding peak of the textual baseline reported in Table 3.3, where the Custom 1 configuration of Chapter 3 reached an F_1-score of 0.7942 at $\alpha = 0.65$ with a precision of 0.8271 and recall of 0.7638. At medium and high thresholds, the textual baseline therefore retains the upper hand in this comparison. The picture, however, changes at low thresholds. At $\alpha = 0.05$, $\mathbf{w}_{F_1}$ achieves a precision of 0.2974 at a recall of 0.9992, against the textual baseline's precision of 0.0798 at a similar recall point, a difference of more than 0.2 in precision while preserving near-perfect recall, which also results in an F_1-score approximately 0.3 higher than the textual baseline. This advantage persists up to $\alpha = 0.15$, with the textual baseline overtaking $\mathbf{w}_{F_1}$ at $\alpha = 0.20$ and moving decisively ahead from $\alpha = 0.25$ onward. The aggregated similarity, by incorporating contextual metadata, is able to filter out a large fraction of the FPs that the textual measure admits at low α values while retaining most of the TPs. This filtering yields a substantially

higher precision than the textual baseline at these thresholds. Even though the textual baseline achieves a higher peak F_1-score at medium and high thresholds, the aggregated similarity's substantial improvement in precision and F_1-score at low thresholds indicates that the metadata provide valuable information that the textual measure alone cannot capture, particularly in cases where the textual similarity is too permissive to distinguish effectively between TPs and FPs.

The behaviour at high α values is also informative. $\mathbf{w}_{F_1}$ never exceeds a precision of 0.7607 across the whole threshold range and reaches only 0.7403 at $\alpha = 1.00$, where the textual baseline reaches a precision of 0.9971 in Table 3.3. The aggregated similarity, by combining heterogeneous criteria, never produces the same proportion of pairs reaching scores at or near 1 as the purely textual measure does, since perfect agreement on text alone is far more common in the corpus than perfect agreement on text and on every available metadata field.

Discussion

Several observations can be drawn from these results. The aggregated similarity does not produce a higher peak F_1-score than the textual baseline of Chapter 3 under either configuration, which may appear surprising given that the incorporation of additional metadata was expected to strengthen the similarity assessment beyond what the textual content alone can provide. A closer examination of the evaluation setting, however, suggests that this outcome cannot readily be interpreted as a failure of the proposed measure.

The aggregated score is a fundamentally more complex similarity than the textual similarity it extends. For any given pair of epigrams, the aggregated score is derived from a different combination of textual and contextual criteria, with weights that depend on both the rank order of the available similarities and the extent of missing metadata for that pair. The metadata themselves are sparse: epigrams that are unrelated according to the textual similarity may still share metadata values that contribute positively to their aggregated score, and vice versa. The aggregated similarity therefore cannot be interpreted uniformly across pairs in the way that the textual similarity can. The α-cut evaluation, by design, reduces this heterogeneous relation to a binary partition: either the aggregated similarity meets the threshold or it does not, and all similarities are treated alike, with no regard for the composition of the score that led to that outcome. This strict treatment may obscure the contribution of the measure, and more nuanced evaluation techniques that do not depend on a fixed threshold partition, such as clustering, may be better suited to revealing it. Indeed, the advantage that $\mathbf{w}_{F_1}$ demonstrates at low α values, where it achieves substantially higher precision than the textual baseline while preserving near-perfect recall, may be an early indication of the measure's ability to surface finer-grained groupings that the textual similarity alone cannot distinguish. This potential is explored in Chapter 6, where the proposed measure is evaluated in terms of its ability to produce meaningful clusters that align with the expert-curated types.

A second observation concerns the role of the confidence score, which is not used in the α-cut evaluation. The aggregated similarity is only one of the two outputs of the proposed measure, and the confidence score, reflecting how much of the original information contributed to that value, is equally central to the method. Pairs with a high aggregated similarity but a low confidence score may warrant a different interpretation from

pairs whose identical similarity score rests on a full set of evidence, yet the α-cut treats the two alike. This distinction cannot be recovered from the evaluation, since the expert-curated types record which epigrams belong together and not how well each grouping is supported by the available metadata. Introducing a second threshold on the confidence would restrict the evaluation to the pairs that meet it, so that the reported scores would no longer be comparable with the textual baseline of Chapter 3. Techniques that reason jointly over similarity and confidence are therefore better placed to exploit both outputs of the measure. The practical value of the confidence score for querying and manual exploration is demonstrated in Section 4.5.5, and its role in further analytical tasks is explored in Chapters 5 and 6.

A third observation concerns the ground truth against which the evaluation is performed. The expert-curated epigram types are compiled manually by a small group of scholars and group epigrams primarily on the basis of textual similarity, with metadata serving as a secondary indicator when the textual content is ambiguous or incomplete, as discussed in Sections 1.3 and 3.5.1. A measure that systematically incorporates contextual metadata may therefore produce groupings that reflect a genuinely different, and potentially more fine-grained, view of the corpus than the one captured by the expert-curated types. The fact that $\mathbf{w}_{F_1}$ achieves a peak F_1-score of 0.7192 suggests that the proposed measure recovers a substantial portion of the expert-curated groupings, and the difference with the textual baseline may in part reflect a divergence from these groupings rather than a weakness of the proposed approach. The expert types are themselves crisp partitions, assigning each epigram to exactly one type or to none, even though the corpus exhibits partial textual similarities, semantic variants, and incomplete texts that do not fit neatly into a sharp categorisation. The scores reported here are therefore best read as a comparison against one defensible scholarly interpretation of the corpus, not as an absolute measure of quality. Optimising the weight vector against these types, as done for $\mathbf{w}_{F_1}$, accordingly amounts to calibrating the measure towards current expert judgement rather than towards an objective standard, and a differently weighted configuration would encode a different, equally legitimate analytical perspective on the corpus. More broadly, the motivation for developing these techniques is not to reproduce existing expert groupings but to provide scholars with flexible tools for exploring the corpus on the basis of their own analytical criteria, whether textual, contextual, or a combination of both. By making the textual measure, the subset of metadata, the similarity measures for each field, and the aggregation weights all configurable, the framework enables researchers to tailor the analysis to their specific research questions and interests. The evaluation against the expert-curated types serves to demonstrate that the proposed measure recovers these groupings to a meaningful degree; it is not intended as the sole criterion by which the measure should be judged.

These observations suggest that the α-cut F_1-score reported here provides a partial and somewhat conservative view of what the proposed measure offers. They also clarify how the reported scores should be read. They are a best case with respect to the empirical tuning of the weight vector, but also a conservative estimate of the performance of the measure, since the evaluation rewards agreement with a primarily text-based interpretation that the measure, by incorporating contextual metadata, is deliberately designed to look beyond. The present results document its behaviour under one particular evaluation method and identify where it adds value relative to the textual baseline. The alternatives within this evaluation are limited, since the expert-curated types form a partition while the measure produces graded pairwise scores, so that any comparison with them requires those

scores to be turned into groups, either through a threshold as in the α-cut or through clustering. The latter is pursued in Chapter 6 on the same subset and against the same expert-curated types, where the resulting partitions are additionally assessed through measures that correct for the agreement expected by chance, and through an internal quality measure that requires no ground truth at all. The following subsection turns from the quantitative evaluation to the use of the resulting similarity-confidence pairs in practice.

4.5.5 Practical Applicability

In addition to the quantitative evaluation, and in line with the approach of Sections 2.6 and 3.5.4, the practical applicability of the proposed measure is demonstrated through querying and visualisation in the graph database environment. For this demonstration, all 45 epigrams belonging to epigram type 1982 [22] are selected for deeper analysis. This type is chosen for its medium size and for the variation in metadata availability across its members, allowing for an illustrative demonstration of the confidence scores. The similarities are computed between all pairs using the same textual and contextual criteria as in the preceding experiments: the Custom 1 configuration for the textual similarity from Chapter 3, the geographical location similarity at the `city` granularity, and the Jaccard similarity for the genre, metre, scribe, and subject metadata fields. The average weight vector $\mathbf{w}_{\text{avg}} = \left(\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}\right)$ is used rather than the empirical vector, because the equal weights cause each missing field to reduce the confidence by exactly $\frac{1}{6}$, producing a more varied distribution of confidence scores across pairs. The epigrams and their textual similarities are stored in the Neo4j graph database [50] as described in Sections 2.6 and 3.5.4, and the similarity-confidence pairs are stored on the `CONTEXT_SIMILAR_TO` edges, with the membership grade represented as a `similarity` property and the confidence score as a `confidence` property.

Confidence-Based Similarity Filtering

A first way to exploit the similarity-confidence pairs is to filter query results on both scores simultaneously, so that only relationships whose similarity and confidence both meet a given minimum are returned. Listing 4.1 shows a Cypher query [51] that retrieves all pairs of epigrams whose aggregated similarity meets or exceeds 0.9 and whose confidence meets or exceeds 0.5. Under $\mathbf{w}_{\text{avg}}$, a confidence of 0.5 corresponds to at least three of the six inputs being available for that pair, that is, the textual similarity together with at least two metadata fields. The query uses `OPTIONAL MATCH` for the relationship so that epigrams without a qualifying connection are still returned and appear as isolated nodes in the visualisation.

```
MATCH (e1:Epigram {type: 1982})
MATCH (e2:Epigram {type: 1982})
OPTIONAL MATCH (e1)-[cs:CONTEXT_SIMILAR_TO]->(e2)
WHERE cs.similarity >= 0.9 AND cs.confidence >= 0.5
RETURN e1, e2, cs
```

Listing 4.1: Cypher query to retrieve all epigrams from epigram type 1982 and the context-aware relationships between them that meet or exceed a similarity of 0.9 and a confidence of 0.5.

The result of this query is visualised in Figure 4.4 using the visualisation capabilities of Neo4j Bloom [52]. A large, densely connected group of epigrams is visible on the right of the figure, corresponding to those pairs for which both the similarity and confidence thresholds are met. The remaining epigrams on the left are either entirely isolated or form only small, sparsely connected groups, indicating that their relationships with the rest of the epigrams in the type either fall below the similarity threshold, below the confidence threshold, or both. The separation between the connected group and the other nodes is a direct consequence of the joint filtering: relationships that would survive the similarity threshold alone are suppressed when the confidence is insufficient, exposing which connections rest on adequate metadata support and which do not. This distinction can guide further scholarly investigation: epigrams that belong to the same expert-curated type but fail to connect to the main group may warrant closer inspection to determine whether their isolation reflects genuine textual and contextual dissimilarity, whether it is attributable to missing metadata that could be completed through further research, or whether it suggests a possible revision of the type assignment.

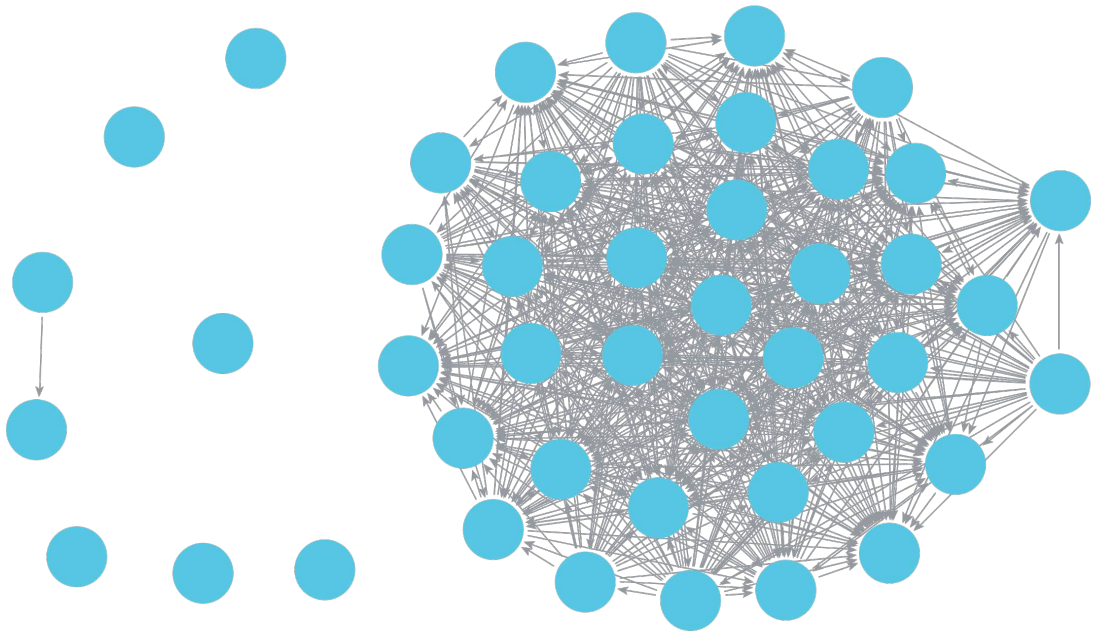


Figure 4.4: Visualisation of the result of the query in Listing 4.1 using the visualisation capabilities of Neo4j Bloom. Pairs of epigrams from epigram type 1982 whose aggregated similarity meets or exceeds 0.9 and whose confidence meets or exceeds 0.5 are connected by an edge. The `CONTEXT_SIMILAR_TO` relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of similarity.

Confidence-Enhanced Visualisation

A complementary use of the confidence scores is to retain all relationships above the similarity threshold while encoding the confidence visually, so that the reliability of each connection is immediately apparent. Listing 4.2 shows a Cypher query that retrieves all pairs of epigrams with an aggregated similarity of at least 0.9, regardless of their confidence, after which Neo4j Bloom is used to colour each relationship according to its confidence score along a gradient from dark red (low confidence) to bright green (high confidence).

```
MATCH (e1:Epigram {type: 1982})
MATCH (e2:Epigram {type: 1982})
OPTIONAL MATCH (e1)-[cs:CONTEXT_SIMILAR_TO]->(e2)
WHERE cs.similarity >= 0.9
RETURN e1, e2, cs
```

Listing 4.2: Cypher query to retrieve all epigrams from epigram type 1982 and the context-aware relationships between them that meet or exceed a similarity of 0.9, for visualisation with confidence-based colouring.

The result is visualised in Figure 4.5. Compared to Figure 4.4, additional relationships now appear: the previously unconnected epigrams on the left of the figure are connected to the main group and to one another through edges that were previously suppressed by the confidence filter. Their dark red colour indicates that these relationships are computed from limited metadata, and they should therefore be interpreted with appropriate caution. The relationships within the densely connected group on the right are predominantly orange to yellow, reflecting a moderate but consistent confidence of approximately 0.5: for most pairs in this group, three of the six inputs, typically genre and metre alongside the textual similarity, are available, while location, scribe, and subject remain unrecorded. The single green edge visible on the far right of the figure stands out as the only high-confidence relationship in the subset, corresponding to a pair for which a more complete set of metadata is available, so that the aggregated score carries a higher reliability. This contrast illustrates precisely the kind of distinction that the confidence score is designed to surface. Two relationships may have similar aggregated similarity scores yet differ substantially in their confidence, indicating that one rests on a more complete set of metadata and may therefore be more trustworthy. Two epigrams remain unconnected to the main group even in this visualisation, indicating that their similarity scores with the rest of the epigrams in the type fall below the similarity threshold and that they may display more variation in their textual or metadata content.

Together, these two examples demonstrate how the simultaneous availability of a similarity score and a confidence score extends the analytical toolkit beyond what the textual similarity of Chapter 3 alone can provide. The confidence score allows scholars to calibrate how much trust to place in any given relationship, to identify records for which metadata are systematically absent, and to prioritise further data collection or verification efforts. These capabilities are further exploited in the subsequent chapters of this thesis, where the similarity-confidence pairs serve as the foundation for context-aware ranking and clustering techniques that reason jointly over both outputs of the proposed measure.

4.6 Conclusions and Future Work

This chapter proposed a novel context-aware and responsible similarity measure for short texts, building on the hierarchical orthographic similarity measure of Chapter 3 by enriching it with contextual information drawn from the metadata associated with each text. Through an adapted OWA-based fusion mechanism, the textual similarity is combined with similarity scores computed over the available metadata into a single aggregated score for each pair of texts. To accommodate veracity issues, and particularly missing metadata, the aggregation operates solely on the available inputs, with the weight vector renormalised accordingly, ensuring that the aggregated similarity remains meaningful regardless of the extent of missing information for any given pair.

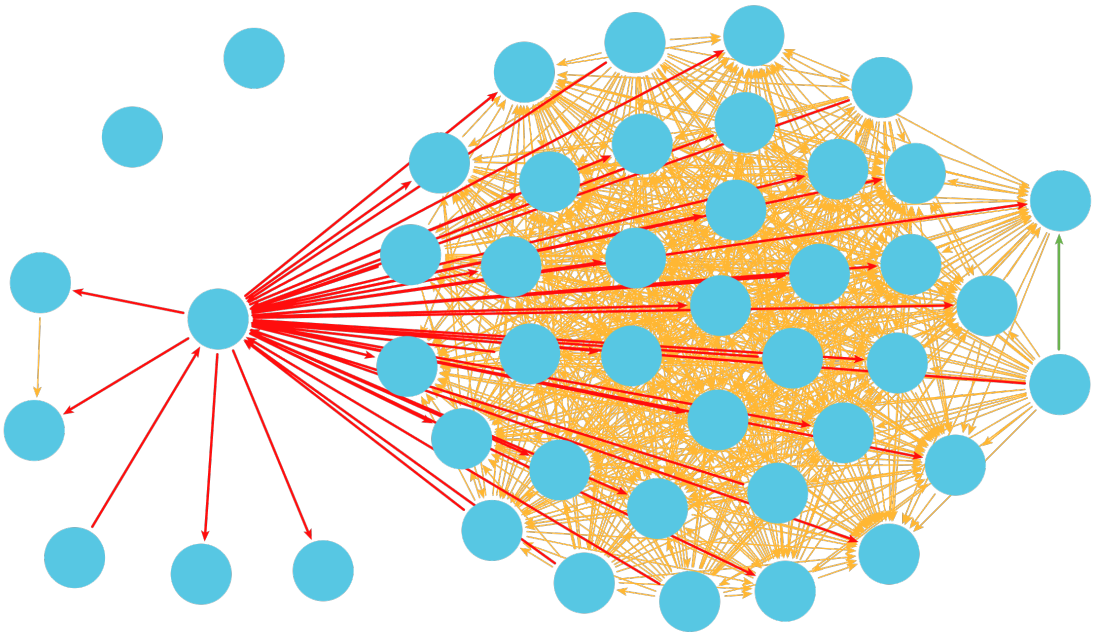


Figure 4.5: Visualisation of the result of the query in Listing 4.2 using the visualisation capabilities of Neo4j Bloom. All `CONTEXT_SIMILAR_TO` relationships between pairs of epigrams from epigram type 1982 with an aggregated similarity of at least 0.9 are shown. Each relationship is coloured according to its confidence score: dark red indicates low confidence, bright green indicates high confidence, and yellow indicates moderate confidence. The relationships appear as directed edges due to the storage model of Neo4j but can be interpreted as undirected in the context of similarity.

A confidence score, obtained as one minus the share of the original weight vector that is left unused due to the missing metadata, is introduced alongside each aggregated similarity to quantify the reliability of the aggregated value. Together, these two values form a similarity-confidence pair (s, c) that captures both the degree of resemblance between two texts and the trustworthiness of that assessment given the available metadata. These pairs are stored as properties of edges in the fuzzy LPG of Chapter 3, extending the graph-based text model of Chapters 2 and 3 with a new layer of context-aware and responsible similarity information.

What distinguishes the proposed approach from existing techniques for handling missing metadata is the combination of an adapted OWA-based fusion mechanism and the separation between the aggregated similarity and the confidence score. Traditional techniques for handling missing metadata have notable drawbacks: marginalisation excludes incomplete metadata fields from the computation entirely, discarding values that are available for some records, while imputation fills in missing values under assumptions that may not hold in practice and introduces errors that propagate silently into any subsequent analysis. The proposed fusion mechanism avoids both strategies by using all metadata that are available for any given pair and by making the effect of the missing information explicit through the confidence score rather than absorbing it into the aggregated similarity. The confidence score is therefore not a by-product of the aggregation but an independent output of equal standing, available for downstream techniques that reason jointly over similarity and reliability. Since the aggregation takes generic similarity scores as its inputs, no particular textual or metadata similarity measure is presupposed, and any combination of measures suited to a given corpus can be used. The mechanism

fixes neither the number of criteria nor their division between textual content and metadata, so that a corpus compared on several textual measures is accommodated in the same way as one compared on a single textual measure and a collection of metadata fields.

We evaluated the proposed measure through experiments on the DBBE corpus, using the same α-cut methodology as in Chapter 3, in which pairs of epigrams whose aggregated similarity meets or exceeds a threshold α are classified as similar and compared against the expert-curated epigram types. The empirically determined weight vector achieved a peak F_1-score below that of the textual baseline from Chapter 3, but the aggregated similarity demonstrated a substantial advantage at low α values, where it achieved markedly higher precision at near-perfect recall. This advantage at low thresholds suggests that the metadata provide discriminative information that the textual content alone cannot capture, and that evaluation techniques better suited to heterogeneous similarity scores, such as clustering, may reveal the full contribution of the contextual information more clearly. We further demonstrated how the simultaneous availability of similarity and confidence scores enables richer querying and more interpretable visualisation than textual similarity alone can support. Importantly, the primary purpose of the proposed framework is not to reproduce existing expert groupings but to equip researchers with a configurable tool, in which the textual measure, the selection of metadata fields, the metadata-specific similarity measures, and the aggregation weights can all be tailored to a specific research question. The evaluation against the expert-curated types therefore confirms that the measure recovers these groupings to a meaningful degree, without being the sole criterion by which it should be judged.

The proposed measure is subject to a number of limitations. The confidence score is restricted to the extent of missing information and does not capture other veracity concerns that may affect the reliability of the aggregated similarity, such as uncertainty or inconsistency in the recorded metadata values. Moreover, because the OWA weights attach to rank positions rather than to specific criteria, the weight vector expresses how much the i-th largest similarity contributes rather than how important a given metadata field is. The contribution of each criterion to an individual score remains traceable, but the weights themselves say nothing about which fields matter most. Furthermore, the weight vector is determined empirically by maximising the F_1-score on the evaluation subset, so the reported scores constitute a best case for this corpus rather than an estimate of performance on unseen data. The transfer of the tuned vector to corpora with different metadata availability profiles is therefore not demonstrated, although the vector is intended as an adjustable parameter that is recalibrated per corpus and per analytical goal rather than reused as given. Computing aggregated similarities for all pairs of texts scales quadratically with corpus size, a challenge already identified for the textual similarity measure in Section 3.6 and equally applicable here. The experiments are conducted on a single corpus, and the degree to which the observed behaviour transfers to other domains, languages, or metadata structures remains an open question.

Each of these limitations opens a direction for further investigation. Extending the confidence score to incorporate veracity concerns beyond missingness, such as the uncertainty of individual metadata values, would produce a richer reliability indicator and bring the framework closer to a more responsible treatment of veracity issues. This direction aligns naturally with the theory of L-grades [49], of which the (s, c)-pairs produced here are an instance and which extends to broader forms of graded reasoning under veracity concerns, explored in the context of ranking in Chapter 5. Learning-based approaches to weight vector determination could be

investigated as an alternative to the current empirical search, with the goal of producing weight vectors that are more robust to different metadata availability profiles and generalisable across corpora. Distinguishing mandatory from optional criteria, in the manner of the LSP decision method, would further allow the aggregation to express settings in which certain criteria must be available for a comparison to be meaningful at all. Downstream analytical techniques that reason jointly over similarity and confidence, such as novel clustering methods, could further serve both as more informative evaluations of the proposed measure and as tools for surfacing finer-grained groupings that the textual similarity alone cannot distinguish. The similarity-confidence pairs produced in this chapter serve as the direct foundation for Chapter 6, where they are used to partition the corpus into coherent groups in an unsupervised manner, and the framework's generic design equally invites application to analytical techniques beyond those explored in this thesis.

References

[1] Christopher D Manning. "Introduction to Information Retrieval". Syngress Publishing, 2008. DOI: https://doi.org/10.1017/CBO9780511809071.

[2] Paul Jaccard. "Étude Comparative de la Distribution Florale dans une Portion des Alpes et des Jura". In: *Bulletin de la Société Vaudoise des Sciences Naturelles* 37 (1901), pp. 547–579.

[3] Vladimir I Levenshtein. "Binary Codes Capable of Correcting Deletions, Insertions, and Reversals". In: *Soviet Physics Doklady*. Vol. 10. 8. 1966, pp. 707–710.

[4] Charu C. Aggarwal and ChengXiang Zhai. "A Survey of Text Clustering Algorithms". In: *Mining Text Data*. Boston, MA: Springer US, 2012, pp. 77–128. isbn: 978-1-4614-3223-4. DOI: https://doi.org/10.1007/978-1-4614-3223-4_4.

[5] Jesús Bobadilla, Fernando Ortega, Antonio Hernando and Abraham Gutiérrez. "Recommender Systems Survey". In: *Knowledge-Based Systems* 46 (2013), pp. 109–132. issn: 0950-7051. DOI: https://doi.org/https://doi.org/10.1016/j.knosys.2013.03.012.

[6] Wael H Gomaa, Aly A Fahmy et al. "A Survey of Text Similarity Approaches". In: *International Journal of Computer Applications* 68.13 (2013), pp. 13–18. DOI: https://doi.org/10.5120/11638-7118.

[7] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis and Vassilios S. Verykios. "Duplicate Record Detection: A Survey". In: *IEEE Transactions on Knowledge and Data Engineering* 19.1 (2007), pp. 1–16. DOI: https://doi.org/10.1109/TKDE.2007.250581.

[8] Dimas Wibisono Prakoso, Asad Abdi and Chintan Amrit. "Short Text Similarity Measurement Methods: A Review". In: *Soft Computing* 25.6 (2021), pp. 4699–4723. issn: 1433-7479. DOI: https://doi.org/10.1007/s00500-020-05479-2.

[9] Rania Albalawi, Tet Hin Yeap and Morad Benyoucef. "Using Topic Modeling Methods for Short-Text Data: A Comparative Analysis". In: *Frontiers in Artificial Intelligence* Volume 3 (2020). DOI: https://doi.org/10.3389/frai.2020.00042.

[10] Majid Hameed Ahmed, Sabrina Tiun, Nazlia Omar and Nor Samsiah Sani. "Short Text Clustering Algorithms, Application and Challenges: A Survey". In: *Applied Sciences* 13.1 (2023). issn: 2076-3417. DOI: https://doi.org/10.3390/app13010342.

[11] Ge Song, Yunming Ye, Xiaolin Du, Xiaohui Huang and Shifu Bie. "Short Text Classification: A Survey". In: *Journal of Multimedia* 9.5 (2014). DOI: https://doi.org/10.4304/jmm.9.5.635-643.

[12] Alexandre Magueresse, Vincent Carles and Evan Heetderks. *Low-Resource Languages: A Review of Past Work and Future Challenges*. 2020. DOI: https://doi.org/10.48550/arXiv.2006.07264.

[13] Partha Pakray, Alexander Gelbukh and Sivaji Bandyopadhyay. "Natural Language Processing Applications for Low-Resource Languages". In: *Natural Language Processing* 31.2 (2025), pp. 183–197. DOI: https://doi.org/10.1017/nlp.2024.33.

[14] Gediminas Adomavicius and Alexander Tuzhilin. "Context-Aware Recommender Systems". In: *Recommender Systems Handbook*. Boston, MA: Springer US, 2011, pp. 217–253. DOI: https://doi.org/10.1007/978-0-387-85820-3_7.

[15] MK Vijaymeena and K Kavitha. "A Survey on Similarity Measures in Text Mining". In: *Machine Learning and Applications: An International Journal* 3.2 (2016), pp. 19–28. DOI: https://doi.org/10.5121/mlaij.2016.3103.

[16] M. Yasser Chuttur. "An Analysis of Problems in Metadata Records". In: *Journal of Library Metadata* 11.2 (2011), pp. 51–62. DOI: https://doi.org/10.1080/19386389.2011.570654.

[17] Roderick JA Little and Donald B Rubin. "Statistical Analysis with Missing Data". John Wiley & Sons, 2019.

[18] Tlamelo Emmanuel, Thabiso Maupong, Dimane Mpoeleng, Thabo Semong, Banyatsang Mphago and Oteng Tabona. "A Survey on Missing Data in Machine Learning". In: *Journal of Big Data* 8.1 (2021), p. 140. issn: 2196-1115. DOI: https://doi.org/10.1186/s40537-021-00516-9.

[19] Pedro J. García-Laencina, José-Luis Sancho-Gómez and Aníbal R. Figueiras-Vidal. "Pattern Classification with Missing Data: A Review". In: *Neural Computing and Applications* 19.2 (2010), pp. 263–282. issn: 1433-3058. DOI: https://doi.org/10.1007/s00521-009-0295-6.

[20] Ronald R. Yager. "On Ordered Weighted Averaging Aggregation Operators in Multicriteria Decisionmaking". In: *IEEE Transactions on Systems, Man, and Cybernetics* 18.1 (1988), pp. 183–190. issn: 2168-2909. DOI: https://doi.org/10.1109/21.87068.

[21] Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, Maxime Deforche, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout and Colin Swaelens. "The Database of Byzantine Book Epigrams Project : Principles, Challenges, Opportunities". eng. In: *Journal of Data Mining and Digital Humanities* On the Way to the Future of Digital Manuscript Studies (2023), p. 41. issn: 2416-5999. DOI: https://doi.org/10.46298/jdmdh.10244.

[22] Kristoffel Demoen, Gilbert Bentein, Klaas Bentein, Floris Bernard, Julián Bértola, Julie Boeten, Mathijs Clement, Cristina Cocola, Eline Daveloose, Sien De Groot, Pieterjan De Potter, Ilse De Vos, Krystina Kubina, Hanne Lauwers, Paulien Lemay, Renaat Meesters, Marjolein Morbé, Delphine Nachtergaele, Marthe Nemegeer, Joachim Nielandt, Mace Ojala, Lisa-Lou Péchillon, Raf Praet, Rachele Ricceri, Anne-Sophie Rouckhout, Jeroen Schepens, Febe Schollaert, Lev Shadrin, Nina Sietis, Dimitrios Skrekas, Colin Swaelens, Maria Tomadaki, Sarah-Helena Van den Brande, Merel Van Nieuwerburgh, Lotte Van Olmen, Noor Vanhoe and Nina Vanhoutte. *Database of Byzantine Book Epigrams (DBBE)*. Version 1.0.0. Zenodo, Feb. 2023. DOI: https://doi.org/10.5281/zenodo.7682523.

[23] Peter Christen. "Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection". Springer, 2012. DOI: https://doi.org/10.1007/978-3-642-31164-2.

[24] Ivan P. Fellegi and Alan B. Sunter. "A Theory for Record Linkage". In: *Journal of the American Statistical Association* 64.328 (1969), pp. 1183–1210. DOI: https://doi.org/10.1080/01621459.1969.10501049.

[25] Antoon Bronselaer and Guy De Tré. "A Possibilistic Approach to String Comparison". In: *IEEE Transactions on Fuzzy Systems* 17.1 (2009), pp. 208–223. DOI: https://doi.org/10.1109/TFUZZ.2008.2008025.

[26] Antoon Bronselaer and Guy De Tré. "Properties of Possibilistic String Comparison". In: *IEEE Transactions on Fuzzy Systems* 18.2 (2010), pp. 312–325. DOI: https://doi.org/10.1109/TFUZZ.2010.2041353.

[27] Antoon Bronselaer, Axel Hallez and Guy De Tré. "Evaluation in the Possibilistic Framework for Object Matching". In: *Proceedings of the 12th International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU)*. 2008, pp. 1701–1708.

[28] Gleb Beliakov, Ana Pradera and Tomasa Calvo. "Aggregation Functions: A Guide for Practitioners". Springer, 2007. DOI: https://doi.org/10.1007/978-3-540-73721-6.

[29] Michel Grabisch, Jean-Luc Marichal, Radko Mesiar and Endre Pap. "Aggregation Functions". Cambridge University Press, 2009. DOI: https://doi.org/10.1017/CBO9781139644150.

[30] Jozo J. Dujmović and Henrik L. Larsen. "Generalized Conjunction/Disjunction". In: *International Journal of Approximate Reasoning* 46.3 (2007), pp. 423–446. issn: 0888-613X. DOI: https://doi.org/https://doi.org/10.1016/j.ijar.2006.12.011.

[31] Jozo J. Dujmovic. "Continuous Preference Logic for System Evaluation". In: *IEEE Transactions on Fuzzy Systems* 15.6 (2007), pp. 1082–1099. DOI: https://doi.org/10.1109/TFUZZ.2007.902041.

[32] Jozo J. Dujmovic. "Soft Computing Evaluation Logic: The LSP Decision Method and Its Applications". John Wiley & Sons, 2018. DOI: https://doi.org/10.1002/9781119256489.

[33] Gustave Choquet. "Theory of Capacities". In: *Annales de l'Institut Fourier* 5 (1954), pp. 131–295. DOI: https://doi.org/10.5802/aif.53.

[34] Michio Sugeno. "Fuzzy Measures and Fuzzy Integrals: A Survey". In: *Fuzzy Automata and Decision Processes*. North-Holland, 1977, pp. 89–102. DOI: https://doi.org/10.1016/B978-1-4832-1450-4.50027-4.

[35] Michel Grabisch. "The Application of Fuzzy Integrals in Multicriteria Decision Making". In: *European Journal of Operational Research* 89.3 (1996), pp. 445–456. DOI: https://doi.org/https://doi.org/10.1016/0377-2217(95)00176-X.

[36] Jean-Luc Marichal. "On Choquet and Sugeno Integrals as Aggregation Functions". In: *Fuzzy Measures and Integrals - Theory and Applications*. Physica-Verlag, 2000.

[37] Paul N. Bennett, Ryen W. White, Wei Chu, Susan T. Dumais, Peter Bailey, Fedor Borisyuk and Xiaoyuan Cui. "Modeling the Impact of Short- and Long-Term Behavior on Search Personalization". In: *Proceedings of the 35th International ACM SIGIR Conference on Research and Development in Information Retrieval*. SIGIR '12. Association for Computing Machinery, 2012, pp. 185–194. DOI: https://doi.org/10.1145/2348283.2348312.

[38] Ramya Elizabeth Thomas and Shamsuddin S. Khan. "Improved Clustering Technique Using Metadata for Text Mining". In: *2016 International Conference on Communication and Electronics Systems (ICCES)*. 2016, pp. 1–5. DOI: https://doi.org/10.1109/CESYS.2016.7889835.

[39] Donald B. Rubin. "Inference and missing data". In: *Biometrika* 63.3 (1976), pp. 581–592. DOI: https://doi.org/10.1093/biomet/63.3.581.

[40] Stef van Buuren. "Flexible Imputation of Missing Data". CRC Press, 2018. DOI: https://doi.org/10.1201/9780429492259.

[41] Della Murbarani Prawidya Murti, Utomo Pujianto, Aji Prasetya Wibawa and Muhammad Iqbal Akbar. "K-Nearest Neighbor (K-NN) Based Missing Data Imputation". In: *2019 5th International Conference on Science in Information Technology (ICSITech)*. 2019, pp. 83–88. DOI: https://doi.org/10.1109/ICSITech46713.2019.8987530.

[42] Daniel J. Stekhoven and Peter Bühlmann. "MissForest—Non-Parametric Missing Value Imputation for Mixed-Type Data". In: *Bioinformatics* 28.1 (2012), pp. 112–118. issn: 1367-4803. DOI: https://doi.org/10.1093/bioinformatics/btr597.

[43] Thomas N. Herzog, Fritz Scheuren and William E. Winkler. "Data Quality and Record Linkage Techniques". Springer, 2007. DOI: https://doi.org/10.1007/0-387-69505-2.

[44] Jozo Dujmovic. "The Problem of Missing Data in LSP Aggregation". In: *Advances in Computational Intelligence*. Springer, 2012, pp. 336–346. isbn: 978-3-642-31718-7. DOI: https://doi.org/10.1007/978-3-642-31718-7_35.

[45] Dimitar Filev and Ronald R. Yager. "On the Issue of Obtaining OWA Operator Weights". In: *Fuzzy Sets and Systems* 94.2 (1998), pp. 157–169. issn: 0165-0114. DOI: https://doi.org/10.1016/S0165-0114(96)00254-0.

[46] Ronald R. Yager. "Quantifier Guided Aggregation Using OWA Operators". In: *International Journal of Intelligent Systems* 11.1 (1996), pp. 49–73. DOI: https://doi.org/10.1002/(SICI)1098-111X(199601)11:1<49::AID-INT3>3.0.CO;2-Z.

[47] Lee R Dice. "Measures of the Amount of Ecologic Association Between Species". In: *Ecology* 26.3 (1945), pp. 297–302. DOI: https://doi.org/10.2307/1932409.

[48] Yannis Vassiliou. "Null Values in Data Base Management a Denotational Semantics Approach". In: *Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data*. SIGMOD '79. Boston, Massachusetts: Association for Computing Machinery, 1979, pp. 162–169. isbn: 089791001X. DOI: https://doi.org/10.1145/582095.582123.

[49] Guy De Tré, Milan Peelman and Jozo Dujmović. "Logic Reasoning under Data Veracity Concerns". In: *International Journal of Approximate Reasoning* 161 (2023), p. 108977. issn: 0888-613X. DOI: https://doi.org/10.1016/j.ijar.2023.108977.

[50] Neo4j. "Neo4j Graph Database". url: https://neo4j.com/ (visited on 27/03/2026).

[51] Neo4j. "Cypher Query Language". url: https://neo4j.com/docs/cypher-manual/current/ (visited on 01/04/2026).

[52] Neo4j. “Neo4j Bloom”. url: https://neo4j.com/bloom/ (visited on 03/04/2026).

5

Ranking of Satisfaction-Confidence Pairs

"Virtually every important aspect of programming
arises somewhere in the context of sorting or searching."

— **Donald E. Knuth**

This chapter presents techniques for ranking pairs of satisfaction and confidence grades. The satisfaction component reflects how well a result meets a criterion, while the confidence component reflects the trustworthiness of that assessment. Such pairs arise naturally in data processing techniques in which both the quality of a result and the reliability of that assessment are of interest, including the context-aware similarity measure of Chapter 4, where each aggregated similarity is accompanied by a confidence score reflecting the extent of missing metadata. Such a similarity-confidence pair is a satisfaction-confidence pair in which the aggregated similarity takes the role of the satisfaction grade. These satisfaction-confidence pairs are formalised as *L-grades* [1], a framework that pairs satisfaction and confidence as independent components to support veracity handling in logic reasoning. L-grades can be integrated into query answering, decision support, and data analysis applications to enable better explainable and responsible reasoning under circumstances where data veracity is an issue. Because the two components of an L-grade are orthogonal, no single natural ordering exists, and various ranking strategies can be employed. Three such approaches are proposed here: *lexicographic ordering*, in which one component is prioritised over the other; *non-weighted global preference*, in which both components are conjunctively combined with equal importance; and *weighted global preference*, in which a provided weight reflects the relative importance of confidence versus satisfaction. These ranking approaches are complemented by a distance measure that quantifies the strength of a preference between two L-grades and by an approximate equality comparator, ultimately providing domain experts with the ability to rank and compare satisfaction-confidence pairs in a way that aligns with their specific requirements. These techniques are applied to the Database of Byzantine Book Epigrams (DBBE) corpus in two use cases, the second of which also introduces a geographical querying technique in which L-grades express how closely a recorded location matches a queried one within a hierarchical geographical structure.

The development and study of these ranking techniques address the following research question:

> **RQ 4.B:** How can query results and candidate interconnections be ranked based on both their similarity score and associated confidence score to provide more informative and transparent results for domain experts?

The content of this chapter is based on parts of the following publications:

- **Maxime Deforche** and Guy De Tré. *Ranking L-grades for Veracity Handling in Logic Reasoning.* Accepted for publication in Proceedings of the Joint World Congress of the International Fuzzy Systems Association (IFSA) and the North American Fuzzy Information Processing Society (NAFIPS), 2025.
- **Maxime Deforche**, Guy De Tré, Paolo Fosci, and Giuseppe Psaila. *On Veracity Handling in Document Stores: A Novel Technique Based on L-graded Logic Running on the J-CO Framework.* Published in Proceedings of the 21st International Conference on Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU), Communications in Computer and Information Science, vol 3020, 2026.
  DOI: 10.1007/978-3-032-28997-1_23

The remainder of this fifth chapter is structured as follows. Section 5.1 motivates the need for ranking techniques that jointly consider satisfaction and confidence, and outlines the approach proposed in this chapter. Related work on models for handling veracity and confidence in data, as well as on ranking and preference ordering techniques, is discussed in Section 5.2. Section 5.3 provides the necessary preliminaries on L-grades and their mathematical foundation, as well as the concepts of ranking and order relations. The proposed ranking techniques are presented in Section 5.4. This section covers the three ranking approaches, the cases of full dominance that any ranking must respect, and a distance measure with an associated approximate equality comparator. Section 5.5 demonstrates the practical applicability of the ranking techniques through two use cases based on the DBBE corpus, the first of which builds on the context-aware similarity measure of Chapter 4. Finally, Section 5.6 summarises the contributions of this chapter, discusses limitations, and outlines directions for future work.

5.1 Introduction

Handling data veracity, the extent to which data can be trusted to faithfully reflect reality, is widely recognised as one of the central challenges in modern data management and processing, especially in the context of big data [2–5]. A lack of trust in data sources propagates through processing and analysis to computational outputs, potentially affecting the reliability of query results, analytical conclusions, and decision support systems [4]. In practice, available resources are often insufficient to fully resolve such concerns, whether they stem from incomplete data, uncertain values, information compiled from heterogeneous sources of varying quality, or any other source of imperfections that could affect confidence in the data [6, 7]. It is therefore important to develop techniques that assess the known or estimated veracity of input data and report the resulting veracity of output data to users, enabling them to make informed decisions about how to use and interpret results.

Assessments that pair a result with a measure of its trustworthiness arise in many data processing contexts, including flexible query answering [8], decision support [9, 10], and data analysis. In the preceding chapter, we addressed this need by pairing an aggregated similarity score with a confidence score that reflects the extent of missing metadata, resulting in a satisfaction-confidence pair for every comparison between two texts.

Results of these processes often need to be ranked in a meaningful order, either for presentation to stakeholders or for subsequent applications that require ordered inputs. Ranking based on a single score is a well-understood problem, but when each result is characterised by two independent components, a satisfaction grade and a confidence grade, ranking becomes considerably more challenging. A result with high satisfaction but low confidence may or may not be preferable to one with moderate satisfaction and high confidence, depending on the preferences and requirements of the stakeholder. Consider, for instance, a user searching for a hotel: one option has an excellent average rating but is based on only a handful of reviews, giving it low confidence, while another has a good but lower rating supported by hundreds of reviews, giving it high confidence. Whether the first option should be ranked above the second depends on the user, who may place greater value on the rating itself or on its reliability. Both components are orthogonal, capturing different aspects of a result, and a ranking technique must therefore make a trade-off between them. Existing ranking techniques in information retrieval typically operate on a single relevance score [11, 12]. The ranking methods in Multi-Criteria Decision Making (MCDM), while capable of handling multiple criteria, do not inherently account for the distinct semantics of a satisfaction-confidence pair, in which one component qualifies the trustworthiness of the other [10, 13]. Moreover, when results are presented to users who may not have a background in the underlying mathematical techniques, the resulting rankings must be interpretable and explainable [14–16]. Users should be able to understand why one result is ranked above another and how both satisfaction and confidence contributed to that ordering. This combination of requirements motivates the development of novel ranking techniques that accommodate both components while remaining interpretable.

In this chapter, we address this challenge by formalising satisfaction-confidence pairs as *L-grades* [1], a framework proposed for handling data veracity in logic reasoning that uses the simplest form of Zadeh's Z-numbers [17]. An L-grade $\ell = (s, c)$ consists of two truth values $s, c \in [0, 1]$, where s represents the degree of satisfaction, suitability, or similarity, and c represents the degree of confidence in s. We propose and study three ranking approaches, each of which imposes a total ordering or a total preordering on the set of L-grades. *Lexicographic ordering* ranks L-grades by giving priority to one of the two components, selected by the stakeholder, and uses the other component to break ties. *Non-weighted global preference* combines both components conjunctively into a single preference value, treating satisfaction and confidence as equally important. *Weighted global preference* replaces this conjunctive combination by a linear combination in which a stakeholder-assigned weight reflects the relative importance of confidence versus satisfaction. In addition, we introduce thresholds that exclude L-grades falling below a minimum satisfaction or confidence from the ranking, a distance measure that quantifies the strength of a preference between two L-grades, and an approximate equality comparator that identifies L-grades sufficiently close to one another.

The principal novelty of the proposed approach lies in the systematic study of ranking techniques specifically designed for L-grades, including formal proofs that each proposed ordering is a total order, or a total preorder in the case of the global preference functions, and, outside a small number of identified boundary cases, consistent

with the cases of full dominance that any reasonable ranking must respect. By offering multiple ranking approaches, the proposed techniques accommodate a range of stakeholder preferences rather than imposing a single fixed ordering, thereby providing a flexible framework that can be tailored to the specific needs of different applications and use cases. The explicit separation of satisfaction and confidence in L-grades, which was one of Zadeh's original motivations for introducing Z-numbers in information modelling [17], enables a clear and intuitive presentation of results, as both components remain separately accessible throughout the ranking process. This transparency ensures that the resulting rankings are easy to explain, even to users without expertise in the underlying formalism. The combination of formal properties, flexibility in accommodating preferences, and the interpretability of the underlying model represents a contribution to the development of ranking methods under data veracity concerns.

We demonstrate the practical applicability of the proposed ranking techniques through two use cases, both grounded in the Database of Byzantine Book Epigrams (DBBE) corpus [18, 19]. In the first use case, we rank the context-aware similarity-confidence pairs produced by the measure of Chapter 4. Different ranking approaches enable researchers to explore the similarities between epigrams from complementary perspectives, prioritising either high textual and contextual resemblance, high reliability of the underlying metadata, or a balanced combination of both. The second use case addresses the ranking of query results obtained from a geographical querying technique based on L-grades, in which epigrams are retrieved based on their association with a queried location within a hierarchical geographical structure. Each result is assigned a satisfaction grade and a confidence grade determined by the hierarchical relationship between the queried location and the location recorded for the epigram. Together, these two use cases illustrate how the proposed ranking techniques enrich retrieval and analysis with veracity awareness, enabling domain experts to calibrate the trade-off between relevance and trustworthiness according to their specific analytical needs.

5.2 Related Work

The ranking techniques proposed in this chapter draw on two topics of prior work: models that pair a satisfaction or membership degree with an associated measure of reliability or uncertainty, and techniques for ranking results that carry multiple evaluation criteria. This section reviews the approaches most directly relevant to the ranking of L-grades.

L-grades explicitly handle veracity by separating satisfaction and confidence into two independently interpretable truth values [1]. This design adopts a principle introduced by Zadeh through the concept of Z-numbers [17], in which the reliability of a fuzzy restriction is modelled as a separate component rather than being absorbed into the restriction itself. Other models for handling confidence or uncertainty alongside a membership degree exist, differing primarily in whether they treat this second component implicitly or explicitly. Type-2 fuzzy sets [20–22], including the widely used special case of interval type-2 fuzzy sets [23, 24], replace the crisp membership degree with a fuzzy set or interval within the unit interval. This captures uncertainty about the membership value, but embeds it within the membership representation rather than separating it into a distinct component. Intuitionistic fuzzy sets [25] take a different approach by assigning both a membership degree and a non-membership degree to each element, which together must sum to at most one; the residual is interpreted

as a hesitation margin reflecting the uncertainty about the membership of the element. Pythagorean fuzzy sets [26] relax this constraint further, requiring only that the sum of the squares of the two degrees be at most one, thereby permitting a broader range of combinations. R-sets [27] and intervals [7] have also been used to impose bounds on criterion satisfaction to reflect uncertainty in the data. A further step is taken by suitability distributions, where the satisfaction degree of a result is not fixed to one value but given as a possibility distribution over the degrees it could take [28]. The distribution retains all possible suitability degrees, but makes results harder to compare, so applications tend to reduce each distribution to a single representative degree before ranking [29]. A pair of values also arises in Dempster-Shafer theory [30, 31], which separates belief, a lower bound on the probability of a proposition, from plausibility, an upper bound, thereby characterising the certainty of an assessment within a probabilistic rather than a fuzzy logic setting. More broadly, the theory of imprecise probabilities generalises this idea by using intervals or sets of multiple probabilities instead of a single, exact probability value, allowing the model to explicitly reflect a lack of information or data [32, 33]. Compared to these models, L-grades reduce both components to crisp truth values, facilitating logic reasoning while keeping confidence directly accessible for inspection and further processing [1], and share with Z-numbers the underlying principle that reliability should be modelled explicitly rather than implicitly [17, 34, 35].

Ranking methods have been proposed for Z-numbers specifically, typically by converting the Z-number into a crisp value or a simpler fuzzy number before applying conventional fuzzy ranking techniques [36–38]. These conversion-based approaches are effective for general Z-numbers, whose fuzzy-valued components resist direct comparison, but the simplicity of L-grades, whose components are numerical truth values, permits ranking techniques that operate directly on the two-component structure without requiring an intermediate conversion step. None of them, however, addresses ranking in the specific context of satisfaction-confidence pairs, where one component quantifies the trustworthiness of the other.

Beyond the models themselves, a separate body of work addresses the problem of ranking in a more general context. In information retrieval, classical models such as Term Frequency-Inverse Document Frequency (TF-IDF) [39] and BM25 [40] assign each document a single relevance score, and learning-to-rank methods extend this paradigm by training models to predict relevance from richer feature representations [12]. In all of these cases, the output is a single score per document on which ranking is well-defined. When multiple criteria must be considered simultaneously, the field of MCDM provides a broad range of methods [10]. Scoring functions such as weighted sums reduce multiple criteria to a single aggregate, outranking methods such as ELECTRE [41] and PROMETHEE [42] construct pairwise preference relations between alternatives, and lexicographic ordering [13] ranks by the most important criterion first and uses the remaining criteria to break ties. Distance-based techniques such as TOPSIS [43] rank alternatives based on their proximity to ideal and anti-ideal solutions, while Pareto dominance [44] yields a partial order when no criterion weights are available, preferring one alternative over another only if it is at least as good on all criteria and strictly better on at least one. The ranking of fuzzy quantities has been studied extensively as well, with methods based on defuzzification, centroids, and expected values, among others [45, 46]. Similar scoring approaches have been developed for intuitionistic fuzzy values [47] and type-2 fuzzy sets [21], typically by mapping the membership representation to a single real number before ranking. These techniques provide the mathematical foundations that inspire the ranking approaches for L-grades proposed in this chapter, several of which draw directly on them. However, none of

them has been applied to satisfaction-confidence pairs, where the two components have asymmetric roles and where the choice of ranking strategy must account for their independence rather than treating them as homogeneous criteria.

Taken together, the work surveyed in this section addresses the two topics on which this chapter builds: models for handling confidence and veracity, and techniques for ranking multi-valued assessments. The models reviewed in the first topic offer several ways to embed uncertainty or reliability into a satisfaction or membership degree, but do not address how the resulting values should be ranked. Z-number ranking methods do consider ordering, yet they reduce the pair to a single value or apply techniques designed for the more complex general Z-number structure, neither of which preserves the two-component semantics of L-grades. The ranking techniques reviewed in the second topic provide a broad range of ordering methods, several of which serve as the mathematical foundation for the approaches proposed in this chapter, but they have not been studied in the specific context of paired satisfaction and confidence grades. By developing total orders and total preorders over L-grades that respect the orthogonality of satisfaction and confidence while accommodating different stakeholder preferences, the ranking techniques proposed in this chapter bridge these two topics. The formal concepts on which these orderings are built are introduced in Section 5.3.

5.3 Preliminaries

This section provides the formal background needed to present the ranking techniques proposed in this chapter. Two topics are central: the L-grade framework, which combines a satisfaction grade and a confidence grade into a single structure suitable for logic reasoning under data veracity concerns, and the concepts that formalise the notion of ranking and order relations. Section 5.3.1 introduces L-grades, including their definition, semantics, logic operators, and aggregators. Section 5.3.2 recalls the concepts of partial orders, total orders, and total preorders that underpin the proposed ranking techniques.

5.3.1 L-grades

In one of his last research contributions, Zadeh proposed the notion of a *Z-number* Z, defined by an ordered pair of fuzzy numbers, $Z = (A, B)$ [17]. Fuzzy numbers are fuzzy sets used to represent imprecise numerical values, such as "about 1.5"; they are usually characterised by a central value at which membership equals one and a gradual decline in membership as values deviate from that central value. A linguistic variable X is a variable whose values are expressed in natural language rather than as precise numbers [20]. In the Z-number framework, the first fuzzy number A represents a fuzzy restriction $R(X)$ on the possible (numerical) values of X, constraining them through a fuzzy set rather than a crisp condition, whereas the second fuzzy number B represents a measure of the reliability one has in A. Typically, both A and B are described in natural language. For instance, if X represents the travel time by train from *Brussels* to *Paris*, then $Z = (\text{"about 1h 40 min"}, \text{"quite certain"})$ expresses a fuzzy restriction on that travel time together with a reliability assessment. The key insight underlying Z-numbers is that the reliability of data should not be modelled by implicitly adjusting the uncertainty of a value, for instance by widening the fuzzy restriction, but

should instead be treated as a separate, independently interpretable component [17, 34]. Both components therefore remain independently accessible, allowing a user to inspect and reason about the reliability of an assessment without first having to disentangle it from that assessment.

Definition and Semantics

In line with Zadeh's proposal, De Tré et al. [1] proposed using the simplest form of Z-numbers to model truth values in logic reasoning under data veracity concerns, which frequently arise in the context of big data [2]. Such simplified Z-numbers are called *L-grades*, where the "L" stands for logic. Definition 5.1 provides their formal description.

Definition 5.1 (L-grade [1]). An L-grade ℓ is defined to be a Z-number where both fuzzy numbers are crisp singletons that each contain a single value of the unit interval $[0, 1]$ that is interpreted as a truth value and is further processed using fuzzy logic, i.e.,

$$\ell = (\{(s, 1)\}, \{(c, 1)\}), \quad \text{where } s, c \in [0, 1]. \tag{5.1}$$

In the remainder, we use the short notation $\ell = (s, c)$. The set of all L-grades is denoted by $\mathbb{L}$, i.e.,

$$\mathbb{L} = [0, 1]^2. \tag{5.2}$$

The first truth value s is interpreted as a *satisfaction grade* (or *suitability grade*), denoting, for example, to what extent a data value stored in a database satisfies a given (fuzzy) query criterion. The second truth value c is interpreted as a *confidence grade* that reflects the degree of trust one has in s. Both s and c take gradual values in the unit interval, ranging from 0 (no satisfaction or no confidence) to 1 (full satisfaction or full confidence), and model degrees of truth of fuzzy propositions. While the satisfaction grade s follows from a direct evaluation of the criterion, the confidence grade c is usually derived from a separate assessment of the trustworthiness of that evaluation. The two truth values of an L-grade should not be confused with the components of a Possibilistic Truth Value (PTV), which give the degree of possibility that a proposition is true and the degree of possibility that it is false [48–50]. In a PTV, both components describe a single assessment of the proposition and constrain one another, whereas in an L-grade the two grades originate from separate evaluations and vary independently. In the context of this thesis, the similarity-confidence pairs produced by the context-aware similarity measure of Chapter 4 can be considered instances of L-grades: the aggregated similarity between two texts serves as the satisfaction component s, and the confidence component c reflects the total weight of the metadata available for that comparison. L-grades generalise this specific instance to any setting in which both a degree of criterion satisfaction and a degree of trust in that assessment are relevant.

Two L-grades $\ell_1 = (s_1, c_1)$ and $\ell_2 = (s_2, c_2)$ are equal if and only if their corresponding components are equal, i.e.,

$$(s_1 = s_2) \wedge (c_1 = c_2) \iff \ell_1 = \ell_2. \tag{5.3}$$

The following example illustrates how L-grades make the distinction between satisfaction and confidence explicit.

> **Example 5.1.** Consider a flexible query that asks for hotels with a high rating. A hotel with a rating of 9.2 out of 10 based on 500 reviews might be assigned the L-grade $\ell_1 = (0.92, 0.95)$, reflecting high satisfaction and high confidence. A newly listed hotel with a rating of 9.8 but based on only 3 reviews could receive $\ell_2 = (0.98, 0.22)$, indicating very high satisfaction but low confidence. The L-grade makes this distinction explicit: the second hotel scores higher on the criterion, but the trustworthiness of that score is considerably lower. The confidence grade here is illustrative and could in real applications be derived from the number of reviews, the trust in certain reviewers, or other factors affecting the reliability of the rating.

Logic Operators

To facilitate reasoning with L-grades, logic operators are required. In each of these operators, the resulting satisfaction grade s is computed using conventional fuzzy logic techniques, whereas the resulting confidence grade c should reflect how the confidence in the satisfaction grades of the arguments carries over to the result [1]. This requirement is called the *sibling principle*, and operators for L-grades that satisfy it are called *sibling operators*. Since each confidence grade c_i expresses the trust one has in its own satisfaction grade s_i and in nothing else, only those arguments that contribute to the resulting satisfaction should contribute to the resulting confidence. For negation, conjunction, and disjunction, the basic logic operators for L-grades, this idea is realised by taking the highest confidence grade among the arguments that fully determine the outcome, and the lowest otherwise, since it cannot then be established which argument contributes to which extent [1].

The negation of an L-grade $\ell = (s, c) \in \mathbb{L}$ is defined by

$$\neg(s, c) = (N(s), c), \tag{5.4}$$

where N denotes the standard fuzzy negation $N : [0,1] \to [0,1] : x \mapsto 1 - x$. The confidence grade is preserved under negation, since negating the satisfaction does not alter the trust one has in the original assessment.

Conjunction and disjunction for L-grades are defined in terms of a Triangular Norm (t-norm) $\top$ and its dual Triangular Conorm (t-conorm) $\bot$, respectively, which are standard operators in fuzzy logic that generalise the classical AND and OR operations to the unit interval [51]. A t-norm is a binary operation on $[0,1]$ that is commutative, associative, monotone, and has neutral element 1; its dual t-conorm satisfies the same properties with neutral element 0. The simplest instances are the minimum t-norm and the maximum t-conorm.

The conjunction of two L-grades $(s_1, c_1), (s_2, c_2) \in \mathbb{L}$ is defined by

$$\top((s_1, c_1),\, (s_2, c_2)) = \begin{cases} \left(\top(s_1, s_2),\, \max_{s_i = 0} c_i\right) & \text{if } s_1 = 0 \vee s_2 = 0, \\ (\top(s_1, s_2),\, \min(c_1, c_2)) & \text{if } s_1 \neq 0 \wedge s_2 \neq 0, \end{cases} \tag{5.5}$$

where $\max_{s_i=0} c_i$ denotes the maximum confidence grade c_i among those arguments whose satisfaction grade s_i equals 0, with $i \in \{1, 2\}$. When at least one satisfaction grade is zero, those arguments fully determine the conjunction result and their confidence accordingly becomes the most relevant. When no satisfaction grade is zero, both arguments contribute to the result and the minimum confidence is taken, reflecting that the least confident argument limits the overall trust in the result.

The disjunction of two L-grades $(s_1, c_1), (s_2, c_2) \in \mathbb{L}$ is defined by

$$\perp((s_1, c_1),\, (s_2, c_2)) = \begin{cases} \big(\perp(s_1, s_2),\, \max_{s_i=1} c_i\big) & \text{if } s_1 = 1 \vee s_2 = 1, \\ (\perp(s_1, s_2),\, \min(c_1, c_2)) & \text{if } s_1 \neq 1 \wedge s_2 \neq 1, \end{cases} \tag{5.6}$$

where $\max_{s_i=1} c_i$ denotes the maximum confidence grade c_i among those arguments whose satisfaction grade s_i equals 1, with $i \in \{1, 2\}$. The reasoning mirrors that of conjunction: when at least one argument fully satisfies the disjunction, its confidence determines the result; otherwise, the minimum confidence applies. Example 5.2 illustrates how these operators work in practice.

> **Example 5.2.** Consider three L-grades $\ell_1 = (0.8, 0.9)$, $\ell_2 = (0.6, 0.7)$, and $\ell_3 = (0, 0.5)$, and let $\top$ and $\perp$ be the minimum t-norm and maximum t-conorm respectively. The negation of ℓ_1 yields $\neg\ell_1 = (0.2, 0.9)$: the satisfaction is inverted while the confidence is preserved. For the conjunction of ℓ_1 and ℓ_2, since neither satisfaction grade is zero, the minimum applies to both components: $\top(\ell_1, \ell_2) = (\min(0.8, 0.6),\, \min(0.9, 0.7)) = (0.6, 0.7)$. The conjunction of ℓ_1 and ℓ_3, however, activates the zero case: since $s_3 = 0$, the conjunction result is $\top(\ell_1, \ell_3) = (\min(0.8, 0),\, 0.5) = (0, 0.5)$, where the confidence of the zero-satisfaction argument determines the resulting confidence. For the disjunction of ℓ_1 and ℓ_2, since neither satisfaction grade is one, the minimum confidence applies: $\perp(\ell_1, \ell_2) = (\max(0.8, 0.6),\, \min(0.9, 0.7)) = (0.8, 0.7)$.

Aggregators

Beyond the basic logic operators, aggregation operators are required to combine multiple L-grades into a single representative L-grade. Here the sibling principle takes a more specific form: the confidence grade c_i of each argument should contribute to the computation of the resulting confidence c to the same extent as its corresponding satisfaction grade s_i contributes to the computation of the resulting satisfaction s. Aggregation operators that follow it are called *sibling aggregators*, and both Weighted Mean (WM)- and Ordered Weighted Averaging (OWA)-based instances have been defined for L-grades [1]. These aggregators extend the standard WM operator and the OWA operator [52], the latter described in Definition 4.1, to operate on L-grades by computing the satisfaction component through the conventional aggregation and deriving the confidence component from the weights of the contributing arguments.

The WM aggregator for L-grades is defined by

$$\mathrm{WM}((\ell_1, \ell_2, \ldots, \ell_n),\, \mathbf{w}) = \left(\sum_{i=1}^{n} w_i \cdot s_i,\; \sum_{i=1}^{n} w_i \cdot c_i \right), \tag{5.7}$$

where $\mathbf{w}$ is a weight vector and s_i and c_i are the satisfaction and confidence grades of the i-th argument, respectively. Both the satisfaction and confidence components are computed as weighted means of their respective grades, since in a WM each input occupies a fixed position and its weight directly reflects its contribution, satisfying the sibling principle without reordering.

The OWA aggregator for L-grades is defined by

$$\mathrm{OWA}((\ell_1, \ell_2, \ldots, \ell_n),\, \mathbf{w}) = \left(\sum_{i=1}^{n} w_i \cdot s_{\rho(i)},\; \sum_{i=1}^{n} w_i \cdot c_{\rho(i)} \right), \tag{5.8}$$

where $\mathbf{w}$ is a weight vector, s_i and c_i are the satisfaction and confidence grades of the i-th argument, respectively, and ρ is a permutation that orders the L-grades by decreasing satisfaction, so that $s_{\rho(1)} \geq s_{\rho(2)} \geq \ldots \geq s_{\rho(n)}$. Here, the confidence component is computed as a weighted sum of the confidence grades, where the weight assigned to each confidence grade corresponds to the weight of its sibling satisfaction grade in the sorted order. This is a direct application of the sibling principle: the confidence of each argument contributes to the aggregate confidence to the same extent as its satisfaction contributes to the aggregate satisfaction. In both aggregators, the weight vector $\mathbf{w} = (w_1, w_2, \ldots, w_n)$ satisfies $\sum_{i=1}^{n} w_i = 1$ and $w_i \geq 0$ for all i, ensuring that the resulting satisfaction and confidence grades remain within the unit interval.

The OWA aggregator for L-grades, as originally proposed in [1], sorts the L-grades exclusively by satisfaction and applies the same permutation to the confidence grades. The ranking techniques proposed in this chapter open the possibility of defining novel alternative OWA-based aggregators that sort L-grades according to a different ordering.

A more detailed treatment of the logic operators and aggregators for L-grades, including their formal properties and proofs, can be found in [1]. The concepts that formalise the notion of ranking and order relations are introduced next.

5.3.2 Order Relations

Ranking a set of elements requires a formal notion of preference among them. The relevant concepts from order theory [53] are briefly described here, as they underpin the ranking techniques proposed in Section 5.4.

To rank elements in any set S, a binary relation R is defined as a subset of $S \times S$ that captures the preference between elements. For two elements $x, y \in S$, we write $x\,R\,y$ to denote that the pair (x, y) belongs to R. Three properties of binary relations are central to the definitions that follow:

(i) the relation R is *reflexive* if $\forall\, x \in S :\; x\,R\,x$;

(ii) the relation R is *antisymmetric* if $\forall\, x, y \in S :\; x\,R\,y \;\wedge\; y\,R\,x \implies x = y$;

(iii) the relation R is *transitive* if $\forall\, x, y, z \in S :\; x\,R\,y \;\wedge\; y\,R\,z \implies x\,R\,z$.

Based on these properties, the following types of order relations can be distinguished.

> **Definition 5.2** (Partial order). A partial order on a set S is a binary relation $\preceq$ that is reflexive, antisymmetric, and transitive. The pair $(S, \preceq)$ is called a partially ordered set, or *poset*.

In a partial order, not every pair of elements need be comparable: two elements $x, y \in S$ are comparable if $x \preceq y$ or $y \preceq x$, and incomparable otherwise. When a complete ranking is required, however, a total order is needed to ensure that every pair of elements is comparable.

Definition 5.3 (Total order). A total order on a set S is a partial order $\preceq$ that is additionally *total*, meaning that every pair of elements is comparable,

$$\forall\, x, y \in S :\; x \preceq y \;\vee\; y \preceq x. \tag{5.9}$$

When the antisymmetry condition is relaxed, a weaker structure is obtained.

Definition 5.4 (Total preorder). A *total preorder*, also called a *total weak order*, on a set S is a binary relation $\preceq$ that is reflexive, transitive, and total. A total preorder permits *ties*: two distinct elements $x, y \in S$ may satisfy both $x \preceq y$ and $y \preceq x$.

Total preorders arise naturally when elements are ranked through a function that maps them onto a totally ordered set, since such a function does not need to be injective and distinct elements may therefore receive the same value.

Every order relation $\preceq$ has a converse $\succeq$, defined by $x \succeq y \iff y \preceq x$, which reverses the direction of comparison while preserving all properties. From these non-strict relations, the corresponding strict orders are derived by excluding the cases in which the comparison also holds in the opposite direction, i.e., when both elements are equal or tied,

$$x \prec y \iff x \preceq y \;\wedge\; \neg(y \preceq x), \qquad x \succ y \iff x \succeq y \;\wedge\; \neg(y \succeq x). \tag{5.10}$$

For a total order, where ties cannot occur, this amounts to excluding equality. The strict orders $\prec$ and $\succ$ are *irreflexive* and *asymmetric*, but remain transitive.

In the context of L-grades, we write $\ell_i \succeq \ell_j$ to denote that ℓ_i is *preferred or equally preferred* to ℓ_j, and $\ell_i \succ \ell_j$ to denote that ℓ_i is *strictly preferred* over ℓ_j. Conversely, $\ell_i \preceq \ell_j$ denotes that ℓ_i is *less preferred or equally preferred* to ℓ_j, and $\ell_i \prec \ell_j$ that ℓ_i is *strictly less preferred*. Equality of two L-grades, $\ell_i = \ell_j$, is defined component-wise as in Equation (5.3). Because s and c are orthogonal, cases such as the one in Example 5.1 arise in which one L-grade has higher satisfaction but lower confidence than another, leaving both incomparable under a partial order. The ranking techniques proposed in Section 5.4 resolve these incomparabilities by imposing total orders or total preorders on $\mathbb{L}$ through different mechanisms, each reflecting a different set of stakeholder preferences.

5.4 Ranking L-grades

With the formal concepts from Section 5.3 in place, we now turn to the ranking of L-grades. When data objects resulting from a decision-making or flexible querying process each carry an associated L-grade, these objects

must be presented to stakeholders in a meaningful order, or the L-grades themselves must be compared as part of the data processing technique under consideration. Section 5.4.1 first identifies the cases in which one L-grade can be unambiguously preferred over another and shows that these cases yield only a partial ordering on $\mathbb{L}$. Three ranking approaches that each impose a total (pre)ordering on $\mathbb{L}$ are then proposed: ranking based on lexicographic ordering in Section 5.4.2, and non-weighted and weighted ranking based on global preference in Section 5.4.3. Section 5.4.4 then introduces a distance measure that quantifies the strength of a preference between two L-grades, as well as an approximate equality comparator.

5.4.1 Full Dominance and Partial Ordering

Given two L-grades $\ell_i = (s_i, c_i)$ and $\ell_j = (s_j, c_j)$, in general ℓ_i might be preferred over ℓ_j (denoted $\ell_i \succ \ell_j$), ℓ_j might be preferred over ℓ_i (denoted $\ell_i \prec \ell_j$), or both might have equal preference (denoted $\ell_i = \ell_j$). In line with what has been established in the preliminaries, the ranking relations to be defined are subject to the following properties:

$$\ell_i \preceq \ell_j \iff \ell_j \succeq \ell_i, \tag{5.11}$$

$$\ell_i \prec \ell_j \iff \ell_i \preceq \ell_j \wedge \neg(\ell_j \preceq \ell_i), \tag{5.12}$$

$$\ell_i \succ \ell_j \iff \ell_i \succeq \ell_j \wedge \neg(\ell_j \succeq \ell_i). \tag{5.13}$$

From a stakeholder's point of view, both components s and c of an L-grade $\ell = (s, c)$ are orthogonal, as illustrated by the two-dimensional structure of $\mathbb{L}$ shown in Figure 5.1. A ranking technique should therefore make a trade-off between satisfaction and confidence. In general, it can be safely assumed that all stakeholders simultaneously want high satisfaction and high confidence. Likewise, L-grades with low satisfaction and low confidence are usually less preferred. The preference $\ell_i \succ \ell_j$ can be easily assigned in the following cases of *full dominance*:

$$(s_i > s_j) \wedge (c_i \geq c_j) \implies \ell_i \succ \ell_j, \tag{5.14}$$

$$(s_i = s_j) \wedge (c_i > c_j) \implies \ell_i \succ \ell_j. \tag{5.15}$$

These cases of dominance impose a partial ordering on $\mathbb{L}$, as defined in Section 5.3.2. Cases where $s_i > s_j$ and $c_i < c_j$ remain challenging, because the preferred L-grade depends on how the stakeholder weighs satisfaction against confidence. Additionally, in comparison problems we are not only interested in the rank order, but also in how strong the preference of ℓ_i over ℓ_j is. The following subsections propose ranking approaches that resolve these incomparabilities by imposing a total (pre)ordering on $\mathbb{L}$, complemented by a distance measure that reflects preference strength.

5.4.2 Ranking Based on Lexicographic Ordering

In *lexicographic ordering*, L-grades are ordered based on the value of one of their two components, s or c, selected by the stakeholder. L-grades with an equal value for the selected component are further ordered by

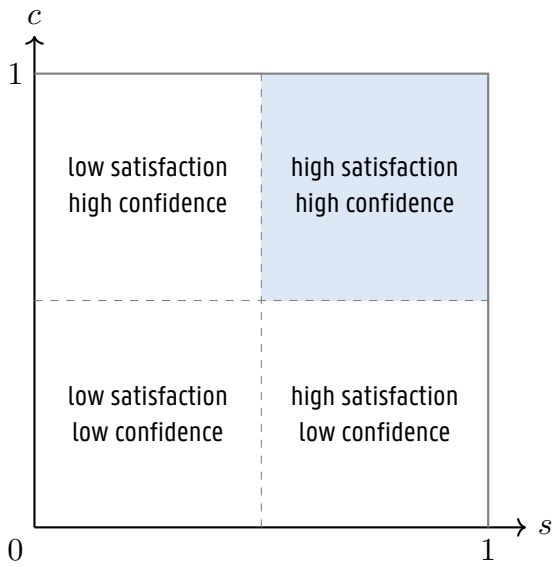


Figure 5.1: Visualisation of $\mathbb{L}$. Both components s (satisfaction) and c (confidence) range from 0 to 1. The shaded region in the upper-right quadrant represents the area of simultaneously high satisfaction and high confidence, which is generally preferred by stakeholders.

their other component. The stakeholder therefore needs to indicate whether satisfaction or confidence should serve as the primary ordering criterion.

If stakeholders give preference to satisfaction, we obtain the following order relation:

$$\ell_i \succ^{s}_{\text{lex}} \ell_j \iff \big(s_i > s_j\big) \vee \big(s_i = s_j \wedge c_i > c_j\big). \tag{5.16}$$

When confidence is prioritised, the order relation becomes:

$$\ell_i \succ^{c}_{\text{lex}} \ell_j \iff \big(c_i > c_j\big) \vee \big(c_i = c_j \wedge s_i > s_j\big). \tag{5.17}$$

The corresponding non-strict relations are obtained by additionally including equality,

$$\ell_i \succeq^{s}_{\text{lex}} \ell_j \iff \ell_i \succ^{s}_{\text{lex}} \ell_j \ \vee \ \ell_i = \ell_j, \tag{5.18}$$

$$\ell_i \succeq^{c}_{\text{lex}} \ell_j \iff \ell_i \succ^{c}_{\text{lex}} \ell_j \ \vee \ \ell_i = \ell_j. \tag{5.19}$$

The same convention applies to all order relations defined in this chapter: each strict relation $\succ$ is accompanied by a non-strict counterpart $\succeq$ defined analogously.

Proposition 5.1. The order relations $\succeq^{s}_{\text{lex}}$ and $\succeq^{c}_{\text{lex}}$ are total orders on $\mathbb{L}$ and support the cases of full dominance expressed by Equations (5.14) and (5.15).

Proof. We prove the claims for $\succeq^{s}_{\text{lex}}$; the argument for $\succeq^{c}_{\text{lex}}$ is analogous with the roles of s and c interchanged. We first show that $\succeq^{s}_{\text{lex}}$ is a total order on $\mathbb{L}$ by verifying reflexivity, antisymmetry, transitivity, and totality, and then verify that it supports the cases of full dominance.

Reflexivity.
For any $\ell \in \mathbb{L}$, the equality $\ell = \ell$ holds, so $\ell \succeq^{s}_{\text{lex}} \ell$ by the equality clause of Equation (5.18).

Antisymmetry.
Proof by contradiction. Let $\ell_1 = (s_1, c_1)$ and $\ell_2 = (s_2, c_2)$ be two elements of $\mathbb{L}$ and suppose $\ell_1 \succeq^{s}_{\text{lex}} \ell_2$ and $\ell_2 \succeq^{s}_{\text{lex}} \ell_1$ with $\ell_1 \neq \ell_2$. Then $\ell_1 \succ^{s}_{\text{lex}} \ell_2$ and $\ell_2 \succ^{s}_{\text{lex}} \ell_1$ must both hold. From $\ell_1 \succ^{s}_{\text{lex}} \ell_2$, either $s_1 > s_2$ or $(s_1 = s_2 \wedge c_1 > c_2)$ holds. From $\ell_2 \succ^{s}_{\text{lex}} \ell_1$, either $s_2 > s_1$ or $(s_2 = s_1 \wedge c_2 > c_1)$ holds. Both conditions cannot hold simultaneously: if $s_1 > s_2$ it contradicts both branches of $\ell_2 \succeq^{s}_{\text{lex}} \ell_1$, and if $s_1 = s_2 \wedge c_1 > c_2$ it contradicts $c_2 > c_1$. Therefore ℓ_1 must be equal to ℓ_2, establishing antisymmetry.

Transitivity.
Let $\ell_1 = (s_1, c_1), \ell_2 = (s_2, c_2)$, and $\ell_3 = (s_3, c_3)$ be three elements of $\mathbb{L}$ and suppose $\ell_1 \succeq^{s}_{\text{lex}} \ell_2$ and $\ell_2 \succeq^{s}_{\text{lex}} \ell_3$. If at least one of the two relations is an equality, then $\ell_1 \succeq^{s}_{\text{lex}} \ell_3$ follows immediately. Otherwise, $\ell_1 \succ^{s}_{\text{lex}} \ell_2$ and $\ell_2 \succ^{s}_{\text{lex}} \ell_3$ must both hold. From $\ell_1 \succ^{s}_{\text{lex}} \ell_2$ follows that $s_1 > s_2$ or that $(s_1 = s_2 \wedge c_1 > c_2)$; from $\ell_2 \succ^{s}_{\text{lex}} \ell_3$ follows that $s_2 > s_3$ or that $(s_2 = s_3 \wedge c_2 > c_3)$. In each of the four combinations, either $s_1 > s_3$ or $(s_1 = s_3 \wedge c_1 > c_3)$ follows from the transitivity of $>$ and $=$ on $\mathbb{R}$, so $\ell_1 \succ^{s}_{\text{lex}} \ell_3$, establishing transitivity.

Totality.
Let $\ell_1 = (s_1, c_1)$ and $\ell_2 = (s_2, c_2)$ be arbitrary L-grades. Exactly one of $s_1 > s_2$, $s_1 = s_2$, or $s_1 < s_2$ holds. If $s_1 > s_2$, then $\ell_1 \succ^{s}_{\text{lex}} \ell_2$ follows from Equation (5.16). If $s_1 < s_2$, then $\ell_2 \succ^{s}_{\text{lex}} \ell_1$ also follows from Equation (5.16). If $s_1 = s_2$, then either $c_1 > c_2$, so $\ell_1 \succ^{s}_{\text{lex}} \ell_2$; or $c_1 < c_2$, so $\ell_2 \succ^{s}_{\text{lex}} \ell_1$; or $c_1 = c_2$, so $\ell_1 = \ell_2$. In every case, $\ell_1 \succeq^{s}_{\text{lex}} \ell_2 \vee \ell_2 \succeq^{s}_{\text{lex}} \ell_1$, proving totality.

Full dominance.
Let $\ell_1 = (s_1, c_1)$ and $\ell_2 = (s_2, c_2)$ be two L-grades.

- Consider Equation (5.14), which establishes $(s_1 > s_2) \wedge (c_1 \geq c_2)$. Whenever $s_1 > s_2$ then the first term of Equation (5.16) holds, which gives $\ell_1 \succ^{s}_{\text{lex}} \ell_2$.
- Consider Equation (5.15), which establishes $(s_1 = s_2) \wedge (c_1 > c_2)$. The second term of Equation (5.16) gives $\ell_1 \succ^{s}_{\text{lex}} \ell_2$.

Since $\succeq^{s}_{\text{lex}}$ is reflexive, antisymmetric, transitive, and total, it is a total order on $\mathbb{L}$. □

Example 5.3 illustrates how the choice of primary component in lexicographic ordering can lead to different rankings of the same set of L-grades.

Example 5.3. Consider the following L-grades: $\ell_1 = (1, 1), \ell_2 = (1, 0.8), \ell_3 = (0.8, 0.8), \ell_4 = (0.6, 0.8), \ell_5 = (0.2, 0.8), \ell_6 = (0.8, 0.6), \ell_7 = (0.8, 0.2), \ell_8 = (0.8, 1), \ell_9 = (0, 1),$

$\ell_{10} = (1, 0)$, and $\ell_{11} = (0, 0)$. Ranking these L-grades using the lexicographic ordering relations $\succ^s_{\text{lex}}$ and $\succ^c_{\text{lex}}$ yields:

$$\ell_1 \succ^s_{\text{lex}} \ell_2 \succ^s_{\text{lex}} \ell_{10} \succ^s_{\text{lex}} \ell_8 \succ^s_{\text{lex}} \ell_3 \succ^s_{\text{lex}} \ell_6 \succ^s_{\text{lex}} \ell_7 \succ^s_{\text{lex}} \ell_4 \succ^s_{\text{lex}} \ell_5 \succ^s_{\text{lex}} \ell_9 \succ^s_{\text{lex}} \ell_{11}$$

and

$$\ell_1 \succ^c_{\text{lex}} \ell_8 \succ^c_{\text{lex}} \ell_9 \succ^c_{\text{lex}} \ell_2 \succ^c_{\text{lex}} \ell_3 \succ^c_{\text{lex}} \ell_4 \succ^c_{\text{lex}} \ell_5 \succ^c_{\text{lex}} \ell_6 \succ^c_{\text{lex}} \ell_7 \succ^c_{\text{lex}} \ell_{10} \succ^c_{\text{lex}} \ell_{11}.$$

With satisfaction-based ordering, $\ell_{10} = (1, 0)$ ranks third because its satisfaction is maximal despite having zero confidence. Under confidence-based ordering, the same L-grade drops to tenth place, illustrating how the choice of primary component fundamentally changes the ranking.

In practice, stakeholders may require minimum levels of satisfaction and confidence to accept an object with an associated L-grade. This can be achieved by asking the stakeholders to provide a threshold value $0 \leq \tau_s \leq 1$ for satisfaction and a threshold value $0 \leq \tau_c \leq 1$ for confidence. Objects with an associated L-grade $\ell = (s, c)$ for which $s < \tau_s$ or $c < \tau_c$ are then considered unacceptable and discarded from further processing or presentation.

Example 5.4. Reconsider the L-grades from Example 5.3. Assume that a stakeholder does not accept objects with associated L-grades that have a satisfaction lower than 0.8 or a confidence lower than 0.5. This can be imposed with thresholds $\tau_s = 0.8$ and $\tau_c = 0.5$. Employing these thresholds ensures that the objects with L-grades $\ell_4 = (0.6, 0.8)$, $\ell_5 = (0.2, 0.8)$, $\ell_7 = (0.8, 0.2)$, $\ell_9 = (0, 1)$, $\ell_{10} = (1, 0)$, and $\ell_{11} = (0, 0)$ are no longer considered in further processing and presentation. The rankings of Example 5.3 are then restricted to:

$$\ell_1 \succ^s_{\text{lex}} \ell_2 \succ^s_{\text{lex}} \ell_8 \succ^s_{\text{lex}} \ell_3 \succ^s_{\text{lex}} \ell_6$$

and

$$\ell_1 \succ^c_{\text{lex}} \ell_8 \succ^c_{\text{lex}} \ell_2 \succ^c_{\text{lex}} \ell_3 \succ^c_{\text{lex}} \ell_6.$$

Notice that $\ell_{10} = (1, 0)$, which ranked third under satisfaction-based ordering in Example 5.3, is now excluded because its confidence falls below the threshold.

5.4.3 Ranking Based on Global Preference

A stakeholder might also prefer to rank L-grades based on a global preference function $p : \mathbb{L} \to [0, 1]$ that reflects a trade-off between satisfaction and confidence. In what follows, we make a distinction between *non-weighted* and *weighted* global preference functions.

Non-Weighted Global Preference

A basic choice for p is

$$\begin{aligned} p_{\text{basic}} : \mathbb{L} &\to [0,1] \\ (s,c) &\mapsto s \cdot c, \end{aligned} \tag{5.20}$$

which combines satisfaction and confidence conjunctively through the product, treating both as equally important. In principle any t-norm could be used here, but the resulting order relation preserves full dominance if and only if it is a strict t-norm (i.e., strictly increasing in each argument on $]0,1]$). The product t-norm satisfies this requirement, whereas non-strict t-norms, like the minimum t-norm do not. For example, $\min(1, 0.8) = \min(0.9, 0.8)$ which incorrectly ties two L-grades of which the first strictly dominates the second. Using p_{basic}, we define the following order relation:

$$\ell_i \succ^{\text{basic}}_{\text{global}} \ell_j \iff p_{\text{basic}}(\ell_i) > p_{\text{basic}}(\ell_j). \tag{5.21}$$

Proposition 5.2. The order relation $\succeq^{\text{basic}}_{\text{global}}$ is a total preorder on $\mathbb{L}$. It supports the cases of full dominance expressed by Equations (5.14) and (5.15), except when $c_i = c_j = 0$ in Equation (5.14) or when $s_i = s_j = 0$ in Equation (5.15).

Proof. We first show that $\succeq^{\text{basic}}_{\text{global}}$ is a total preorder and then verify the cases of full dominance.

Total preorder.
Equation (5.20) maps each L-grade $\ell = (s,c)$ to a single real value $p_{\text{basic}}(\ell) = s \cdot c$ in the interval $[0,1]$. The order relation $\succeq^{\text{basic}}_{\text{global}}$ corresponds to the standard order $\geq$ on $\mathbb{R}$ applied to these preference values and therefore inherits reflexivity, transitivity, and totality from $\geq$. Antisymmetry does not hold: for instance, $p_{\text{basic}}((1, 0.8)) = 0.8 = p_{\text{basic}}((0.8, 1))$, yet $(1, 0.8) \neq (0.8, 1)$. The relation is therefore a total preorder but not a total order.

Full dominance.
Let $\ell_1 = (s_1, c_1)$ and $\ell_2 = (s_2, c_2)$ be two L-grades.

- Consider Equation (5.14), which establishes $(s_1 > s_2) \wedge (c_1 \geq c_2)$. If $c_1 > 0$, then $s_1 \cdot c_1 > s_2 \cdot c_2$, since $c_1 > 0$ preserves the strict inequality through multiplication and $c_1 \geq c_2$ ensures the second inequality. Hence $\ell_1 \succ^{\text{basic}}_{\text{global}} \ell_2$ and dominance is reflected. However, if $c_1 = c_2 = 0$, then $s_1 \cdot 0 = s_2 \cdot 0$, so $p_{\text{basic}}(\ell_1) = p_{\text{basic}}(\ell_2)$ and dominance is not reflected.
- Consider Equation (5.15), which establishes $(s_1 = s_2) \wedge (c_1 > c_2)$. If $s_1 = s_2 > 0$, then $s_1 \cdot c_1 > s_2 \cdot c_2$, so $\ell_1 \succ^{\text{basic}}_{\text{global}} \ell_2$ and dominance is reflected. However, if $s_1 = s_2 = 0$, then $0 \cdot c_1 = 0 \cdot c_2$ and dominance is again not reflected.

□

The boundary cases in which dominance is not reflected by p_{basic} arise when one of the two components is zero for both L-grades under comparison. Since 0 is the annihilator of every t-norm [51], these cases are inherent to conjunctive combination and cannot be avoided by selecting a different operator. Two strategies can mitigate

this limitation. First, the threshold mechanism introduced in the previous subsection can exclude L-grades with very low satisfaction or confidence from the ranking. Second, lexicographic ordering can be applied as a tiebreaker among L-grades that receive the same global preference value.

The following example illustrates how the non-weighted global preference function produces a different ranking from the lexicographic orderings of Example 5.3.

Example 5.5. Applying preference function p_{basic} to the L-grades $\ell_1, \ell_2, \ldots, \ell_{11}$ from Example 5.3 yields the following global preference values: $p_{\text{basic}}(\ell_1) = 1$, $p_{\text{basic}}(\ell_2) = 0.8$, $p_{\text{basic}}(\ell_3) = 0.64$, $p_{\text{basic}}(\ell_4) = 0.48$, $p_{\text{basic}}(\ell_5) = 0.16$, $p_{\text{basic}}(\ell_6) = 0.48$, $p_{\text{basic}}(\ell_7) = 0.16$, $p_{\text{basic}}(\ell_8) = 0.8$, $p_{\text{basic}}(\ell_9) = 0$, $p_{\text{basic}}(\ell_{10}) = 0$, and $p_{\text{basic}}(\ell_{11}) = 0$. The resulting ranking is:

$$\begin{aligned}\ell_1 \succ^{\text{basic}}_{\text{global}} \ell_2 =^{\text{basic}}_{\text{global}} \ell_8 \succ^{\text{basic}}_{\text{global}} \ell_3 \succ^{\text{basic}}_{\text{global}} \ell_4 =^{\text{basic}}_{\text{global}} \ell_6 \\ \succ^{\text{basic}}_{\text{global}} \ell_5 =^{\text{basic}}_{\text{global}} \ell_7 \succ^{\text{basic}}_{\text{global}} \ell_9 =^{\text{basic}}_{\text{global}} \ell_{10} =^{\text{basic}}_{\text{global}} \ell_{11}.\end{aligned}$$

Here, $=^{\text{basic}}_{\text{global}}$ denotes an equal global preference value, i.e., a tie under the preorder, rather than component-wise equality of the L-grades. Unlike the lexicographic orderings of Example 5.3, the non-weighted global preference function produces ties between L-grades whose product $s \cdot c$ is identical, such as $\ell_2 = (1, 0.8)$ and $\ell_8 = (0.8, 1)$. The three-way tie $\ell_9 =^{\text{basic}}_{\text{global}} \ell_{10} =^{\text{basic}}_{\text{global}} \ell_{11}$ illustrates the boundary limitation identified above: $\ell_9 = (0, 1)$ and $\ell_{10} = (1, 0)$ each have a zero component that annihilates the product, placing them at the same level as $\ell_{11} = (0, 0)$ despite one component being maximal in each case.

Weighted Global Preference

Alternatively, stakeholders can provide a weight $w \in [0, 1]$ denoting the relative importance of confidence versus satisfaction, which is used to define a weighted global preference function:

$$\begin{aligned} p_{\text{weighted}} : \mathbb{L} \times [0,1] &\to [0,1] \\ ((s,c),\, w) &\mapsto (1-w) \cdot s + w \cdot c. \end{aligned} \tag{5.22}$$

Using p_{weighted}, the following order relation $\succ^{\text{weighted}}_{\text{global}}$ is obtained:

$$\ell_i \succ^{\text{weighted}}_{\text{global}} \ell_j \iff p_{\text{weighted}}(\ell_i, w) > p_{\text{weighted}}(\ell_j, w). \tag{5.23}$$

Proposition 5.3. The order relation $\succeq^{\text{weighted}}_{\text{global}}$ is a total preorder on $\mathbb{L}$. For $w \in]0, 1[$, it supports the cases of full dominance expressed by Equations (5.14) and (5.15). At the boundary value $w = 0$, only the dominance case of Equation (5.14) is fully supported; at $w = 1$, Equation (5.14) is supported only when $c_i > c_j$, and Equation (5.15) is fully supported.

Proof. We first show that $\succeq^{\text{weighted}}_{\text{global}}$ is a total preorder and then verify the cases of full dominance.

Total preorder.
As with $\succeq^{\text{basic}}_{\text{global}}$, each L-grade is mapped to a real value in $[0,1]$ and the order relation corresponds to $\geq$ on $\mathbb{R}$, so reflexivity, transitivity, and totality are inherited directly. Antisymmetry does not hold: for instance, with $w = 0.8$, the L-grades $(0.8, 0.8)$ and $(0, 1)$ both receive $p_{\text{weighted}} = 0.8$ while being distinct. The relation is therefore a total preorder but not a total order.

Full dominance.
Let $\ell_1 = (s_1, c_1)$ and $\ell_2 = (s_2, c_2)$ be two L-grades, and write $\Delta = p_{\text{weighted}}(\ell_1, w) - p_{\text{weighted}}(\ell_2, w) = (1-w)(s_1 - s_2) + w(c_1 - c_2)$ as the difference between their weighted preferences. The sign of Δ fully determines the relative ordering: $\Delta > 0$ yields $\ell_1 \succ^{\text{weighted}}_{\text{global}} \ell_2$, and $\Delta < 0$ yields $\ell_2 \succ^{\text{weighted}}_{\text{global}} \ell_1$. When $\Delta = 0$, the relation yields $\ell_1 =^{\text{weighted}}_{\text{global}} \ell_2$, either because the two L-grades are equally preferable or because the choice of w suppresses a difference that exists between them. We now verify under which conditions on w the dominance cases are reflected, i.e., yield $\Delta > 0$ for $\ell_1 \succ^{\text{weighted}}_{\text{global}} \ell_2$.

- Consider Equation (5.14), which establishes $(s_1 > s_2) \wedge (c_1 \geq c_2)$. For $w < 1$, the first term $(1-w)(s_1 - s_2) > 0$ and the second term $w(c_1 - c_2) \geq 0$, so $\Delta > 0$ and dominance is reflected. For $w = 1$, the first term vanishes and $\Delta = c_1 - c_2 \geq 0$, which is zero when $c_1 = c_2$, so dominance is not guaranteed.
- Consider Equation (5.15), which establishes $(s_1 = s_2) \wedge (c_1 > c_2)$. For $w > 0$, the second term $w(c_1 - c_2) > 0$ and the first term vanishes, so $\Delta > 0$ and dominance is reflected. For $w = 0$, both terms vanish and $\Delta = 0$, so dominance is not reflected.

For $w \in]0, 1[$, both cases of full dominance are therefore supported. At $w = 0$, only Equation (5.14) is supported. At $w = 1$, Equation (5.15) is fully supported, while Equation (5.14) is supported only when $c_1 > c_2$. □

The boundary behaviour is intuitive: when $w = 0$, the preference function reduces to s alone, so differences in confidence are ignored; when $w = 1$, it reduces to c alone, so differences in satisfaction are ignored. For any weight $w \in]0, 1[$, both components contribute and both dominance cases are fully supported.

A weight of $w = 0.8$, which places more emphasis on confidence than on satisfaction, has the following effect.

Example 5.6. Applying preference function p_{weighted} with weight $w = 0.8$ to the L-grades $\ell_1, \ell_2, \ldots, \ell_{11}$ from Example 5.3 yields the following global preference values: $p_{\text{weighted}}(\ell_1, 0.8) = 1$, $p_{\text{weighted}}(\ell_2, 0.8) = 0.84$, $p_{\text{weighted}}(\ell_3, 0.8) = 0.8$, $p_{\text{weighted}}(\ell_4, 0.8) = 0.76$, $p_{\text{weighted}}(\ell_5, 0.8) = 0.68$, $p_{\text{weighted}}(\ell_6, 0.8) = 0.64$, $p_{\text{weighted}}(\ell_7, 0.8) = 0.32$, $p_{\text{weighted}}(\ell_8, 0.8) = 0.96$, $p_{\text{weighted}}(\ell_9, 0.8) = 0.8$, $p_{\text{weighted}}(\ell_{10}, 0.8) = 0.2$, and $p_{\text{weighted}}(\ell_{11}, 0.8) = 0$. The resulting ranking is:

$$\ell_1 \succ^{\text{weighted}}_{\text{global}} \ell_8 \succ^{\text{weighted}}_{\text{global}} \ell_2 \succ^{\text{weighted}}_{\text{global}} \ell_3 =^{\text{weighted}}_{\text{global}} \ell_9 \succ^{\text{weighted}}_{\text{global}} \ell_4 \succ^{\text{weighted}}_{\text{global}} \ell_5 \succ^{\text{weighted}}_{\text{global}} \ell_6 \succ^{\text{weighted}}_{\text{global}} \ell_7 \succ^{\text{weighted}}_{\text{global}} \ell_{10} \succ^{\text{weighted}}_{\text{global}} \ell_{11}.$$

Here, $=^{\text{weighted}}_{\text{global}}$ denotes an equal weighted preference value, i.e., a tie under the preorder, rather than component-wise equality of the L-grades. With $w = 0.8$, confidence contributes four times as much as satisfaction. Compared to the non-weighted ranking of Example 5.5, L-grade $\ell_8 = (0.8, 1)$ now ranks above $\ell_2 = (1, 0.8)$ because its higher confidence outweighs its lower satisfaction under this weight. Likewise, $\ell_9 = (0, 1)$, which has zero satisfaction but full confidence, ties with $\ell_3 = (0.8, 0.8)$, illustrating the strong emphasis on confidence imposed by the chosen weight.

5.4.4 Distance Between L-grades

Beyond the information that, based on a given preference relation $\succ$, an L-grade ℓ_1 is preferred over another L-grade ℓ_2, a stakeholder might also be interested in the *strength* of that preference. In other words, how strongly is ℓ_1 preferred over ℓ_2? We use the distance between ℓ_1 and ℓ_2 as a measure of the strength of the preference between them, the preference of ℓ_1 over ℓ_2 if $\ell_1 \succ \ell_2$ and that of ℓ_2 over ℓ_1 otherwise: the larger the distance, the stronger the preference.

Since L-grades can be represented as points in the two-dimensional Euclidean subspace shown in Figure 5.1, the Euclidean distance provides a natural measure of the separation between two L-grades. The distance d between two L-grades $\ell_1 = (s_1, c_1)$ and $\ell_2 = (s_2, c_2)$ is defined by

$$\begin{aligned} d : \mathbb{L} \times \mathbb{L} &\to \left[0, \sqrt{2}\right] \\ ((s_1, c_1),\, (s_2, c_2)) &\mapsto \sqrt{(s_1 - s_2)^2 + (c_1 - c_2)^2}. \end{aligned} \tag{5.24}$$

As a restriction of the standard Euclidean metric to $\mathbb{L} = [0, 1]^2$, the distance d inherits the standard metric properties: non-negativity, identity of indiscernibles, symmetry, and the triangle inequality. The maximum distance $\sqrt{2}$ results from computing the distance between the L-grades $(0, 0)$ and $(1, 1)$. When two L-grades are ranked equally, the distance no longer expresses a preference strength, but still indicates how far apart they lie in $\mathbb{L}$.

Example 5.7. Considering the ranking obtained in Example 5.6, the distances between consecutively ranked L-grades are:

$$\begin{aligned}
\ell_1 &\succ^{\text{weighted}}_{\text{global}} \ell_8 \to d((1,1),\,(0.8,1)) &&= 0.20 \\
\ell_8 &\succ^{\text{weighted}}_{\text{global}} \ell_2 \to d((0.8,1),\,(1,0.8)) &&\approx 0.28 \\
\ell_2 &\succ^{\text{weighted}}_{\text{global}} \ell_3 \to d((1,0.8),\,(0.8,0.8)) &&= 0.20 \\
\ell_3 &=^{\text{weighted}}_{\text{global}} \ell_9 \to d((0.8,0.8),\,(0,1)) &&\approx 0.82 \\
\ell_9 &\succ^{\text{weighted}}_{\text{global}} \ell_4 \to d((0,1),\,(0.6,0.8)) &&\approx 0.63 \\
\ell_4 &\succ^{\text{weighted}}_{\text{global}} \ell_5 \to d((0.6,0.8),\,(0.2,0.8)) &&= 0.40 \\
\ell_5 &\succ^{\text{weighted}}_{\text{global}} \ell_6 \to d((0.2,0.8),\,(0.8,0.6)) &&\approx 0.63 \\
\ell_6 &\succ^{\text{weighted}}_{\text{global}} \ell_7 \to d((0.8,0.6),\,(0.8,0.2)) &&= 0.40
\end{aligned}$$

$$\ell_7 \succ^{\text{weighted}}_{\text{global}} \ell_{10} \to d((0.8, 0.2),\, (1, 0)) \qquad \approx 0.28$$
$$\ell_{10} \succ^{\text{weighted}}_{\text{global}} \ell_{11} \to d((1, 0),\, (0, 0)) \qquad = 1.00$$

These distances reveal that consecutively ranked L-grades are not necessarily equidistant. The preference of ℓ_{10} over ℓ_{11}, for instance, is by far the strongest of the ranking ($d = 1.00$), whereas the preference of ℓ_1 over ℓ_8 is comparatively weak ($d = 0.20$), despite both being consecutive in the ranking of Example 5.6. The tied pair $\ell_3 =^{\text{weighted}}_{\text{global}} \ell_9$ is particularly noteworthy: although both L-grades receive the same global preference value, their distance in $\mathbb{L}$ is 0.82, indicating that they occupy very different positions in the satisfaction-confidence space despite being ranked equally.

The distance d between two L-grades can also be used to define *proximity*, which is useful in applications that tolerate small differences between L-grades rather than enforcing a strict total ordering. Two L-grades ℓ_1 and ℓ_2 are considered *approximately equal*, denoted $\ell_1 \approx \ell_2$, if they are sufficiently close to each other. This can be formalised through a decreasing fuzzy membership function $\mu_{\approx} : \left[0, \sqrt{2}\right] \to [0, 1]$, defined on the range of d. The approximate equality comparator $\approx$ for L-grades is then defined by

$$\begin{aligned} \approx\colon \mathbb{L} \times \mathbb{L} &\to [0, 1] \\ (\ell_1, \ell_2) &\mapsto \mu_{\approx}(d(\ell_1, \ell_2)). \end{aligned} \tag{5.25}$$

when $\mu_{\approx}(d(\ell_1, \ell_2)) > 0$, the L-grades ℓ_1 and ℓ_2 are considered approximately equal, with the degree of proximity given by the value of $\mu_{\approx}(d(\ell_1, \ell_2))$.

A simple linear membership function $\mu_{\approx}$ can be used to model this comparator.

Example 5.8. An example of a fuzzy membership function $\mu_{\approx} : \left[0, \sqrt{2}\right] \to [0, 1]$ for $\approx$ is shown in Figure 5.2. The presented function is defined by

$$\mu_{\approx}(d) = \begin{cases} 1 - \dfrac{d}{\delta}, & \text{if } d \leq \delta, \\ 0, & \text{otherwise,} \end{cases} \tag{5.26}$$

where $0 < \delta \leq \sqrt{2}$ is a threshold value above which a given distance is no longer considered sufficient for proximity. With $\delta = 0.2$: $((0.8, 0.8) \approx (0.8, 0.8)) = 1$, $((0.8, 0.8) \approx (0.8, 0.7)) = 0.5$, and $((0.8, 0.8) \approx (0.8, 0.5)) = 0$.

In practice, this comparator is particularly useful for grouping query results into tiers of approximately equally ranked items rather than imposing a strict total ordering. The choice of the threshold δ and the shape of the membership function $\mu_{\approx}$ can be adapted to the requirements of the application.

The ranking techniques and distance measure introduced in this section provide a comprehensive set of methods for ordering, comparing, and grouping L-grades. Their practical applicability is demonstrated in the next section through two use cases based on the DBBE corpus.

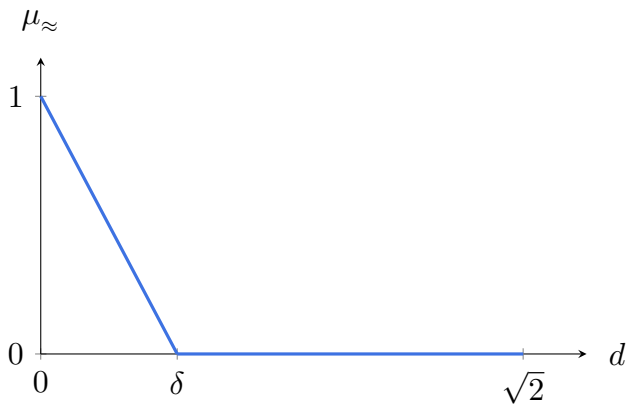


Figure 5.2: Example of a membership function $\mu_{\approx}$ to model proximity $\approx$ between L-grades. The function decreases linearly from 1 at $d = 0$ to 0 at $d = \delta$, and remains 0 for all distances beyond δ.

5.5 Practical Applicability

The ranking techniques proposed in this chapter are designed to support any application in which data objects are characterised by satisfaction-confidence pairs that need to be ordered according to stakeholder preferences. In this section, we demonstrate this through two use cases grounded in the Database of Byzantine Book Epigrams (DBBE) corpus [18, 19]. The first use case applies the ranking techniques to the similarity-confidence pairs produced by the context-aware similarity measure of Chapter 4, showing how different ranking approaches produce distinct orderings of the similarity-confidence pairs between a reference epigram and the other epigrams in the corpus. The second use case introduces a geographical querying technique that assigns L-grades to epigrams based on their association with a queried location in a hierarchical geographical structure, and demonstrates how the proposed ranking approaches order the retrieved results.

5.5.1 Ranking Similarity-Confidence Pairs

The context-aware and responsible similarity measure proposed in Chapter 4 produces, for each pair of texts, a similarity-confidence pair (s, c). These pairs are in fact instances of L-grades. The epigrams and their associated metadata are modelled in a Neo4j graph database [54] following the graph-based text representation of Chapter 2. The similarity and confidence values are stored on the `CONTEXT_SIMILAR_TO` edges linking compared texts, as `similarity` and `confidence` properties, respectively, as described in Section 4.4.3. This storage model allows the context-aware similarity between epigrams to be retrieved through graph queries and the proposed ranking techniques to be applied directly within the query to order the resulting L-grades.

In the following demonstration, we use the subset of 496 epigrams from the DBBE corpus employed in the experiments of Chapters 3 and 4. The similarity-confidence pairs were computed using the experimental configuration described in Section 4.5, with the average weight vector rather than the empirically determined one, since its equal weights yield a wider spread of confidence values across pairs, as noted in Section 4.5.5. In particular, the textual similarity from Chapter 3 was computed using the Custom 1 parameter configuration, and

the metadata similarities for location (with `city` granularity), genre, metre, scribe, and subject were computed as described in Chapter 4.

Since both components of the L-grade are stored as numerical edge properties, the ranking techniques can be implemented directly in the Cypher query language [55] using its `ORDER BY` clause. Lexicographic ordering by satisfaction, for instance, is achieved by sorting first on the `similarity` property and then on the `confidence` property, both in descending order. Listing 5.1 shows a query that retrieves all context-aware similarity-confidence pairs between a reference epigram with identifier 17752 and the remaining epigrams in the subset, and orders them from highest to lowest preference by the lexicographic ordering $\succeq^{s}_{\text{lex}}$. The query returns the identifier of the other epigram alongside the similarity and confidence values for the `CONTEXT_SIMILAR_TO` relationship between it and the reference epigram. The top ten rows of the output are shown in the left part of Table 5.1.

```
MATCH (e1:Epigram {id: 17752})-[cs:CONTEXT_SIMILAR_TO]-(e2:Epigram)
RETURN e2.id, cs.similarity, cs.confidence
ORDER BY cs.similarity DESC, cs.confidence DESC
```

Listing 5.1: Cypher query to retrieve all context-aware similarity-confidence pairs between the epigram with identifier 17752 and the other epigrams, and order them by the lexicographic ordering $\succeq^{s}_{\text{lex}}$.

The non-weighted global preference function p_{basic} can similarly be computed within the query by multiplying the similarity and confidence properties and ordering on the resulting value. Listing 5.2 shows the Cypher query that retrieves all context-aware similarity-confidence pairs and orders them by the non-weighted global preference $\succeq^{\text{basic}}_{\text{global}}$. The query returns the same properties as the previous one, but additionally computes the global preference value for each pair and returns it as a separate column, which is then used for ordering. The top ten rows of the output are displayed in the right part of Table 5.1.

```
MATCH (e1:Epigram {id: 17752})-[cs:CONTEXT_SIMILAR_TO]-(e2:Epigram)
RETURN e2.id, cs.similarity, cs.confidence,
    ↪ cs.similarity * cs.confidence AS globPref
ORDER BY globPref DESC
```

Listing 5.2: Cypher query to retrieve all context-aware similarity-confidence pairs between the epigram with identifier 17752 and the other epigrams, and order them by the non-weighted global preference $\succeq^{\text{basic}}_{\text{global}}$.

Both orderings agree on the top four candidates, where the similarity is perfect ($s = 1$) or very high ($s = 0.75$), paired with moderate to high confidence (between 0.667 and 0.833). This agreement is expected, since the lexicographic ordering prioritises similarity, and the global preference function, while combining both components, still assigns a high preference to candidates whose similarity is perfect or near-perfect even when confidence is not maximal. Moreover, each of these four candidates either equals or fully dominates the next in the ranking, as each has at least as high a similarity and at least as high a confidence as its successor, so that no reordering is possible regardless of which proposed technique is applied.

From position five onward, however, the two orderings diverge significantly. The lexicographic ordering ranks candidates with higher similarity but lower confidence above those with lower similarity but higher confidence. The global preference function, by multiplying both components, tends to favour candidates with a better

Table 5.1: Top ten context-aware similarity-confidence pairs between the reference epigram with identifier 17752 and the other epigrams. The results ordered by the lexicographic ordering $\succeq^{s}_{\text{lex}}$ are displayed in the left group of columns, while the results ordered by the non-weighted global preference $\succeq^{\text{basic}}_{\text{global}}$ are displayed in the right group. The rank column is added for reference and is not part of the query output. The similarity, confidence, and global preference values are rounded to three decimal places.

rank	$\succeq^{\mathbf{s}}_{\text{lex}}$ (Listing 5.1)			$\succeq^{\text{basic}}_{\text{global}}$ (Listing 5.2)			
	id	**similarity**	**confidence**	**id**	**similarity**	**confidence**	**globPref**
1	17754	1.000	0.833	17754	1.000	0.833	0.833
2	20718	1.000	0.667	20718	1.000	0.667	0.667
3	32893	1.000	0.667	32893	1.000	0.667	0.667
4	20915	0.750	0.667	20915	0.750	0.667	0.500
5	22372	0.699	0.500	19670	0.575	0.667	0.383
6	26328	0.698	0.500	21112	0.526	0.667	0.351
7	19670	0.575	0.667	26545	0.526	0.667	0.351
8	20021	0.553	0.500	29749	0.526	0.667	0.351
9	24646	0.553	0.500	34326	0.421	0.833	0.351
10	22027	0.548	0.333	22372	0.699	0.500	0.349

balance between similarity and confidence, even if their similarity alone is not the highest. This divergence is reflected in the composition of the remaining rows. Only two candidates, with identifiers 22372 and 19670, appear in both rankings after the top four, and they do so at different positions: 22372 drops from fifth under lexicographic ordering to tenth under global preference, while 19670 rises from seventh to fifth. All other rows contain entirely different epigrams. The contrast illustrates how the choice of ranking technique leads to different interpretations of the same L-grades, and how the proposed techniques enable researchers to explore the context-aware relatedness between epigrams under different trade-off preferences.

The distance measure introduced in Section 5.4.4 provides additional information that complements the rank order by quantifying how close two L-grades are in the unit square $\mathbb{L}$. Listing 5.3 shows a Cypher query that, given two reference epigrams and the L-grade between them, retrieves the similarity-confidence pairs between the first reference epigram and all other epigrams. For each of these pairs, the query computes the Euclidean distance to the reference L-grade and orders the results by increasing distance. This enables researchers who have identified an interesting pair of epigrams with particular similarity-confidence values to find other epigrams whose relationship with the reference epigram exhibits similar characteristics, even if they do not rank highly according to one of the preference relations. In this query, the first reference epigram has identifier 17752, the second has identifier 26545, and the L-grade between them is $(0.526, 0.667)$. The top ten rows of the output are shown in Table 5.2, where the rank column is added for reference and is not part of the query output.

```
MATCH (e1:Epigram {id: 17752})-[cs1:CONTEXT_SIMILAR_TO]-
    ↪ (e2:Epigram {id: 26545})
MATCH (e1)-[cs2:CONTEXT_SIMILAR_TO]-(e3:Epigram)
```

```
WHERE e3.id <> e2.id
WITH e3, cs2, sqrt((cs1.similarity - cs2.similarity)^2
  ↪ + (cs1.confidence - cs2.confidence)^2) AS distance
RETURN e3.id, cs2.similarity, cs2.confidence, distance
ORDER BY distance ASC
```

Listing 5.3: Cypher query to compute the Euclidean distance between the L-grade of a reference pair of epigrams and the L-grades of all other pairs involving the first epigram, ordered by increasing distance.

The results in Table 5.2 show that the epigrams with identifiers 21112 and 29749 have the same L-grade as the reference pair, yielding a distance of zero. The remaining candidates in the top ten all have the same confidence (0.667) and differ only in similarity. This pattern reflects the discrete nature of the confidence values in this particular dataset, where confidence is determined by the proportion of the six inputs, the textual similarity and the five metadata fields, that are available for a pair. The approximate equality comparator defined in Section 5.4.4 can be expressed in Cypher in a similar manner, by evaluating the membership function $\mu_{\approx}$ of Equation (5.26) on the computed distance for a chosen threshold δ. Listing 5.4 shows a Cypher query that extends the query of Listing 5.3 to compute this membership value, following Equation (5.26), for approximate equality with respect to the reference pair, using $\delta = 0.1$. The computed membership values are displayed in the last column of Table 5.2. Filtering the results of such a query on the membership grade would allow researchers to retrieve all epigrams whose similarity-confidence pair is approximately equal to a given reference, finding groups of similar L-grades rather than imposing a strict ordering among them.

```
MATCH (e1:Epigram {id: 17752})-[cs1:CONTEXT_SIMILAR_TO]-
  ↪ (e2:Epigram {id: 26545})
MATCH (e1)-[cs2:CONTEXT_SIMILAR_TO]-(e3:Epigram)
WHERE e3.id <> e2.id
WITH e3, cs2, sqrt((cs1.similarity - cs2.similarity)^2
  ↪ + (cs1.confidence - cs2.confidence)^2) AS distance
WITH e3, cs2, distance, CASE WHEN distance <= 0.1
  ↪ THEN (1 - distance/0.1) ELSE 0 END as membership
RETURN e3.id, cs2.similarity, cs2.confidence, distance, membership
ORDER BY distance ASC
```

Listing 5.4: Cypher query to compute the Euclidean distance between the L-grade of a reference pair of epigrams and the L-grades of all other pairs involving the first epigram, ordered by increasing distance. The membership values for approximate equality with respect to the reference pair are also computed, following Equation (5.26) with $\delta = 0.1$.

The four queries demonstrated in this subsection illustrate how the ranking techniques and the distance measure can be implemented directly within the query language of the graph database, without requiring external computation or post-processing. By changing a single `ORDER BY` clause, a researcher can switch between ranking by similarity, by the balance of similarity and confidence, or by proximity to a reference L-grade. Each perspective reveals different aspects of the relationships between epigrams. The resulting rankings can be employed in various ways, such as presenting the most relevant interconnections to researchers during analysis or prioritising them for subsequent computational processing. Thresholds on satisfaction and confidence, as introduced in Section 5.4.2, can be added to these queries through `WHERE` clauses to restrict the results to

Table 5.2: Top ten context-aware similarity-confidence pairs with the smallest Euclidean distance to the reference pair $(0.526, 0.667)$ between the epigrams with identifiers 17752 and 26545. The membership values are computed following Equation (5.26) with $\delta = 0.1$. The similarity, confidence, distance, and membership values are rounded to three decimal places.

rank	id	similarity	confidence	distance	membership
1	21112	0.526	0.667	0.000	1.000
2	29749	0.526	0.667	0.000	1.000
3	25355	0.523	0.667	0.003	0.966
4	19670	0.575	0.667	0.049	0.515
5	26050	0.450	0.667	0.077	0.234
6	22634	0.425	0.667	0.101	0.000
7	25362	0.411	0.667	0.116	0.000
8	19746	0.409	0.667	0.117	0.000
9	26591	0.409	0.667	0.118	0.000
10	19350	0.401	0.667	0.125	0.000

candidates meeting minimum requirements on both components, further tailoring the output to the researcher's needs.

The second use case, presented in the following subsection, demonstrates the ranking techniques in a different application setting, where L-grades arise not from similarity computation between all epigrams in a database, but from a query process that assigns L-grades to results.

5.5.2 Ranking Geographical Query Results

In the preceding subsection, L-grades arose from the context-aware similarity measure of Chapter 4, in which the satisfaction component reflects the degree of resemblance between two texts and the confidence component reflects the extent of available metadata. L-grades can equally arise from other data processing techniques in which both the suitability of a result and the reliability of that assessment are of interest. In this subsection, we demonstrate this through a second use case in which the DBBE corpus is queried for epigrams associated with a particular geographical location. Each retrieved epigram is assigned an L-grade based on the hierarchical relationship between the queried location and the location recorded for the epigram. The satisfaction component captures how closely the recorded location matches the query, while the confidence component reflects the reliability of that assessment given the granularity of the available geographical information. The proposed ranking techniques then order these L-grades, enabling researchers not only to identify the epigrams most closely associated with the queried location but also to explore results linked to broader or related geographical levels, with the trade-off between geographical suitability and reliability made explicit through the ranking.

Geographical Hierarchy

As described in Section 4.5.2, geographical metadata in the DBBE are organised into a hierarchy that ranges from broad geographical regions to specific locations. This hierarchy comprises five levels of granularity below the root: `country`, `part` (a major subdivision of a country), `region`, `city`, and `site` (a specific location within a city, such as a monastery). A virtual root node representing the entire world sits at depth 0, and the five named levels occupy depths 1 through 5. Each epigram in the corpus is associated with exactly one node in this hierarchy, namely the most specific node consistent with the geographical information recorded for it. For epigrams without any recorded geographical information, this is the root node, reflecting that they could have originated anywhere. The associated node corresponds to the place where the epigram was written or copied, which may differ from the location where the manuscript containing the epigram was produced or where the manuscript is currently held.

The granularity at which an epigram is associated with a geographical location varies across the corpus. Some branches of the hierarchy in the DBBE do not span all five levels: for instance, a `city` may be recorded directly under a `part` without a known `region` in between. In such cases, intermediate levels are filled by propagating the value of the closest known ancestor downward, following the same filling mechanism used for the geographical similarity computation in Chapter 4, so that every geographical location lies at the end of a complete path from the root. Figure 5.3 shows a fragment of this hierarchy containing the branches relevant to the illustrative example in this section.

The geographical hierarchy in the DBBE represents only the locations from which epigrams are known to have originated, rather than a comprehensive geographical hierarchy of the entire world. It is therefore incomplete both in breadth, as many geographical locations are not represented, and in depth, as some branches do not span all five levels. Despite this limited scope, the hierarchy captures the geographical hotspots of the corpus and is sufficient for querying and analysing the geographical distribution of epigrams. When new location data become available in the DBBE, the hierarchy can be extended by adding new nodes and edges, and the proposed ranking techniques continue to apply without modification. The limited breadth of the hierarchy should, however, be taken into account when defining the satisfaction function, since the number of effective endpoints visible in the hierarchy may underestimate the true number of possible locations at a given depth. The satisfaction function introduced in the next subsection therefore includes a parameter u that compensates for this underestimation.

L-grade Computation for Geographical Queries

When querying for epigrams associated with a specific geographical location q, each epigram in the corpus whose recorded location is p is assigned an L-grade $\ell = (s, c)$. The satisfaction component s reflects the degree to which the epigram's location matches the queried location, while the confidence component c quantifies the reliability of that assessment based on the hierarchical distance between q and p. We write $\mathrm{desc}(x)$ for the set of descendants of a node x in the hierarchy, $\mathrm{anc}(x)$ for the set of its ancestors, and $\mathrm{depth}(x)$ for the depth of x, with the depth of the root node equal to $\mathrm{depth}(\texttt{world}) = 0$.

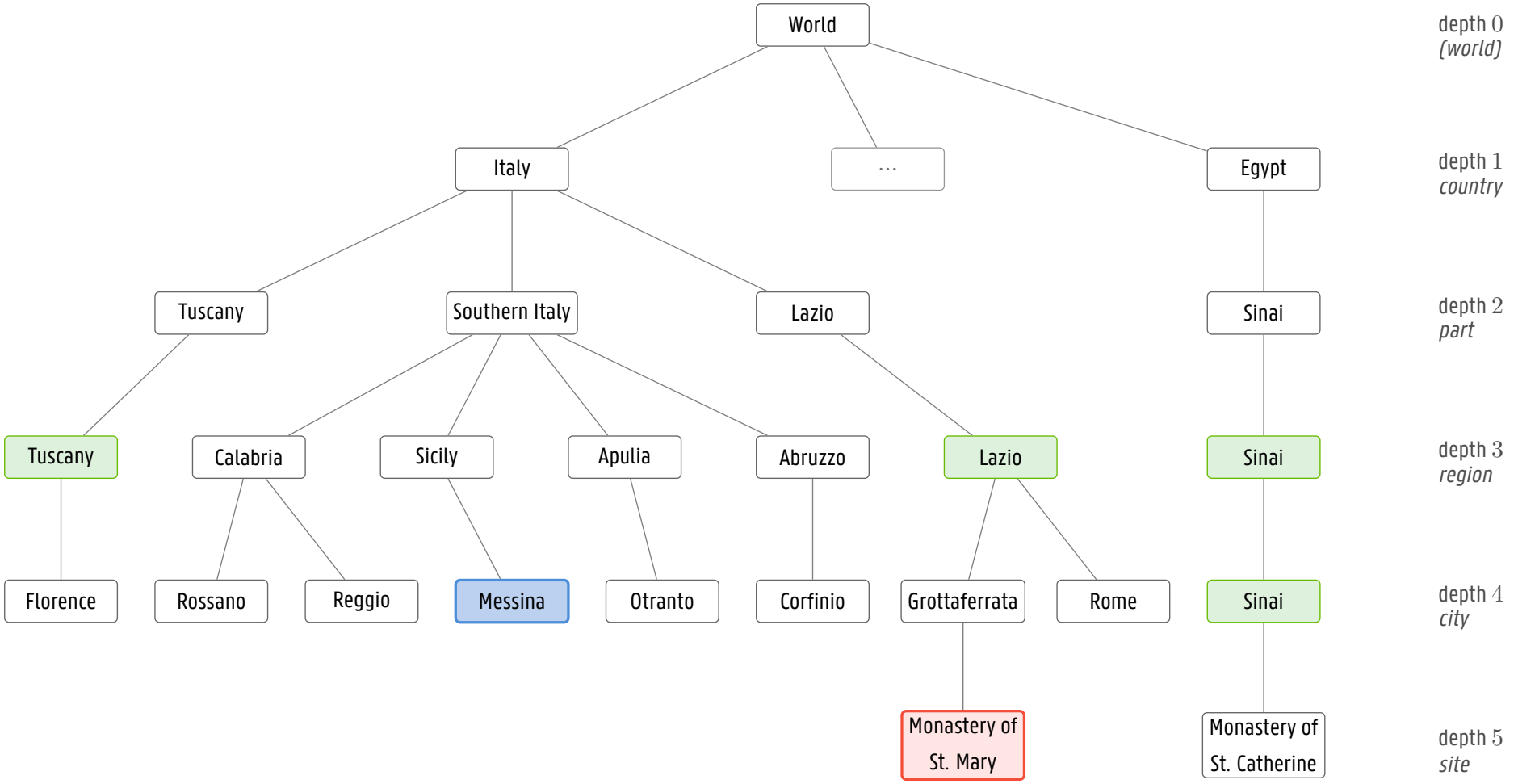


Figure 5.3: Fragment of the geographical hierarchy used in the DBBE, containing the branches relevant to the illustrative example. Each row corresponds to a depth level and its associated granularity, from the virtual world root at depth 0 to specific sites at depth 5. Nodes shown in green denote values inherited from the closest known ancestor when the corresponding granularity level is absent in the original hierarchy, following the filling mechanism described in Section 4.5.2. The queried site in Example 5.9, *Monastery of St. Mary*, is highlighted in red. The queried city in the illustrative example, *Messina*, is highlighted in blue.

To define the satisfaction when p is an ancestor of q, we introduce the notion of *effective endpoints* of a node p at a target depth $d = \text{depth}(q)$, denoted $\text{end}(p, d)$. This set is obtained by following each path from p towards depth d in the subtree rooted at p: paths that reach depth d contribute the node at that depth, while paths that terminate at a depth $d' < d$ contribute their deepest available node instead. This mechanism mirrors the propagation rule used for the geographical similarity computation in Section 4.5.2, ensuring that the satisfaction remains well-defined regardless of whether all paths in the hierarchy extend to the queried depth. Example 5.9 illustrates the concept of effective endpoints on the hierarchy fragment shown in Figure 5.3.

> **Example 5.9.** Consider the geographical hierarchy from Figure 5.3. Let q be the queried location *Monastery of St. Mary* at depth 5 and let p, an ancestor of q, be *Italy* at depth 1. The effective endpoints of p at depth $\text{depth}(q) = 5$ comprise all descendants of p at that depth together with the deepest available nodes along paths that do not reach it. Since only the path through *Grottaferrata* reaches depth 5 (at *Monastery of St. Mary*), while all other paths terminate at depth 4, the effective endpoints are $\text{end}(p, 5) = \{$*Monastery of St. Mary*, *Florence*, *Rossano*, *Reggio*, *Messina*, *Otranto*, *Corfinio*, *Rome*$\}$.

Using the effective endpoints and a parameter $u \geq 0$ that compensates for the limited breadth of the hierarchy, the satisfaction degree is defined by

$$s = \begin{cases} 1 & \text{if } p = q \ \vee \ p \in \text{desc}(q), \\ \dfrac{1}{|\text{end}(p,\ \text{depth}(q))| + u} & \text{if } p \in \text{anc}(q), \\ 0 & \text{otherwise.} \end{cases} \tag{5.27}$$

Three cases arise. When the epigram's location p coincides with the queried location q or lies below it in the hierarchy (i.e., p is a descendant of q), the geographical query is fully satisfied and $s = 1$. The latter indicating that the epigram is associated with a more specific location than the query, which is considered a perfect match. When p is an ancestor of q, the epigram is associated with a coarser level that contains q among its descendants, making the match only partial. In this case, the satisfaction is inversely proportional to the number of effective endpoints reachable from p at $\text{depth}(q)$, reflecting that the epigram could have originated at any of those locations with equal likelihood. Since p is an ancestor of q, the path from p to q reaches $\text{depth}(q)$, so q is itself an effective endpoint and the denominator is at least 1, which keeps the satisfaction well-defined. Because the hierarchy records only locations from which epigrams are known, the number of effective endpoints may underestimate the true number of possible locations at the target depth, and a positive value of u compensates for this by reducing the satisfaction for ancestors more aggressively. When $u = 0$, the satisfaction is based solely on the observed hierarchy. When p and q lie on entirely different branches of the hierarchy, there is no geographical overlap and $s = 0$. We consider only queries for locations that are recorded in the hierarchy, so that the satisfaction is always well-defined.

The confidence grade is defined by

$$c = \begin{cases} \dfrac{1}{2^{\mathsf{depth}(q)-\mathsf{depth}(p)}} & \text{if } p \in \mathsf{anc}(q), \\ 1 & \text{otherwise.} \end{cases} \tag{5.28}$$

The confidence captures the reliability of the satisfaction assessment. When p is an ancestor of q, the confidence decreases exponentially with the number of hierarchical levels separating the two, reflecting that a coarser recorded location provides progressively less certainty about where the epigram truly originated. The base of 2 ensures that the confidence halves with each additional level of separation, providing an increasingly strong penalty for distant ancestors. In all other cases, the confidence is full. When p equals or is a descendant of q, the geographical information fully supports the result, and when p and q lie on different branches, the conclusion that no geographical overlap exists is itself fully reliable.

Illustrative Examples

To illustrate the geographical querying technique and the subsequent application of the ranking approaches, we query the subset of 496 epigrams used in the experiments of Chapters 3 and 4 for those associated with the city of *Messina*, located at depth 4 in the hierarchy under *Sicily*, *Southern Italy*, and *Italy*, as shown in Figure 5.3. For this demonstration, the parameter u is set to 0, so that the satisfaction is based solely on the observed hierarchy. As reported in Table 4.2, fewer than one in six epigrams in this subset carry a recorded location, so most epigrams are associated with the root, or `world`, node. The L-grades computed using Equations (5.27) and (5.28) for each epigram in the dataset fall into six distinct groups, summarised in Table 5.3 together with the number of epigrams assigned to each.

Table 5.3: Distribution of epigrams across L-grade groups when querying for Messina (depth 4, granularity `city`) with $u = 0$. Each row corresponds to a distinct satisfaction-confidence pair that is assigned to at least one epigram in the dataset. The number of effective endpoints at depth 4 under each location p is indicated alongside the resulting L-grade components and the number of epigrams in that group. The six groups and their associated L-grades are denoted ℓ_1 through ℓ_6 for use in the subsequent ranking demonstrations.

Group	Location p	Relationship to q	$\lvert\mathrm{end}(p,4)\rvert$	s	c	Count
ℓ_1	*Messina*	$p = q$	—	1.0000	1.0000	2
ℓ_2	*Sicily*	$p \in \mathsf{anc}(q)$	1	1.0000	0.5000	1
ℓ_3	*Southern Italy*	$p \in \mathsf{anc}(q)$	5	0.2000	0.2500	2
ℓ_4	*Italy*	$p \in \mathsf{anc}(q)$	8	0.1250	0.1250	4
ℓ_5	*World*	$p \in \mathsf{anc}(q)$	26	0.0385	0.0625	424
ℓ_6	Other branches	$p \neq q \wedge p \notin \mathsf{anc}(q) \cup \mathsf{desc}(q)$	—	0.0000	1.0000	63

The two epigrams located at *Messina* receive the pair $(1, 1)$: the geographical query is fully satisfied with full confidence. The single epigram associated with *Sicily* also receives full satisfaction ($s = 1$), because *Messina* is the only `city`-level endpoint under *Sicily* in this hierarchy, making the denominator in Equation (5.27) equal to

1. Its confidence, however, is reduced to $c = 0.50$, reflecting the one-level separation between the recorded location p and the queried location q.

Moving further up the hierarchy, the two epigrams at the level of *Southern Italy* receive $s = 0.20$, reflecting the five `city`-level endpoints under *Southern Italy* (*Rossano*, *Reggio*, *Messina*, *Otranto*, and *Corfinio*), and $c = 0.25$ due to the two-level separation. At the level of *Italy*, the four epigrams receive $s = 0.125$ (eight `city`-level endpoints) and $c = 0.125$ (three-level separation). The 424 epigrams without geographical information, associated with the root, receive the lowest satisfaction ($s = \frac{1}{26} \approx 0.0385$, corresponding to twenty-six `city`-level endpoints in the full DBBE hierarchy) and the lowest confidence ($c = \frac{1}{16} = 0.0625$, reflecting a four-level separation), capturing both the limited match and the unreliability of an assessment at this distance. Finally, the 63 epigrams associated with known locations on branches unrelated to *Messina* receive $s = 0$ and $c = 1$, confirming with full confidence that they do not match the queried location.

Ranking the six groups by the lexicographic ordering $\succ^{c}_{\text{lex}}$ over their L-grades with confidence as the primary component yields

$$\ell_1 \succ^{c}_{\text{lex}} \ell_6 \succ^{c}_{\text{lex}} \ell_2 \succ^{c}_{\text{lex}} \ell_3 \succ^{c}_{\text{lex}} \ell_4 \succ^{c}_{\text{lex}} \ell_5.$$

Group $\ell_6 = (0, 1)$ ranks second, immediately after the ideal $\ell_1 = (1, 1)$, because its confidence is maximal. Under this ordering, the 63 epigrams known to be located on a different branch would be presented to the researcher ahead of the epigram in ℓ_2 ($s = 1, c = 0.5$) and the two epigrams in ℓ_3 ($s = 0.20$), even though those 63 epigrams are known not to match the queried location. This illustrates that the lexicographic ordering using confidence as the primary component may not immediately align with a researcher's expectations for relevance in a geographical retrieval task. An alternative interpretation is that this ordering presents the results with full confidence first, namely the confirmed match (ℓ_1) and the confirmed non-matches (ℓ_6), followed by results with decreasing confidence, allowing the researcher to explore the results in order of reliability.

Applying the weighted global preference function p_{weighted} with $w = 0.7$ (placing substantially more emphasis on confidence) yields the preference values $p_{\text{weighted}}(\ell_1) = 1.00$, $p_{\text{weighted}}(\ell_2) = 0.65$, $p_{\text{weighted}}(\ell_3) \approx 0.24$, $p_{\text{weighted}}(\ell_4) = 0.125$, $p_{\text{weighted}}(\ell_5) \approx 0.06$, and $p_{\text{weighted}}(\ell_6) = 0.70$, producing the ranking

$$\ell_1 \succ^{\text{weighted}}_{\text{global}} \ell_6 \succ^{\text{weighted}}_{\text{global}} \ell_2 \succ^{\text{weighted}}_{\text{global}} \ell_3 \succ^{\text{weighted}}_{\text{global}} \ell_4 \succ^{\text{weighted}}_{\text{global}} \ell_5.$$

The same issue arises: ℓ_6 again ranks second above ℓ_2, because the weighted combination allocates 70% of its weight to confidence, and ℓ_6 has full confidence despite zero satisfaction. The non-weighted global preference p_{basic} avoids this specific issue, since the product reduces any L-grade with a zero component to zero, giving $p_{\text{basic}}(\ell_6) = 0 \cdot 1 = 0$, but as noted in Section 5.4.3, this comes at the cost of losing the ability to distinguish between L-grades that both have a satisfaction or confidence of zero.

The threshold mechanism of Section 5.4.2 resolves this issue. Applying a satisfaction threshold $\tau_s > 0$ removes ℓ_6 from consideration, since its satisfaction is zero. In addition, a confidence threshold can be applied to further restrict the results. Setting $\tau_s = 0.10$ and $\tau_c = 0.10$, for instance, retains only ℓ_1, ℓ_2, ℓ_3, and ℓ_4, discarding

the 424 root-level epigrams (ℓ_5) alongside the 63 confirmed non-matches (ℓ_6). The weighted ranking among the remaining candidates,

$$\ell_1 \succ^{\text{weighted}}_{\text{global}} \ell_2 \succ^{\text{weighted}}_{\text{global}} \ell_3 \succ^{\text{weighted}}_{\text{global}} \ell_4,$$

is now free of potentially problematic orderings and presents the researcher with a focused set of candidates in decreasing order of their combined geographical satisfaction and reliability. Adjusting the thresholds allows stakeholders to control the breadth of the results: stricter thresholds yield a smaller but more reliable set, while more permissive thresholds include more uncertain candidates.

Together, the two use cases presented in this section demonstrate that the ranking techniques proposed in this chapter are applicable to L-grades arising from fundamentally different data processing contexts: context-aware similarity computation and veracity-aware geographical querying. In both cases, the availability of multiple ranking approaches, complemented by thresholds, a distance measure, and the approximate equality comparator, provides stakeholders with a flexible set of tools for exploring, filtering, and interpreting results according to their analytical needs. The choice of ranking approach is not prescribed by the data but by the research question: a researcher prioritising reliability will rank differently from one prioritising suitability, and the proposed techniques make this choice explicit and transparent. The contributions, limitations, and future directions arising from these techniques are discussed in the following section.

5.6 Conclusions and Future Work

This chapter proposed and studied ranking techniques for L-grades, ordered pairs of satisfaction and confidence grades that arise in data processing contexts where both the quality of a result and the trustworthiness of that assessment are of interest. By explicitly separating satisfaction from confidence, L-grades enable veracity-aware reasoning in settings where data are uncertain or incomplete [1]. Because the two components of an L-grade are orthogonal, no single natural ordering exists, and the choice of ranking strategy reflects a trade-off between satisfaction and confidence that depends on the preferences of the stakeholder or the requirements of the application. Three ranking approaches were proposed to resolve this trade-off. Lexicographic ordering ranks L-grades by prioritising one of the two components, selected by the stakeholder, and uses the other component to break ties, yielding a total order on $\mathbb{L}$. Non-weighted global preference combines both components conjunctively into a single preference value through their product, yielding a total preorder. Weighted global preference instead combines the two components through a weighted sum, in which a stakeholder-provided weight reflects the relative importance of confidence versus satisfaction, allowing the balance between the two components to be controlled continuously and also yielding a total preorder. These ranking approaches are complemented by a distance measure that quantifies the strength of a preference between two L-grades using Euclidean distance in $\mathbb{L}$, and by an approximate equality comparator that identifies L-grades sufficiently close to one another through a fuzzy membership function defined on this distance. In addition, thresholds on satisfaction and confidence can be applied prior to ranking to restrict the result set to candidates meeting minimum requirements on both components.

The principal contribution of this chapter is the systematic study of ranking techniques specifically designed for L-grades, including formal proofs that each proposed ordering is a total order (or a total preorder in the case of the global preference functions) and, outside a small number of identified boundary cases, consistent with the cases of full dominance that any reasonable ranking must respect. Unlike existing ranking methods for Z-numbers, which reduce the Z-number to a single crisp value or a simpler fuzzy number before ranking, the proposed techniques operate directly on the two-component structure of L-grades, thereby preserving the independent interpretability of satisfaction and confidence throughout the ranking process. By offering multiple ranking approaches rather than prescribing a single fixed ordering, the proposed techniques provide a flexible framework that accommodates a range of stakeholder preferences, while the explicit separation of both components ensures that the resulting rankings remain easy to communicate to users without expertise in the underlying formalism.

We demonstrated the practical applicability of the proposed techniques through two use cases grounded in the DBBE corpus of Byzantine book epigrams. In the first use case, the ranking techniques were applied to the similarity-confidence pairs produced by the context-aware similarity measure of Chapter 4. The results showed how the choice of ranking approach affects the ordering of similarity-confidence pairs between epigrams, with the ranking implemented directly within the Cypher query language of a Neo4j graph database. The second use case introduced a geographical querying technique based on L-grades, in which epigrams are retrieved based on their association with a queried location in a hierarchical geographical structure and assigned satisfaction and confidence grades determined by the hierarchical relationship between the queried and recorded locations. Both use cases therefore confirm that the proposed techniques apply to L-grades regardless of the data processing technique that produced the L-grades, and that the choice of ranking approach has a tangible impact on the presentation and interpretation of results.

The proposed techniques are subject to a number of limitations. First, the ranking approaches are defined for L-grades, in which both components of a Z-number are reduced to crisp singletons in the unit interval; extending the proposed orderings to more general Z-numbers, where both components are fuzzy numbers, would require adapting the ranking definitions accordingly. Second, the global preference functions, while yielding total preorders, are not total orders: distinct L-grades can receive the same preference value, resulting in ties that must be resolved by an additional criterion or accepted as equivalent. Third, the Euclidean distance treats both components equally, which may not align with the preferences of a stakeholder who considers one component more important than the other; a weighted distance measure could address this asymmetry. Finally, the use cases are demonstrated on a single corpus, and evaluating the ranking techniques in other application domains would strengthen the evidence for their generalisability.

These limitations, together with the contributions of this chapter, suggest several directions for future work. A first direction concerns the OWA aggregator for L-grades. As noted in Section 5.3.1, the current OWA aggregator sorts L-grades exclusively by decreasing satisfaction before applying the weight vector to both components [1]. The ranking techniques proposed in this chapter provide alternative total orderings that could serve as the sorting criterion, yielding a family of OWA-based aggregators parameterised by the chosen ranking approach. Studying the formal properties of these alternative aggregators and evaluating their behaviour in practical applications constitutes a promising avenue for further investigation. A second direction is the development of a

weighted distance measure that incorporates the relative importance of satisfaction and confidence, analogously to the weighted global preference function, so that the strength of a preference can be assessed in a manner consistent with the stakeholder's priorities. A third direction is the extension to more general Z-numbers noted above, which would broaden the applicability of the proposed techniques beyond the L-grade setting to the wider family of Z-number-based frameworks.

This chapter and those preceding it have addressed the representation, similarity computation, context-aware fusion, and ranking of short texts and their associated satisfaction-confidence pairs. The remaining analytical objective, the clustering of short texts into meaningful groups based on their context-aware and responsible similarity and confidence scores, is explored in Chapter 6.

References

[1] Guy De Tré, Milan Peelman and Jozo Dujmović. "Logic Reasoning under Data Veracity Concerns". In: *International Journal of Approximate Reasoning* 161 (2023), p. 108977. issn: 0888-613X. DOI: https://doi.org/10.1016/j.ijar.2023.108977.

[2] Jinchuan Chen, Yueguo Chen, Xiaoyong Du, Cuiping Li, Jiaheng Lu, Suyun Zhao and Xuan Zhou. "Big Data Challenge: A Data Management Perspective". In: *Frontiers of Computer Science* 7.2 (2013), pp. 157–164. DOI: https://doi.org/10.1007/s11704-013-3903-7.

[3] Tatiana Lukoianova and Victoria L. Rubin. "Veracity Roadmap: Is Big Data Objective, Truthful and Credible?" In: *Proceedings of the 25th Annual Conference of the North American Society for Knowledge Organization (NASKO)*. Vol. 24. 1. 2014, pp. 4–15. DOI: https://doi.org/10.7152/acro.v24i1.14671.

[4] Diego De Siqueira Braga, Marco Niemann, Bernd Hellingrath and Fernando Buarque De Lima Neto. "Survey on Computational Trust and Reputation Models". In: *ACM Comput. Surv.* 51.5 (2018). DOI: https://doi.org/10.1145/3236008.

[5] Rob Kitchin. "Big Data, New Epistemologies and Paradigm Shifts". In: *Big Data & Society* 1.1 (2014). DOI: https://doi.org/10.1177/2053951714528481.

[6] Min Chen, Shiwen Mao and Yunhao Liu. "Big Data: A Survey". In: *Mobile Networks and Applications* 19.2 (2014), pp. 171–209. DOI: https://doi.org/10.1007/s11036-013-0489-0.

[7] Vladik Kreinovich and Rujira Ouncharoen. "Fuzzy (and Interval) Techniques in the Age of Big Data: An Overview with Applications to Environmental Science, Geosciences, Engineering, and Medicine". In: *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems* 23.Suppl. 1 (2015), pp. 75–89. DOI: https://doi.org/10.1142/S0218488515400061.

[8] Troels Andreasen, Gloria Bordogna, Guy De Tré, Janusz Kacprzyk, Henrik Legind Larsen and Sławomir Zadrożny. "The Power and Potentials of Flexible Query Answering Systems: A Critical and Comprehensive Analysis". In: *Data & Knowledge Engineering* 149 (2024), p. 102246. issn: 0169-023X. DOI: https://doi.org/10.1016/j.datak.2023.102246.

[9] Gustavo Grander, Luciano Ferreira da Silva and Ernesto Del Rosário Santibañez Gonzalez. "Big Data as a Value Generator in Decision Support Systems: A Literature Review". In: *Revista de Gestão* 28.3 (2021), pp. 205–222. issn: 1809–2276. DOI: https://doi.org/10.1108/REGE-03-2020-0014.

[10] Ching-Lai Hwang and Kwangsun Yoon. "Multiple Attribute Decision Making: Methods and Applications". Springer, 1981. DOI: https://doi.org/10.1007/978-3-642-48318-9.

[11] Christopher D Manning. "Introduction to Information Retrieval". Syngress Publishing, 2008. DOI: https://doi.org/10.1017/CBO9780511809071.

[12] Tie-Yan Liu. "Learning to Rank for Information Retrieval". In: *Foundations and Trends in Information Retrieval* 3.3 (June 2009), pp. 225–331. issn: 1554-0669. DOI: https://doi.org/10.1561/1500000016.

[13] Peter C. Fishburn. "Lexicographic Orders, Utilities and Decision Rules: A Survey". In: *Management Science* 20.11 (1974), pp. 1442–1471. DOI: https://doi.org/10.1287/mnsc.20.11.1442.

[14] Emrullah Şahin, Naciye Nur Arslan and Durmuş Özdemir. "Unlocking the Black Box: An in-Depth Review on Interpretability, Explainability, and Reliability in Deep Learning". In: *Neural Computing and Applications* 37.2 (2025), pp. 859–965. issn: 1433-3058. DOI: https://doi.org/10.1007/s00521-024-10437-2.

[15] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador Garcia, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, Raja Chatila and Francisco Herrera. "Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges Toward Responsible AI". In: *Information Fusion* 58 (2020), pp. 82–115. issn: 1566-2535. DOI: https://doi.org/10.1016/j.inffus.2019.12.012.

[16] Tim Miller. "Explanation in Artificial Intelligence: Insights from the Social Sciences". In: *Artificial Intelligence* 267 (2019), pp. 1–38. issn: 0004-3702. DOI: https://doi.org/10.1016/j.artint.2018.07.007.

[17] Lotfi A. Zadeh. "A Note on Z-Numbers". In: *Information Sciences* 181.14 (2011), pp. 2923–2932. issn: 0020-0255. DOI: https://doi.org/10.1016/j.ins.2011.02.022.

[18] Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, Maxime Deforche, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout and Colin Swaelens. "The Database of Byzantine Book Epigrams Project : Principles, Challenges, Opportunities". eng. In: *Journal of Data Mining and Digital Humanities* On the Way to the Future of Digital Manuscript Studies (2023), p. 41. issn: 2416-5999. DOI: https://doi.org/10.46298/jdmdh.10244.

[19] Kristoffel Demoen, Gilbert Bentein, Klaas Bentein, Floris Bernard, Julián Bértola, Julie Boeten, Mathijs Clement, Cristina Cocola, Eline Daveloose, Sien De Groot, Pieterjan De Potter, Ilse De Vos, Krystina Kubina, Hanne Lauwers, Paulien Lemay, Renaat Meesters, Marjolein Morbé, Delphine Nachtergaele, Marthe Nemegeer, Joachim Nielandt, Mace Ojala, Lisa-Lou Péchillon, Raf Praet, Rachele Ricceri, Anne-Sophie Rouckhout, Jeroen Schepens, Febe Schollaert, Lev Shadrin, Nina Sietis, Dimitrios Skrekas, Colin Swaelens, Maria Tomadaki, Sarah-Helena Van den Brande, Merel Van Nieuwerburgh, Lotte Van Olmen, Noor Vanhoe and Nina Vanhoutte. *Database of Byzantine Book Epigrams (DBBE)*. Version 1.0.0. Zenodo, Feb. 2023. DOI: https://doi.org/10.5281/zenodo.7682523.

[20] L.A. Zadeh. "The Concept of a Linguistic Variable and Its Application to Approximate Reasoning—I". In: *Information Sciences* 8.3 (1975), pp. 199–249. issn: 0020-0255. DOI: https://doi.org/10.1016/0020-0255(75)90036-5.

[21] Jerry M. Mendel. "Type-2 Fuzzy Sets and Systems: An Overview". In: *IEEE Computational Intelligence Magazine* 2.1 (2007), pp. 20–29. DOI: https://doi.org/10.1109/MCI.2007.380672.

[22] J.M. Mendel and R.I.B. John. "Type-2 Fuzzy Sets Made Simple". In: *IEEE Transactions on Fuzzy Systems* 10.2 (2002), pp. 117–127. DOI: https://doi.org/10.1109/91.995115.

[23] Jerry M. Mendel, Robert I. John and Feilong Liu. "Interval Type-2 Fuzzy Logic Systems Made Simple". In: *IEEE Transactions on Fuzzy Systems* 14.6 (2006), pp. 808–821. DOI: https://doi.org/10.1109/TFUZZ.2006.879986.

[24] Qilian Liang and J.M. Mendel. "Interval Type-2 Fuzzy Logic Systems: Theory and Design". In: *IEEE Transactions on Fuzzy Systems* 8.5 (2000), pp. 535–550. DOI: https://doi.org/10.1109/91.873577.

[25] Krassimir T. Atanassov. "Intuitionistic Fuzzy Sets". In: *Intuitionistic Fuzzy Sets: Theory and Applications*. Heidelberg: Physica-Verlag HD, 1999, pp. 1–137. isbn: 978-3-7908-1870-3. DOI: https://doi.org/10.1007/978-3-7908-1870-3_1.

[26] Ronald R. Yager. "Pythagorean Membership Grades in Multicriteria Decision Making". In: *IEEE Transactions on Fuzzy Systems* 22.4 (2014), pp. 958–965. DOI: https://doi.org/10.1109/TFUZZ.2013.2278989.

[27] Hamidreza Seiti, Ashkan Hafezalkotob and Luis Martínez. "R-Sets, Comprehensive Fuzzy Sets Risk Modeling for Risk-Based Information Fusion and Decision-Making". In: *IEEE Transactions on Fuzzy Systems* 29.2 (2021), pp. 385–399. DOI: https://doi.org/10.1109/TFUZZ.2019.2955061.

[28] Robin De Mol, Antoon Bronselaer and Guy De Tré. "Evaluating flexible criteria on uncertain data". In: *Fuzzy Sets and Systems* 328 (2017), pp. 122–140. DOI: https://doi.org/10.1016/j.fss.2017.07.012.

[29] Robin De Mol and Guy De Tré. "Applying Suitability Distributions in a Geological Context". In: *Information Processing and Management of Uncertainty in Knowledge-Based Systems. Theory and Foundations*. Vol. 853. Communications in Computer and Information Science. Cham: Springer, 2018, pp. 278–288. DOI: https://doi.org/10.1007/978-3-319-91473-2_24.

[30] Glenn Shafer. "A Mathematical Theory of Evidence". Princeton University Press, 1976. isbn: 9780691100425. DOI: https://doi.org/10.1515/9780691214696.

[31] Arthur P. Dempster. "Upper and Lower Probabilities Induced by a Multivalued Mapping". In: *Classic Works of the Dempster-Shafer Theory of Belief Functions*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 57–72. isbn: 978-3-540-44792-4. DOI: https://doi.org/10.1007/978-3-540-44792-4_3.

[32] Peter Walley. "Statistical Reasoning with Imprecise Probabilities". Chapman and Hall, 1991.

[33] Thomas Augustin, Frank P. A. Coolen, Gert De Cooman and Matthias C. M. Troffaes. "Introduction to Imprecise Probabilities". John Wiley & Sons, 2014. DOI: https://doi.org/10.1002/9781118763117.

[34] R. A. Aliev and Vladik Kreinovich. "Z-Numbers and Type-2 Fuzzy Sets: A Representation Result". In: *Intelligent Automation & Soft Computing* 0.0 (2017), pp. 1–5. DOI: https://doi.org/10.1080/10798587.2017.1330310.

[35] Vladik Kreinovich, Olga Kosheleva and Michael Zakharevich. "Z-Numbers: How They Describe Student Confidence and How They Can Explain (and Improve) Laplacian and Schroedinger Eigenmap Dimension Reduction in Data Analysis". In: *Fuzzy Approaches for Soft Computing and Approximate Reasoning:*

Theories and Applications: Dedicated to Bernadette Bouchon-Meunier. Cham: Springer International Publishing, 2021, pp. 285–297. isbn: 978-3-030-54341-9. DOI: https://doi.org/10.1007/978-3-030-54341-9_24.

[36] Thanh-Lam Nguyen. "Methods in Ranking Fuzzy Numbers: A Unified Index and Comparative Reviews". In: *Complexity* 2017.1 (2017), p. 3083745. DOI: https://doi.org/10.1155/2017/3083745.

[37] Ahmad Syafadhli Abu Bakar and Alexander Gegov. "Multi-Layer Decision Methodology for Ranking Z-Numbers". In: *International Journal of Computational Intelligence Systems* 8.2 (2015), pp. 395–406. DOI: https://doi.org/10.1080/18756891.2015.1017371.

[38] Somayeh Ezadi and Tofigh Allahviranloo. "New Multi-Layer Method for Z-Number Ranking Using Hyperbolic Tangent Function and Convex Combination". In: *Intelligent Automation & Soft Computing* 0.0 (2017), pp. 1–7. DOI: https://doi.org/10.1080/10798587.2017.1367146.

[39] Gerard Salton and Christopher Buckley. "Term-Weighting Approaches in Automatic Text Retrieval". In: *Information Processing & Management* 24.5 (1988), pp. 513–523. issn: 0306-4573. DOI: https://doi.org/https://doi.org/10.1016/0306-4573(88)90021-0.

[40] Stephen Robertson and Hugo Zaragoza. "The Probabilistic Relevance Framework: BM25 and Beyond". In: *Foundations and Trends in Information Retrieval* 4.1-2 (Sept. 2009), pp. 1–174. issn: 1554-0669. DOI: https://doi.org/10.1561/1500000019.

[41] Bernard Roy. "The Outranking Approach and the Foundations of Electre Methods". In: *Theory and Decision* 31.1 (1991), pp. 49–73. issn: 1573-7187. DOI: https://doi.org/10.1007/BF00134132.

[42] J. P. Brans and Ph. Vincke. "A Preference Ranking Organisation Method". In: *Management Science* 31.6 (1985), pp. 647–656. DOI: https://doi.org/10.1287/mnsc.31.6.647.

[43] Hamed Taherdoost and Mitra Madanchian. "A Comprehensive Survey and Literature Review on TOPSIS". In: *International Journal of Service Science, Management, Engineering, and Technology* 15.1 (2024). issn: 1947-959X. DOI: https://doi.org/10.4018/IJSSMET.347947.

[44] Matthias Ehrgott. "Multicriteria Optimization". Springer Science & Business Media, 2005. DOI: https://doi.org/10.1007/3-540-27659-9.

[45] Xuzhu Wang and Etienne E. Kerre. "Reasonable Properties for the Ordering of Fuzzy Quantities (I)". In: *Fuzzy Sets and Systems* 118.3 (2001), pp. 375–385. issn: 0165-0114. DOI: https://doi.org/https://doi.org/10.1016/S0165-0114(99)00062-7.

[46] Matteo Brunelli and József Mezei. "How Different Are Ranking Methods for Fuzzy Numbers? A Numerical Study". In: *International Journal of Approximate Reasoning* 54.5 (2013), pp. 627–639. issn: 0888-613X. DOI: https://doi.org/https://doi.org/10.1016/j.ijar.2013.01.009.

[47] Zeshui Xu. "Intuitionistic Fuzzy Aggregation Operators". In: *IEEE Transactions on Fuzzy Systems* 15.6 (2007), pp. 1179–1187. DOI: https://doi.org/10.1109/TFUZZ.2006.890678.

[48] Henri Prade. "Possibility sets, fuzzy sets and their relation to Łukasiewicz logic". In: *Proceedings of the 12th International Symposium on Multiple-Valued Logic*. 1982, pp. 223–227.

[49] Gert De Cooman. "Towards a Possibilistic Logic". In: *Fuzzy Set Theory and Advanced Mathematical Applications*. Ed. by Da Ruan. International Series in Intelligent Technologies. Dordrecht, The Netherlands: Kluwer Academic Publishers, 1995, pp. 89–133. DOI: https://doi.org/10.1007/978-1-4615-2201-0_5.

[50] Guy De Tré. "Extended possibilistic truth values". In: *International Journal of Intelligent Systems* 17.4 (2002), pp. 427–446.

[51] George J. Klir and Bo Yuan. "Fuzzy Sets and Fuzzy Logic: Theory and Applications". Kluwer Academic Publishers, 1995.

[52] Ronald R. Yager. "On Ordered Weighted Averaging Aggregation Operators in Multicriteria Decisionmaking". In: *IEEE Transactions on Systems, Man, and Cybernetics* 18.1 (1988), pp. 183–190. issn: 2168-2909. DOI: https://doi.org/10.1109/21.87068.

[53] Brian A Davey and Hilary A Priestley. "Introduction to Lattices and Order". Cambridge university press, 2002. DOI: https://doi.org/10.1017/CBO9780511809088.

[54] Neo4j. "Neo4j Graph Database". url: https://neo4j.com/ (visited on 27/03/2026).

[55] Neo4j. "Cypher Query Language". url: https://neo4j.com/docs/cypher-manual/current/ (visited on 01/04/2026).

6

Context-Aware and Responsible Short Text Clustering

"You shall know a word by the company it keeps."

— **John R. Firth**

This chapter presents techniques for clustering short texts based on the context-aware similarity measure developed in Chapter 4. This measure fuses textual and metadata-based similarities through an adapted Ordered Weighted Averaging (OWA) fusion mechanism and pairs each aggregated score with a confidence score that reflects its reliability given the extent of missing metadata. Three clustering approaches are applied to the resulting similarity scores, namely K-Medoids, Hierarchical Agglomerative Clustering (HAC), and Fuzzy C-Medoids (FCMdd). For each approach, three strategies for integrating confidence into the clustering process and outcome are explored: a post-hoc visualisation that colours each text according to the confidence of its cluster assignment, a non-weighted combination that multiplies similarity and confidence before clustering, and a weighted combination that balances the two components through an adjustable weight parameter. The latter two strategies are inspired by the global preference techniques proposed in Chapter 5, while the first preserves the orthogonality of the similarity and confidence grades. Together, the three clustering approaches and the confidence integration strategies provide domain experts with a range of complementary tools for organising short texts into meaningful groups while making the reliability of each cluster assignment explicit.

The development and evaluation of these clustering techniques address the following research question:

> **RQ 4.C:** How can short texts be clustered based on context-aware similarity scores and confidence scores, in a way that handles missing metadata gracefully and makes the reliability of cluster assignments explicit?

The content of this chapter is based on parts of the following publication:

- **Maxime Deforche**, Ilse De Vos, and Guy De Tré. *A Novel Approach to Context-Aware and Responsible Short Text Clustering.* Published in Proceedings of the 16th International Conference on Flexible Query Answering Systems (FQAS), Lecture Notes in Computer Science, vol 16119, 2025.
  DOI: 10.1007/978-3-032-05607-8_4

The remainder of this sixth chapter is structured as follows. Section 6.1 situates the clustering of short texts within the broader landscape of text mining and motivates the need for clustering techniques that operate on context-aware similarity scores and incorporate confidence information. Related work on short text clustering, metadata-enhanced clustering, and the incorporation of uncertainty or confidence into the clustering process is discussed in Section 6.2. Section 6.3 provides the necessary preliminaries on the three clustering algorithms employed in this chapter, including their mathematical foundations and the determination of their parameters. The proposed clustering method, which operates on the aggregated similarity scores, and the three strategies for integrating confidence are presented in Section 6.4. Section 6.5 reports on the experimental evaluation using the Database of Byzantine Book Epigrams (DBBE) dataset, comparing the three clustering approaches under different similarity inputs and confidence integration strategies against a baseline that relies only on textual similarity. This section also presents illustrative examples that demonstrate the practical implications of incorporating context-aware similarity and confidence into the clustering process, along with the insights that the different approaches and strategies can offer. Finally, Section 6.6 summarises the contributions of this chapter, discusses limitations, and outlines directions for future work.

6.1 Introduction

Clustering is one of the most widely used unsupervised techniques in data mining and Machine Learning (ML), aiming to partition a collection of objects into groups such that objects within the same group are more similar to one another than to those in other groups [1, 2]. Because clustering does not require labelled training data, it is particularly valuable for the exploratory analysis of datasets where the goal is to discover meaningful patterns rather than to reproduce a known classification. In information retrieval, for instance, clustering is commonly applied to organise search results, discover thematic groups, and detect near-duplicate or related documents [3, 4]. When the objects under consideration are texts, the clustering process typically operates on a similarity or distance measure computed over the textual content, whether through surface-level measures such as edit distances [5, 6] or through the comparison of vector representations obtained from embedding techniques [7, 8]. The quality of the resulting clusters depends directly on how well the chosen measure captures the relationships between texts [1, 9, 10]. Where the underlying measure incorporates not only textual content but also contextual information, clustering can surface connections that purely textual approaches would miss, making it a valuable tool for the exploration of text corpora.

Clustering short texts, however, presents particular challenges, especially when Natural Language Processing (NLP) techniques are used to compute similarities between texts. The limited length of such texts results in fewer discriminative features and sparser representations, which diminishes the effectiveness of traditional representation methods such as Term Frequency-Inverse Document Frequency (TF-IDF) [11] and Bag of Words (BoW) [12], and consequently the quality of the similarities computed from them [13–15]. These difficulties are

even more pronounced when the texts are written in a low-resource language, for which pre-trained models, annotated corpora, and linguistic tools are scarce or unavailable [16, 17]. Orthographic similarity measures are therefore a natural starting point for clustering short texts in low-resource languages [6]. A potential approach to increasing the discriminative power of such measures is to incorporate contextual metadata, such as authorship, location, or genre [18]. In many real-world corpora, such metadata are only partially available, and traditional approaches either discard the affected fields entirely for all records, even those for which values are present, or impute the missing values under assumptions that may not hold in practice [19, 20]. In Chapter 4, we addressed this challenge by proposing a context-aware similarity measure that fuses textual and metadata-based similarities through an adapted OWA mechanism, producing for each pair of texts an aggregated similarity score alongside a confidence score that reflects the reliability of the aggregation given the extent of missing metadata. Traditional clustering algorithms, however, operate on a single similarity or dissimilarity measure and treat every entry as equally reliable. They do not provide a mechanism to incorporate the confidence scores into the clustering process or to make the reliability of cluster assignments transparent to domain experts.

In this chapter, we address this need by investigating clustering techniques that operate on the context-aware similarity scores from Chapter 4 and that additionally either make the confidence apparent in the clustering outcome or integrate the accompanying confidence scores into the clustering process. We consider three clustering approaches, each representing a different paradigm: *K-Medoids* [21], a technique that selects actual data points as cluster centres and assigns all other points to the nearest cluster centre; *Hierarchical Agglomerative Clustering (HAC)* [22], which builds a hierarchy of nested partitions through successive merging; and *Fuzzy C-Medoids (FCMdd)* [23], a fuzzy version of K-Medoids that assigns each object a degree of membership to every cluster centre rather than a single hard assignment to one medoid. Furthermore, we explore three strategies for integrating the confidence score into the clustering process or its interpretation. The first strategy is a post-hoc visualisation in which each text is coloured according to the confidence of its cluster assignment, providing an immediate visual indication of the reliability of each individual assignment or of an entire cluster without altering the clustering itself. The second strategy multiplies the similarity and confidence scores prior to clustering, producing a single combined input in which low confidence directly reduces the effective similarity, following the non-weighted global preference function proposed in Chapter 5. The third strategy replaces this combination by a linear combination in which a weight parameter controls the relative importance of similarity and confidence, inspired by the weighted global preference function proposed in Chapter 5. In addition to the confidence integration strategies, all three clustering approaches are applied to both the purely textual similarity from Chapter 3 and the context-aware similarity from Chapter 4, enabling a direct assessment of the added value of incorporating contextual metadata into the clustering input.

The novelty of this chapter is twofold. The first contribution lies in the application of multiple clustering paradigms to context-aware similarity scores produced by a measure specifically designed to handle missing metadata through an adapted aggregation mechanism, thereby enabling the clustering of short texts in settings where textual content alone provides limited discriminative power and where contextual metadata are incomplete. By operating on the aggregated scores produced by the OWA-based fusion mechanism of Chapter 4, the clustering approaches inherit the ability to incorporate all available metadata without requiring either

imputation, which introduces assumptions about the missing data, or marginalisation, which discards available information entirely. The second contribution concerns the integration of confidence into the clustering process and its interpretation. Rather than relying only on the similarity scores, the proposed strategies explicitly integrate the confidence score, either by providing an interpretable layer of reliability information on top of the clustering result or by adjusting the similarity input to the clustering algorithm. This integration allows domain experts not only to obtain clusters, but also to assess which assignments rest on complete contextual evidence and which are based on limited metadata, supporting more informed analysis and downstream decision-making.

We evaluate the proposed clustering approaches and confidence integration strategies through experiments on the DBBE corpus [24, 25], a collection of short poems written in a low-resource language and characterised by a high degree of missing or incomplete metadata. As a baseline, clustering is first performed on the purely textual similarity scores from Chapter 3, after which the context-aware similarity scores from Chapter 4 are used to measure the impact of incorporating metadata. The non-weighted and weighted confidence integration strategies are then applied to examine how confidence affects the clustering outcome. To assess clustering quality, the resulting clusters are compared against the expert-curated epigram types using precision, recall, and F_1-score, as well as the clustering-specific quality measures Adjusted Rand Index (ARI) and Adjusted Mutual Information (AMI), complemented by the silhouette coefficient, which assesses cohesion and separation against the dissimilarity structure alone and therefore requires no ground truth. Alongside the quantitative evaluation, illustrative examples highlight the distinctive properties of each clustering approach, including the fuzzy membership degrees produced by FCMdd, the hierarchical structure of HAC, and the crisp partitions of K-Medoids, as well as the confidence-based visualisation that reveals the reliability of individual cluster assignments.

6.2 Related Work

The approaches proposed in this chapter draw on three areas of prior research: methods for clustering short texts, particularly in settings where linguistic resources are limited; strategies that incorporate additional information, such as metadata or background knowledge, into the clustering process; and techniques that account for uncertainty or confidence in cluster assignments and in the underlying input data. This section reviews the work most directly relevant to these three themes.

The problem of clustering short texts has received considerable attention in the text mining and NLP communities, driven by the rapid growth of brief user-generated content on social media, messaging platforms, and other digital channels [1, 13, 15]. The central difficulty lies in the sparsity of short text representations: traditional representation methods such as TF-IDF [11] and BoW [12] produce high-dimensional vectors in which most entries are zero. This affects texts of any length, but where two long documents on the same subject still overlap in a substantial number of terms, two short texts often share none at all, which leaves distance- or similarity-based clustering algorithms with little basis for distinguishing meaningful groups [15]. Several lines of research have tried to address this sparsity. Topic-modelling approaches adapted for short documents include the Biterm Topic Model (BTM) [26], which captures word co-occurrence patterns across a corpus rather than within individual documents, and the Gibbs Sampling Dirichlet Multinomial Mixture Model (GSDMMM) [27], which

assigns each short text to a single topic to avoid sparse per-document topic distributions. The latter approach has also been adapted to online settings [28], and has been compared with other mixture-model approaches for short text [29]. Embedding-based methods, such as BERT [8] and Sentence-BERT [30], represent texts in a continuous space on which standard clustering algorithms can operate, and have substantially improved short text clustering performance [14, 31]. Deep Learning (DL) methods take a more integrated approach by jointly learning the text representation and the cluster assignments, for instance through self-taught convolutional architectures [32], contrastive-learning-based approaches [33], or autoencoder-based frameworks that optimise a clustering objective alongside the reconstruction loss [34, 35]. Despite these advances, the methods described above share two notable limitations from the perspective of this chapter. First, nearly all of them rely on learned representations that require large training corpora or pre-trained models in the target language, making them difficult to apply to texts in low-resource languages for which such tools are unavailable [16, 17]. Second, they operate exclusively on the textual content of the documents, disregarding any contextual metadata that may accompany the texts.

When such contextual information is available, several strategies have been proposed to integrate it into the clustering process. In constrained or semi-supervised clustering, prior knowledge derived from metadata can be expressed as pairwise constraints indicating that two objects should (*must-link*) or should not (*cannot-link*) belong to the same cluster [36, 37]. Such constraints are typically derived from labelled examples or domain expertise and are incorporated either by modifying the objective function that the clustering algorithm optimises or by learning an adapted distance metric [36, 38, 39]. These constraints are, however, typically binary: two objects either must or must not be co-clustered, with no way to express a graded relationship or to quantify the reliability of the constraint itself. A related technique is multi-view clustering, in which each object is described by multiple feature sets or similarity matrices and the goal is to find a partition that is consistent across all views [40, 41]. Multi-view methods combine views through co-training, co-regularisation, multi-kernel learning, or shared latent representations, and have been applied to a wide range of domains [40]. Metadata can also be treated as additional features alongside the textual content, either by concatenating metadata feature vectors with textual embeddings [42] or by learning a joint representation that captures both modalities, typically using DL techniques [43]. While constrained and multi-view clustering address the general problem of incorporating multiple sources of information, they typically assume either that the auxiliary information takes the form of binary constraints or that all views are available for all objects. Neither assumption holds in the setting of this chapter, where the metadata, and therefore the metadata-based similarity scores, are frequently incomplete or unavailable. In this thesis, contextual information is integrated at the level of the similarity measure itself, through the OWA-based fusion mechanism of Chapter 4, so that the clustering methods operate on precomputed similarity scores that already incorporate both textual and contextual information. This design makes the approach independent of the specific similarity measures used and allows any clustering algorithm that accepts similarity or dissimilarity scores as input to benefit from the contextual enrichment without modification.

A third relevant area of research concerns the modelling of uncertainty or confidence in the clustering process. The most prominent family of approaches is fuzzy or soft clustering, which replaces hard assignments with membership degrees that reflect the degree of association of each object with each cluster [44]. A high membership value indicates a strong association with a cluster, while a low value reflects a weak one. The Fuzzy

C-Means (FCM) algorithm [44] is the fuzzy variant of K-Means, in which the membership degrees for each data point are constrained to sum to one and the objective function minimises a weighted sum of distances to cluster centres. For settings in which only precomputed dissimilarities are available, the FCMdd algorithm adapts this paradigm by selecting actual data objects as cluster representatives and operating directly on dissimilarity scores [23, 45]. Possibilistic C-Means (PCM) [46, 47] relaxes the normalisation constraint of FCM, allowing each membership to reflect the absolute compatibility of an object with a cluster independently of its compatibility with other clusters, so that an outlier can receive low membership in all clusters simultaneously. This distinction is conceptually important: in possibilistic clustering, low membership values signal that an object does not fit well into any cluster, whereas in fuzzy clustering a low membership in one cluster is compensated by a higher membership in another. Evidential clustering extends the modelling of uncertainty further by employing belief functions from Dempster-Shafer theory [48] to represent partial knowledge about cluster assignments. The Evidential C-Means (ECM) algorithm [49] assigns a mass function over all subsets of clusters to each object, capturing ambiguity between clusters. These soft clustering paradigms address the question of how confidently an object can be assigned to a cluster, but they derive this confidence from the geometric structure of the data, specifically from the pairwise dissimilarities between objects and cluster centres, rather than from an external measure of data quality or reliability.

Closely related is the clustering of data with missing values, where certain features are unavailable for some objects. The standard responses, as discussed in Chapter 4 in the context of similarity computation, are imputation and marginalisation [20, 50]. In the clustering setting, imputation-based methods fill in the missing features before clustering, using strategies ranging from simple mean substitution to model-based approaches such as expectation-maximisation [51], while marginalisation-based methods adapt the distance computation to use only the features available for all objects [52]. In this thesis, missing metadata are handled at the similarity computation stage presented in Chapter 4, where the OWA-based aggregation mechanism produces both an aggregated similarity score and an associated confidence score that reflects the extent of missing metadata. The clustering algorithms in this chapter therefore do not need to handle missing values internally, and the challenge shifts to exploiting the associated confidence scores within the clustering process or its interpretation.

Taken together, the work surveyed in this section addresses the three themes on which this chapter builds: short text clustering, the incorporation of contextual information, and the handling of uncertainty. Existing short text clustering methods have made substantial progress through richer representations and deeper models, but remain largely dependent on language-specific resources and do not incorporate contextual metadata. Constrained and metadata-enhanced clustering approaches offer ways to integrate additional information, yet they either rely on binary constraints that cannot express graded reliability or treat metadata as raw features rather than as part of the similarity computation. Soft clustering paradigms, including fuzzy, possibilistic, and evidential approaches, provide mechanisms for expressing uncertainty in cluster assignments, but derive this uncertainty from pairwise dissimilarities rather than from an independent measure of data quality or reliability. When missing values are present, existing methods handle them through imputation or marginalisation at the clustering level, whereas the novel approach proposed in this chapter resolves incompleteness at the similarity computation stage and instead propagates a confidence score into the clustering process. This chapter brings these threads together by clustering based on precomputed context-aware similarities, investigating multiple

clustering paradigms, and integrating an external confidence score, derived from the completeness of the metadata, into both the clustering input and the interpretation of the clustering output. The formal definitions and properties of the three clustering algorithms employed in this chapter are introduced in the following section.

6.3 Preliminaries

This section provides the formal background on clustering and on the specific methods employed in this chapter. Clustering is an unsupervised learning technique that partitions a set of objects into groups, called *clusters*, such that objects within a cluster are more similar to one another than to objects in other clusters [2, 53]. Unlike supervised classification, it requires no labelled training data, which makes it well suited to the exploratory discovery of structure in unlabelled datasets.

Clustering algorithms can be broadly categorised into several paradigms [2, 53]. Partition-based methods, such as K-Means and K-Medoids, divide the data into a fixed number of non-overlapping clusters by optimising an objective function, for instance the total dissimilarity between data points and their assigned cluster centres. Hierarchical methods, such as HAC, produce a nested sequence of partitions, represented as a tree structure, by iteratively merging or splitting clusters without requiring a predefined number of clusters. Fuzzy or soft clustering methods, such as FCM, relax the assumption that each object belongs to exactly one cluster, instead assigning it a degree of membership to every cluster, which can capture uncertainty or ambiguity in cluster assignments. Many other paradigms exist, including density-based methods, which identify clusters as high-density regions in the data space; model-based methods, which assume that the data are generated by a mixture of underlying probability distributions and seek to estimate the parameters of these distributions; and graph-based methods, which represent the data as a graph and partition it based on connectivity or community structure [53].

The three clustering algorithms employed in this chapter are chosen because they are well-known representatives of distinct clustering paradigms and because they can operate on precomputed similarity scores rather than on feature vectors or Euclidean distances. Each of them takes as input a matrix that represents the pairwise dissimilarities between the objects to be clustered, and none of them requires those dissimilarities to satisfy the properties of a metric, which makes them directly compatible with the output of the context-aware similarity measure of Chapter 4. They do require the matrix to be symmetric, meaning that the dissimilarity between two objects does not depend on the order in which they are compared. Section 6.4.1 establishes that the matrices derived from the graph satisfy this condition. The following subsections introduce each algorithm, including its formal definition, iterative procedure, key parameters, and the methods used to determine those parameters.

6.3.1 K-Medoids

K-Medoids is a partition-based clustering algorithm that divides a set of n objects into k clusters by selecting k actual data objects as cluster representatives, called *medoids* [21]. Unlike K-Means [54, 55], which represents each cluster by a centroid that may not correspond to any observed data point, K-Medoids restricts the cluster centres

to members of the dataset. This restriction makes K-Medoids applicable in settings where only precomputed dissimilarities are available instead of feature vectors or Euclidean distances, since it requires no feature-space representation and no assumption that the dissimilarities satisfy the properties of a metric.

Definition 6.1 (K-Medoids Clustering [21]). Let k denote the desired number of clusters, let $X = \{x_1, x_2, \dots, x_n\}$ be a set of n objects, and let $D = \left[d_{i,j}\right]$ be a symmetric dissimilarity matrix in which $d_{i,j} = d(x_i, x_j) \in [0,1]$ denotes the dissimilarity between x_i and x_j for $i, j = 1, 2, \dots, n$, with $d_{i,i} = 0$ for all i. The K-Medoids algorithm seeks a set of k medoids $M = \{m_1, m_2, \dots, m_k\} \subseteq X$ that minimises the total dissimilarity between each object and the medoid closest to it,

$$J_{\text{KMedoids}} = \sum_{i=1}^{n} \min_{j \in \{1,\dots,k\}} d(x_i, m_j). \tag{6.1}$$

The selected medoids partition X into k clusters $C = \{C_1, C_2, \dots, C_k\}$, such that $X = \bigcup_{j=1}^{k} C_j$ and $C_p \cap C_q = \emptyset$ for all $p \neq q$, in which every object belongs to the cluster of the medoid closest to it,

$$C_j = \left\{ x_i \in X \mid d(x_i, m_j) \leq d(x_i, m_\ell),\ \forall \ell \in \{1, \dots, k\} \right\}, \tag{6.2}$$

where ties are resolved in favour of the medoid with the smallest index.

The most widely used implementation of K-Medoids is the Partitioning Around Medoids (PAM) algorithm [21], which proceeds in two phases. In the *build* phase, an initial set of k medoids is constructed through a greedy heuristic, although implementations also commonly allow random or manual initialisation. In the *swap* phase, the algorithm iteratively considers all pairs (m_j, x_i) where m_j is a current medoid and x_i is a non-medoid, and swaps the pair that produces the largest decrease in the total dissimilarity J_{KMedoids}. The swap phase terminates when no further swap reduces the total dissimilarity, at which point a local optimum has been reached, or when a maximum number of iterations is exceeded. Because the swap step evaluates all $k \cdot (n-k)$ candidate swaps at each iteration, and evaluating each candidate requires updating the assignment cost across all n objects, the computational complexity of a single iteration is $\mathcal{O}(k \cdot n^2)$.

The number of clusters k must be specified in advance. The choice of k can be guided by domain knowledge, for instance by setting k equal to the known number of groups in a ground-truth partition, which enables a direct comparison between the predicted and true partitions. When k is not known a priori, it must be determined from the data. The *elbow method* [56] computes the total dissimilarity J_{KMedoids} for a range of values of k and selects the point at which it stops decreasing significantly, corresponding to a bend (or elbow) in the curve when plotted. Another method, the *silhouette method* [57], evaluates how well each object fits its assigned cluster relative to the nearest alternative cluster. For each object x_i belonging to cluster C_j, the silhouette value $s(x_i)$ is defined by

$$s(x_i) = \frac{b(x_i) - a(x_i)}{\max(a(x_i),\ b(x_i))}, \tag{6.3}$$

where $a(x_i)$ is the average dissimilarity of x_i to all other objects in its own cluster C_j,

$$a(x_i) = \frac{1}{|C_j| - 1} \sum_{\substack{x_\ell \in C_j \\ \ell \neq i}} d(x_i, x_\ell), \tag{6.4}$$

and $b(x_i)$ is the smallest average dissimilarity of x_i to any cluster to which it does not belong,

$$b(x_i) = \min_{\substack{C_p \in C \\ C_p \neq C_j}} \frac{1}{|C_p|} \sum_{x_\ell \in C_p} d(x_i, x_\ell). \tag{6.5}$$

For a singleton cluster, where $|C_j| = 1$ and $a(x_i)$ is undefined, the silhouette value is set to 0 by convention. The silhouette value $s(x_i)$ ranges from -1 to 1. A value close to 1 indicates that x_i is well matched to its own cluster and poorly matched to neighbouring clusters, while a value near 0 indicates that x_i lies on the boundary between two clusters. A negative value suggests that x_i may have been assigned to the wrong cluster. The overall quality of a clustering outcome with k clusters can be summarised by the average silhouette value, commonly referred to as the *silhouette coefficient*, defined as the mean of $s(x_i)$ over all objects in X. To determine the optimal k, the clustering is performed for a range of values of k and the value that maximises the silhouette coefficient is selected. Beyond its use for selecting k, the silhouette coefficient can also serve as a quality measure for any given partition, making it useful for comparing different clustering outcomes when no ground-truth partition is available.

Although the silhouette coefficient is introduced here in the context of K-Medoids, it can be computed for any partition, and therefore equally for the partitions extracted from the HAC hierarchy and for the crisp partitions derived from the fuzzy memberships of FCMdd. It is used as the primary method for determining k across all three clustering approaches in this chapter.

6.3.2 Hierarchical Agglomerative Clustering

Hierarchical Agglomerative Clustering (HAC) is a hierarchical clustering method that produces a nested sequence of partitions by iteratively merging the two most similar clusters until all objects belong to a single cluster [2, 22]. Unlike partition-based methods, which produce a single flat partition for a given k, HAC builds a complete hierarchy of clusterings, from n singleton clusters to a single all-encompassing cluster, and any partition at an intermediate level can be extracted by cutting the hierarchy at the desired number of clusters. The algorithm operates in a bottom-up fashion: it begins with each object in its own cluster and, at each step, identifies and merges the pair of clusters with the smallest inter-cluster dissimilarity according to a chosen *linkage criterion*. This process is repeated for $n - 1$ steps, each reducing the number of clusters by one, until only a single cluster remains.

> **Definition 6.2** (Hierarchical Agglomerative Clustering (HAC) [22]). Let $X = \{x_1, x_2, \dots, x_n\}$ be a set of n objects, let $D = \left[d_{i,j}\right]$ be a symmetric dissimilarity matrix in which $d_{i,j} = d(x_i, x_j) \in [0, 1]$ denotes the dissimilarity between x_i and x_j for $i, j = 1, 2, \dots, n$, with $d_{i,i} = 0$ for all i, and let Δ be a

> linkage criterion that assigns a dissimilarity to every pair of disjoint subsets of X. HAC produces a sequence of partitions $P_n, P_{n-1}, \dots, P_1$ of X, where $P_n = \{\{x_1\}, \{x_2\}, \dots, \{x_n\}\}$ and $P_1 = \{X\}$. At each step t, $1 \leq t \leq n-1$, the two distinct clusters $C_p, C_q \in P_{n-t+1}$ that minimise the inter-cluster dissimilarity $\Delta(C_p, C_q)$ are merged into a single cluster $C_p \cup C_q$, yielding the partition P_{n-t}, with ties resolved in favour of the pair with the smallest indices. Every partition P_k in the sequence contains exactly k clusters.

The linkage criterion Δ determines how the dissimilarity between two clusters is computed from the pairwise dissimilarities between the objects they contain. Its choice determines the behaviour of the algorithm and has a significant impact on the resulting hierarchy and on the quality of the extracted partitions. Three widely used linkage criteria are defined as follows [22, 58], for two disjoint clusters C_p and C_q:

- *Single linkage* defines the inter-cluster dissimilarity as the minimum dissimilarity between any pair of objects from the two clusters,

$$\Delta_{\text{single}}(C_p, C_q) = \min_{\substack{x_i \in C_p \\ x_j \in C_q}} d(x_i, x_j). \tag{6.6}$$

  It tends to produce elongated clusters and is sensitive to chaining effects, where a sequence of close objects can bridge otherwise distinct groups [22].

- *Complete linkage* takes the opposite approach by using the maximum dissimilarity,

$$\Delta_{\text{complete}}(C_p, C_q) = \max_{\substack{x_i \in C_p \\ x_j \in C_q}} d(x_i, x_j). \tag{6.7}$$

  This criterion favours compact, roughly equally sized clusters but can be sensitive to outliers, since a single distant pair can prevent two otherwise similar clusters from merging [22].

- *Average linkage* computes the mean dissimilarity over all pairs of objects from the two clusters,

$$\Delta_{\text{average}}(C_p, C_q) = \frac{1}{|C_p| \cdot |C_q|} \sum_{\substack{x_i \in C_p \\ x_j \in C_q}} d(x_i, x_j). \tag{6.8}$$

  It offers a compromise between single and complete linkage, reducing sensitivity to both chaining effects and outliers, and is often preferred for its balance [22].

Additional linkage criteria, such as Ward's method [59], exist but rely on assumptions about the distance structure that may not hold for arbitrary dissimilarity matrices.

The complete merge history of HAC can be visualised as a *dendrogram*, a tree diagram in which the leaves represent individual objects and the height at which two branches join indicates the inter-cluster dissimilarity of the corresponding merge. The dendrogram provides a compact representation of the entire hierarchy and allows domain experts to visually identify natural groupings at different levels of granularity. An illustration of a dendrogram is shown in Figure 6.1. To obtain a flat partition of k clusters, the dendrogram is cut at a

horizontal level that intersects exactly k branches, yielding the partition P_k. As for K-Medoids, k can be chosen from domain knowledge or from the silhouette coefficient computed with Equation (6.3) on the partitions obtained by cutting the dendrogram at different levels, and additionally by visual inspection of the dendrogram for prominent gaps in the merge distances.

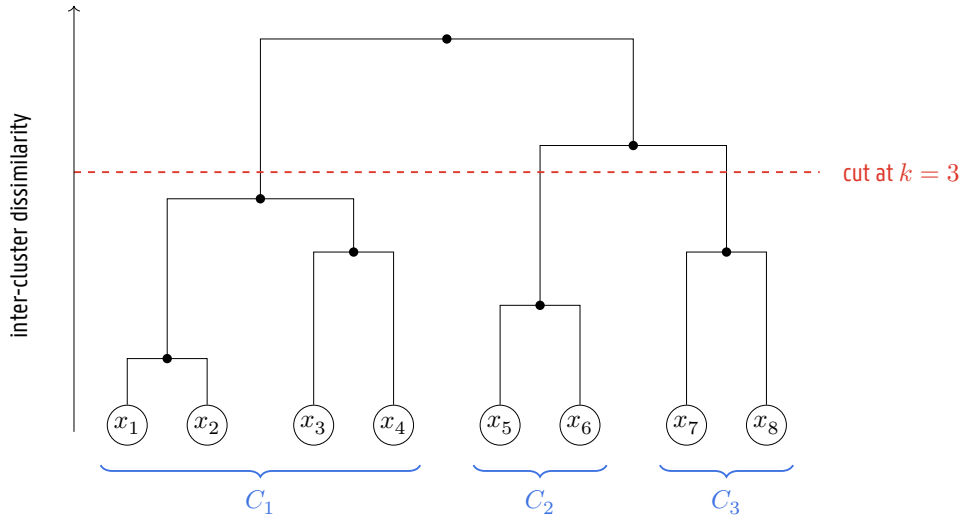


Figure 6.1: Illustration of a dendrogram produced by HAC for eight objects. Each leaf represents an object, and the merge height indicates the inter-cluster dissimilarity at which the corresponding clusters were joined. The dashed line marks a horizontal cut yielding $k = 3$ clusters, labelled C_1, C_2, and C_3.

The computational complexity of HAC is $\mathcal{O}(n^3)$ for a naive implementation and $\mathcal{O}(n^2 \log n)$ with optimised data structures [58], making it more expensive than a single K-Medoids iteration for large datasets when the number of clusters is known. A key advantage, however, is that the algorithm is deterministic and produces the full hierarchy in a single pass. K-Medoids must be re-run for each value of k, whereas HAC allows partitions for any value of k to be extracted without re-executing the algorithm. In this chapter, we use average linkage as the default criterion, since it provides a balanced trade-off between the extremes of single and complete linkage and does not impose distributional assumptions on the dissimilarity matrix.

6.3.3 Fuzzy C-Medoids

The Fuzzy C-Medoids (FCMdd) algorithm [23, 45] extends the K-Medoids approach by replacing hard cluster assignments with fuzzy memberships, allowing each object to belong to multiple or all clusters simultaneously with varying degrees. Like K-Medoids, it selects actual data objects as cluster representatives (medoids) and operates directly on a dissimilarity matrix, making it applicable to relational data. In contrast to K-Medoids, the algorithm assigns each object x_i a membership degree $u_{i,j} \in [0, 1]$ for every cluster j, subject to the constraint that the memberships of each object across all clusters sum to one.

> **Definition 6.3** (Fuzzy C-Medoids (FCMdd) [23, 45]). Let k denote the desired number of clusters, let $X = \{x_1, x_2, \dots, x_n\}$ be a set of n objects, let $D = [d_{i,j}]$ be a symmetric dissimilarity matrix in which $d_{i,j} = d(x_i, x_j) \in [0, 1]$ denotes the dissimilarity between x_i and x_j for $i, j = 1, 2, \dots, n$, with $d_{i,i} = 0$ for all i, and let $q > 1$ be the *fuzzification exponent* that reflects the degree of fuzziness in

the partition. The FCMdd algorithm seeks a set of k medoids $M = \{m_1, m_2, \dots, m_k\} \subseteq X$ and a membership matrix $U = [u_{i,j}]$, where $u_{i,j} \in [0, 1]$ denotes the membership degree of x_i in the cluster represented by m_j for $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, k$, that minimise the total dissimilarity between the objects and the medoids, weighted by the membership degrees,

$$J_{\text{FCMdd}} = \sum_{i=1}^{n} \sum_{j=1}^{k} u_{i,j}^{q} \cdot d(x_i, m_j), \tag{6.9}$$

subject to the constraint

$$\sum_{j=1}^{k} u_{i,j} = 1, \quad \forall\, i \in \{1, \dots, n\}. \tag{6.10}$$

The fuzzification exponent q controls how sharply the membership degrees are distributed across clusters. When q approaches 1, the memberships converge to the hard assignments of K-Medoids, with each object receiving a membership of 1 in its nearest cluster and 0 in all others. As q increases, the memberships become more uniform across clusters, yielding a softer partition in which most objects have non-negligible membership in multiple clusters. The value $q = 2$ is the most commonly used default in the literature [23, 44], although domain-specific tuning may yield better results.

The FCMdd algorithm alternates between two update steps until convergence. Given a fixed set of medoids M, the membership degrees are updated by

$$u_{i,j} = \frac{1}{\sum_{\ell=1}^{k} \left(\frac{d(x_i, m_j)}{d(x_i, m_\ell)} \right)^{\frac{1}{q-1}}}, \quad \forall i \in \{1, \dots, n\},\ j \in \{1, \dots, k\}. \tag{6.11}$$

The ratios in Equation (6.11) are undefined when x_i coincides with a medoid. In that case, the object is assigned full membership in the corresponding cluster and zero membership in all others. In the medoid update step, the medoid m_j of each cluster is updated by selecting the object that minimises the weighted sum of dissimilarities,

$$m_j = \operatorname*{arg\,min}_{x_\ell \in X} \sum_{i=1}^{n} u_{i,j}^{q} \cdot d(x_i, x_\ell). \tag{6.12}$$

The algorithm iterates between Equations (6.11) and (6.12) until there is no change in medoids between successive iterations, or until a maximum number of iterations is reached. As in K-Medoids, the initial medoids can be selected randomly, specified manually, or determined through a greedy heuristic.

The FCMdd algorithm also requires the number of clusters k to be specified in advance. Since FCMdd produces a fuzzy partition, the silhouette coefficient cannot be applied directly to the membership matrix. Instead, a hard partition is first obtained by assigning each object to the cluster for which it has the highest membership degree,

$$C'_j = \left\{ x_i \in X \mid j = \operatorname*{arg\,max}_{\ell \in \{1, \dots, k\}} u_{i,\ell} \right\}, \tag{6.13}$$

where ties are resolved in favour of the cluster with the smallest index, and the silhouette coefficient is then computed on this crisp partition. The value of k that maximises the silhouette coefficient of the crisp partition is selected, following the procedure described in Section 6.3.1.

The fuzzy membership degrees produced by FCMdd provide information that is not available from hard clustering methods. An object with a high membership degree in a single cluster and low degrees in all others can be considered a strong member of that cluster, whereas an object with similar membership degrees across multiple clusters lies at the boundary between those clusters. This distinction is valuable for the interpretation of clustering results, as it allows domain experts to identify ambiguous assignments and to understand the degree to which each object belongs to its assigned cluster. The method for integrating the confidence scores from Chapter 4 into the three clustering approaches is presented in Section 6.4.

6.4 Context-Aware and Responsible Clustering of Short Texts

This section presents the proposed method for clustering short texts based on the context-aware similarity and confidence scores from Chapter 4, together with three strategies for integrating confidence into the clustering process or its interpretation. We first describe how the pairwise similarity and confidence scores are extracted from the graph-based data model and organised into the matrix form required by the clustering algorithms of Section 6.3. We then introduce the three confidence integration strategies: a post-hoc visualisation that colours each text according to the confidence of its cluster assignment, a non-weighted combination that multiplies similarity and confidence prior to clustering, and a weighted combination that balances the two components through an adjustable weight parameter. Both combination strategies are inspired by the global preference functions proposed in Chapter 5 and produce a modified dissimilarity matrix that serves as input to the clustering algorithms.

6.4.1 Similarity and Confidence Score Extraction

The clustering algorithms introduced in Section 6.3 operate on a symmetric dissimilarity matrix derived from the pairwise relationships between texts. In the previous chapters, however, the relatedness between texts has been represented as similarity scores and stored by edge properties in the graph-based data model of Chapter 2. Two types of similarity edges are relevant to this chapter.

The first type corresponds to the purely textual similarity computed by the hierarchical orthographic similarity measure of Chapter 3. These edges, labelled `SIMILAR_TO`, connect text nodes and carry a membership grade representing the orthographic similarity between two text elements. In this chapter, we are primarily interested in clustering entire texts, so we focus on the `SIMILAR_TO` edges that connect top-level text nodes, which represent complete texts. When clustering is performed on textual similarity alone, these edges provide the sole input, and no confidence information is available.

The second type corresponds to the context-aware similarity computed by the OWA-based fusion mechanism of Chapter 4. As described in Section 4.4.3, the pairwise similarity and confidence scores are stored respectively

as the membership grade and the `confidence` property of the edge labelled `CONTEXT_SIMILAR_TO` between top-level text nodes, where each edge carries both an aggregated similarity $s_{i,j} \in [0,1]$ and its associated confidence score $c_{i,j} \in [0,1]$. When clustering is performed on the context-aware similarity, both scores are available for the confidence integration strategies described in the remainder of this section.

To apply the clustering algorithms, the pairwise scores are extracted from the graph into matrix form. Let $T = \{t_1, t_2, \dots, t_n\}$ be the set of n texts to be clustered. The *similarity matrix* $M_S = [s_{i,j}]$ stores the similarity score $s_{i,j} \in [0,1]$ between each pair of texts t_i and t_j for $i, j = 1, 2, \dots, n$, with $s_{i,i} = 1$ for all i, since identical texts are fully similar. In the case of textual similarity, the entries of M_S are derived from the `SIMILAR_TO` edges, while for context-aware similarity they are obtained from the `CONTEXT_SIMILAR_TO` edges. When clustering on the context-aware similarity, the *confidence matrix* $M_C = [c_{i,j}]$ stores the corresponding confidence score $c_{i,j} \in [0,1]$ between each pair for $i, j = 1, 2, \dots, n$, with $c_{i,i} = 1$ for all i, reflecting full confidence in the self-similarity of each text. The entries of M_C are derived from the `confidence` property of the `CONTEXT_SIMILAR_TO` edges and reflect the extent of available metadata used in the similarity aggregation, as defined in Section 4.4.2. Both matrices are symmetric, since the hierarchical orthographic similarity measure is symmetric by Proposition 3.2 and the aggregated similarity and confidence scores inherit this property, as established in Section 4.4.3, so that $s_{i,j} = s_{j,i}$ and $c_{i,j} = c_{j,i}$ for all i and j. The *dissimilarity matrix* M_D is obtained by complementing the similarity scores:

$$M_D = [d_{i,j}], \quad \text{where } d_{i,j} = 1 - s_{i,j} \text{ for } i, j = 1, 2, \dots, n. \tag{6.14}$$

Since this complement is applied per element, M_D is symmetric as well and satisfies the requirement of the three clustering algorithms introduced in Section 6.3. It serves as the default clustering input. In what follows, the notations $d(t_i, t_j) = d_{i,j}$ and $c(t_i, t_j) = c_{i,j}$ are used to refer to the dissimilarity and confidence scores between texts t_i and t_j, respectively, as stored in M_D and M_C. When one of the combination-based confidence integration strategies is applied, a modified dissimilarity matrix replaces M_D, as described in Section 6.4.2.

6.4.2 Confidence Integration Strategies

The dissimilarity matrix M_D derived from the context-aware similarity already incorporates the contextual metadata through the OWA-based fusion mechanism, but it does not reflect the confidence scores that accompany the aggregated similarities. The two are computed from disjoint inputs, as set out in Section 4.4.2. The aggregated similarity follows from the values of the criteria that were available, whereas the confidence follows from the total weight of those that were not, so that no value of $s_{i,j}$ constrains $c_{i,j}$. The confidence matrix M_C therefore remains unused if the clustering algorithms are applied directly to M_D. To exploit this additional information, we propose three strategies that differ in whether they modify the clustering input or add an interpretive layer to its output.

The first strategy, *post-hoc confidence visualisation*, leaves the clustering process unchanged and instead augments the clustering outcome with a visual layer that indicates the confidence of each cluster assignment. The second strategy, a *non-weighted combination* of similarity and confidence, replaces each similarity score

by the product of the similarity and confidence scores, producing a modified dissimilarity matrix that serves as input to the clustering algorithms. The third strategy, a *weighted combination* of similarity and confidence, replaces the product by a linear combination in which a weight parameter controls the relative contribution of each component. These strategies provide domain experts with multiple means of integrating the confidence information: the visualisation offers an interpretive layer that surfaces the reliability of cluster assignments, while the two combination strategies integrate the confidence directly into the clustering input, allowing the resulting clusters to reflect not only the similarity between texts, which already incorporates the available context, but also the reliability of those similarity scores.

Post-Hoc Confidence Visualisation

The first strategy does not alter the clustering algorithm or its input. Instead, after clustering has been performed on the dissimilarity matrix M_D, each cluster assignment is given a colour on a gradient from red (low confidence) to yellow (medium confidence) to green (high confidence), providing a visual indication of its reliability. The colour is determined by the confidence score between the text and the *representative* of its assigned cluster, where the notion of representative depends on the clustering method. Where the assignment is made by comparing a text to the representatives, the confidence to the representative is the reliability of the comparison that determines that assignment, which makes it the natural quantity to display. This confidence reflects nothing of the confidence between the text and the remaining members of its cluster. If the reliability of a cluster as a whole is of interest rather than the reliability of an individual assignment, the confidence scores over all members would have to be aggregated, by taking their average, their minimum, or any other aggregation function that is appropriate for the application context.

For K-Medoids, the representative of each cluster C_j is the medoid m_j selected by the algorithm, where $1 \leq j \leq k$. The confidence of the assignment of text t_i to cluster C_j is then the confidence score $c\big(t_i, m_j\big)$ stored in M_C. A high value indicates that the similarity between t_i and m_j was computed from a substantial proportion of the available metadata, whereas a low value reflects an assessment that rests on limited contextual metadata.

For FCMdd, the medoids are likewise available, since the algorithm selects a set of actual data objects as cluster centres. When the fuzzy partition is transformed into a crisp assignment using Equation (6.13), each text t_i is assigned to the cluster C_j for which its membership degree is highest. The confidence of this assignment is defined in the same way as for K-Medoids, namely the confidence score $c\big(t_i, m_j\big)$ stored in M_C. Because FCMdd additionally provides the membership degrees $u_{i,j}$, the visualisation can convey two complementary dimensions of reliability for each text. The membership degree reflects how strongly the text is associated with its assigned cluster based on the dissimilarity structure, so that a high membership indicates a reliable assignment, whereas the confidence score captures the completeness of the metadata underlying the similarity between the text and its medoid. Both dimensions could be aggregated into a single reliability score for each cluster assignment, through an average, pessimistic (minimum), or optimistic (maximum) combination, but this is not pursued in this chapter, since the two dimensions may be of interest to domain experts in their own right and may be interpreted differently depending on the application context.

For HAC, the situation differs, since the algorithm does not produce medoids as part of its output. After cutting the dendrogram at k clusters, a flat partition $\{C_1, C_2, \dots, C_k\}$ is obtained, but no cluster representative is identified by the algorithm itself. To enable a consistent visualisation across all three methods, we determine a *post-hoc medoid* for each cluster C_j by selecting the object with the smallest total dissimilarity to all other objects in the cluster:

$$m_j = \begin{cases} \underset{t_\ell \in C_j}{\arg\min} \sum_{\substack{t_i \in C_j \\ i \neq \ell}} d(t_i, t_\ell), & \text{if } \left|C_j\right| > 1 \\ t_\ell, & \text{if } C_j = \{t_\ell\}. \end{cases} \tag{6.15}$$

This post-hoc medoid is the most centrally located object in the cluster according to the dissimilarity matrix and serves the same role as the medoid in K-Medoids and FCMdd. The confidence of the assignment of text t_i to cluster C_j is then the confidence score $c\big(t_i, m_j\big)$ stored in M_C, consistent with the other two methods. Because the partition is formed by successive merges rather than by comparison to a representative, this confidence serves as a proxy for the reliability of the grouping rather than as the reliability of the comparison that produced it. A cluster containing a single text is its own representative, so the confidence displayed for that text is maximal by construction and reflects the isolation of the text rather than the completeness of its metadata.

The post-hoc visualisation strategy provides domain experts with an interpretable layer of reliability information without affecting the clustering outcome. Clusters in which most assignments are coloured green suggest that the underlying similarity scores were computed from relatively complete metadata, indicating that the cluster assignments are more reliable. Conversely, clusters dominated by red or yellow assignments rest on limited contextual evidence and may warrant closer inspection. The colouring describes a particular partition and therefore depends on the number of clusters k, since changing k changes the composition of the clusters and with it the representative against which each text is compared. This dependence is not specific to HAC, as the medoids selected by K-Medoids and FCMdd likewise change with k. This visualisation is applicable regardless of which dissimilarity matrix was used as input, whether M_D, derived from the aggregated similarity alone, or one of the modified matrices introduced in the following subsections.

Non-Weighted Combination of Similarity and Confidence

The second strategy integrates the confidence scores directly into the clustering input by combining each similarity score with its associated confidence score into a single value before computing the dissimilarity matrix. The combined score for each pair of texts t_i and t_j is defined by the product of the similarity and confidence, following the non-weighted global preference function of Equation (5.20):

$$p_{i,j} = s_{i,j} \cdot c_{i,j}. \tag{6.16}$$

By combining both components conjunctively, the product ensures that a high combined score requires both a high similarity and a high confidence: if either component is low, the product decreases accordingly.

The modified dissimilarity matrix M'_D is then derived by complementing the combined scores:

$$M'_D = \left[d'_{i,j}\right], \quad \text{where } d'_{i,j} = 1 - p_{i,j} \text{ for } i, j = 1, 2, \ldots, n. \tag{6.17}$$

Since $s_{i,i} = 1$ and $c_{i,i} = 1$, it follows that $d'_{i,i} = 0$, preserving the required diagonal property. The modified matrix M'_D replaces M_D as input to any of the three clustering algorithms, and the clustering proceeds identically to the base case.

The effect of this combination on the clustering outcome can be understood through its behaviour at the boundaries. When the confidence is maximal ($c_{i,j} = 1$), the combined score reduces to the similarity itself ($p_{i,j} = s_{i,j}$), and the dissimilarity is unchanged. When the confidence is low, the product diminishes the effective similarity, thereby increasing the dissimilarity between the two texts. Two texts with high similarity but low confidence will consequently be treated as less similar by the clustering algorithm, reducing the likelihood that they are assigned to the same cluster. This strategy effectively shifts the clustering objective from grouping texts by similarity alone to grouping them by a joint criterion that rewards both high similarity and high confidence, prioritising pairs supported by more complete metadata. Such behaviour is desirable when the goal is to produce clusters whose assignments are supported by sufficient contextual metadata, at the potential cost of separating pairs whose similarity is plausible but insufficiently supported by the available metadata.

Weighted Combination of Similarity and Confidence

The third strategy replaces the conjunctive product of the non-weighted combination by a linear combination in which a weight parameter $w \in [0, 1]$ controls the relative importance of the similarity and confidence components. The combined score for each pair of texts t_i and t_j is defined by the weighted global preference function of Equation (5.22):

$$p^w_{i,j} = (1 - w) \cdot s_{i,j} + w \cdot c_{i,j}. \tag{6.18}$$

This formulation expresses the trade-off between similarity and confidence through a linear combination governed by the weight w.

The modified dissimilarity matrix M^w_D is obtained by complementing the combined scores:

$$M^w_D = \left[d^w_{i,j}\right], \quad \text{where } d^w_{i,j} = 1 - p^w_{i,j} \text{ for } i, j = 1, 2, \ldots, n. \tag{6.19}$$

As with the non-weighted combination, the diagonal entries satisfy $d^w_{i,i} = 0$ for all i and all w, since $s_{i,i} = c_{i,i} = 1$.

The boundary behaviour of the weight parameter determines the character of the resulting dissimilarity matrix. When $w = 0$, the combined score reduces to the similarity alone ($p^w_{i,j} = s_{i,j}$), and the modified dissimilarity matrix is equal to the standard matrix M_D of Equation (6.14), so that the clustering is driven entirely by similarity. When $w = 1$, the combined score reduces to the confidence alone ($p^w_{i,j} = c_{i,j}$), and the clustering groups texts based solely on their pairwise confidence, regardless of the actual similarity between them. For intermediate values of w, both components contribute, with larger w placing greater emphasis on confidence.

A moderate weight, for instance $w = 0.3$, causes the clustering to favour pairs that are both similar and well supported by metadata, while still permitting high-similarity pairs with moderate confidence to remain close in the dissimilarity space.

The choice of w is a modelling decision that reflects the priorities of the researcher. In settings where the completeness of metadata varies substantially across the corpus and where the researcher wishes to ensure that cluster assignments are supported by sufficient contextual metadata, a higher weight may be appropriate. Conversely, where the textual similarity is already highly discriminative and the confidence scores serve primarily as supplementary information, a lower weight preserves the dominant role of similarity. The experimental evaluation in Section 6.5 examines the effect of different weight values on clustering quality.

Together, the three confidence integration strategies offer a set of options for integrating the confidence scores from Chapter 4 into the clustering process. The post-hoc visualisation preserves the original clustering and overlays confidence as an interpretive aid; the non-weighted combination modifies the clustering input by treating similarity and confidence as equally important; and the weighted combination provides continuous control over the balance between the two components. Because the two combination strategies alter the dissimilarity matrix, they shift the clustering objective from pure similarity-based grouping towards a joint criterion that accounts for both similarity and confidence, and the clusters they produce may consequently differ from those obtained on similarity alone. Both combinations are computed elementwise from the symmetric matrices M_S and M_C, so the modified matrices M'_D and M^w_D are symmetric as well. All three strategies are applicable to each of the three clustering algorithms introduced in Section 6.3, yielding a flexible framework for the context-aware and responsible clustering of short texts. The experimental evaluation of this framework is presented in the following section.

6.5 Experiments

This section evaluates the clustering approaches and confidence integration strategies proposed in Section 6.4 through experiments on the DBBE corpus. The evaluation is structured around four objectives: (i.) analysing the difference between the α-cut grouping procedure and clustering, (ii.) assessing the impact of incorporating contextual metadata into the similarity input used for clustering, (iii.) examining the effect of integrating confidence scores into the clustering process, and (iv.) investigating how certain properties of clustering algorithms and the expected number of clusters can lead to new insights into the dataset. Section 6.5.1 describes the dataset and the ground truth used in the evaluation. Section 6.5.2 presents the experimental setup, including the similarity inputs, the confidence integration configurations, the procedure for determining the number of clusters, the clustering parameters, and the quality measures used to assess the clustering performance. The results are reported in Section 6.5.3, and the practical applicability of the proposed framework is demonstrated through illustrative examples in Section 6.5.4. Finally, Section 6.5.5 discusses and interprets these findings.

6.5.1 Dataset

The experiments are conducted on the same subset of 496 epigrams from the DBBE corpus [24, 25] introduced in Section 3.5.1 and used throughout the preceding chapters. The epigrams are drawn from 21 expert-curated epigram types and have been preprocessed as described in Section 2.6.1. The metadata associated with these epigrams are the same as those described in Section 4.5.1 and include values for location, scribe, metre, genre, and subject. As discussed there, many of the metadata values are missing or incomplete, and the OWA-based fusion mechanism of Chapter 4 is therefore employed to produce context-aware similarity scores that incorporate the available metadata while accounting for their incompleteness through the accompanying confidence scores. Since most metadata are associated with the top-level text nodes and we are interested in clustering entire texts, the similarity and confidence scores used in these experiments are those computed between the top-level text nodes with the label `Epigram`. The graph-based representation of this subset, including the textual similarities from Chapter 3 and the context-aware similarity-confidence pairs from Chapter 4, is stored in a Neo4j [60] graph database, which serves as the source from which the similarity and confidence matrices are extracted, as described in Section 6.4.1.

The 21 epigram types serve as the ground truth for the evaluation. Each epigram belongs to exactly one type, forming a crisp partition of the dataset into $k_{\text{GT}} = 21$ non-overlapping groups. As noted in Sections 3.5.1 and 4.5.4, these types are compiled manually by a small group of researchers, who group epigrams primarily on the basis of textual similarity, with metadata serving as a secondary indicator when the textual content is ambiguous or incomplete. The expert-curated partition therefore represents a single interpretation of the corpus rather than an absolute ground truth, and a clustering that differs from it may still capture valid groupings based on fine-grained textual differences, contextual metadata, or confidence information. This observation motivates not only the evaluation of clustering quality against the expert-curated types but also the analysis of how different clustering configurations could lead to new insights into the dataset.

6.5.2 Experimental Setup

Before presenting the results of the experiments, we first describe the experimental setup. This setup encompasses the selection of similarity inputs and weight vector configurations, the confidence integration configurations, the procedure for determining the number of clusters, the clustering parameters, and the quality measures used to evaluate the clustering results.

Similarity Inputs and Weight Vector Configurations

The experiments employ several similarity inputs derived from the graph-based data model. These inputs differ in whether contextual metadata are incorporated and, when they are incorporated, in the weight vectors used to aggregate the similarity components in the OWA-based fusion mechanism of Chapter 4.

The first input is the *textual baseline*, in which the similarity matrix $M_{S_{\text{text}}}$ is derived from the textual similarity scores of Chapter 3 using the Custom 1 parameter configuration. Since no contextual metadata are incorporated into this similarity measure, no weight vector and no confidence scores are associated with this input. This

configuration serves as the reference against which all context-aware configurations are compared to assess the impact of incorporating contextual metadata and confidence information into the clustering process.

The remaining inputs use the context-aware similarity from Chapter 4, computed through the OWA-based fusion mechanism with different weight vectors. The underlying similarity components include the textual similarity (with the Custom 1 parameter configuration) and the five metadata fields (location, scribe, metre, genre, and subject), computed following the same procedure as in Section 4.5. These components are aggregated using the OWA operator, and the weight vector determines the contribution of each rank position to the aggregated similarity and, consequently, to the confidence score. Three weight vector configurations are evaluated:

- $\mathbf{w}_{\text{avg}}$: The average weight vector $\mathbf{w}_{\text{avg}} = \left(\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}\right)$, which distributes the weight equally across all six rank positions, reducing the OWA aggregation to the arithmetic mean of the textual similarity and the five metadata similarities. The resulting similarity and confidence matrices are denoted $M_{S_{\text{avg}}}$ and $M_{C_{\text{avg}}}$, respectively.
- $\mathbf{w}_{F_1}$: The optimal weight vector from Section 4.5.4, namely $\mathbf{w}_{F_1} = (0.005, 0.200, 0.770, 0.000, 0.005, 0.020)$, determined by maximising the F_1-score in the α-cut evaluation of Chapter 4. The resulting similarity and confidence matrices are denoted $M_{S_{F_1}}$ and $M_{C_{F_1}}$, respectively.
- $\mathbf{w}_{\text{cluster}}^{\text{strategy}}$: A clustering-specific optimised weight vector, determined separately for each clustering algorithm and each confidence integration strategy, as described below. The superscript denotes the confidence strategy (*ctx* for context-aware without confidence, *nw* for non-weighted, and *w* for weighted) and the subscript denotes the clustering algorithm (KMedoids, HAC, or FCMdd), yielding nine optimised weight vectors in total. The similarity and confidence matrices computed from each optimised weight vector are denoted $M_{S_{\text{cluster}}}^{\text{strategy}}$ and $M_{C_{\text{cluster}}}^{\text{strategy}}$, respectively.

The $\mathbf{w}_{\text{avg}}$ and $\mathbf{w}_{F_1}$ weight vectors are used to allow a comparison with the results from Chapter 4 and to assess whether optimised method-specific OWA weight vectors are required.

Confidence Integration Configurations

The confidence integration strategies from Section 6.4.2 are evaluated together with their own optimised weight vectors. Each strategy receives a separate optimised weight vector per clustering algorithm, ensuring that the OWA aggregation is tuned for the specific way in which the similarity and confidence scores are combined. This design allows the effect of confidence integration to be assessed under the best conditions for each strategy, revealing the maximal potential of each method combination. As in Section 4.5.3, the scores obtained under these optimised vectors should therefore be read as a best case for this corpus rather than as an estimate of performance on unseen data. These optimised best-case vectors and parameters characterise what the proposed methods can achieve, while remaining adjustable to the priorities of the researchers who apply them. Held-out protocols such as cross-validation are likewise not employed here, following the same rationale as in Section 4.5.3.

Three sets of configurations are evaluated per clustering algorithm, in addition to the textual baseline, which does not involve confidence integration or weight vector tuning.

Context-Aware Similarity without Confidence Integration. In the first set, the clustering algorithms are applied to the dissimilarity matrix M_D derived from the context-aware similarity alone, as defined in Equation (6.14), without incorporating the confidence scores. This set is evaluated for all three weight vector configurations ($\mathbf{w}_{\text{avg}}$, $\mathbf{w}_{F_1}$, and $\mathbf{w}^{\text{ctx}}_{\text{cluster}}$), yielding three configurations per clustering algorithm. The optimised weight vector $\mathbf{w}^{\text{ctx}}_{\text{cluster}}$ for each clustering algorithm is determined using the two-stage grid search procedure described below.

Non-Weighted Confidence Integration. In the second set, the dissimilarity matrix M'_D is derived from the product of the similarity and confidence scores, as defined in Equation (6.17). Since the context-aware comparison in the first set already establishes the effect of weight vector choice, the non-weighted strategy is evaluated only with its own optimised weight vector $\mathbf{w}^{\text{nw}}_{\text{cluster}}$, yielding one configuration per clustering algorithm. The optimised weight vector $\mathbf{w}^{\text{nw}}_{\text{cluster}}$ for each clustering algorithm is determined through the same two-stage grid search procedure as for the context-aware strategy, using the modified dissimilarity matrix M'_D instead of M_D.

Weighted Confidence Integration. In the third set, the dissimilarity matrix M^w_D is derived from the weighted combination of Equation (6.19). The OWA weight vector and the confidence weight w are jointly optimised through the extended grid search procedure described below, yielding an optimised weight vector $\mathbf{w}^{\text{w}}_{\text{cluster}}$ and an associated optimal confidence weight for each clustering algorithm. To assess the sensitivity of the clustering quality to the confidence weight, the results are reported for all $w \in \{0.1, 0.2, \dots, 1.0\}$ using the optimised OWA vector $\mathbf{w}^{\text{w}}_{\text{cluster}}$, yielding ten configurations per clustering algorithm. The value $w = 0$ is excluded since it reduces to the context-aware configuration without confidence integration.

Weight Vector Optimisation

The optimised weight vectors for the context-aware and non-weighted strategies are determined through the same two-stage grid search procedure as in Section 4.5.3, adapted to the clustering setting. In the first stage, a coarse grid search with steps of 0.05 is performed over all weight vectors of length six whose components are drawn from $\{0, 0.05, 0.10, \dots, 1.00\}$ and sum to 1. For each candidate vector, the context-aware similarity and confidence matrices are computed, the dissimilarity matrix is derived according to the relevant strategy (M_D for the context-aware strategy, M'_D for the non-weighted strategy), and the clustering algorithm is applied with k_{GT}. The weight vector that yields the highest pairwise F_1-score, defined below, is retained. In the second stage, a finer grid search with steps of 0.005 is performed within a ± 0.025 range around each component of the retained vector, again retaining the vector that maximises the F_1-score. The fine-grid bounds are narrower than those used in Section 4.5.3 to accommodate the higher computational cost of clustering relative to the α-cut evaluation.

For the weighted strategy, the first stage is extended to include the confidence weight w. For each combination of candidate OWA vector and $w \in \{0.1, 0.2, \dots, 1.0\}$, the dissimilarity matrix M^w_D is derived and the clustering is applied with k_{GT}. The combination that yields the highest F_1-score determines both the retained

OWA vector and the optimal confidence weight. The second stage then refines only the OWA vector within the same ± 0.025 range at the fixed optimal confidence weight, yielding the final optimised weight vector $\mathbf{w}^{\text{w}}_{\text{cluster}}$.

For FCMdd, the fuzzification exponent q is selected in the same search rather than fixed in advance. The first stage evaluates each candidate OWA vector in combination with q ranging over $\{1.1, 1.2, \dots, 2.5\}$, and the combination yielding the highest F_1-score with k_{GT} is retained. This is done separately for each confidence integration strategy, so that the reported results represent the best case for FCMdd under each of them. For the context-aware strategy, the selected q is reused for the $\mathbf{w}_{\text{avg}}$ and $\mathbf{w}_{F_1}$ configurations, which are not themselves optimised in this chapter. For the textual baseline, which involves no weight vector, q is selected on its own as the value in $\{1.1, 1.2, \dots, 2.5\}$ that yields the highest F_1-score with k_{GT}.

Overview of Experimental Configurations

The post-hoc visualisation strategy from Section 6.4.2 does not alter the clustering input and is therefore not included as a separate configuration; it is demonstrated through illustrative examples in Section 6.5.4. Together, the experiments comprise 15 configurations per clustering algorithm: the textual baseline, three context-aware configurations without confidence integration ($\mathbf{w}_{\text{avg}}$, $\mathbf{w}_{F_1}$, and $\mathbf{w}^{\text{ctx}}_{\text{cluster}}$), one non-weighted configuration ($\mathbf{w}^{\text{nw}}_{\text{cluster}}$), and ten weighted configurations ($\mathbf{w}^{\text{w}}_{\text{cluster}}$ evaluated at each $w \in \{0.1, 0.2, \dots, 1.0\}$). Unlike the other strategies, the weighted combination is reported at all ten values of w rather than at its optimum alone, since w is also a modelling decision that reflects the priorities of the researcher, as set out in Section 6.4.2, and to illustrate the effect of varying w on the clustering quality. Reporting the full range shows how the clustering quality varies across the choices a researcher might make, rather than only at the value that agrees best with the expert-curated types. This yields a total of $15 \times 3 = 45$ configurations across the three clustering algorithms.

Number of Clusters

All configurations described in the preceding subsections are evaluated with the number of clusters set to $k_{\text{GT}} = 21$, corresponding to the number of expert-curated epigram types in the ground truth (GT). Clustering using k_{GT} enables a direct comparison between the predicted and expert-curated partitions through the quality measures described in Section 6.5.2 and addresses the question of how well the clustering recovers the expert-curated groupings under each similarity input and confidence strategy.

The expert-curated partition, however, represents a single interpretation of the corpus, and the underlying similarity structure may support alternative partitions with different numbers of clusters, which could be equally valid and lead to new insights into the corpus. To investigate this, Section 6.5.4 presents a separate analysis in which the number of clusters is determined by the data rather than prescribed by the ground truth. In that analysis, the silhouette coefficient, computed from Equation (6.3), is maximised over $k \in \{10, 11, \dots, 30\}$ for a representative subset of configurations, and the resulting optimal number of clusters is compared against k_{GT}. Alternative partitions that differ from the expert-curated types may capture groupings that are not reflected in the expert types but are nonetheless supported by the similarity and confidence data, offering domain experts a complementary perspective on the corpus.

Clustering Parameters

Most of the clustering parameters are set consistently across all experiments to ensure comparability. For K-Medoids, the PAM algorithm is used [21], with the initial medoids selected through *min-max seeding*: the first medoid is chosen as the object with the smallest total dissimilarity to all other objects, and each subsequent medoid is selected as the object whose minimum distance to the already selected medoids is maximal. This deterministic initialisation avoids the variability associated with random seeding and ensures reproducibility. K-Medoids is run for a maximum of 300 iterations or until convergence, defined by no change in the medoids between iterations. For HAC, average linkage is used, as discussed in Section 6.3.2. For FCMdd, the algorithm iterates until the medoids do not change or until a maximum of 300 iterations is reached. The initial medoids for FCMdd are selected using the same min-max seeding procedure as for K-Medoids to ensure consistency in the initialisation across these two algorithms. The fuzzification exponent q is the one parameter that is not fixed across the experiments, since it is selected jointly with the optimised weight vector as described in Section 6.5.2.

Quality Measures

The quality of the clustering results is assessed through five external measures that compare the predicted partition against the expert-curated ground truth, complemented by the silhouette coefficient as an internal quality measure. As in the α-cut evaluation of Sections 3.5.2 and 4.5.3, the external measures are based on pairwise relationships between the epigrams. For all possible pairs of epigrams, pairs are classified as True Positives (TP) if both epigrams are assigned to the same cluster and belong to the same expert-curated epigram type, as False Positives (FP) if they are assigned to the same cluster but belong to different epigram types, as False Negatives (FN) if they are assigned to different clusters but belong to the same epigram type, and as True Negatives (TN) if they are assigned to different clusters and belong to different epigram types. In the case of FCMdd, which produces a fuzzy partition, the predicted partition is first transformed into a crisp partition by assigning each epigram to the cluster for which it has the highest membership degree, as defined by Equation (6.13), before applying the same pairwise classification.

Precision, recall, and F_1-score. From these pairwise counts, precision, recall, and F_1-score are computed using Equations (3.25), (3.24) and (3.26), as defined in Section 3.5.2. Precision reflects the proportion of epigram pairs assigned to the same cluster that also belong to the same expert-curated epigram type, recall reflects the proportion of pairs belonging to the same type that are correctly assigned to the same cluster, and the F_1-score provides a harmonic mean that balances the two. These measures allow a direct comparison with the α-cut evaluation of Chapter 4, but they do not account for the agreement expected by chance, meaning that random cluster assignments of epigrams can yield non-zero scores [61]. The ARI and AMI measures address this limitation.

Adjusted Rand Index. The standard Rand Index (RI) [62] measures the overall agreement between two clustering outcomes, or partitions, as the proportion of pairs on which the two partitions agree. Let $U = \left\{U_1, U_2, \dots, U_{|U|}\right\}$ and $V = \left\{V_1, V_2, \dots, V_{|V|}\right\}$ be two partitions of the same set of n objects, and let

$n_{i,j} = \left|U_i \cap V_j\right|$ denote the number of objects in the intersection of U_i and V_j. These entries form the contingency table of the two partitions, summarising how the objects are distributed across the clusters in U and V. Let $u_i = |U_i|$ and $v_j = \left|V_j\right|$ denote the sizes of the clusters in U and V, respectively. In these experiments, U represents the predicted partition from the clustering algorithm and V the expert-curated partition. The TP, FP, FN, and TN counts follow the same definitions as in the precision and recall calculations. The standard RI is defined by

$$\mathrm{RI}(U, V) = \frac{\mathrm{TP} + \mathrm{TN}}{\binom{n}{2}}, \tag{6.20}$$

where $\binom{n}{2} = \mathrm{TP} + \mathrm{FN} + \mathrm{FP} + \mathrm{TN}$ is the total number of pairs among the n objects. The RI ranges from 0 to 1, with 1 indicating perfect agreement. However, as with precision and recall, the standard RI does not correct for the agreement expected by chance [61].

The Adjusted Rand Index (ARI) [61] addresses this limitation by adjusting the RI for the expected value under random assignment of objects to clusters, and is defined by

$$\mathrm{ARI}(U, V) = \frac{\mathrm{RI}(U, V) - \mathbb{E}[\mathrm{RI}(U, V)]}{1 - \mathbb{E}[\mathrm{RI}(U, V)]}, \tag{6.21}$$

where $\mathbb{E}[\mathrm{RI}(U, V)]$ is the expected value of the RI under random assignment and the 1 in the denominator represents the maximum possible value of the RI. The value 1 in the denominator is the maximum of the Rand index, attained when the two partitions coincide. The ARI can be computed directly from the contingency table by

$$\mathrm{ARI}(U, V) = \frac{\sum_{i=1}^{|U|}\sum_{j=1}^{|V|}\binom{n_{i,j}}{2} - \frac{\sum_{i=1}^{|U|}\binom{u_i}{2}\cdot\sum_{j=1}^{|V|}\binom{v_j}{2}}{\binom{n}{2}}}{\frac{1}{2}\left[\sum_{i=1}^{|U|}\binom{u_i}{2} + \sum_{j=1}^{|V|}\binom{v_j}{2}\right] - \frac{\sum_{i=1}^{|U|}\binom{u_i}{2}\cdot\sum_{j=1}^{|V|}\binom{v_j}{2}}{\binom{n}{2}}}, \tag{6.22}$$

or, equivalently, expressed in terms of the pairwise counts as [63]

$$\mathrm{ARI}(U, V) = \frac{2 \cdot (\mathrm{TP} \cdot \mathrm{TN} - \mathrm{FN} \cdot \mathrm{FP})}{(\mathrm{TP} + \mathrm{FN}) \cdot (\mathrm{FN} + \mathrm{TN}) + (\mathrm{TP} + \mathrm{FP}) \cdot (\mathrm{FP} + \mathrm{TN})}. \tag{6.23}$$

The ARI takes a value of 1 when the two partitions are identical, 0 when the agreement is no better than random assignment, and negative values when the agreement is worse than random assignment. The pairwise formulation makes the relationship between the ARI and the pairwise precision, recall, and F_1-score explicit: all four measures are derived from the same TP, FP, FN, and TN counts, but the ARI corrects for the agreement expected under random assignment, enabling meaningful comparisons across configurations that may produce different numbers of clusters.

Adjusted Mutual Information. Standard Mutual Information (MI) [64] measures the statistical dependence between the ground truth and the clustering results: if the cluster assignments closely correspond to the

expert-curated epigram types, the MI will be high, whereas if they are independent, the MI will be low. Using the same notation as above, the MI is defined by

$$\mathrm{MI}(U,V) = \sum_{i=1}^{|U|}\sum_{j=1}^{|V|} \frac{n_{i,j}}{n} \ln\left(\frac{n \cdot n_{i,j}}{u_i \cdot v_j}\right). \tag{6.24}$$

The MI is non-negative and equals zero when the two partitions are independent. However, as with the standard RI, the standard MI does not account for the agreement expected by chance, and its upper bound depends on the entropies of the individual partitions, making direct comparison across configurations with different numbers of clusters difficult.

The Adjusted Mutual Information (AMI) [65] addresses both limitations by adjusting the MI for the expected value under random assignment and by normalising the score by the average entropy of both partitions, and is defined by

$$\mathrm{AMI}(U,V) = \frac{\mathrm{MI}(U,V) - \mathbb{E}[\mathrm{MI}(U,V)]}{\mathrm{avg}(H(U),H(V)) - \mathbb{E}[\mathrm{MI}(U,V)]}, \tag{6.25}$$

where $H(U) = -\sum_{i=1}^{|U|} \frac{u_i}{n} \ln\left(\frac{u_i}{n}\right)$ and $H(V) = -\sum_{j=1}^{|V|} \frac{v_j}{n} \ln\left(\frac{v_j}{n}\right)$ are the entropies of the two partitions, and $\mathbb{E}[\mathrm{MI}(U,V)]$ is the expected value of the MI under random assignment. The AMI takes a value of 1 when the two partitions are identical, 0 when the agreement is no better than random assignment, and negative values when the agreement is worse than random assignment. Like the ARI, the AMI corrects for chance and enables meaningful comparisons across configurations with different numbers of clusters. The implementation of the AMI from the scikit-learn library [66] is used in the experiments.

Silhouette coefficient. The silhouette coefficient, computed from Equation (6.3), serves a dual role in these experiments. It is used both to determine the optimal number of clusters, as described in Section 6.5.2, and as an internal quality measure that reflects how well the clustering fits the dissimilarity structure independently of any external ground truth. Unlike the five external measures above, the silhouette coefficient does not require a reference partition and instead evaluates the cohesion and separation of the clusters based on the dissimilarity matrix alone, providing a complementary perspective on clustering quality that is not tied to the expert-curated types.

6.5.3 Results

This subsection reports the results of the experiments described in Section 6.5.2. We first present the clustering results obtained on the textual baseline, which provide a reference point for the subsequent analyses and situate the clustering relative to the α-cut evaluation of Chapters 3 and 4. We then examine the effect of incorporating contextual metadata into the similarity input, and finally the effect of the two confidence integration strategies that modify the clustering input. Throughout, the results obtained with $k_{\mathrm{GT}} = 21$ are compared against the expert-curated epigram types.

Textual Baseline Clustering

The first set of experiments applies the three clustering algorithms to the textual baseline dissimilarity matrix $M_{D_{\text{text}}}$ with $k_{\text{GT}} = 21$. The matrix $M_{D_{\text{text}}}$ is obtained from $M_{S_{\text{text}}}$ using Equation (6.14) and incorporates no contextual metadata or confidence information. Table 6.1 presents the results for K-Medoids, HAC, and FCMdd, reporting the precision, recall, F_1-score, ARI, AMI, and silhouette coefficient for each algorithm. The fuzzification exponent q for FCMdd is selected as the value yielding the highest F_1-score with k_{GT} on the textual baseline, following the procedure described in Section 6.5.2, which gives $q = 1.2$.

Table 6.1: Quality measures for the clustering results obtained from the textual baseline dissimilarity matrix $M_{D_{\text{text}}}$ with $k_{\text{GT}} = 21$. The FCMdd clustering is performed with the optimised fuzzification exponent $q = 1.2$. The highest score for each quality measure is highlighted in bold.

Method	Precision	Recall	$\mathbf{F_1}$	ARI	AMI	Silhouette
K-Medoids	**0.8590**	0.8211	0.8396	0.8260	0.8988	0.6761
HAC	0.5690	**0.9783**	0.7195	0.6881	0.9151	**0.7001**
FCMdd	0.8305	0.9103	**0.8685**	**0.8566**	**0.9293**	0.6934

On the textual baseline, the three algorithms display distinct behaviours on the precision-recall trade-off. K-Medoids results in the highest precision at 0.8590 but the lowest recall at 0.8211, reflecting its tendency to form compact clusters around well-separated medoids: the epigrams grouped together are mostly of the same expert-curated epigram type, but a number of genuinely related epigrams are split across separate clusters. HAC with average linkage shows the opposite behaviour, achieving the highest recall at 0.9783 against a much lower precision of 0.5690. The successive merging of the most similar clusters absorbs many epigrams into a few large clusters, which captures most same-type pairs but also admits many pairs drawn from different epigram types, substantially lowering the precision. FCMdd strikes the most even balance between the two extremes, with a precision of 0.8305 and a recall of 0.9103, and consequently yields the highest F_1-score of 0.8685.

The chance-corrected measures further characterise the performance while exposing a more subtle distinction between the algorithms. FCMdd records the highest ARI and AMI at 0.8566 and 0.9293 respectively, confirming that its clusters align most closely with the expert-curated epigram types once agreement by chance is discounted. K-Medoids follows on the ARI at 0.8260, consistent with its strong F_1-score, but records the lowest AMI of the three methods at 0.8988. HAC, by contrast, ranks last on the ARI at 0.6881 yet second on the AMI at 0.9151, a discrepancy that follows from the differing sensitivities of the two measures. The pairwise ARI heavily penalises the oversized clusters produced by average linkage, since each such cluster includes epigrams from several expert-curated types and therefore generates a large number of same-cluster pairs that belong to different epigram types. The information-theoretic AMI, however, remains high because knowing the cluster to which an epigram is assigned still conveys substantial information about its expert-curated type, even though average linkage produces clusters of very unequal size, with a few dominant clusters alongside many small ones. The silhouette coefficient, which assesses cohesion and separation of clusters against the dissimilarity structure alone and is therefore independent of the expert-curated epigram types, is highest for HAC at 0.7001,

indicating that its partition fits the geometry of the textual dissimilarities better than those of K-Medoids and FCMdd, even though it agrees least with the ground truth on a pairwise basis.

These results can be placed in direct comparison with the α-cut evaluations of Chapters 3 and 4, since both evaluation settings rest on the same pairwise precision, recall, and F_1-score against the same expert-curated epigram types, as noted in Section 6.5.2. The textual baseline clustering operates on exactly the Custom 1 textual similarity configuration whose α-cut evaluation in Chapter 3 reached a peak F_1-score of 0.7942 on the epigram subset. Clustering this same similarity surpasses that peak for two of the three algorithms: K-Medoids and FCMdd reach F_1-scores of 0.8396 and 0.8685, respectively, exceeding the best textual α-cut result by a clear margin, while HAC reaches 0.7195 and remains well below it.

This improvement is consistent with the nature of the two evaluation settings. The α-cut applies a single global threshold to every pair, so a similarity value that is appropriate for one region of the corpus may be too permissive or too strict for another. Clustering with k_{GT} instead partitions the epigrams into exactly as many groups as there are expert-curated types, and lets each grouping decision adapt to the local structure of the dissimilarity matrix. An epigram with a generally high dissimilarity to all other epigrams may therefore still be correctly assigned, because it is more similar to the epigrams in its cluster than to those in the others, even if none of its similarity values exceeds the α-cut threshold. Every pair of epigrams within a cluster moreover counts as a match, whereas the α-cut counts only the pairs whose similarity exceeds the threshold, so two epigrams that are both similar to a third may be counted as unrelated. A grouping that adapts to local structure is therefore more naturally aligned with the partition-style ground truth than a fixed-threshold cut, which matches the expectation raised in the discussion of Chapter 4 that evaluation techniques not tied to a single threshold would reveal value in the similarity scores that the α-cut discards.

Considering the α-cut evaluation, a connection can be drawn to single linkage, one of the linkage criteria for HAC introduced in Section 6.3.2 but not used in these experiments. Single linkage joins two clusters as soon as one pair of epigrams across them is similar enough, so two epigrams end up in the same cluster whenever a chain of sufficiently similar epigrams runs between them. This is what the α-cut produces as well, once the pairs it retains are followed through such chains. The two therefore group the corpus in the same way. They differ only in what they count as a positive pair, the clustering counting every pair within a group and the α-cut only the pairs that exceed the α threshold. Average linkage behaves differently, since it merges on the average dissimilarity between two clusters rather than on a single close pair, avoiding the chaining effect of single linkage and producing more compact clusters.

The higher clustering scores were attributed partly to the fact that every pair within a cluster counts as a match, whereas the α-cut counts only the pairs above the threshold. Single linkage puts that explanation to the test, since it forms a complete partition as well while grouping epigrams similarly to the α-cut. If forming a complete partition were the source of the improvement, single linkage should show it too. Applied to the textual similarities with k_{GT}, as an additional check outside the configurations of Section 6.5.2, it yields a precision of 0.5536, a recall of 0.9938, and an F_1-score of 0.7111, with an ARI of 0.6781 and an AMI of 0.9153. Almost every related pair is recovered, but barely half of the pairs placed together belong to the same type, as a few bridging epigrams merge distinct types into one large cluster and every pair inside it is then counted as a match.

The F_1-score stays below the 0.7195 of average linkage and below the best α-cut value of 0.7942, so the improvement does not appear. Forming a complete partition is therefore not by itself what raised the scores, and the stronger results of K-Medoids and FCMdd follow from how those methods decide which epigrams belong together. Single linkage serves here only to isolate this effect and is not carried into the remaining experiments, which use average linkage throughout for the reasons given in Section 6.3.2.

With the textual baseline established as a reference point, the following subsection examines whether incorporating contextual metadata into the similarity input alters these clustering outcomes.

Context-Aware Similarity Clustering

This subsection turns to the context-aware similarity and assesses how the inclusion of contextual metadata affects the clustering results. For each clustering algorithm, the context-aware similarity matrices computed with the average weight vector $\mathbf{w}_{\text{avg}}$, the empirical weight vector $\mathbf{w}_{F_1}$ from Chapter 4, and the method-specific optimised weight vector $\mathbf{w}^{\text{ctx}}_{\text{cluster}}$ are used as input, and the resulting partitions are compared against the expert-curated epigram types using the same quality measures as for the textual baseline. The corresponding dissimilarity matrices are derived through Equation (6.14), the number of clusters is fixed with $k_{\text{GT}} = 21$, and for FCMdd the fuzzification exponent $q = 1.8$ is selected jointly with the optimised weight vector and reused across this method's weight vector configurations, following Section 6.5.2. Table 6.2 reports the results for the three algorithms under each input.

Table 6.2: Quality measures for the clustering results obtained from the context-aware dissimilarity matrix M_D with $k_{\text{GT}} = 21$, for each clustering algorithm and for each of the three weight vector configurations: the average weight vector $\mathbf{w}_{\text{avg}}$, the empirical weight vector $\mathbf{w}_{F_1}$ from Chapter 4, and the method-specific optimised weight vector $\mathbf{w}^{\text{ctx}}_{\text{cluster}}$. For FCMdd, the fuzzification exponent $q = 1.8$ is selected jointly with the optimised weight vector and reused across the three weight vector configurations. The highest score for each quality measure is highlighted in bold.

Weight vector	Method	Precision	Recall	$\mathbf{F_1}$	ARI	AMI	Silhouette
$\mathbf{w}_{\text{avg}}$	K-Medoids	0.8064	0.8641	0.8342	0.8193	0.9082	0.5813
	HAC	0.4919	0.9757	0.6541	0.6130	0.8682	0.5793
	FCMdd	0.3667	0.9601	0.5307	0.4695	0.8074	0.5936
$\mathbf{w}_{F_1}$	K-Medoids	0.7471	0.7504	0.7487	0.7269	0.8409	0.5817
	HAC	0.5020	0.9299	0.6520	0.6117	0.8507	**0.6752**
	FCMdd	0.6716	0.7912	0.7265	0.7007	0.8417	0.5468
$\mathbf{w}^{\text{ctx}}_{\text{KMedoids}}$	K-Medoids	**0.8172**	0.9000	0.8566	0.8435	0.9171	0.6110
$\mathbf{w}^{\text{ctx}}_{\text{HAC}}$	HAC	0.5227	**0.9818**	0.6822	0.6452	0.8868	0.5635
$\mathbf{w}^{\text{ctx}}_{\text{FCMdd}}$	FCMdd	0.7876	0.9545	**0.8630**	**0.8499**	**0.9286**	0.6119

The method-specific optimised weight vectors are

$$\mathbf{w}^{\text{ctx}}_{\text{KMedoids}} = (0.025, 0.330, 0.050, 0.045, 0.225, 0.325),$$

$$\mathbf{w}^{\text{ctx}}_{\text{HAC}} = (0.065, 0.070, 0.110, 0.125, 0.575, 0.055),$$
$$\mathbf{w}^{\text{ctx}}_{\text{FCMdd}} = (0.325, 0.095, 0.090, 0.050, 0.115, 0.325).$$

These vectors place substantial weight on scattered rank positions, and none of them resembles $\mathbf{w}_{F_1}$ or the vectors retained for the other methods. That they differ between the methods is expected, since each is optimised against a different clustering algorithm, and the three algorithms use the dissimilarity structure in different ways, so the weighting that best supports one algorithm is not necessarily the best for another. As set out in Section 6.5.2, these vectors are the best-performing configurations found on the evaluation subset, and the same holds for those reported in the following paragraphs.

Across the weight vector configurations, K-Medoids remains precise and comparatively balanced and HAC remains strongly recall-oriented, whereas FCMdd no longer occupies a balanced middle ground: its precision varies considerably with the weight vector, falling as low as 0.3667 under $\mathbf{w}_{\text{avg}}$. The method-specific optimised vectors yield the strongest results for every algorithm. Under these vectors K-Medoids improves on its textual baseline, raising the F_1-score from 0.8396 to 0.8566, the ARI from 0.8260 to 0.8435, and the AMI from 0.8988 to 0.9171. FCMdd, by contrast, remains close to its baseline, with an F_1-score of 0.8630 against 0.8685 and similarly marginal changes in the ARI and AMI, so the contextual metadata neither help nor harm the results. HAC performs worse: its best context-aware F_1-score of 0.6822 falls below the textual baseline value of 0.7195, with the ARI and AMI also lower, so the contextual metadata do not improve the merging behaviour of average linkage. Incorporating contextual metadata therefore clearly benefits K-Medoids, leaves FCMdd at roughly its textual baseline, and does not help HAC, indicating that the metadata refine the recovery of the expert-curated epigram types for K-Medoids while the other two approaches draw no additional benefit from them on their own. The inclusion of context also shifts the clustering objective slightly, since the expert-curated epigram types are grounded primarily in textual resemblance and only secondarily in context, so the external measures register its effect only partially. This does not, however, mean that clustering texts together with their metadata is unhelpful, but rather that the selected metadata are not ideal for recovering the expert-curated types, or that these metadata support other valuable clustering outcomes that align less closely with the expert types while capturing more meaningful metadata relationships.

The comparison across weight vectors shows that $\mathbf{w}_{F_1}$, optimised for the α-cut evaluation of Chapter 4, does not consistently improve performance in the clustering setting. For FCMdd it clearly outperforms the average vector, raising the F_1-score from 0.5307 to 0.7265, whereas for K-Medoids it underperforms the average vector, lowering the F_1-score from 0.8342 to 0.7487. Only the per-method optimised vectors, which differ significantly across the three algorithms and are tuned directly against the clustering objective, perform well throughout, confirming that the OWA weights are best selected for the specific clustering method rather than carried over from the α-cut setting.

The silhouette coefficients are also uniformly lower than on the textual baseline across all configurations, which shows that the aggregated similarity, combining textual and metadata components, yields a less compact dissimilarity structure even where its agreement with the expert-curated types is comparable. The following subsection examines the effect of integrating the confidence scores that accompany the context-aware similarities into the clustering input.

Confidence Integration

The final set of experiments incorporates the confidence scores from the similarity-confidence pairs of Chapter 4 into the clustering input through the two combination strategies of Section 6.4.2: the non-weighted combination, which multiplies similarity and confidence, and the weighted combination, which balances the two through a weight w. Both strategies modify the dissimilarity matrix, and in doing so they change what the clustering optimises. Where the preceding experiments grouped epigrams by similarity alone, the confidence-integrated configurations group them by a joint criterion that also rewards reliable, metadata-supported relationships. The expert-curated epigram types are based on textual and, secondarily, contextual resemblance, but they carry no notion of how complete the underlying metadata are. A configuration that incorporates confidence is therefore optimising a partly different objective from the one the external measures reward, since it places more weight on reliable relationships supported by metadata rather than primarily on textual resemblance. Relative to the confidence-free results, the external measures could therefore move in either direction. The F_1-score, ARI, and AMI could decrease as a consequence of this shift in objective, or they could increase if the confidence scores successfully identify and amplify the relationships that are actually informative for recovering the expert-curated types. A decrease, moreover, does not necessarily indicate a worse result: it may instead reflect a more meaningful partition that better captures the underlying structure of the corpus, even if it aligns less closely with the expert-curated types. Where the number of clusters is not fixed in advance, this can even take the form of additional clusters that isolate low-confidence epigrams, which may itself be a valuable outcome in certain research settings. We report the external measures throughout for continuity, but interpret them in light of this caveat.

Non-Weighted Confidence Integration. The non-weighted strategy replaces the similarity score of each pair with the product of its similarity and confidence, defined in Equation (6.16), before deriving the dissimilarity matrix M'_D through Equation (6.17). Table 6.3 reports the results for the method-specific optimised weight vectors $\mathbf{w}^{\text{nw}}_{\text{cluster}}$, with the fuzzification exponent selected for this strategy as $q = 1.1$.

Table 6.3: Quality measures for the clustering results obtained from the non-weighted dissimilarity matrix M'_D with $k_{\text{GT}} = 21$ for each clustering algorithm, using the method-specific optimised weight vectors $\mathbf{w}^{\text{nw}}_{\text{cluster}}$ for the context-aware similarity input. For FCMdd, the fuzzification exponent $q = 1.1$ is selected jointly with the optimised weight vector for this strategy. The highest score for each quality measure is highlighted in bold.

Method	Precision	Recall	$\mathbf{F_1}$	ARI	AMI	Silhouette
K-Medoids	0.8036	0.8895	0.8444	0.8302	**0.8985**	0.0532
HAC	0.4903	**0.9657**	0.6504	0.6090	0.8582	**0.3235**
FCMdd	**0.8217**	0.8907	**0.8548**	**0.8417**	0.8970	0.2024

The method-specific optimised weight vectors are

$$\begin{aligned}
\mathbf{w}^{\text{nw}}_{\text{KMedoids}} &= (0.185, 0.055, 0.035, 0.030, 0.570, 0.125),\\
\mathbf{w}^{\text{nw}}_{\text{HAC}} &= (0.025, 0.130, 0.425, 0.005, 0.040, 0.375),\\
\mathbf{w}^{\text{nw}}_{\text{FCMdd}} &= (0.475, 0.090, 0.160, 0.050, 0.170, 0.055).
\end{aligned}$$

Relative to the confidence-free results of Table 6.2, the non-weighted combination lowers the F_1-score, ARI, and AMI by a small margin for all three algorithms. The F_1-score moves from 0.8566 to 0.8444 for K-Medoids and from 0.8630 to 0.8548 for FCMdd, with the ARI and AMI declining as well, while HAC decreases from 0.6822 to 0.6504. Here HAC remains the most recall-oriented, while FCMdd now has the highest precision of the three methods. FCMdd remains the strongest method under the non-weighted combination, retaining the highest F_1-score and ARI, though both now fall slightly below its textual baseline. K-Medoids also remains close to its confidence-free result, with a small decrease in the F_1-score and ARI, which remain above its textual baseline, and a drop in AMI, which stays higher than that of the other two methods. The reductions are consistent with the conjunctive nature of the product: by increasing the dissimilarity of pairs whose metadata are incomplete, the strategy separates some genuinely related pairs and so lowers recall for all three methods, while its effect on precision is smaller and varies between them. In line with the framing above, these modest decreases in the external measures reflect the partial reorientation of the clustering objective towards reliable relationships rather than a degradation of the clustering itself, since the expert-curated epigram types reward similarity alone and are indifferent to the completeness of the metadata.

The silhouette coefficient, by contrast, drops sharply for all three algorithms, from between 0.56 and 0.61 under the optimised context-aware vectors to 0.0532 for K-Medoids, 0.2024 for FCMdd, and 0.3235 for HAC. This collapse is a direct consequence of the product: because the confidence scores are frequently low where metadata are missing, the product $s_{i,j} \cdot c_{i,j}$ pushes a large proportion of the dissimilarities towards their maximum, compressing the dissimilarity range and reducing the contrast between within-cluster and between-cluster distances on which the silhouette depends. This is to be expected in a corpus with a large amount of missing metadata, especially when the ground truth is based primarily on textual resemblance and is compiled by domain experts who can recognise relationships even where the metadata are incomplete. The clusters therefore become far less compact in the modified dissimilarity space, even though their agreement with the expert-curated types, as measured by the chance-corrected external measures, remains close to that of the confidence-free configurations. The non-weighted combination thus integrates confidence at a substantial cost to internal cohesion while preserving most of the external agreement, which motivates the weighted combination examined next, where the weight w provides explicit control over how strongly the confidence reshapes the dissimilarity matrix.

Weighted Confidence Integration. The weighted strategy replaces the fixed product of the non-weighted combination with the combination $(1 - w) \cdot s_{i,j} + w \cdot c_{i,j}$ of Equation (6.18), introducing a weight w that controls how strongly confidence influences the dissimilarity. Table 6.4 reports the results for $w \in \{0.1, 0.2, \dots, 1.0\}$ using the method-specific optimised weight vectors $\mathbf{w}^{\mathsf{W}}_{\text{cluster}}$ for the context-aware similarity input, with the fuzzification exponent selected for this strategy as $q = 1.1$.

The method-specific optimised weight vectors are

$$\begin{aligned}
\mathbf{w}^{\mathsf{W}}_{\text{KMedoids}} &= (0.010, 0.650, 0.125, 0.025, 0.065, 0.125), \\
\mathbf{w}^{\mathsf{W}}_{\text{HAC}} &= (0.225, 0.025, 0.165, 0.240, 0.120, 0.225), \\
\mathbf{w}^{\mathsf{W}}_{\text{FCMdd}} &= (0.475, 0.170, 0.125, 0.115, 0.105, 0.010).
\end{aligned}$$

Table 6.4: Quality measures for the clustering results obtained from the weighted dissimilarity matrix M_D^w with $k_{\text{GT}} = 21$ for each clustering algorithm and weight $w \in \{0.1, 0.2, \ldots, 1.0\}$, using the method-specific optimised weight vectors $\mathbf{w}_{\text{cluster}}^{\text{w}}$ for the context-aware similarity input. For FCMdd, the fuzzification exponent $q = 1.1$ is selected jointly with the optimised weight vector for this strategy. The highest score for each quality measure is highlighted in bold.

Method	Weight w	Precision	Recall	$\mathbf{F_1}$	ARI	AMI	Silhouette
K-Medoids	0.1	0.8138	0.8735	0.8426	0.8284	0.9120	**0.5448**
	0.2	0.8097	0.8564	0.8324	0.8174	0.9050	0.4635
	0.3	0.8097	0.8564	0.8324	0.8174	0.9050	0.4036
	0.4	**0.8511**	0.8654	0.8582	0.8458	0.9062	0.3664
	0.5	0.8282	0.8429	0.8354	0.8210	0.8842	0.3068
	0.6	0.7932	0.8809	0.8348	0.8196	0.8931	0.2526
	0.7	0.7729	0.8702	0.8187	0.8020	0.8778	0.1976
	0.8	0.7625	0.8613	0.8089	0.7913	0.8665	0.1341
	0.9	0.7082	0.8426	0.7696	0.7477	0.8429	0.0806
	1.0	0.0779	0.8991	0.1434	−0.0041	−0.0095	0.0340
HAC	0.1	0.5254	0.9806	0.6842	0.6476	0.8863	0.4760
	0.2	0.5254	0.9806	0.6842	0.6476	0.8863	0.4237
	0.3	0.5193	0.9708	0.6766	0.6391	0.8818	0.3962
	0.4	0.5170	**0.9865**	0.6784	0.6408	0.8852	0.3428
	0.5	0.5132	0.9801	0.6737	0.6355	0.8798	0.3002
	0.6	0.4130	0.8650	0.5591	0.5057	0.7596	0.1903
	0.7	0.3034	0.8546	0.4478	0.3741	0.6723	0.1253
	0.8	0.2161	0.8029	0.3405	0.2456	0.5353	0.1308
	0.9	0.2145	0.8012	0.3384	0.2431	0.5263	0.1681
	1.0	0.0933	0.4563	0.1549	0.0258	0.1501	0.1639
FCMdd	0.1	0.3838	0.9520	0.5471	0.4890	0.8336	0.5327
	0.2	0.3836	0.9509	0.5466	0.4885	0.8335	0.5423
	0.3	0.3836	0.9509	0.5466	0.4885	0.8335	0.4927
	0.4	0.8254	0.9401	**0.8790**	**0.8678**	**0.9354**	0.3927
	0.5	0.7245	0.8431	0.7793	0.7586	0.8567	0.3364
	0.6	0.6318	0.7501	0.6858	0.6560	0.7919	0.2732
	0.7	0.5950	0.6556	0.6238	0.5895	0.7537	0.2092
	0.8	0.6000	0.7778	0.6774	0.6455	0.7702	0.0555
	0.9	0.2762	0.5579	0.3695	0.2941	0.5708	0.0519
	1.0	0.0779	0.8991	0.1434	−0.0041	−0.0095	0.1264

The weighted combination produces different behaviour across the three algorithms as the confidence weight w increases. K-Medoids is the most stable, holding an F_1-score between 0.83 and 0.86 for w up to 0.6, peaking at 0.8582 at $w = 0.4$, higher than its textual baseline, and declining only gradually thereafter before collapsing at $w = 1.0$. HAC is more sensitive, remaining near an F_1-score of 0.68 for w up to 0.5 and reaching its best value of 0.6842 at $w = 0.1$ and $w = 0.2$, after which it deteriorates steeply, falling to 0.4478 at $w = 0.7$ and below 0.16 at $w = 1.0$. FCMdd behaves non-monotonically, performing poorly for small weights with an F_1-score near 0.55 for $w \leq 0.3$, then rising abruptly to its peak of 0.8790 at $w = 0.4$ before declining again for larger weights. The weak FCMdd results at low w are associated with a low precision and a high recall, 0.3838 and 0.9520 at $w = 0.1$, which points to a small number of oversized clusters until the partition reshuffles at $w = 0.4$. The identical values across adjacent weights, visible for several methods, reflect ranges of w over which the discrete partition does not change.

At a moderate confidence weight, the weighted combination slightly improves over both the confidence-free and the non-weighted configurations for the two medoid-based methods. FCMdd with $w = 0.4$ results in an F_1-score of 0.8790, an ARI of 0.8678, and an AMI of 0.9354, each the highest value recorded for that method across all experiments in this chapter, and in the case of the F_1-score the highest of any configuration evaluated. K-Medoids likewise reaches its best F_1-score of 0.8582 at $w = 0.4$, marginally above its confidence-free value of 0.8566. These results confirm that, for a suitably moderate weight, integrating confidence can amplify the relationships that are informative for recovering the expert-curated epigram types rather than merely shifting the objective away from them, realising the favourable outcome anticipated at the start of this subsection. The gain is small in absolute terms. For FCMdd it is consistent across the F_1-score, ARI, and AMI; for K-Medoids it appears in the F_1-score and ARI, while the AMI edges down slightly. In both cases the best result is reached at the same weight $w = 0.4$.

The behaviour at $w = 1.0$ is the extreme case in which similarity drops out of the combination entirely. At this weight the combined score reduces to the confidence alone, so the clustering groups epigrams purely by the completeness of their shared metadata, disregarding their textual and contextual resemblance. The resulting partitions have no meaningful relation to the expert-curated epigram types: K-Medoids and FCMdd both record a slightly negative ARI of -0.0041 and AMI of -0.0095, indicating agreement marginally worse than random assignment, while HAC barely exceeds chance with an ARI of 0.0258. A negative chance-corrected score is the expected outcome here, since the confidence quantifies the reliability of a similarity rather than the resemblance between two epigrams, and grouping by reliability alone provides no basis for recovering the epigram types. This confirms that confidence is valuable as a modifier of a similarity-based objective but cannot serve as a clustering criterion on its own.

As in the non-weighted case, the silhouette coefficient declines overall as w increases for all three algorithms, though the decline is monotone only for K-Medoids, with HAC and FCMdd recovering slightly at the highest weights. Placing more weight on confidence progressively compresses the dissimilarity range and reduces the internal compactness of the clusters, independently of how well the resulting partition agrees with the expert-curated epigram types. The size of this compression reflects the low confidence values, and thus the high proportion of missing metadata, present throughout the corpus.

Taken together, the confidence integration experiments show that the non-weighted product, and the weighted combination at high w, trade external agreement and internal cohesion for an objective that favours reliable relationships, whereas a moderate weight around $w = 0.4$ secures a small improvement over the confidence-free configurations for the two medoid-based methods, K-Medoids and FCMdd. Confidence nonetheless remains an independent measure of how far the available metadata support each relationship, and folding it into the clustering input can complicate the interpretation of the resulting partition. The post-hoc confidence visualisation of Section 6.5.4 therefore offers a more natural alternative, allowing researchers to assess the reliability of each assignment without altering the clustering objective. A complete interpretation of these findings, together with the qualitative behaviour of the three algorithms, is taken up in Section 6.5.5, once the practical applicability of the framework has been demonstrated. The following subsection presents those demonstrations, illustrating the post-hoc confidence visualisation and the distinctive outputs of each clustering approach on the DBBE corpus.

6.5.4 Practical Applicability

In addition to the quantitative evaluation, and in line with the approach of Sections 2.6, 3.5.4, 4.5.5 and 5.5, this subsection demonstrates the practical applicability of the proposed clustering framework as a hands-on research tool for corpus exploration. The quality measures of Section 6.5.3 summarise how closely each configuration recovers the expert-curated epigram types, but they do not show how the framework can be used to explore the corpus and its underlying structure, or how the confidence scores can be used to inform the interpretation of the clusters. This subsection therefore illustrates three applications in particular: (i.) the post-hoc confidence visualisation, which overlays the confidence scores onto a crisp partition to show the reliability of each assignment; (ii.) the fuzzy memberships of FCMdd, which provide a graded membership of every epigram to every cluster; and (iii.) the choice of the number of clusters, which can be informed by the silhouette coefficient and the HAC hierarchy, potentially revealing other valid groupings of the corpus beyond the expert-curated epigram types. Additionally, we present a concrete example of inspecting the resulting clusters and their metadata relationships to gain further insights into the corpus and flag situations that require further attention from domain experts.

All demonstrations are performed on the full subset of 496 epigrams, except when specified otherwise. For simplicity, the context-aware similarity scores used for clustering are those without confidence integration. To keep the figures and tables small and interpretable, each demonstration highlights only a selection of the clusters or epigrams, but the same principles apply to the full clustering output. K-Medoids illustrates the post-hoc confidence visualisation and FCMdd the graded memberships, after which the choice of the number of clusters is discussed using the silhouette coefficient as well as using the HAC hierarchy. The additional example of inspecting the clusters and their metadata relationships is performed on the K-Medoids clustering, but the same approach can be applied to the other algorithms.

Post-Hoc Confidence Visualisation

Just as we visualised the confidence scores in Section 4.5.5 to show the reliability of the similarity scores, we can visualise the confidence of each cluster assignment to convey the reliability of the clustering itself. To perform this visualisation, we use the Neo4j graph database [60], which already stores the epigrams and their metadata as described in the previous chapters. We first include the cluster assignments in the database by connecting each epigram node to the epigram node representing the medoid of its assigned cluster. These relationships, labelled `IN_CLUSTER`, can be enriched with the confidence score between the epigram and its medoid as a `confidence` property, so that the confidence of each assignment is directly accessible in the graph database. The post-hoc confidence visualisation of Section 6.4.2 can then be illustrated using the visualisation capabilities of Neo4j Bloom [67]. There, the `IN_CLUSTER` relationships can be coloured using a gradient from red through yellow to green according to their `confidence` property, so that green denotes an assignment computed from substantial metadata (high confidence) and red an assignment resting on limited metadata (low confidence). After performing K-Medoids clustering on the full subset of 496 epigrams with $k_{\text{GT}} = 21$ clusters based on context-aware similarity scores computed using the $\mathbf{w}^{\text{ctx}}_{\text{KMedoids}}$ OWA weight vector, we can query the resulting cluster assignments using the Cypher [68] query in Listing 6.1. The query retrieves the epigrams and their assigned cluster medoids, along with the `IN_CLUSTER` relationships, which carry the confidence scores. The query result can then be visualised in Neo4j Bloom, which is configured to assign the appropriate colours to the relationships between epigrams and their assigned cluster medoids based on their confidence scores. A small subset of the resulting visualisation is shown in Figure 6.2, where the distinct groups of epigrams are the clusters and the edges connect each epigram to the medoid of its assigned cluster, coloured according to the confidence of that assignment.

```
MATCH (ep:Epigram)-[r:IN_CLUSTER]->(med:Epigram)
RETURN ep, med, r
```

Listing 6.1: Cypher query to retrieve the epigrams and their cluster assignments. The query matches each epigram node with its assigned cluster medoid through the `IN_CLUSTER` relationship and returns the epigram, the medoid, and the relationship.

The four clusters displayed in Figure 6.2 are selected from the smaller clusters of the partition. Due to the large amount of missing or incomplete metadata in the DBBE corpus, many of the assignments are supported by limited metadata and therefore often have low to medium confidence, which is reflected in a majority of red and yellow relationships in the figure. Only a single relationship has a high confidence score, shown in green, indicating that the corresponding assignment is supported by a relatively complete set of metadata. This colouring allows a domain expert to read the reliability of each assignment directly from the partition, distinguishing clusters that are well grounded in contextual evidence from those that are not, without altering the clustering itself. The confidence values must be interpreted together with the weight vector. Placing a high weight on the lower-ranked similarity scores often leads to lower confidence scores, since these scores draw on metadata fields that are frequently missing in the corpus. The same visualisation can be produced for the FCMdd clustering, where the `IN_CLUSTER` relationships connect each epigram either to the medoid of its highest-membership cluster or to the medoids of all clusters with an associated membership degree, and the confidence colouring can be read together with the fuzzy memberships, allowing the reliability of each

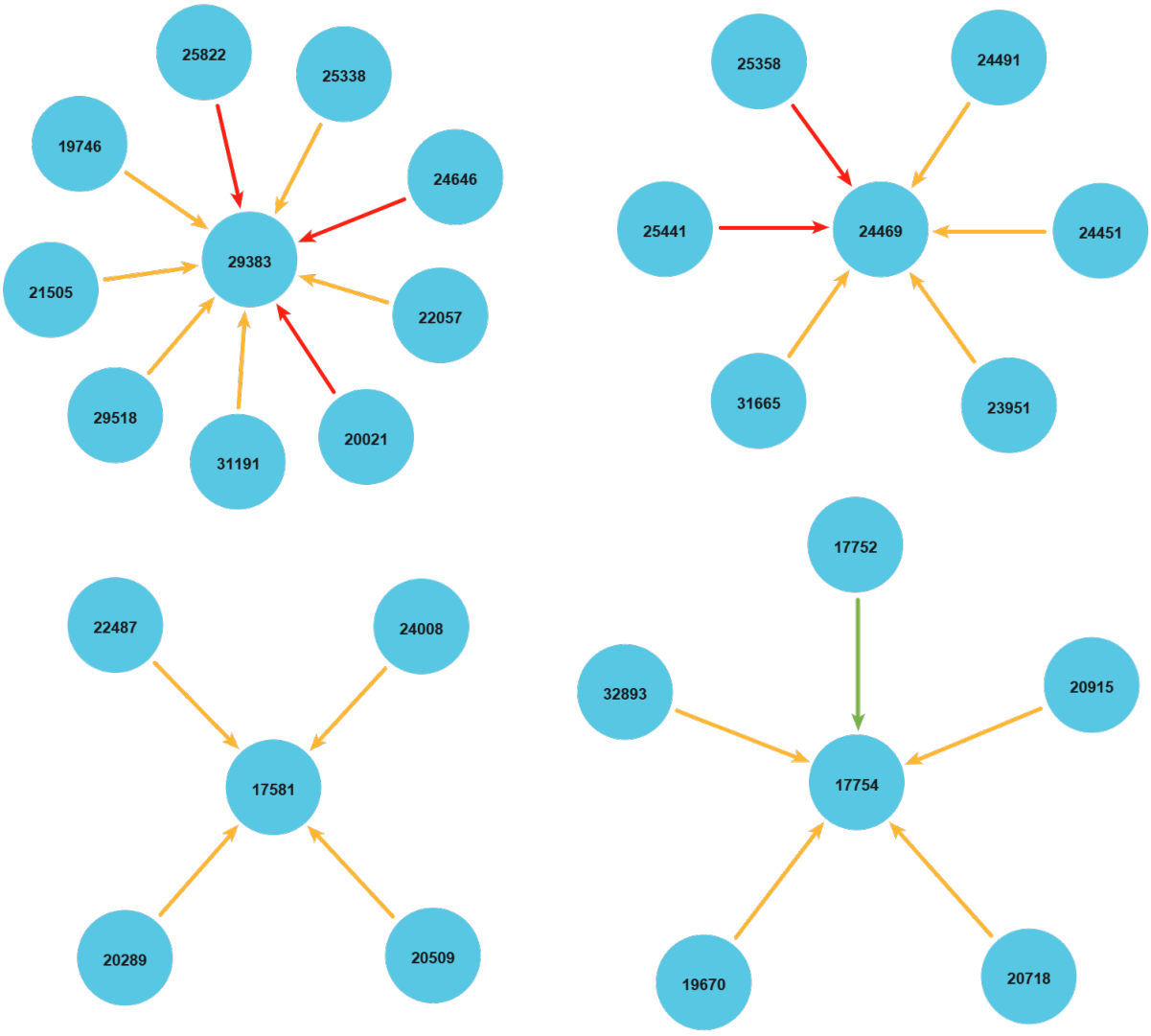


Figure 6.2: Post-hoc confidence visualisation of a selection of the smaller K-Medoids clusters, obtained by clustering the full subset on the context-aware similarity computed with the $\mathbf{w}^{\text{ctx}}_{\text{KMedoids}}$ OWA weight vector using K-Medoids with $k_{\text{GT}} = 21$ clusters. The relationships between each epigram and the epigram representing the medoid of its cluster are coloured according to the confidence of their assignments using a gradient from red (low confidence, $c < 0.45$) through yellow (medium confidence) to green (high confidence, $c > 0.65$).

assignment to be interpreted alongside the graded memberships. For HAC, the confidence colouring can likewise be applied to the relationships between each epigram and the medoid of the cluster to which it belongs at a given level of the hierarchy, determined by Equation (6.15).

Fuzzy Cluster Memberships

Where K-Medoids assigns each epigram to exactly one cluster, FCMdd produces a graded membership of every epigram to every cluster, as defined in Section 6.3.3. To illustrate this, we compute the FCMdd clustering of the full subset with $k_{\text{GT}} = 21$ clusters based on the context-aware similarity computed with the $\mathbf{w}^{\text{ctx}}_{\text{FCMdd}}$ OWA weight vector using the fuzzification exponent $q = 1.8$, the value optimised for the confidence-free context-aware similarity input, which also allows a moderate degree of graded membership. The resulting cluster assignments can then be included in the graph database, where each epigram node is connected to every cluster medoid through an `IN_CLUSTER` relationship. This relationship carries a `membership` property holding the membership degree of the epigram to that cluster, so that the graded memberships are directly accessible, and a `confidence` property holding the confidence score between the epigram and that cluster's medoid, so that the reliability of each assignment can be read alongside the memberships. When we randomly select four medoids from the 21 clusters produced by FCMdd and query the resulting assignments using Listing 6.2, we can retrieve the membership degrees of every epigram to those four clusters. A small subset of these results is reported in Table 6.5, where the membership degrees of a selection of epigrams to each of the four clusters are listed.

```
MATCH (ep:Epigram)-[r1:IN_CLUSTER]->(med1:Epigram {id: 18056})
MATCH (ep)-[r2:IN_CLUSTER]->(med2:Epigram {id: 18236})
MATCH (ep)-[r3:IN_CLUSTER]->(med3:Epigram {id: 22027})
MATCH (ep)-[r4:IN_CLUSTER]->(med4:Epigram {id: 33571})
RETURN ep.id AS epigram, r1.membership AS membership_C1,
  ↪ r2.membership AS membership_C2, r3.membership AS membership_C3,
  ↪ r4.membership AS membership_C4
```

Listing 6.2: Cypher query to retrieve epigrams and their cluster assignments to a subset of four selected medoids. The query matches each epigram node with each of the four selected medoids through the `IN_CLUSTER` relationship and returns the epigram alongside the membership degrees to those medoids.

The memberships in Table 6.5 show that FCMdd captures the graded nature of the epigram types, ranging from unambiguous to genuinely shared assignments. Epigrams 17149 and 19784 sit at one extreme, each belonging fully to a single cluster with a membership of 1, exactly as a crisp partition would assign them, whereas epigram 21503 sits at the other extreme, dividing its membership almost evenly across the four clusters shown, indicating that it could reasonably be placed in several groups at once. The reported values cover only the four selected clusters, denoted C_1 to C_4, while the membership of each epigram is distributed over all 21 clusters and sums to one, so for some epigrams the highest membership value lies in a cluster that is not among the four shown. This graded view is particularly informative when an epigram could genuinely belong to multiple types. In such cases the confidence scores can be read together with the memberships to assess the reliability of each membership, which can be important for further interpretation or analysis.

Table 6.5: Subset of the query results from Listing 6.2. The table lists the membership degrees of a selection of epigrams to four selected clusters, rounded to four decimal places for readability. For each epigram, the highest membership degree among the four selected clusters is highlighted in bold.

epigram	membership_C1	membership_C2	membership_C3	membership_C4
17149	**1.0000**	0.0000	0.0000	0.0000
24074	**0.6849**	0.1069	0.0282	0.0431
31962	**0.3161**	0.2067	0.0671	0.0935
21503	**0.1467**	0.1441	0.1009	0.1403
18861	0.0928	0.0948	0.0741	**0.4158**
17113	0.0568	0.0578	**0.1206**	0.0577
28796	0.0570	**0.8375**	0.0154	0.0212
19784	0.0000	0.0000	**1.0000**	0.0000
⋮	⋮	⋮	⋮	⋮

The epigram with id 31962 can be interpreted as an intermediate case, with a membership of 0.3161 to cluster C_1 and 0.2067 to cluster C_2, which suggests that it shares characteristics of both clusters. A closer look at its content and metadata, shown in Table 6.6, reveals that the first two verses of epigram 31962 correspond to the shared opening and closing verses of the epigrams in types 2148, 2150, and 4245, which were discussed in Sections 2.6 and 3.5.4 and represent the two-verse, three-verse, and longer variants built around that common pair, respectively. The third verse of epigram 31962, however, is unique to this epigram and does not match the shared verses of any of those types, which may explain why it is not fully assigned to any of the clusters representing those types. These shared verses explain the divided membership: cluster C_1 is represented by the medoid with epigram id 18056, which belongs to the two-verse type 2148 in the ground truth, while cluster C_2 is represented by the medoid with epigram id 18236, which belongs to the three-verse type 2150. Both representative epigrams also share the same genre and metre as epigram 31962, which further accounts for the shared membership. For a domain expert, such partial memberships flag epigrams whose classification is ambiguous and which may benefit from closer manual inspection, something a conventional, crisp partition would not reveal.

Number of Clusters

The preceding examples fixed the number of clusters at $k_{\text{GT}} = 21$ to enable a direct comparison with the expert-curated types. As noted in Section 6.5.2, however, the expert-curated partition represents a single interpretation of the corpus, and the similarity structure may support alternative partitions at a different number of clusters. This is especially the case when the OWA weight vectors or confidence integration strategies are not optimised for the ground truth, but instead reflect a different prioritisation of the similarity or confidence scores that may be more appropriate for other research questions or applications. Here we show that a different number of clusters can indeed be optimal for the dissimilarity structure of the corpus and the clustering method, and that this number can be determined from the data using the silhouette coefficient, which can be computed for any partition and therefore does not require a ground truth for evaluation. Furthermore, we examine how

Table 6.6: Textual content and metadata of epigram 31962 [25] after preprocessing. The dashes in the location and subject rows indicate that no value is recorded in the DBBE.

Metadata	Values for epigram 31962
Text	1. ωσπερ ξενι χαιρουσιν ιδιν πατριδα 2. ουτως και ι γραφοντες βιβλιου τελος 3. κιριε βιθι του δουλου σου ιωαννου
Location	—
Genre	Scribe-related
Metre	Dodecasyllable
Scribe	Ioannes
Subject	—

the dendrogram produced by HAC can be used to explore the hierarchical structure of the corpus and identify potential alternative partitions at different levels of the hierarchy. Since the optimised weight vectors for all three clustering methods are tuned to the best partition using $k_{\text{GT}} = 21$ clusters, we instead use the context-aware similarity, without confidence integration, computed with the $\mathbf{w}_{\text{avg}}$ OWA weight vector, which is not optimised for any particular number of clusters. We compute the partitions of all three algorithms on the full subset for all $k \in \{10, 11, \dots, 30\}$ and evaluate the silhouette coefficient of each partition to determine the optimal number of clusters. We first consider all the quality measures as a function of k to examine their behaviour as the number of clusters changes, taking the K-Medoids partition as an example; the results are shown in Table 6.7. These measures are reported for one method only, since they only serve to show how agreement with the expert-curated types varies with the number of clusters rather than to compare the three algorithms. The silhouette coefficient, which is the measure used to select k and the only one that can be determined without a ground truth, is reported for all three algorithms in Figure 6.3.

The quality measures in Table 6.7 display the trade-off between precision and recall as a result of varying the number of clusters. At low k, the partition consists of a few large clusters that capture most same-type pairs but also merge two or more different epigram types into the same cluster, giving a high recall, above 0.96 for $k \leq 15$, against a comparatively low precision. As k increases, the clusters become smaller and more homogeneous, so precision rises overall, reaching 0.9408 at $k = 28$, while recall falls below 0.72. The F_1-score and the two chance-corrected measures balance these tendencies and peak at intermediate values of k, with the F_1-score, ARI, and AMI all highest at $k = 15$. Notably, none of the quality measures that are based on the ground truth is maximised at the expert-curated $k_{\text{GT}} = 21$: the agreement with the expert epigram types is in fact slightly higher at several smaller values of k, which indicates that, for this weight vector, the similarity structure aligns better with the expert types at fewer clusters.

The silhouette coefficient, being independent of the ground truth, can be used to select the number of clusters directly from the dissimilarity structure. Figure 6.3 plots the silhouette coefficient of all three clustering algorithms as a function of k, with the optimal k of each method indicated by a vertical line and the expert-curated $k_{\text{GT}} = 21$ shown for reference. For K-Medoids, the silhouette is maximised at $k_{\text{KMedoids}} = 15$, which

Table 6.7: Quality measures for the K-Medoids partition of the full subset of 496 epigrams for all $k \in \{10, 11, \dots, 30\}$, using the context-aware similarity computed with the $\mathbf{w}_{\text{avg}}$ OWA weight vector, without confidence integration. The highest score for each quality measure is highlighted in bold.

k	Precision	Recall	$\mathbf{F_1}$	ARI	AMI	Silhouette
10	0.6699	0.9665	0.7913	0.7696	0.8727	0.5413
11	0.7090	0.9665	0.8180	0.7995	0.8888	0.5394
12	0.7290	0.9669	0.8312	0.8144	0.8994	0.5620
13	0.7412	0.9685	0.8397	0.8238	0.9122	0.5738
14	0.7661	0.9685	0.8555	0.8413	0.9229	0.5956
15	0.7861	**0.9689**	**0.8680**	**0.8552**	**0.9337**	**0.6081**
16	0.7840	0.9566	0.8617	0.8484	0.9272	0.6049
17	0.7772	0.9198	0.8426	0.8276	0.9162	0.5774
18	0.8161	0.9206	0.8652	0.8527	0.9302	0.5908
19	0.8139	0.9075	0.8582	0.8451	0.9236	0.5981
20	0.8072	0.8689	0.8369	0.8222	0.9123	0.5749
21	0.8064	0.8641	0.8342	0.8193	0.9082	0.5813
22	0.8388	0.8232	0.8309	0.8164	0.8954	0.5752
23	0.8437	0.8232	0.8333	0.8191	0.8986	0.5777
24	0.9009	0.8027	0.8490	0.8367	0.8953	0.4867
25	0.8365	0.7676	0.8006	0.7841	0.8824	0.5158
26	0.8767	0.7357	0.8000	0.7843	0.8725	0.5232
27	0.8973	0.7218	0.8000	0.7847	0.8706	0.5230
28	**0.9408**	0.7135	0.8115	0.7976	0.8717	0.4746
29	0.9406	0.7120	0.8105	0.7965	0.8693	0.4729
30	0.9402	0.7067	0.8069	0.7927	0.8662	0.4679

coincides with the peak of the F_1-score, ARI, and AMI in Table 6.7 and therefore indicates that, for this method and weight vector, the most internally cohesive K-Medoids partition is also the one in closest agreement with the expert types. HAC reaches its highest silhouette at $k_{\text{HAC}} = 10$, the smallest number of clusters considered, consistent with the merging behaviour of average linkage, which favours a small number of large clusters. FCMdd is optimal at $k_{\text{FCMdd}} = 16$, close to the K-Medoids optimum, reflecting the similar medoid-based character of the two methods. In all three cases the data-driven optimum lies below $k_{\text{GT}} = 21$, which reinforces the observation that the corpus supports a grouping into fewer clusters than the expert-curated partition under this weight vector, while none of the methods identifies exactly the same number of clusters, since each responds differently to the dissimilarity structure.

HAC offers an additional perspective on the number of clusters. Rather than selecting a single number of clusters, HAC produces a hierarchy of nested partitions, represented by a dendrogram as described in Section 6.3.2, which traces the merging of clusters across all numbers of clusters, from one cluster containing all epigrams at the

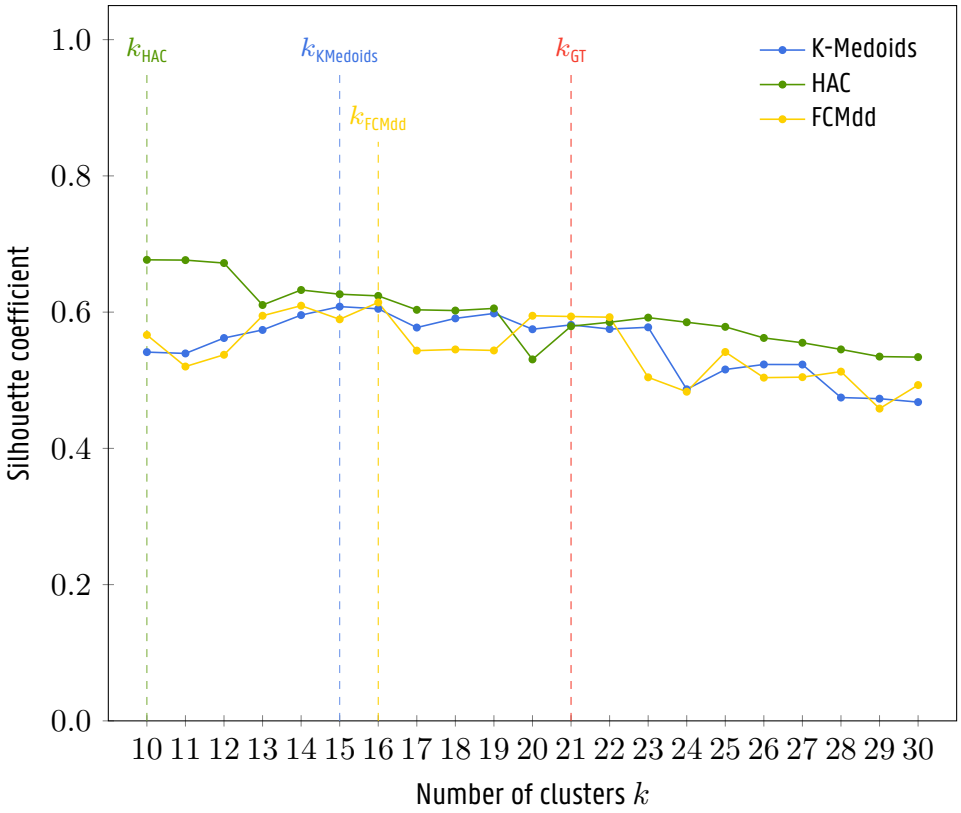


Figure 6.3: Silhouette coefficient of the three clustering algorithms applied to the full subset as a function of the number of clusters k, using the context-aware similarity computed with $\mathbf{w}_{\text{avg}}$. The vertical lines mark the silhouette-optimal number of clusters for each method, $k_{\text{KMedoids}} = 15$, $k_{\text{HAC}} = 10$, and $k_{\text{FCMdd}} = 16$, and the expert-curated ground truth granularity $k_{\text{GT}} = 21$.

top of the hierarchy down to the singleton clusters at the bottom. The silhouette coefficient can be computed for each partition in the hierarchy, so that the optimal number of clusters can be determined from the hierarchy itself, without rerunning the clustering for each value of k.

As an illustration, consider a small subset of thirteen epigrams drawn from two expert-curated types, the types with identifiers 2311 and 4155, comprising five and eight epigrams respectively. Computing the HAC hierarchy of these epigrams from the context-aware similarity with the $\mathbf{w}_{\text{HAC}}^{\text{ctx}}$ OWA weight vector yields the dendrogram in Figure 6.4, which traces the merges from thirteen singleton clusters at the bottom up to a single cluster at the top. Evaluating the silhouette coefficient at each level of the hierarchy selects $k_{\text{HAC}} = 2$ as the optimal number of clusters, shown by the dashed line in the figure. Cutting the dendrogram at that level separates the subset into a cluster of the five epigrams of type 2311 and a cluster of the eight epigrams of type 4155, reproducing the expert-curated types exactly.

In this small case the data-driven number of clusters coincides with the expert-curated one, whereas for the full subset the best number of clusters, based on the silhouette coefficient, lies below it, as seen earlier. The dendrogram itself carries information that a single partition does not: the height of each merge gives the inter-cluster dissimilarity at which the two clusters are joined under the chosen linkage criterion, so reading the hierarchy from coarse to fine-grained groupings shows how distinct the clusters are at every level and the

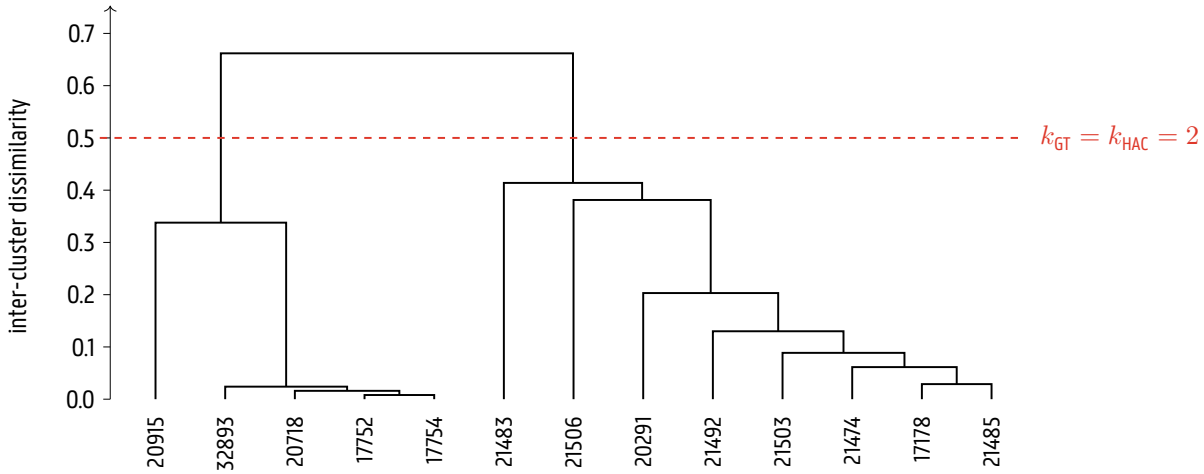


Figure 6.4: Dendrogram produced by HAC for the thirteen epigrams from the types with identifiers 2311 (5 epigrams) and 4155 (8 epigrams), with the epigram ids shown along the horizontal axis. The input is based on the context-aware similarity computed with $\mathbf{w}^{\text{ctx}}_{\text{HAC}}$. The merge height indicates the inter-cluster dissimilarity at which the corresponding clusters were joined. The dashed red line indicates the cut that produces the optimal partition at $k_{\text{HAC}} = 2$ clusters, which coincides with the expert-curated number of clusters $k_{\text{GT}} = 2$ for this subset.

order in which they were merged. This can help a domain expert identify finer or coarser groupings than the expert-curated types provide, since those types represent only one interpretation of the corpus.

The examples so far have drawn on the clustering output itself: the partitions, the hierarchy, the memberships, and the confidence of each assignment. The output can also be combined with the metadata stored in the graph database, which opens a further kind of exploration, illustrated below.

Further Clustering Output Interpretation and Insights

This final example examines the clustering output together with the associated metadata values, to interpret a cluster and to flag metadata that may be missing from individual epigrams. If a subject is recorded for most of the epigrams in a cluster but absent for a few, the cluster membership suggests that those few may share the subject as well, even though it is not recorded for them. Such a finding cannot, however, be accepted without further inspection, since a cluster may also contain epigrams that genuinely do not share the subject. It nonetheless gives a domain expert a concrete starting point, and where the subject is confirmed, the corpus is enriched and later cluster assignments may become more reliable.

To illustrate this, we consider the K-Medoids clustering of the full subset with $k_{\text{GT}} = 21$ clusters on the context-aware similarity with the $\mathbf{w}^{\text{ctx}}_{\text{KMedoids}}$ OWA weight vector. The cluster assignments are stored in the graph database, with each epigram connected to the medoid of its cluster through an `IN_CLUSTER` relationship, as in Section 6.5.4. The database also holds the associated metadata of Section 2.5.1, including the subjects, which are represented by `Person` nodes connected to the epigrams through `SUBJECT_OF` relationships. Querying a single cluster together with the subjects of its epigrams, using Listing 6.3, allows the shared subjects to be inspected directly.

```
MATCH (ep:Epigram)-[ic:IN_CLUSTER]->(med:Epigram {id: 24469})
OPTIONAL MATCH (p:Person)-[so1:SUBJECT_OF]->(ep)
OPTIONAL MATCH (p2:Person)-[so2:SUBJECT_OF]->(med)
RETURN ep, med, ic, so1, so2, p, p2
```

Listing 6.3: Cypher query to retrieve the epigrams of a single cluster together with their subjects. The query matches each epigram assigned to the selected medoid through the `IN_CLUSTER` relationship and, where present, the `Person` nodes that are the subject of the epigram or of the medoid.

The result is shown in Figure 6.5, where the `IN_CLUSTER` relationships are coloured by confidence as in the post-hoc visualisation, so that the reliability of each assignment can be read from the graph, and the `SUBJECT_OF` relationships reveal the subjects shared across the cluster. Five of the seven epigrams in this cluster share the subject *Sophocles*, while the remaining two have no subject recorded. The fact that they are clustered together with the five suggests that the same subject may apply to them, which a domain expert could verify and, if confirmed, add to the corpus.

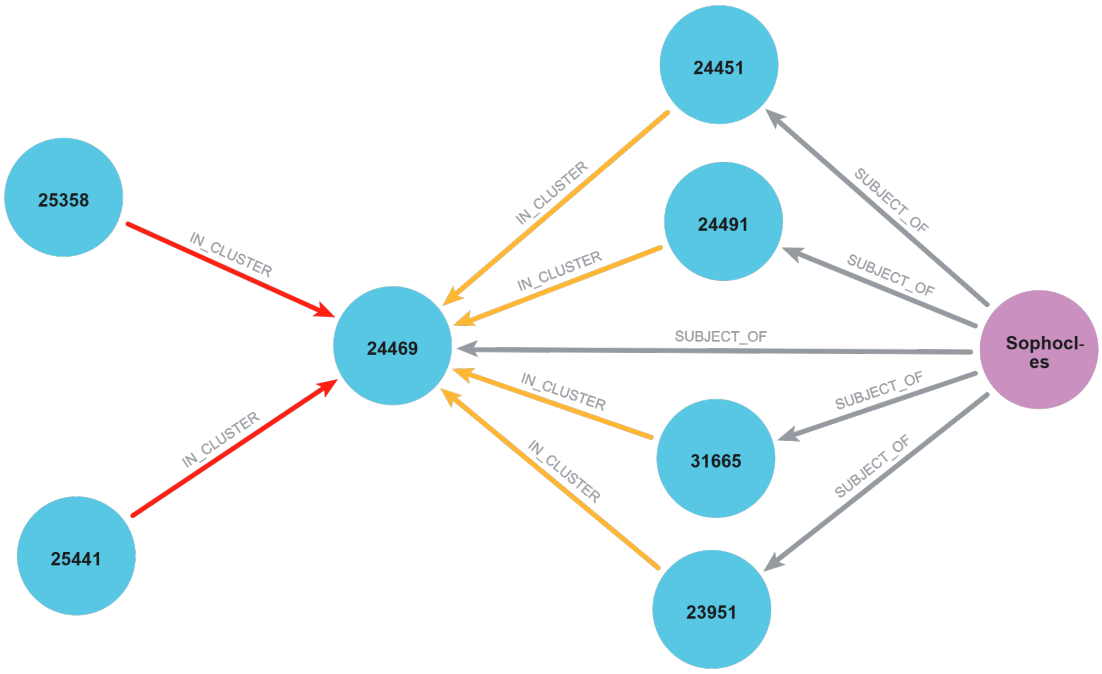


Figure 6.5: Bloom visualisation of the result of Listing 6.3, for the K-Medoids clustering of the full subset on the context-aware similarity computed with the $\mathbf{w}^{\text{ctx}}_{\text{KMedoids}}$ OWA weight vector with $k_{\text{GT}} = 21$ clusters. The relationships between each epigram and the medoid of its cluster are coloured according to the confidence of their assignments, using a gradient from red (low confidence, $c < 0.45$) through yellow (medium confidence) to green (high confidence, $c > 0.65$). The `SUBJECT_OF` relationships connect the epigrams to their recorded subjects.

This example shows how the shared characteristics of a cluster, here the associated subjects, can flag metadata that may be missing from individual epigrams and direct a domain expert towards targeted manual inspection. It is only one of many such use cases: the clustering output can be queried alongside any of the stored metadata or other properties, and potential insights will depend on the corpus and the research question, but it demonstrates that the clustering supports exploration of the corpus beyond the grouping of similar texts.

6.5.5 Discussion

The results of Section 6.5.3 indicate that, on this corpus, clustering can recover the expert-curated epigram types at least as well as the threshold-based evaluation of Chapter 3. On the textual similarity alone, two of the three algorithms exceed the peak F_1-score of that α-cut evaluation, with K-Medoids and FCMdd reaching 0.8396 and 0.8685 against 0.7942. This is not explained by the partition-based nature of the techniques alone, since single linkage, which partitions the corpus in a similar manner to as the α-cut does, stays below the peak α-cut score. The advantage comes instead from the specific clustering criteria that adapt to the local structure of the dissimilarity matrix rather than applying one global threshold to every pair.

Incorporating contextual metadata into the similarity input affects the three methods differently. K-Medoids improves further over its textual baseline on the F_1-score and both chance-corrected measures, FCMdd stays close to its baseline, and HAC falls below it, since the contextual metadata do not improve the merging behaviour of average linkage. The effect of context is therefore real but method-dependent. That FCMdd neither improves nor worsens from context on its own is unsurprising, since the expert-curated types rest primarily on textual resemblance and only secondarily on context, so adding metadata to the similarity shifts the objective slightly away from what the external measures reward.

The confidence integration strategies also act differently on the three methods, and it is here that FCMdd produces its strongest results. Combining similarity and confidence moves the objective towards relationships that are well supported by metadata, and at a moderate weight of $w = 0.4$ this yields the highest F_1-score recorded in the chapter, 0.8790 for FCMdd. Weighting confidence more heavily lowers the agreement with the expert types, and using confidence alone gives partitions no better than a random assignment, which indicates that confidence is useful as a modifier of a similarity-based objective, but not as a clustering criterion on its own. A lower external score under confidence integration does not by itself mean that the partition is worse: it reflects the shift in objective, and may instead point to groupings organised around the reliability of the metadata rather than textual resemblance alone, which the external measures do not reward.

The three approaches also yield outputs that the quality measures do not capture. K-Medoids produces crisp partitions, HAC builds a hierarchy that can be examined at any number of clusters, and FCMdd yields graded memberships that separate clear members from epigrams shared between types, while the post-hoc visualisation makes the reliability of individual assignments visible. The clustering output can moreover be queried together with the stored metadata, which can direct a domain expert to epigrams whose metadata may be incomplete. In the confidence-integrated configurations the fuzzification exponent q is selected close to the crisp limit, because the selection criterion is agreement with the expert types, which are themselves crisp, so the procedure favours the least fuzzy setting that still fits the ground truth. The graded memberships that make FCMdd useful for exploration are more apparent at a higher q than the value that maximises agreement with the expert types, so the configuration most informative to a domain expert is not necessarily the one that scores best against a crisp ground truth. The silhouette-optimal number of clusters likewise differs from the expert-curated $k_{\text{GT}} = 21$, which shows that the similarity structure can reveal other groupings worth investigating.

These findings are subject to the caveats noted throughout the thesis. The evaluation rests on a single corpus and on expert-curated types that represent one defensible interpretation rather than an absolute ground truth, so a divergence from these types may reflect a different grouping rather than an error in the clustering. The weight vectors and the confidence weight are moreover optimised directly on this subset, so the reported scores represent a best case for this corpus and configuration, as set out in Section 6.5.2. Within these limits, the results indicate a consistent advantage on this corpus for clustering over the threshold-based evaluation, and a benefit from incorporating contextual metadata and, at a moderate weight, confidence, over the use of textual similarity alone.

6.6 Conclusions and Future Work

In this chapter, we investigated clustering techniques for short texts that operate on the context-aware similarity and confidence scores introduced in Chapter 4. Three clustering approaches were considered, each representing a different paradigm: K-Medoids, which selects representative objects as cluster centres; HAC, which builds a hierarchy of nested partitions; and FCMdd, a fuzzy counterpart of K-Medoids that assigns each object a graded membership to every cluster rather than a single crisp assignment. The pairwise similarity and confidence scores were extracted from the graph-based data model of Chapter 2 into the matrix form required by these algorithms, and three strategies were proposed for integrating the confidence scores into the clustering or its interpretation: a post-hoc visualisation that colours each text by the reliability of its assignment, a non-weighted combination that multiplies similarity and confidence, and a weighted combination that balances the two through an adjustable weight. Together, these approaches give researchers a set of tools for grouping short texts while making the reliability of each cluster assignment explicit.

The novelty of this chapter is twofold. The first contribution is the application of several clustering paradigms to context-aware similarity scores produced by a measure designed to handle missing metadata through an adapted aggregation mechanism, which allows short texts to be grouped in settings where the textual content alone is only weakly discriminative and where the contextual metadata are incomplete. By operating on the scores produced by the OWA-based fusion of Chapter 4, the clustering process uses all available metadata without resorting to imputation, which introduces assumptions about the missing values, or to marginalisation, which discards information that is not present for all texts. The second contribution is the integration of confidence into the clustering and its interpretation, either by modifying the similarity input or by adding an interpretable layer of reliability, so that a researcher can distinguish assignments that rest on substantial metadata from those that rest on little.

We evaluated these techniques on the Database of Byzantine Book Epigrams, a corpus of short texts in a low-resource language with extensive missing metadata. On the textual similarity alone, clustering recovered the expert-curated epigram types at least as well as the threshold-based evaluation of Chapter 3, with two of the three algorithms exceeding its peak F_1-score. Incorporating contextual metadata improved K-Medoids further, left FCMdd close to its textual baseline, and lowered the performance of HAC. Integrating confidence at a moderate weight gave FCMdd the highest F_1-score recorded in the chapter, 0.8790, whereas relying on confidence alone produced partitions no better than a random assignment. Beyond these scores, the three

approaches offered distinct outputs, from crisp partitions and a navigable hierarchy to graded memberships and a reliability colouring, and a silhouette-based analysis found a number of clusters below the expert-curated $k_{\text{GT}} = 21$, indicating that the similarity structure could support other groupings worth investigating. Taken together, the results favour clustering over the threshold-based evaluation on this corpus, and support the use of contextual metadata and, at a moderate weight, confidence, over textual similarity alone.

The proposed techniques are subject to a number of limitations. First, the evaluation uses a single corpus, and the expert-curated types it compares against are one defensible interpretation rather than an absolute ground truth, so a partition that diverges from them is not necessarily worse, but could indicate a different, equally valid grouping. Second, the expert types rest primarily on textual resemblance, so the external quality measures, which reward agreement with those types, understate the contribution of the confidence-based strategies, whose value lies partly in information the types do not take into account. Third, the benefits of incorporating context and confidence are modest and depend on the method and the configuration: they are not realised by HAC, and the reported gains represent a best case. The weight vectors, and for the weighted strategy the confidence weight, are optimised separately for each method on the evaluation subset itself, through a grid search whose cost grows with the size of the corpus and the resolution of the search.

These limitations suggest several directions for future work. The framework is not restricted to the three algorithms considered here, and other paradigms, such as density-based methods or community detection applied directly to the fuzzy Labelled Property Graph (LPG), could be incorporated and may respond differently to the aggregated scores. The confidence integration could likewise be extended beyond the product and the weighted combination to other operators that combine similarity and reliability, and the weight vectors could be determined through adaptive or more efficient procedures rather than an exhaustive grid search. The development of cluster quality measures that account for confidence would allow the confidence-integrated strategies to be assessed on their own terms rather than only against a confidence-agnostic ground truth, addressing the mismatch between what these strategies optimise and what the current external measures reward. Evaluating the techniques on further corpora, differing in language, text length, and patterns of missing metadata, would strengthen the evidence for their applicability beyond the present case study. Such corpora could also provide multiple ground truths, and one that is based on both text and context would in particular test the confidence-based strategies on the kind of information they incorporate. A further possibility, illustrated in Section 6.5.4, is to use the clustering output to flag metadata that may be missing from individual epigrams, by comparing each epigram against the shared metadata of its cluster; developing this into a systematic procedure for suggesting and validating such additions is left to future work.

Chapter 7 brings together the contributions of this and the preceding chapters, and considers how the tools developed throughout the thesis can support researchers working with short, sparsely annotated texts.

References

[1] Charu C. Aggarwal and ChengXiang Zhai. "A Survey of Text Clustering Algorithms". In: *Mining Text Data*. Boston, MA: Springer US, 2012, pp. 77–128. isbn: 978-1-4614-3223-4. DOI: https://doi.org/10.1007/978-1-4614-3223-4_4.

[2] Anil K. Jain. "Data Clustering: 50 Years Beyond K-Means". In: *Pattern Recognition Letters* 31.8 (2010), pp. 651–666. issn: 0167-8655. DOI: https://doi.org/10.1016/j.patrec.2009.09.011.

[3] Christopher D Manning. "Introduction to Information Retrieval". Syngress Publishing, 2008. DOI: https://doi.org/10.1017/CBO9780511809071.

[4] Weili Wu, Hui Xiong and Shashi Shekhar. "Clustering and Information Retrieval". Vol. 11. Springer Science & Business Media, 2003. DOI: https://doi.org/10.1007/978-1-4613-0227-8.

[5] Jiapeng Wang and Yihong Dong. "Measurement of Text Similarity: A Survey". In: *Information* 11.9 (2020). issn: 2078-2489. DOI: https://doi.org/10.3390/info11090421.

[6] Najlah Gali, Radu Mariescu-Istodor, Damien Hostettler and Pasi Fränti. "Framework for Syntactic String Similarity Measures". In: *Expert Systems with Applications* 129 (2019), pp. 169–185. issn: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2019.03.048.

[7] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado and Jeff Dean. "Distributed Representations of Words and Phrases and Their Compositionality". In: *Advances in Neural Information Processing Systems*. Vol. 26. Curran Associates, Inc., 2013. DOI: https://doi.org/10.48550/arXiv.1310.4546.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova. "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: https://doi.org/10.18653/v1/N19-1423.

[9] Himanshu Suyal, Amit Panwar and Ajit Singh Negi. "Text Clustering Algorithms: A Review". In: *International Journal of Computer Applications* 96.24 (2014), pp. 36–40. issn: 0975-8887. DOI: https://doi.org/10.5120/16946-7075.

[10] Lianyu Hu, Mudi Jiang, Junjie Dong, Xinying Liu and Zengyou He. "Interpretable Clustering: A Survey". In: *arXiv Preprint* (2024). DOI: https://doi.org/10.48550/arXiv.2409.00743.

[11] Gerard Salton and Christopher Buckley. "Term-Weighting Approaches in Automatic Text Retrieval". In: *Information Processing & Management* 24.5 (1988), pp. 513–523. issn: 0306-4573. DOI: https://doi.org/https://doi.org/10.1016/0306-4573(88)90021-0.

[12] Yin Zhang, Rong Jin and Zhi-Hua Zhou. "Understanding Bag-of-Words Model: A Statistical Framework". In: *International Journal of Machine Learning and Cybernetics* 1.1 (2010), pp. 43–52. issn: 1868-808X. DOI: https://doi.org/10.1007/s13042-010-0001-0.

[13] Majid Hameed Ahmed, Sabrina Tiun, Nazlia Omar and Nor Samsiah Sani. “Short Text Clustering Algorithms, Application and Challenges: A Survey”. In: *Applied Sciences* 13.1 (2023). issn: 2076-3417. DOI: https://doi.org/10.3390/app13010342.

[14] Tom Kenter and Maarten de Rijke. “Short Text Similarity with Word Embeddings”. In: *Proceedings of the 24th ACM International on Conference on Information and Knowledge Management*. CIKM ’15. Melbourne, Australia: Association for Computing Machinery, 2015, pp. 1411–1420. isbn: 9781450337946. DOI: https://doi.org/10.1145/2806416.2806475.

[15] Dimas Wibisono Prakoso, Asad Abdi and Chintan Amrit. “Short Text Similarity Measurement Methods: A Review”. In: *Soft Computing* 25.6 (2021), pp. 4699–4723. issn: 1433-7479. DOI: https://doi.org/10.1007/s00500-020-05479-2.

[16] Partha Pakray, Alexander Gelbukh and Sivaji Bandyopadhyay. “Natural Language Processing Applications for Low-Resource Languages”. In: *Natural Language Processing* 31.2 (2025), pp. 183–197. DOI: https://doi.org/10.1017/nlp.2024.33.

[17] Alexandre Magueresse, Vincent Carles and Evan Heetderks. *Low-Resource Languages: A Review of Past Work and Future Challenges*. 2020. DOI: https://doi.org/10.48550/arXiv.2006.07264.

[18] Gediminas Adomavicius and Alexander Tuzhilin. “Context-Aware Recommender Systems”. In: *Recommender Systems Handbook*. Boston, MA: Springer US, 2011, pp. 217–253. DOI: https://doi.org/10.1007/978-0-387-85820-3_7.

[19] Tlamelo Emmanuel, Thabiso Maupong, Dimane Mpoeleng, Thabo Semong, Banyatsang Mphago and Oteng Tabona. “A Survey on Missing Data in Machine Learning”. In: *Journal of Big Data* 8.1 (2021), p. 140. issn: 2196-1115. DOI: https://doi.org/10.1186/s40537-021-00516-9.

[20] Ludmila Himmelspach and Stefan Conrad. “Clustering Approaches for Data with Missing Values: Comparison and Evaluation”. In: *2010 Fifth International Conference on Digital Information Management (ICDIM)*. 2010, pp. 19–28. DOI: https://doi.org/10.1109/ICDIM.2010.5664691.

[21] Leonard Kaufman and Peter J. Rousseeuw. “Partitioning Around Medoids (Program PAM)”. In: *Finding Groups in Data* (2008), pp. 68–125. DOI: https://doi.org/10.1002/9780470316801.ch2.

[22] Fionn Murtagh and Pedro Contreras. “Algorithms for Hierarchical Clustering: An Overview”. In: *WIREs Data Mining and Knowledge Discovery* 2.1 (2012), pp. 86–97. DOI: https://doi.org/10.1002/widm.53.

[23] R. Krishnapuram, A. Joshi, O. Nasraoui and L. Yi. “Low-Complexity Fuzzy Relational Clustering Algorithms for Web Mining”. In: *IEEE Transactions on Fuzzy Systems* 9.4 (2001), pp. 595–607. DOI: https://doi.org/10.1109/91.940971.

[24] Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, Maxime Deforche, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout and Colin Swaelens. “The Database of Byzantine Book Epigrams Project : Principles, Challenges, Opportunities”. eng. In: *Journal of Data Mining and Digital Humanities* On the Way to the Future of Digital Manuscript Studies (2023), p. 41. issn: 2416-5999. DOI: https://doi.org/10.46298/jdmdh.10244.

[25] Kristoffel Demoen, Gilbert Bentein, Klaas Bentein, Floris Bernard, Julián Bértola, Julie Boeten, Mathijs Clement, Cristina Cocola, Eline Daveloose, Sien De Groot, Pieterjan De Potter, Ilse De Vos, Krystina Kubina, Hanne Lauwers, Paulien Lemay, Renaat Meesters, Marjolein Morbé, Delphine Nachtergaele, Marthe Nemegeer, Joachim Nielandt, Mace Ojala, Lisa-Lou Péchillon, Raf Praet, Rachele Ricceri, Anne-Sophie Rouckhout, Jeroen Schepens, Febe Schollaert, Lev Shadrin, Nina Sietis, Dimitrios Skrekas, Colin Swaelens, Maria Tomadaki, Sarah-Helena Van den Brande, Merel Van Nieuwerburgh, Lotte Van Olmen, Noor Vanhoe and Nina Vanhoutte. *Database of Byzantine Book Epigrams (DBBE)*. Version 1.0.0. Zenodo, Feb. 2023. DOI: https://doi.org/10.5281/zenodo.7682523.

[26] Xiaohui Yan, Jiafeng Guo, Yanyan Lan and Xueqi Cheng. "A Biterm Topic Model for Short Texts". In: *Proceedings of the 22nd International Conference on World Wide Web*. New York, NY, USA: Association for Computing Machinery, 2013, pp. 1445–1456. isbn: 9781450320351. DOI: https://doi.org/10.1145/2488388.2488514.

[27] Jianhua Yin and Jianyong Wang. "A Dirichlet Multinomial Mixture Model-Based Approach for Short Text Clustering". In: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: Association for Computing Machinery, 2014, pp. 233–242. isbn: 9781450329569. DOI: https://doi.org/10.1145/2623330.2623715.

[28] Jianhua Yin and Jianyong Wang. "A Text Clustering Algorithm Using an Online Clustering Scheme for Initialization". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York, NY, USA: Association for Computing Machinery, 2016, pp. 1995–2004. isbn: 9781450342322. DOI: https://doi.org/10.1145/2939672.2939841.

[29] Rania Albalawi, Tet Hin Yeap and Morad Benyoucef. "Using Topic Modeling Methods for Short-Text Data: A Comparative Analysis". In: *Frontiers in Artificial Intelligence* Volume 3 (2020). DOI: https://doi.org/10.3389/frai.2020.00042.

[30] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 3982–3992. DOI: https://doi.org/10.18653/v1/D19-1410.

[31] Dejiao Zhang, Feng Nan, Xiaokai Wei, Shang-Wen Li, Henghui Zhu, Kathleen McKeown, Ramesh Nallapati, Andrew O. Arnold and Bing Xiang. "Supporting Clustering with Contrastive Learning". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 5419–5430. DOI: https://doi.org/10.18653/v1/2021.naacl-main.427.

[32] Jiaming Xu, Bo Xu, Peng Wang, Suncong Zheng, Guanhua Tian, Jun Zhao and Bo Xu. "Self-Taught Convolutional Neural Networks for Short Text Clustering". In: *Neural Networks* 88 (2017), pp. 22–31. issn: 0893-6080. DOI: https://doi.org/10.1016/j.neunet.2016.12.008.

[33] Qiang Xu, HaiBo Zan and ShengWei Ji. "A Lightweight Mixup-Based Short Texts Clustering for Contrastive Learning". In: *Frontiers in Computational Neuroscience* Volume 17 (2024). issn: 1662-5188. DOI: https://doi.org/10.3389/fncom.2023.1334748.

[34] Amir Hadifar, Lucas Sterckx, Thomas Demeester and Chris Develder. "A Self-Training Approach for Short Text Clustering". In: *Proceedings of the 4th Workshop on Representation Learning for NLP (RepL4NLP-2019)*. Florence, Italy: Association for Computational Linguistics, 2019, pp. 194–199. DOI: https://doi.org/10.18653/v1/W19-4322.

[35] Md Rashadul Hasan Rakib, Norbert Zeh, Magdalena Jankowska and Evangelos Milios. "Enhancement of Short Text Clustering by Iterative Classification". In: *Natural Language Processing and Information Systems*. Cham: Springer International Publishing, 2020, pp. 105–117. isbn: 978-3-030-51310-8. DOI: https://doi.org/0.1007/978-3-030-51310-8_10.

[36] Kiri Wagstaff, Claire Cardie, Seth Rogers and Stefan Schrödl. "Constrained K-Means Clustering with Background Knowledge". In: *Proceedings of the 18th International Conference on Machine Learning (ICML)*. 2001, pp. 577–584.

[37] Sugato Basu, Ian Davidson and Kiri Wagstaff, eds. "Constrained Clustering: Advances in Algorithms, Theory, and Applications". Chapman and Hall/CRC, 2008.

[38] Eric Xing, Michael Jordan, Stuart J Russell and Andrew Ng. "Distance Metric Learning with Application to Clustering with Side-Information". In: *Advances in Neural Information Processing Systems*. Vol. 15. MIT Press, 2002.

[39] Ian Davidson and S. S. Ravi. "Agglomerative Hierarchical Clustering with Constraints: Theoretical and Empirical Results". In: *Knowledge Discovery in Databases: PKDD 2005*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 59–70. isbn: 978-3-540-31665-7. DOI: https://doi.org/10.1007/11564126_11.

[40] Guoqing Chao, Shiliang Sun and Jinbo Bi. "A Survey on Multiview Clustering". In: *IEEE Transactions on Artificial Intelligence* 2.2 (2021), pp. 146–168. DOI: https://doi.org/10.1109/TAI.2021.3065894.

[41] Yan Yang and Hao Wang. "Multi-View Clustering: A Survey". In: *Big Data Mining and Analytics* 1.2 (2018), pp. 83–107. DOI: https://doi.org/10.26599/BDMA.2018.9020003.

[42] Ramya Elizabeth Thomas and Shamsuddin S. Khan. "Improved Clustering Technique Using Metadata for Text Mining". In: *2016 International Conference on Communication and Electronics Systems (ICCES)*. 2016, pp. 1–5. DOI: https://doi.org/10.1109/CESYS.2016.7889835.

[43] Junyuan Xie, Ross Girshick and Ali Farhadi. "Unsupervised Deep Embedding for Clustering Analysis". In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. PMLR, 2016, pp. 478–487.

[44] James C. Bezdek. "Pattern Recognition with Fuzzy Objective Function Algorithms". Springer Science & Business Media, 1981. DOI: https://doi.org/10.1007/978-1-4757-0450-1.

[45] R. Krishnapuram, A. Joshi and Liyu Yi. "A Fuzzy Relative of the K-Medoids Algorithm with Application to Web Document and Snippet Clustering". In: *FUZZ-IEEE'99. 1999 IEEE International Fuzzy Systems. Conference Proceedings (Cat. No.99CH36315)*. Vol. 3. 1999, 1281–1286 vol.3. DOI: https://doi.org/10.1109/FUZZY.1999.790086.

[46] R. Krishnapuram and J.M. Keller. "A Possibilistic Approach to Clustering". In: *IEEE Transactions on Fuzzy Systems* 1.2 (1993), pp. 98–110. DOI: https://doi.org/10.1109/91.227387.

[47] R. Krishnapuram and J.M. Keller. "The Possibilistic C-Means Algorithm: Insights and Recommendations". In: *IEEE Transactions on Fuzzy Systems* 4.3 (1996), pp. 385–393. DOI: https://doi.org/10.1109/91.531779.

[48] Glenn Shafer. "A Mathematical Theory of Evidence". Princeton University Press, 1976. isbn: 9780691100425. DOI: https://doi.org/10.1515/9780691214696.

[49] Marie-Hélène Masson and T. Denœux. "ECM: An Evidential Version of the Fuzzy C-Means Algorithm". In: *Pattern Recognition* 41.4 (2008), pp. 1384–1397. issn: 0031-3203. DOI: https://doi.org/10.1016/j.patcog.2007.08.014.

[50] Pedro J. García-Laencina, José-Luis Sancho-Gómez and Aníbal R. Figueiras-Vidal. "Pattern Classification with Missing Data: A Review". In: *Neural Computing and Applications* 19.2 (2010), pp. 263–282. issn: 1433-3058. DOI: https://doi.org/10.1007/s00521-009-0295-6.

[51] Zoubin Ghahramani and Michael Jordan. "Supervised Learning from Incomplete Data via an EM Approach". In: *Advances in Neural Information Processing Systems*. Vol. 6. Morgan-Kaufmann, 1993.

[52] R.J. Hathaway and J.C. Bezdek. "Fuzzy C-Means Clustering of Incomplete Data". In: *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 31.5 (2001), pp. 735–744. DOI: https://doi.org/10.1109/3477.956035.

[53] Dongkuan Xu and Yingjie Tian. "A Comprehensive Survey of Clustering Algorithms". In: *Annals of Data Science* 2.2 (2015), pp. 165–193. issn: 2198-5812. DOI: https://doi.org/10.1007/s40745-015-0040-1.

[54] S. Lloyd. "Least Squares Quantization in PCM". In: *IEEE Transactions on Information Theory* 28.2 (1982), pp. 129–137. DOI: https://doi.org/10.1109/TIT.1982.1056489.

[55] J. B. MacQueen. "Some Methods for Classification and Analysis of Multivariate Observations". In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1. Berkeley, CA: University of California Press, 1967, pp. 281–297.

[56] Robert L. Thorndike. "Who Belongs in the Family?" In: *Psychometrika* 18.4 (1953), pp. 267–276. DOI: https://doi.org/10.1007/BF02289263.

[57] Peter J. Rousseeuw. "Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis". In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65. issn: 0377-0427. DOI: https://doi.org/10.1016/0377-0427(87)90125-7.

[58] Daniel Müllner. "Modern Hierarchical, Agglomerative Clustering Algorithms". In: *arXiv Preprint* (2011). DOI: https://doi.org/10.48550/arXiv.1109.2378.

[59] Joe H. Ward Jr. "Hierarchical Grouping to Optimize an Objective Function". In: *Journal of the American Statistical Association* 58.301 (1963), pp. 236–244. DOI: https://doi.org/10.1080/01621459.1963.10500845.

[60] Neo4j. "Neo4j Graph Database". url: https://neo4j.com/ (visited on 27/03/2026).

[61] Lawrence Hubert and Phipps Arabie. "Comparing Partitions". In: *Journal of Classification* 2.1 (1985), pp. 193–218. issn: 1432-1343. DOI: https://doi.org/10.1007/BF01908075.

[62] William M. Rand. "Objective Criteria for the Evaluation of Clustering Methods". In: *Journal of the American Statistical Association* 66.336 (1971), pp. 846–850. DOI: https://doi.org/10.1080/01621459.1971.10482356.

[63] Matthijs J. Warrens. "On the Equivalence of Cohen's Kappa and the Hubert-Arabie Adjusted Rand Index". In: *Journal of Classification* 25.2 (2008), pp. 177–183. issn: 1432-1343. DOI: https://doi.org/10.1007/s00357-008-9023-7.

[64] Alexander Strehl and Joydeep Ghosh. "Cluster Ensembles: A Knowledge Reuse Framework for Combining Multiple Partitions". In: *Journal of Machine Learning Research* 3 (2002), pp. 583–617.

[65] Nguyen Xuan Vinh, Julien Epps and James Bailey. "Information Theoretic Measures for Clusterings Comparison: Is a Correction for Chance Necessary?" In: *Proceedings of the 26th Annual International Conference on Machine Learning*. ICML '09. New York, NY, USA: Association for Computing Machinery, 2009, pp. 1073–1080. isbn: 9781605585161. DOI: https://doi.org/10.1145/1553374.1553511.

[66] "Scikit-Learn: Adjusted Mutual Information". Scikit-learn. url: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_mutual_info_score.html (visited on 18/05/2026).

[67] Neo4j. "Neo4j Bloom". url: https://neo4j.com/bloom/ (visited on 03/04/2026).

[68] Neo4j. "Cypher Query Language". url: https://neo4j.com/docs/cypher-manual/current/ (visited on 01/04/2026).

7

Conclusions and Future Work

"Just like travellers rejoice in seeing their homeland,
so do writers in reaching the end of the book."

— Translation of DBBE type 2148 [1]

Data are generated today at a scale, speed, and diversity that traditional management and analysis techniques struggle to handle, the defining characteristics of what is commonly referred to as big data. The challenges extend beyond sheer volume: the velocity at which data are generated and must be processed, the variety of forms in which data appear, and the veracity concerns surrounding their trustworthiness complicate the derivation of meaningful insights. Textual data, of all lengths and formats, constitute a considerable portion of these newly generated data. Short texts in particular pose difficulties for standard analytical techniques, as their brevity provides less context than Natural Language Processing (NLP) techniques typically require. Some of these short texts are also subject to veracity concerns, such as incomplete, inconsistent, or completely absent metadata, which further amplify the challenges, as the reliability of any analysis becomes uncertain. Even more difficulties arise when such texts are written in a low-resource language, for which annotated data, linguistic resources, and pre-trained models are scarce. This scarcity renders many established techniques ineffective or inapplicable, as discussed in Chapter 1.

In response, this thesis develops computational techniques for the management and analysis of short texts. These techniques operate without language-specific resources, take into account the structural properties of the texts, and explicitly address the veracity concerns that arise from incomplete metadata. Rather than a collection of isolated methods, the contributions form a coherent whole, in which each component builds upon the results of the preceding one, from the representation of the texts, through the computation of textual similarity and its enrichment using metadata, to the analytical techniques that operate on the resulting scores.

The remainder of this seventh chapter is structured as follows. The main contributions of this thesis, and how they can be used together, are summarised in Section 7.1. In Section 7.2, the research questions formulated in Section 1.4 are revisited. For each question, the underlying problem is restated, the proposed methods and their

novelty are described, an answer is formulated, and the limitations of the presented solutions are discussed. The practical applicability of the developed techniques for the Database of Byzantine Book Epigrams (DBBE) and the broader Interconnected Texts project is examined in Section 7.3, which sets out how the contributions can support the manual curation process and complement the other sub-projects. Section 7.4 positions the proposed techniques within a rapidly evolving landscape increasingly shaped by Large Language Models (LLMs), considering both the tasks in which such models excel and the settings in which the developed methods retain distinct advantages. Finally, Section 7.5 outlines directions for future research, synthesising the perspectives identified in the individual chapters and complementing them with new ones that consider the work as a whole.

7.1 Summary of Contributions

Before answering the research questions individually, we briefly recall the research novelties developed in this thesis, whose value lies as much in how they complement each other as in what each achieves on its own. Chapter 2 has proposed a graph-based data model that decomposes short texts into hierarchically organised discourse units and stores them, together with their contextual metadata, in a Labelled Property Graph (LPG), which can be queried, visualised, and analysed with modern graph databases. This representation provides the foundation for Chapter 3, which has developed a novel hierarchical orthographic similarity measure to quantify the resemblance between textual elements at every level of the hierarchy. This measure extends the LPG with fuzzy edges whose membership grades represent the computed similarities. Chapter 4 has then enriched this textual similarity with metadata-based similarities through an adapted Ordered Weighted Averaging (OWA) fusion mechanism. For each pair of text documents, the highest textual elements in the hierarchy, this yields a similarity-confidence pair that records both the degree of resemblance and the reliability of that assessment given the available metadata. Building on these pairs, formalised as L-grades, Chapter 5 has proposed novel ranking techniques that consider the trade-off between satisfaction and confidence according to stakeholder preferences. In turn, Chapter 6 has investigated novel clustering techniques that group short texts based on these aggregated similarities while making the reliability of each cluster assignment explicit.

All of these techniques build on the graph-based representation. The similarity and fusion steps enrich it with new edges that store the computed similarity and confidence scores, on which the ranking and clustering techniques then operate, so that these scores remain available for querying, visualisation, and further analysis. The graph representation itself makes these scores directly queryable, so that patterns, interconnections, and groupings can be retrieved and visualised alongside the texts and metadata they were computed from. When incorporated into a research tool or a Decision Support System (DSS), these techniques are intended to support rather than replace the manual curation or analysis process, and every suggestion is subject to expert review before being applied. This human-in-the-loop approach relies on the techniques being interpretable and veracity-aware, so that domain experts can understand the basis for each suggestion and better assess its reliability.

Throughout this work, the Database of Byzantine Book Epigrams (DBBE) [2], a corpus of short historical poems written in Byzantine Greek, served as the primary use case and proof of concept for instantiating and evaluating these techniques. This corpus exhibits all the challenges that motivated this thesis, combining short and

orthographically inconsistent texts in a low-resource language with frequently incomplete contextual metadata. Although the developed techniques arose from the challenges of this specific corpus, none of them depends on it. They apply equally to other collections of short texts (such as social media posts, product reviews, or song lyrics) that may be highly interconnected and that carry heterogeneous and often incomplete metadata, regardless of whether those texts are written in a low-resource language.

7.2 Answers to the Research Questions

We now return to the four research questions that motivated this thesis, formulated in Section 1.4. The four questions are addressed in turn, each in its own subsection: the graph-based representation (RQ 1), the hierarchical orthographic similarity measure (RQ 2), the context-aware enrichment with metadata (RQ 3), and the analysis of the represented texts (RQ 4), the last of which is divided into three subquestions on querying, ranking, and clustering. Within each subsection, we restate the underlying problem, summarise the proposed methods and their novelty, formulate an answer supported by the results of the corresponding chapters, and set out the limitations that remain.

Throughout the answers that follow, the reported results are assessed against expert-curated groupings of the corpus, namely the epigram types and verse variants in the DBBE. These groupings represent one scholarly interpretation rather than an objective standard, so agreement with them shows that a method can capture the same structure, whereas disagreement is not automatically an error. The techniques are intended to support exploration and discovery rather than to replace expert judgement, so alignment with the expert grouping is a desirable rather than a necessary property. It is, however, not the only value a method can offer. Because these techniques are configurable, the same settings that align a method with this grouping can be adjusted for other research questions and analytical goals, on this corpus or on a completely different one. A different parameter or weight choice may then surface alternative groupings that are themselves legitimate interpretations. Where a method has parameters tuned to the expert grouping, the reported results reflect a best-case configuration rather than a generalisable one, showing that the technique can be aligned with the interpretation currently available without claiming that the same parameters are optimal in other settings.

7.2.1 Data Management and Textual Representation

The first research question of this thesis concerned the storage and representation of short texts:

> **RQ 1:** To what extent can graph databases provide a suitable and efficient data model for representing short texts in a semi-structured way, supporting further analysis and the identification of patterns and interconnections at multiple levels of granularity?

Before similarity between texts can be computed or any analysis can be performed, the data must be stored in a way that adequately reflects the semi-structured nature of texts, their numerous interconnections, and the heterogeneity of their associated metadata. As discussed in Section 1.4.1, relational databases often fall short in this respect. Their rigid schema requirements are ill-suited to textual data and contextual metadata that

vary greatly in structure and availability, and representing dense networks of interconnections between texts in tabular form leads to complex schemas and time-consuming join operations. A more flexible data model is therefore required.

In response, Chapter 2 proposes a graph-based data model built on an LPG. The model decomposes each text into hierarchically organised discourse units, representing the smallest units as individual nodes and larger units as subgraphs composed of their constituent elements. In the case of the DBBE, the lowest-level elements are individual words, followed by verses, and finally the epigrams themselves as the highest-level textual elements. Identical textual content occurring in different texts is represented by a single shared node, except at the top level, where each full text retains its own node to remain uniquely identifiable. Contextual metadata are modelled as separate nodes connected to the texts they describe and are reused across texts that share them. Additional textual properties, such as Part-of-Speech (POS) tags or lemmas, where available, are attached directly to the textual nodes. A construction algorithm, including a corpus-specific preprocessing step, splits each text document into its hierarchical components and transforms it into this graph-based representation.

The novelty of this model lies in the combination of hierarchical decomposition and node merging within a general-purpose, language-independent representation. Whereas flat text representations reduce texts to sequences of characters or tokens, the proposed model preserves the internal organisation of each text, and the node-merging mechanism transforms shared content from something that must be discovered through pairwise comparison into an inherent property of the graph topology. The schemaless nature of the underlying LPG further accommodates metadata of varying structure and availability.

The findings of Chapter 2 show that a graph database provides a suitable and efficient data model for representing short texts in a semi-structured way. The illustrative examples on the DBBE corpus demonstrate the identification of frequently shared textual elements, the visual exploration of interconnections, and declarative pattern matching, all at multiple levels of granularity. The representation also serves as the foundation on which the similarity measure of Chapter 3 and the analytical techniques of the subsequent chapters are built, showing that it supports further analysis as well as direct exploration. Its storage efficiency is a direct result of the node-merging mechanism: each distinct textual element and each metadata value is stored only once, so recurring content is not duplicated and the interconnections between texts are captured implicitly by shared nodes rather than being explicitly stored as a quadratically growing set of relations. This compact structure also supports efficient retrieval, since related content can be reached by traversing shared nodes and containment relationships directly, rather than through the costly joins a relational schema would require, although query and computation speed were not evaluated quantitatively.

Several limitations should be noted, however. The node-merging mechanism relies on exact string equality after preprocessing, causing near-identical elements to remain unconnected, and the preprocessing steps that mitigate this are rigid and corpus-specific. The model captures only orthographic features and no semantic relationships, and its evaluation is confined to a single corpus. Finally, the model is intended to complement rather than replace an existing database within a polyglot persistence architecture, whose synchronisation and Extract, Transform, Load (ETL) overhead this thesis did not address.

7.2.2 Short Text Similarity

The second research question concerned the comparison of short texts once represented in this graph-based model:

> **RQ 2:** How can an orthographic similarity measure be designed to capture similarities between short texts effectively at multiple levels of granularity, while exploiting the graph representation to account for their structural properties?

Comparing short texts is difficult precisely because of their brevity: they offer little of the context on which semantic NLP techniques rely, and for low-resource languages the annotated data and pre-trained models these techniques require are largely unavailable. Orthographic similarity, which compares texts by their written form alone, remains applicable in this setting, but conventional orthographic measures reduce each text to a single flat string and therefore ignore the internal structure that the graph model of Chapter 2 makes explicit. A measure is needed that operates on this structural representation, compares texts at more than one level of granularity, and depends on no language-specific resources.

To this end, Chapter 3 proposes a hierarchical orthographic similarity measure that operates in a bottom-up fashion over the graph-based representation. At the lowest level of the hierarchy, a string-based measure compares the textual content of the smallest elements using an edit distance inspired by the Damerau-Levenshtein distance, normalised into a similarity score, with a separate cost parameter for each edit operation. At every higher level, a node-based measure compares two elements through an edit distance over their ordered lists of constituent text elements, represented by child nodes, in which the cost of substituting one child for another is derived from the similarity already computed between those children at the level below. This propagation of lower-level scores upward is the central, novel mechanism of the measure, ensuring that the similarity between two complete texts reflects both the resemblance and the arrangement of their constituent parts. The resulting scores are stored as fuzzy similarity edges in the graph, so that the resemblance between any pair of comparable elements becomes directly accessible as part of the representation, and can be inspected, analysed, and queried at any level of the hierarchy.

The novelty of the measure lies in the hierarchical application of edit distances to a graph-structured text representation. Whereas a conventional orthographic measure yields a single score for an entire text, the proposed measure produces a score at every level of the hierarchy, each derived from the scores between the elements at the level below. Because these intermediate scores remain available for inspection, the measure is more interpretable, since a similarity between two texts can be traced to the specific verses or words that account for it, which in turn makes the downstream analyses that build on it easier to explain. Relying only on the written form and structure of the texts, the technique requires no language-specific resources and can be applied to corpora in low-resource languages.

The findings of Chapter 3 show that an effective orthographic similarity measure can indeed be designed along these lines. Evaluated on the DBBE corpus against expert-curated groupings, the measure reaches a peak F_1-score of 0.9704 on the verse subset and 0.7942 on the more structurally variable epigram subset, and it substantially outperforms an existing two-level string-matching baseline on both. This margin reflects the two

properties the research question emphasises. The measure compares texts at several levels of granularity rather than as a single string, and it exploits the graph representation directly, since the node-based edit distance operates on the ordered child nodes of each element, so that containment and order shape the resulting scores.

Each of these results was obtained with one of three cost configurations, each specifying different edit operation costs at every level of the hierarchy. The costs can be tuned by domain experts to reflect either the orthographic characteristics of the corpus or the specific research question at hand, allowing the measure to be used as an exploratory tool adapted to particular analytical goals. Producing and storing a score at every level also proves valuable in practice. Because the scores reside on the graph edges, they can be queried directly, for instance by retaining only the edges above a chosen similarity threshold to reveal groups of related texts, while the interconnections they expose can be explored visually within the graph database.

Several limitations remain. The measure is purely orthographic and captures no semantic relationships, so texts that are related in meaning but not in form are not recognised as similar. Computing pairwise scores at each level requires a quadratic number of comparisons in the number of elements, which may become a bottleneck for very large corpora, although node merging reduces how many elements must be compared. The cost of each individual comparison stays low, since the hierarchical decomposition applies every edit distance to a short sequence of child nodes rather than to an entire text, so the length that drives that cost is a verse or a word rather than the full text. Its strong performance also depends on selecting a cost configuration suited to the corpus, which draws on domain knowledge rather than being determined automatically. Finally, the evaluation is confined to a single corpus whose expert-curated groupings are themselves imperfect, containing incomplete texts and semantic variants.

7.2.3 Context-Aware Short Text Similarity

The third research question concerned the enrichment of the textual similarity measure with contextual metadata:

> **RQ 3:** How can contextual metadata complement a textual similarity measure, even when those metadata are missing or incomplete, such that the trustworthiness of the result is made explicit alongside the similarity score for downstream analysis?

Exploiting the metadata associated with short texts is often complicated by concerns about their veracity, since in practice they may be incomplete, imprecise, or entirely missing, and to an uneven extent across a corpus. The two conventional responses both come with drawbacks. Marginalisation discards incomplete fields from all records, even when they are available for some, whereas imputation fills in the missing values under assumptions that may not hold, thereby introducing additional uncertainty. Combining several similarities into a single score between two texts, moreover, hides the extent to which metadata are available, so that transparency is reduced and the trustworthiness of that score remains implicit. A measure is therefore needed that draws on all available metadata without imputation or marginalisation, and that makes the effect of any missing metadata explicit in the result.

Addressing this need, Chapter 4 proposes a context-aware and responsible similarity measure. For each pair of texts, the textual similarity and a similarity computed for each available metadata field are combined into a single aggregated score through an adapted Ordered Weighted Averaging (OWA) operator, which, in its standard form, weights the similarities by their rank rather than by the field they come from. The adaptation handles missing metadata by aggregating only the available similarities and renormalising their weights accordingly, so that the aggregated score remains meaningful whatever the extent of the missing information, and coincides with the standard operator when nothing is missing. Alongside this similarity score, a confidence score derived from the share of the weight vector left unused due to the missing metadata quantifies how much of the intended information contributed, so that the two together form a similarity-confidence pair recording both the resemblance between two texts and the trustworthiness of that assessment given the available metadata.

The novelty of the approach lies in pairing an adapted OWA fusion mechanism with an explicit confidence score. Rather than being a by-product of the aggregation, the confidence score is an independent output that downstream techniques can reason over jointly with the similarity, so that a high-confidence match can be distinguished from one inferred from little evidence. Because the aggregation is defined over generic similarity inputs, the mechanism is independent of any particular textual or metadata measure and can be used with any combination of similarity scores.

Chapter 4 answers this research question through a fusion mechanism that complements the textual measure in three respects. It enriches the textual score by aggregating it with the metadata similarities under the adapted OWA operator, distinguishing pairs that orthography alone would treat as matches and supplying additional context where the orthography is ambiguous. It remains applicable when metadata are missing or incomplete, since the aggregation uses only the available similarities and renormalises their weights, without imputation or the exclusion of partial records. And it reflects trustworthiness explicitly, pairing each aggregated score with a confidence value derived from the weight left unused due to the missing metadata, so that downstream analysis can take the reliability of each score into account. On the DBBE corpus, the measure does not outperform the purely textual baseline at recovering the expert-curated types, which is consistent with those types being defined primarily by orthography. At lower thresholds of the α-cut evaluation, however, it substantially reduces the false positives the textual baseline admits, yielding higher precision at the same near-perfect recall, which confirms that the metadata contribute information beyond orthography.

These results were obtained using a best-case weight vector, namely the weights that performed best on the evaluation subset. It is not a generalisable configuration; rather, it is intended to show what the measure can achieve when its weights are set well. Because incorporating the metadata shifts the target away from a grouping defined by text alone, this best-case vector does not score as highly as the purely textual baseline, yet it still recovers a grouping both close to the expert-curated types and informed by the metadata. The weights are instead meant to be configured through expert knowledge, steering the same measure towards other groupings and research questions, which makes it an exploratory tool rather than a single-purpose measure.

A number of limitations should be noted. The confidence score captures only the extent of missing metadata, and not other veracity concerns such as uncertainty or inconsistency in the recorded values. Because the OWA weights attach to the ranked similarities rather than to individual fields, the weight vector says nothing about

which metadata fields matter most in general, even though the contribution of each criterion to an individual score remains traceable. The gains from the metadata also depend on how much metadata a corpus records and how discriminative they are. As before, the evaluation rests on a single corpus, so the extent to which these findings hold for other domains, languages, and metadata structures remains open.

7.2.4 Analysis of Short Texts

The fourth research question concerned the analysis of short texts once represented by a graph and enriched with similarity and confidence scores:

> **RQ 4:** To what extent can the graph-based representation, orthographic similarity measures, and context-aware similarity scores developed in this thesis support the analysis of short texts?

The graph-based representation, the orthographic similarity measure, and the context-aware similarity scores with their confidence values are not the end goal, but are intended to support the analysis of a corpus of short texts. The fourth research question asks how far they support that analysis, framing the answer around three concrete objectives: querying the corpus for patterns and interconnections, ranking results by both similarity and confidence, and clustering texts into meaningful groups. Each is addressed below, drawing on the querying examples of Chapters 2, 3 and 4, the ranking techniques of Chapter 5, and the clustering strategies of Chapter 6.

Querying

> **RQ 4.A:** How can (flexible) query answering techniques be applied to a graph-based data model of short texts to support the discovery of patterns and interconnections, with or without considering similarity or metadata?

The first analytical objective concerned the discovery of patterns and interconnections through querying, and was addressed across several chapters of this thesis, as part of their practical applicability sections, rather than in a chapter of its own. Querying is enabled by storing the texts, their metadata, and the computed scores together in a graph database, in particular a Neo4j instance [3], which supports querying through the Cypher query language [4], visual exploration through Neo4j Bloom [5], and advanced graph-based analysis through the Neo4j Graph Data Science (GDS) library [6]. The practical applicability section of Chapter 2 shows the identification of frequently shared textual elements, the visual exploration of interconnections, and declarative pattern matching over the texts, their metadata, or both, all at multiple levels of granularity. The similarity edges of Chapter 3 extend these capabilities by allowing users to retrieve partially similar or structurally related texts whose similarity exceeds a chosen threshold, rather than exact matches alone. The confidence scores of Chapter 4 introduce an additional dimension, allowing results to be filtered or ordered by the aggregated similarity and the confidence attached to it. Together, these scores allow a query to return not only patterns and interconnections but also groupings of texts, selected by a chosen similarity threshold, a confidence threshold, or both. The clustering outputs of Chapter 6 can themselves be stored in the graph, so that they too remain available for querying and visualisation. Finally, the ranking techniques of Chapter 5 allow the results of a

query to be ordered, when a result is a similarity-confidence pair rather than a single score, by using one of the proposed strategies to reflect the stakeholder's priorities.

The novelty here lies in the combination of a graph-based representation with similarity and confidence scores, which together support (flexible) querying of short texts and their metadata. These capabilities together answer the subquestion by supporting a wide range of querying options for the discovery of patterns and interconnections at multiple levels of granularity, with or without similarity and metadata.

The evidence for this answer is demonstrative, through practical examples on the DBBE corpus, rather than measured against a benchmark. The querying process, moreover, inherits the limits of the scores stored in the database, so a query is only as reliable as the scores behind it, which are affected by the orthographic similarity measure, by missing or incomplete metadata, and by the choice of costs and weights.

Ranking

> **RQ 4.B:** How can query results and candidate interconnections be ranked based on both their similarity score and associated confidence score to provide more informative and transparent results for domain experts?

Once each result of a similarity computation or a query carries both a satisfaction grade and a confidence grade, presenting these results in a meaningful order is no longer a matter of sorting on a single number. The two grades are orthogonal: a highly similar pair supported by little metadata may or may not be preferred to a moderately similar one supported by a great deal of metadata, and which of the two is preferable depends on the stakeholder rather than on the data. Standard information-retrieval ranking sorts on a single relevance score, and multi-criteria decision methods, while handling several criteria, do not capture the particular relationship in which one grade qualifies the trustworthiness of the other. A ranking approach is therefore needed that orders results on both scores at once, reflects the stakeholder's priorities or preferences, and remains interpretable to domain experts who need not know the underlying ranking method.

In response, Chapter 5 studies ranking techniques for L-grades, the ordered pairs of a satisfaction grade and a confidence grade produced by the measure of Chapter 4. Because no single ordering of such pairs is natural, except where one L-grade fully dominates the other, three complementary strategies are proposed. Lexicographic ordering prioritises one grade, chosen by the stakeholder, and breaks ties with the other, yielding a total order. Global preference functions instead combine the two grades into a single value, either non-weighted, as the product of the two, or weighted, through a weight by which the stakeholder tunes the relative importance of confidence and satisfaction, resulting in total preorders. These are complemented by a distance measure that quantifies how strong a preference is, an approximate equality comparator for L-grades that lie close together, and thresholds that discard L-grades falling below a minimum satisfaction or confidence from the ranking.

The novelty lies in a systematic set of ranking techniques designed specifically for satisfaction-confidence pairs, each accompanied by a formal proof that it constitutes a total order or total preorder and that it is, outside a small number of identified boundary cases, consistent with the cases in which one L-grade fully dominates the

other. Because the two grades within the pair remain available throughout, the rankings are interpretable, and a domain expert can see why one result outranks another and which grade drives the decision.

These techniques answer the subquestion by ranking on both components of each similarity-confidence pair, and by making the trade-off between them an explicit, adjustable choice rather than a fixed rule. Their applicability is demonstrated on the DBBE corpus in two use cases, ranking the context-aware similarity-confidence pairs of Chapter 4 from complementary perspectives, and ranking the results of a geographical querying technique in which each retrieved epigram carries a satisfaction and a confidence grade derived from its recorded location.

The contribution is formal and demonstrative rather than evaluated against a benchmark, since no ground-truth ordering is available against which such a ranking could be scored. It also places the choice of strategy and weight with the stakeholder, and assumes that the grades being ordered are themselves meaningful, so that a ranking is only ever as good as the grades being ranked.

Clustering

> **RQ 4.C:** How can short texts be clustered based on context-aware similarity scores and confidence scores, in a way that handles missing metadata gracefully and makes the reliability of cluster assignments explicit?

The final objective was to cluster texts in an unsupervised manner by their context-aware similarity, while handling missing metadata gracefully and making the reliability of each grouping explicit. Clustering short texts is difficult in itself, given how little information each text carries, and the requirement to account for missing metadata rules out treating the similarity score as a single clustering criterion, since the confidence score is an essential qualifier of its trustworthiness. A clustering approach is therefore needed that operates on the aggregated context-aware similarity, tolerates the missing metadata behind it, and reports how far each assignment can be trusted.

To that end, Chapter 6 applies and compares three clustering algorithms, namely K-Medoids, Hierarchical Agglomerative Clustering (HAC), and Fuzzy C-Medoids (FCMdd), using the aggregated similarity of Chapter 4, which already handles missing metadata through its adapted OWA aggregation. The accompanying confidence scores play two roles: they can be incorporated into the clustering itself, or used afterwards to make the trustworthiness of each assignment explicit by colouring the cluster assignments according to their confidence, so that weakly supported clusters and individual assignments stand out for expert review. When incorporated into the clustering, the dissimilarity is derived from either the product of the similarity and confidence or a weighted combination of the two, with the weight reflecting the stakeholder's preference for similarity over confidence.

The novelty is twofold: the clustering operates on the fused context-aware similarity, and the reliability of the result is treated explicitly, since the confidence is either integrated into the clustering process or used to visualise the trustworthiness of each assignment. This distinguishes the approach from conventional short-text clustering, which neither incorporates heterogeneous and incomplete metadata in this way nor exposes the reliability of its assignments in the light of the available metadata.

Using the DBBE corpus, clustering on the context-aware similarity improves the recovery of the expert-curated types for K-Medoids, leaves FCMdd close to its textual baseline, and does not help HAC. When the confidence is incorporated into the clustering process, the outcome depends on how it is combined with the similarity. Under the non-weighted combination, the two medoid-based methods, K-Medoids and FCMdd, stay close to their confidence-free results, whereas under a weighted combination at a moderate weight they improve slightly, with FCMdd reaching the highest F_1-score recorded in Chapter 6. In both cases HAC draws no benefit, and weighting confidence too heavily degrades all three. The measured gains are modest and heavily dependent on the configuration, in part because the expert types are based primarily on textual characteristics, so adding context and confidence shifts the target even further than context alone. This shift is not necessarily a drawback, since the expert types are only one interpretation of the corpus, and the context-aware similarity may surface alternative but equally legitimate groupings that the expert types do not capture. For exploratory analysis in particular, the ability to reveal such groupings is a strength rather than a weakness, and the confidence scores let the user either shape the clustering or assess to what extent each assignment can be trusted. The clearer contribution is therefore not a closer agreement with the expert types but context- and reliability-aware clustering, the ability to cluster under incomplete metadata and to make the confidence in every assignment explicit. These clustering techniques thereby answer the subquestion, offering several ways to cluster short texts by their context-aware similarity while handling missing metadata and making the reliability of each assignment explicit.

The reported results rest on best-case weight vectors and other method-specific parameters, tuned separately for each algorithm. The same argument applies as in the previous sections: these results show what can be achieved when the parameters are set well, not the performance of a generalisable configuration. The point is stronger here, since incorporating context and confidence shifts the target away from the expert types even more, so agreement with those types is no longer the best measure of success, though it still shows to what extent the proposed techniques can recover the same structure. What matters more is that the techniques let the user explore the corpus in a context- and reliability-aware manner, surfacing alternative groupings that may be of interest to domain experts.

Taken together, the three objectives show that the components developed in this thesis support the analysis of short texts to a meaningful extent, through flexible querying, veracity-aware ranking, and context- and reliability-aware clustering, each of which keeps the underlying uncertainty visible rather than hiding it. The gains are clearest from a qualitative perspective, since the techniques let domain experts explore the corpus more richly than conventional methods allow and surface alternative groupings of interest. From a purely quantitative perspective they are more modest, since agreement with the expert-curated types is not always improved, nor is it expected to be.

7.3 Practical Applicability for the Database of Byzantine Book Epigrams

Although the techniques in this work were deliberately designed to be generally applicable to the analysis of short texts, they were conceived as part of the Interconnected Texts project, described in Section 1.3.3, and with the DBBE in mind. In this setting, the individual contributions can work together in a pipeline to enable

either exploratory or targeted analysis of the corpus, and to support domain experts in pursuing their research objectives. This pipeline comprises four stages: (i.) the selection and integration of the epigrams and the metadata to be examined (Chapter 2); (ii.) the configuration of the edit operation costs and the computation of the textual similarity measure (Chapter 3); (iii.) the configuration of the weight vector and the computation of the context-aware similarity measure, given a choice of metadata and a similarity measure for them (Chapter 4); and (iv.) the analysis of the results using the ranking and clustering techniques proposed in this work (Chapters 5 and 6), together with the querying and visualisation capabilities of the underlying graph database system. Using this pipeline, researchers can pursue a particular research question that was previously difficult to address manually or through the relational database or its web interface alone. It can uncover new interconnections under many different configurations, reveal patterns, rank relevant or important text elements, propose new groupings through clustering, and flag metadata that are missing for a text but present for most others in its cluster, all in an interpretable and veracity-aware manner that supports the manual curation of the DBBE. The graded similarity scores and fuzzy cluster memberships further allow researchers to explore the corpus in a nuanced way, rather than being limited to a single hard assignment.

The contributions of this thesis are, however, the outcome of only one of the four sub-projects of the Interconnected Texts project. As such, these tools can provide a valuable layer of analysis for the other sub-projects, be integrated into them, or draw on the results of those sub-projects to improve their own analysis. For example, the integrated modelling of contextual metadata supports the manuscript culture sub-project in relating epigrams to their geographical, chronological, and material context [7], while the querying and pattern-matching capabilities of the graph model can support the linguistics sub-project in identifying formulaic patterns and their variations [8]. The NLP sub-project could in turn complement the methods of this work by providing textual metadata, such as lemmatisation or POS tags, to enrich the similarity measure and the subsequent analysis, or additional similarity measures between texts that could be integrated into the OWA-based fusion. The tools presented in this work are thus only one component of a broader, multi-disciplinary effort, and their practical applicability is enhanced by integration with the other sub-projects and the use of their results.

Within the Interconnected Texts project, future work could focus on integrating the sub-projects into a single research tool, in which the contributions of this thesis are complemented by the results of the others, and in which researchers can explore the corpus through adjustable parameters, including similarity thresholds, weight vectors, confidence weights, and cluster counts. Moreover, a graph-based interface could enhance the current DBBE web platform with additional visualisation and exploration capabilities, and a natural language interface could lower the barrier for non-technical researchers to use the tools developed in this thesis. Integrating the current relational database with the graph-based representation and the similarity scores, and keeping them synchronised through an ETL process as texts are added or modified, nonetheless remains a challenge that requires further research.

7.4 Short Text Analysis in the Age of Large Language Models

Over the course of this work, Large Language Models (LLMs) have emerged and profoundly changed the field of NLP and, more broadly, the way in which many researchers interact with textual data [9]. Tasks that previously

required dedicated methods, including semantic similarity estimation, classification, and text generation, can now often be performed by prompting a general-purpose model. Their generality and ease of use raise the question of whether the specialised techniques developed in this thesis remain relevant.

The field of short text analysis is indeed being transformed by LLMs. While analysing short texts has traditionally been difficult in NLP research, because existing methods were not well suited to their brevity and ambiguity [10], an entire research field has since emerged around analysing, clustering, and classifying short texts with LLMs. This work is supported by a growing body of publications and open-source tools [11, 12]. These efforts show promising results and are accessible to a far wider range of researchers than specialised methods are. When low-resource languages are involved, however, the situation changes significantly. Attributed to the limited availability of training data, large performance gaps remain between high- and low-resource languages, and the outputs of LLMs may be unreliable or even misleading [13, 14]. These outputs are also often opaque and difficult to verify or interpret, given the black-box nature of these models and the limited explainability of their decision-making [15, 16]. The techniques developed in this thesis, by contrast, are deterministic, and every score they produce can be traced to its contributing factors: the hierarchical similarity to its lower-level scores, the aggregated similarity to its textual and contextual inputs, and the confidence score to the weight left unused due to the missing metadata. Current LLM-based techniques therefore do not yet address the specific challenges of this setting, though future research may change that.

At the same time, it would be short-sighted to frame LLMs solely as being in competition with this work. One limitation of the techniques developed here is that they are not readily accessible to non-technical researchers, requiring a degree of expertise to use effectively, from formulating Cypher queries to selecting weight vectors and thresholds, and interpreting the results. Rather than requiring dedicated interfaces to be built from the ground up, LLMs could be used to lower this barrier, translating natural language questions into database queries, explaining analytical results in accessible terms, and guiding researchers through parameter choices. LLMs can equally contribute to the analysis itself. Because the aggregation of Chapter 4 is defined over generic similarity inputs, a semantic similarity derived from an LLM could be added as a further input, supplying the semantic dimension the orthographic measure lacks while the framework retains its transparent and veracity-aware handling of the combined scores.

7.5 Future Work

Each contribution chapter closed with directions for future work specific to it, and the extensions tied directly to the Interconnected Texts project were set out in Section 7.3. This final section brings those together and identifies directions that span the entire thesis, either because they are relevant to several contributions or because they become apparent only when the work is considered as a whole.

A first direction concerns the methods themselves, and how they can be extended, improved, or made more robust. The hierarchical similarity measure of Chapter 3 could adopt other orthographic similarity measures at its individual levels, such as the Jaro-Winkler similarity, or could integrate linguistic information such as lemmatisation or POS tags into the similarity computation. Beyond the LLM-derived semantic scores discussed

in Section 7.4, the aggregation of Chapter 4 could be enriched with further similarity inputs, such as measures computed over lemmatised or linguistically annotated text. The proposed OWA aggregation could also be further developed to take similarity-confidence pairs as inputs, ordered by the ranking techniques of Chapter 5, so that confidence and veracity enter the aggregation itself. This would allow veracity to inform the similarity computation directly, for instance by giving less weight to a semantic similarity derived from an LLM when a low-resource language is involved. All of these extensions would be compatible with the existing framework and would expand and improve the set of tools available to researchers, while the interpretability and veracity-aware nature of the techniques would remain intact.

A second direction concerns scalability and generalisation. The pairwise computation of similarities grows quadratically with corpus size, so approximation or optimisation strategies, potentially combined with the indexing and analytics facilities of the graph database, should be investigated for very large corpora. Furthermore, the results in this work rest on a single corpus, the DBBE, and a single expert interpretation of it, the expert-curated types. Future research should apply the techniques to other corpora of short texts, in other low-resource languages and beyond, and evaluate them against other ground truths, to establish how far the findings generalise and where they need adjustment. Given more ground truths, the edit operation costs, the OWA weights, and the clustering parameters could also be learned automatically in a generalisable manner, rather than set by hand or tuned against a single ground truth as they are here. An overarching direction, already outlined in Section 7.3, is the integration of the contributions of this thesis into a single research tool with adjustable parameters, possibly incorporated into the DBBE web platform. Realising this would require the ETL synchronisation process and the user-friendly interface described there. Closer collaboration with domain experts could further refine such a tool, tuning the similarity measures, the aggregation, and the ranking and clustering strategies to better reflect their research questions and priorities, ultimately yielding a higher-quality research tool or DSS.

Taken as a whole, this thesis set out to make short, interconnected texts in a low-resource language easier to analyse computationally. It did so by representing them in a graph, computing a similarity measure that captures their orthographic resemblance, enriching that measure with contextual metadata, and supporting the analysis of the resulting scores through querying, ranking, and clustering. The techniques it developed, from representation through similarity and context-aware fusion to ranking and clustering, form a coherent and interpretable pipeline that keeps the domain expert in the loop and the underlying uncertainty visible. The directions above show how much remains to be explored, but the groundwork is laid for a way of analysing such corpora that is at once flexible and transparent.

Concluding Remarks

Several epigrams from the DBBE, often used in examples throughout this work, were written by scribes to express their joy of finishing a manuscript, similar to the feeling of a traveller reaching their destination after a long journey. After years of research and months of writing, the author feels a similar joy at the end of this work.

References

[1] Kristoffel Demoen, Gilbert Bentein, Klaas Bentein, Floris Bernard, Julián Bértola, Julie Boeten, Mathijs Clement, Cristina Cocola, Eline Daveloose, Sien De Groot, Pieterjan De Potter, Ilse De Vos, Krystina Kubina, Hanne Lauwers, Paulien Lemay, Renaat Meesters, Marjolein Morbé, Delphine Nachtergaele, Marthe Nemegeer, Joachim Nielandt, Mace Ojala, Lisa-Lou Péchillon, Raf Praet, Rachele Ricceri, Anne-Sophie Rouckhout, Jeroen Schepens, Febe Schollaert, Lev Shadrin, Nina Sietis, Dimitrios Skrekas, Colin Swaelens, Maria Tomadaki, Sarah-Helena Van den Brande, Merel Van Nieuwerburgh, Lotte Van Olmen, Noor Vanhoe and Nina Vanhoutte. *Database of Byzantine Book Epigrams (DBBE)*. Version 1.0.0. Zenodo, Feb. 2023. DOI: https://doi.org/10.5281/zenodo.7682523.

[2] Rachele Ricceri, Klaas Bentein, Floris Bernard, Antoon Bronselaer, Els De Paermentier, Pieterjan De Potter, Guy De Tré, Ilse De Vos, Maxime Deforche, Kristoffel Demoen, Els Lefever, Anne-Sophie Rouckhout and Colin Swaelens. "The Database of Byzantine Book Epigrams Project : Principles, Challenges, Opportunities". eng. In: *Journal of Data Mining and Digital Humanities* On the Way to the Future of Digital Manuscript Studies (2023), p. 41. issn: 2416-5999. DOI: https://doi.org/10.46298/jdmdh.10244.

[3] Neo4j. "Neo4j Graph Database". url: https://neo4j.com/ (visited on 27/03/2026).

[4] Neo4j. "Cypher Query Language". url: https://neo4j.com/docs/cypher-manual/current/ (visited on 01/04/2026).

[5] Neo4j. "Neo4j Bloom". url: https://neo4j.com/bloom/ (visited on 03/04/2026).

[6] Neo4j. "Neo4j Graph Data Science". url: https://neo4j.com/product/graph-data-science/ (visited on 10/05/2026).

[7] Eleonora Lauro. "Alongside the Text : Byzantine Metrical Paratexts in Gospel Manuscripts from Medieval Southern Italy". In: *Intersection : Interdisciplinary Approaches to New Testament Text and Manuscript Studies*. Hendrickson, 2025, pp. 175–189. isbn: 9798400508349.

[8] Kyriaki Giannikou. "Formulaic Language in Byzantine Book Epigrams : A Typology Grounded in Ritually-Charged Pragmatic Functions". In: *GREEK ROMAN AND BYZANTINE STUDIES* 66.1 (2026), pp. 1–41. issn: 0017-3916.

[9] Fiona Fui-Hoon Nah, Ruilin Zheng, Jingyuan Cai, Keng Siau and Langtao Chen. "Generative AI and ChatGPT: Applications, Challenges, and AI-Human Collaboration". In: *Journal of Information Technology Case and Application Research* 25.3 (2023), pp. 277–304. DOI: https://doi.org/10.1080/15228053.2023.2233814.

[10] Dimas Wibisono Prakoso, Asad Abdi and Chintan Amrit. "Short Text Similarity Measurement Methods: A Review". In: *Soft Computing* 25.6 (2021), pp. 4699–4723. issn: 1433-7479. DOI: https://doi.org/10.1007/s00500-020-05479-2.

[11] Justin K. Miller and Tristram J. Alexander. "Human-Interpretable Clustering of Short Text Using Large Language Models". In: *Royal Society Open Science* 12.1 (2025), p. 241692. issn: 2054-5703. DOI: https://doi.org/10.1098/rsos.241692.

[12] Vijay Viswanathan, Kiril Gashteovski, Kiril Gashteovski, Carolin Lawrence, Tongshuang Wu and Graham Neubig. “Large Language Models Enable Few-Shot Clustering”. In: *Transactions of the Association for Computational Linguistics* 12 (Apr. 2024), pp. 321–333. issn: 2307-387X. DOI: https://doi.org/10.1162/tacl_a_00648.

[13] Partha Pakray, Alexander Gelbukh and Sivaji Bandyopadhyay. “Natural Language Processing Applications for Low-Resource Languages”. In: *Natural Language Processing* 31.2 (2025), pp. 183–197. DOI: https://doi.org/10.1017/nlp.2024.33.

[14] Solmaz Panahi, John Kelleher and Vasudevan Nedumpozhimana. “When LLMs Annotate: Reliability Challenges in Low-Resource NLI”. In: *Proceedings of the Second Workshop on Language Models for Low-Resource Languages (LoResLM 2026)*. Rabat, Morocco: Association for Computational Linguistics, 2026, pp. 178–188. isbn: 979-8-89176-377-7. DOI: https://doi.org/10.18653/v1/2026.loreslm-1.17.

[15] Dang Minh, H. Xiang Wang, Y. Fen Li and Tan N. Nguyen. “Explainable Artificial Intelligence: A Comprehensive Review”. In: *Artificial Intelligence Review* 55.5 (2022), pp. 3503–3568. issn: 1573-7462. DOI: https://doi.org/10.1007/s10462-021-10088-y.

[16] Emrullah Şahin, Naciye Nur Arslan and Durmuş Özdemir. “Unlocking the Black Box: An in-Depth Review on Interpretability, Explainability, and Reliability in Deep Learning”. In: *Neural Computing and Applications* 37.2 (2025), pp. 859–965. issn: 1433-3058. DOI: https://doi.org/10.1007/s00521-024-10437-2.